МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Информационных систем и технологий
Специальность 6-05-0611-01 «Информационные системы и технологии»

Дисциплина Операционные системы

Лабораторная работа № 2

РАБОТА В КОМАНДНОЙ ОБОЛОЧКЕ

Студент 3 курса 2 группы
Марушко Т. Ф.
Проверил: Ассистент Савельева М. Г.

Минск, 2026

Содержание

Введение....3
1 Выполнение заданий на ОС *Linux*....4
1.1 Подготовка среды и сведения о системе....4
1.2 Уровень 4–5: навигация и операции над файлами и каталогами....5
1.3 Уровни 6–7: копирование, консоль, пакетные сценарии....20
1.4 Исследование переменных окружения....29
1.5 Уровень 8: анализ дерева каталогов и содержимого *PATH*....31
1.6 Уровень 9: монитор процессов в реальном времени....34
1.7 Вывод по разделу....36
2 Выполнение заданий на ОС *Windows*....37
2.1 Подготовка рабочей среды....37
2.2 Уровень 4–5: навигация и операции с файлами....39
2.3 Уровень 4–5: перемещение, переименование, удаление....45
2.4 Уровень 4–5: просмотр, консоль, поиск, сортировка....50
2.5 Уровень 6–7: копирование с заменой, консоль....56
2.6 Уровень 6–7: пакетные сценарии командной оболочки....60
2.7 Уровень 6–7: переменные окружения и *PATH*....65
2.8 Уровень 8: дерево каталогов с размерами файлов....69
2.9 Уровень 8: интерактивный калькулятор на языке *PowerShell*....70
2.10 Уровень 8: конвертер температур на языке *PowerShell*....74
2.11 Уровень 9: монитор процессов в реальном времени....76
2.12 Вывод по разделу....78
Заключение....79
ПРИЛОЖЕНИЕ А....80

Введение

Цель лабораторной работы – освоение приёмов работы в командных оболочках операционных систем семейства *Linux* и *Windows*, получение практических навыков выполнения типовых операций над файлами и каталогами средствами командных оболочек.

Для достижения цели определены задачи:

– изучить способы навигации по файловой системе и структуру каталогов;

– освоить команды создания, копирования, перемещения, переименования и удаления файлов и каталогов;

– изучить механизмы перенаправления ввода-вывода, конвейеры и обработку выходных потоков;

– получить навыки написания пакетных сценариев с параметрами и обработки ошибок;

– освоить разработку сценариев на языке *PowerShell* для операционной системы *Windows* и на оболочке *bash* для операционной системы *Linux*;

– выполнить задания лабораторной работы на двух операционных системах – *Linux* (*Arch Linux ARM*, оболочка *bash*) и *Windows* (*Windows* 11 *ARM*, оболочки *cmd* и *PowerShell*);

– провести исследование переменных окружения и правил наименования команд, оформить результаты исследований и сравнить возможности двух операционных систем.

Работа выполнена на двух виртуальных машинах под управлением *VMware Fusion*. В *Linux* использованы *Arch Linux ARM* и оболочка *bash*, в *Windows* – *Windows* 11 *ARM* и оболочки *cmd* и *PowerShell*. Выполнены задания уровней 4–9; команды и результаты работы сценариев зафиксированы в консоли и на снимках экрана.

1 Выполнение заданий на ОС *Linux*

1.1 Подготовка среды и сведения о системе

Задания лабораторной работы на ОС *Linux* выполнялись в виртуальной машине под управлением *Arch Linux ARM*, запущенной в гипервизоре *VMware Fusion* на том же хосте, что и *Windows*-часть работы. Работа велась в графической оболочке *KDE Plasma* с терминалом *Konsole* под учётной записью пользователя, созданной в лабораторной работе №1. Все задания выполнялись только с клавиатуры, без использования мыши.

Сценарий *prep.sh* приведён в листинге А.1; он создаёт структуру *OS*2, каталоги для копирования, перемещения и удаления, а также файлы-заготовки с требуемыми правами доступа.

Сведения о системе получены командой *uname*, вывод которой приведён ниже: имя узла *alarm*, версия ядра 7.2.8-1-*aarch*64-*ARCH*, архитектура *aarch*64, дистрибутив *Arch Linux ARM*. Эти данные фиксируются один раз в начале работы и далее используются в выводах.

Версия *BIOS* виртуальной машины прочитана из файла */sys/class/dmi/id/bios_version*; получено значение *VMW*201.00*V*.21805430.*BA*64.2305221830. Результат показан на рисунках 1.1 и 1.2.

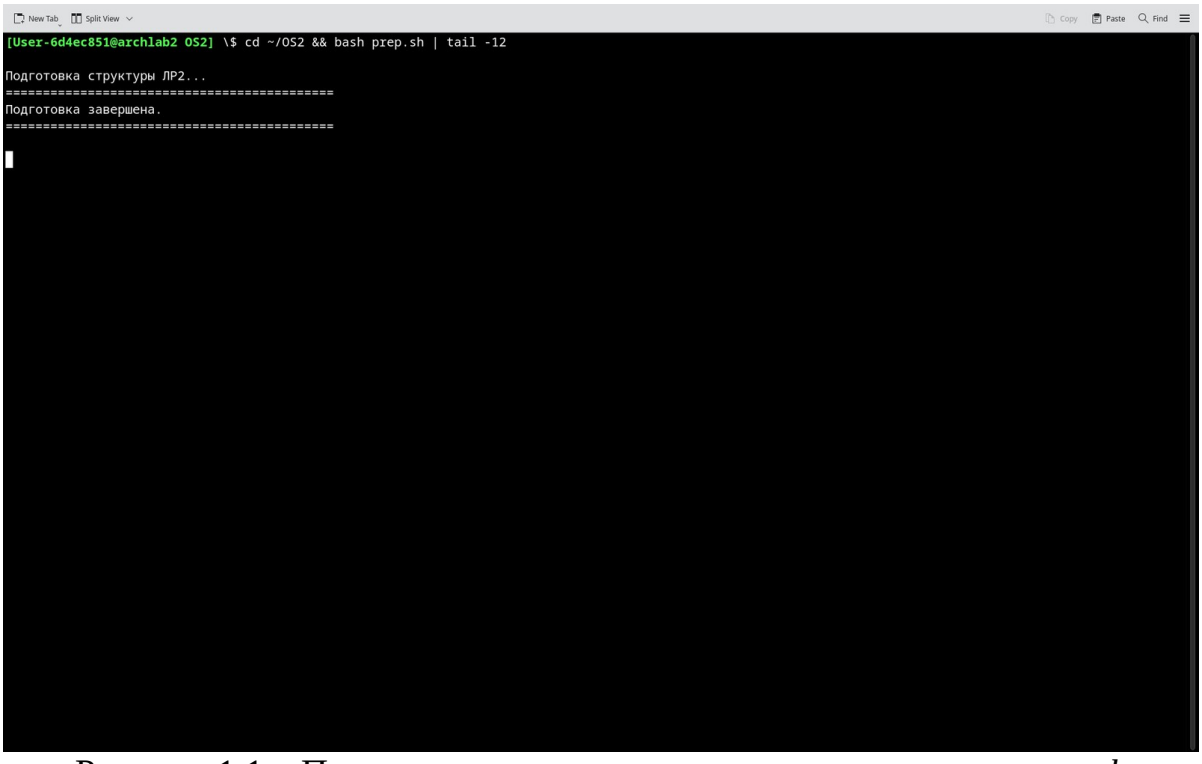


Рисунок 1.1 – Подготовка структуры каталогов сценарием *prep.sh*

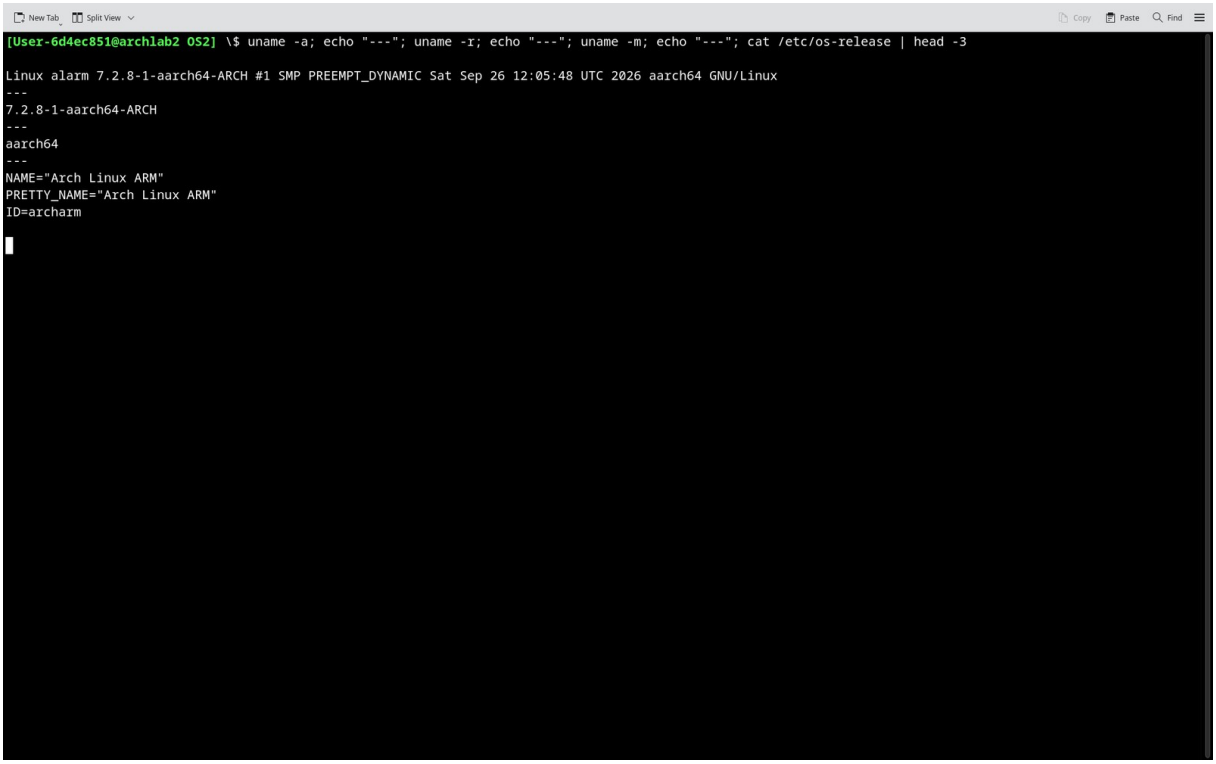


Рисунок 1.2 – Сведения о системе: версия ядра, архитектура, дистрибутив

Среда подготовлена: версия ядра и дистрибутив соответствуют заданию, можно переходить к выполнению уровней.

1.2 Уровень 4–5: навигация и операции над файлами и каталогами

На уровнях 4 и 5 проверяется базовый набор приёмов работы в командной оболочке: навигация по файловой системе, создание и удаление каталогов, копирование, перемещение и переименование файлов, сравнение содержимого, удаление с подтверждением, а также механизмы перенаправления вывода, конвейеры и утилиты *grep*, *sort*, *find*.

Переход в каталог выполняется командой *cd*: абсолютный путь задаётся от домашнего каталога (~/*OS*2/*task*_01*i*/Русский), относительный – перечислением компонентов с двойной точкой (*cd* ../*English*). Проверка текущего каталога выполняется командой *pwd*, а быстрая проверка перехода – выводом *ls*.

Создание каталогов выполняется командой *mkdir*, а полная иерархия каталогов *sub*/*dir*/*full*/*path* создаётся одним вызовом *mkdir -p*: ключ *-p* создаёт все промежуточные каталоги и не завершает работу с ошибкой, если каталог уже существует. Удаление пустого каталога выполняется командой *rmdir*.

Копирование выполняется командой *cp*. Копирование группы файлов по маске (*.*txt*) в другой каталог выполняется одной командой *cp* ~/*OS*2/*tasks_tmp*/*.*txt* ~/*OS*2/*task*_01*a*/. Интерактивный режим с подтверждением перезаписи включается ключом *-i*, а копирование с присвоением нового имени выполняется указанием двух полных путей. Объединение содержимого двух файлов в третий выполняется конвейером *cat* с перенаправлением вывода: *cat txt*1.*txt txt*2.*txt* > *merged*.*txt*. Копирование выполняется в три этапа, результаты показаны на рисунках ниже.

Перемещение и переименование выполняются командой *mv*. Смена расширения у группы файлов реализуется циклом *for* с подстановкой параметра: *for f in *.txt*; *do mv* «$*f*» «${*f*%.*txt*}.*bmp*»; *done* – такая конструкция заменяет суффикс в имени каждого файла, не затрагивая остальные символы. Переименование файла в имя текущего пользователя выполняется подстановкой переменной $*USER*: результат предсказуем и не зависит от конкретного имени учётной записи.

Сравнение двух файлов выполняется командой *diff*, результат которой сохраняется в файле *result.txt* перенаправлением вывода: *diff txt*1.*txt txt*2.*txt* > *result.txt*. Пустой файл результата означает, что файлы идентичны; непустой содержит строки отличий.

Для проверки операций с файлами применялись команды *cp*, *mv*, *diff* и *rm*. Файлы копировались по маске и с новым именем, два файла объединялись командой *cat* с перенаправлением в *merged.txt*. Сравнение выполнено командой *diff txt*1.*txt txt*2.*txt* > *result.txt*; пустой *result.txt* означает отсутствие различий. Файлы и каталоги удалялись командами *rm* и *rmdir*, а рекурсивная очистка выполнялась *rm -rf*.

Постраничный просмотр дерева каталогов выполняется командой *ls -R* с передачей вывода в постраничный просмотрщик *less*: *ls -R ~/ | less*. Такой просмотр удобен для больших деревьев, поскольку позволяет вернуться к предыдущему экрану клавишей *q* или прокручивать вывод клавишами *PgUp* и *PgDn*.

Алфавитный список содержимого каталога сохраняется в файл командой *ls* -1 с перенаправлением вывода: *ls* -1 ~/*OS*2/*task*_01*h* > ~/*OS*2/*task*_01*h*/*user_files.txt*. Ключ -1 выводит по одному элементу в строке, что делает файл пригодным для последующей обработки.

В *Bash* операторы передачи вывода «>» и «>>» создают или дополняют файл; конвейер «|» передаёт стандартный вывод одной команды на стандартный ввод следующей. Например, список студентов вводится командой *cat > students.txt* до *Ctrl+D*, затем фамилии отбираются *grep*, новые строки добавляются через *echo* и «>>», а список сортируется командой *sort*. В *Windows* аналогичный ввод выполняется через *type con* с завершением *Ctrl+Z*.

Отдельно проверяется работа с атрибутами файлов. В *Linux* право исполнения устанавливается командой *chmod*: списочный режим (*chmod +x* скрипт) добавляет бит исполнения владельцу, группам и остальным, а символьный режим (*chmod* 644 файл, *chmod* 755 скрипт) задаёт полный набор прав напрямую. Вывод признака исполняемости в листинге каталога соответствует колонке *x* и используется для проверки успешного назначения прав.

Все команды заданий уровней 4–5 выполнены. Результаты операций приведены на рисунках 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 1.10, 1.11, 1.12, 1.13, 1.14, 1.15, 1.16, 1.17, 1.18, 1.19, 1.20, 1.21, 1.22, 1.23, 1.24, 1.25, 1.26, 1.27 и 1.28.

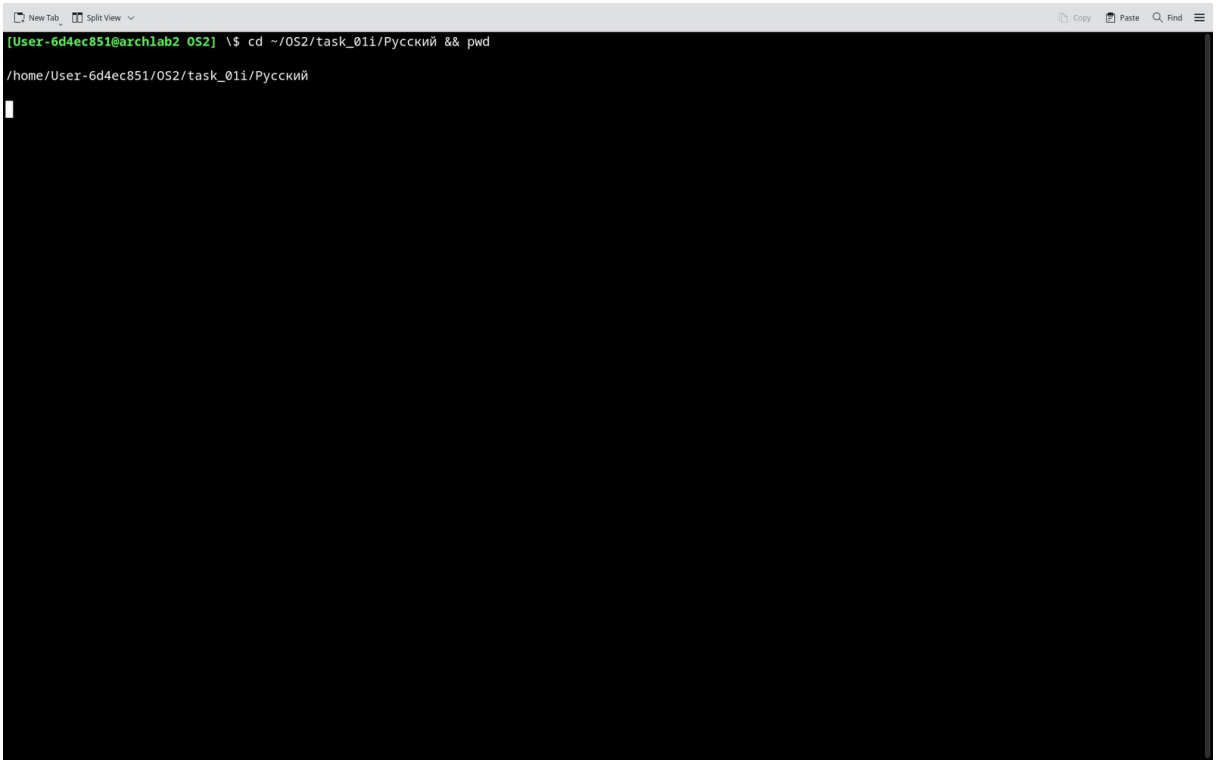


Рисунок 1.3 – Переход в каталог по абсолютному пути

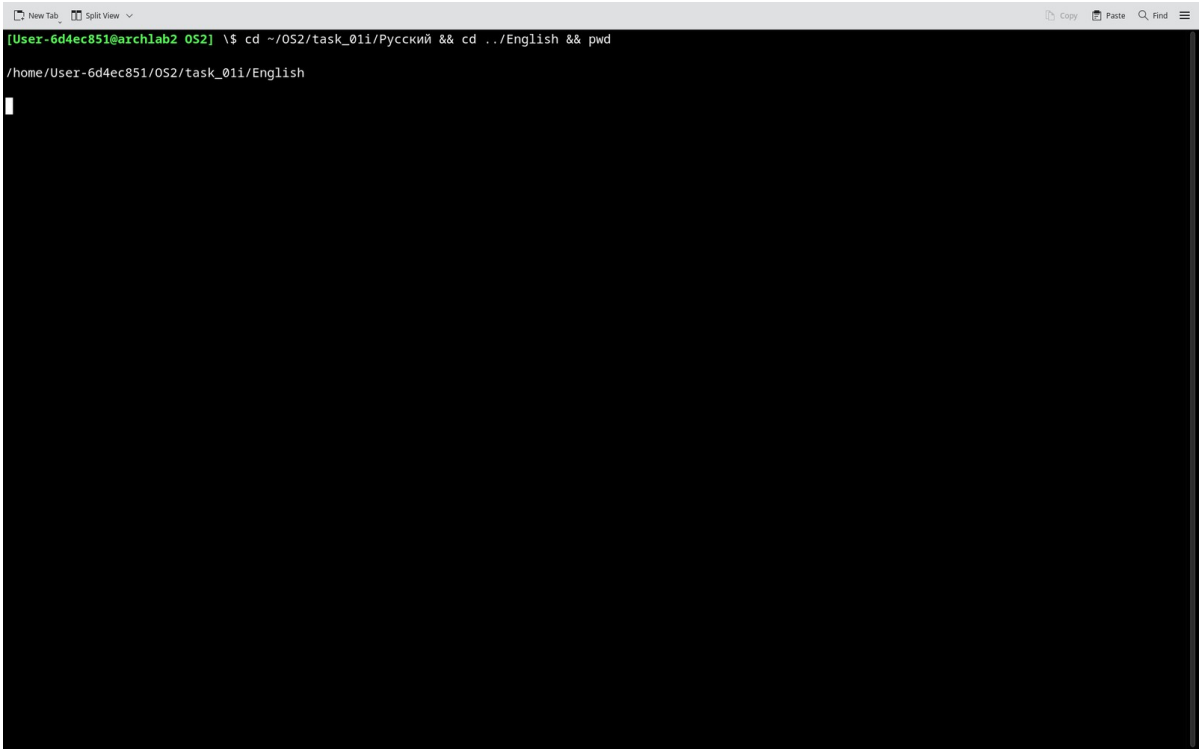


Рисунок 1.4 – Переход в каталог по относительному пути

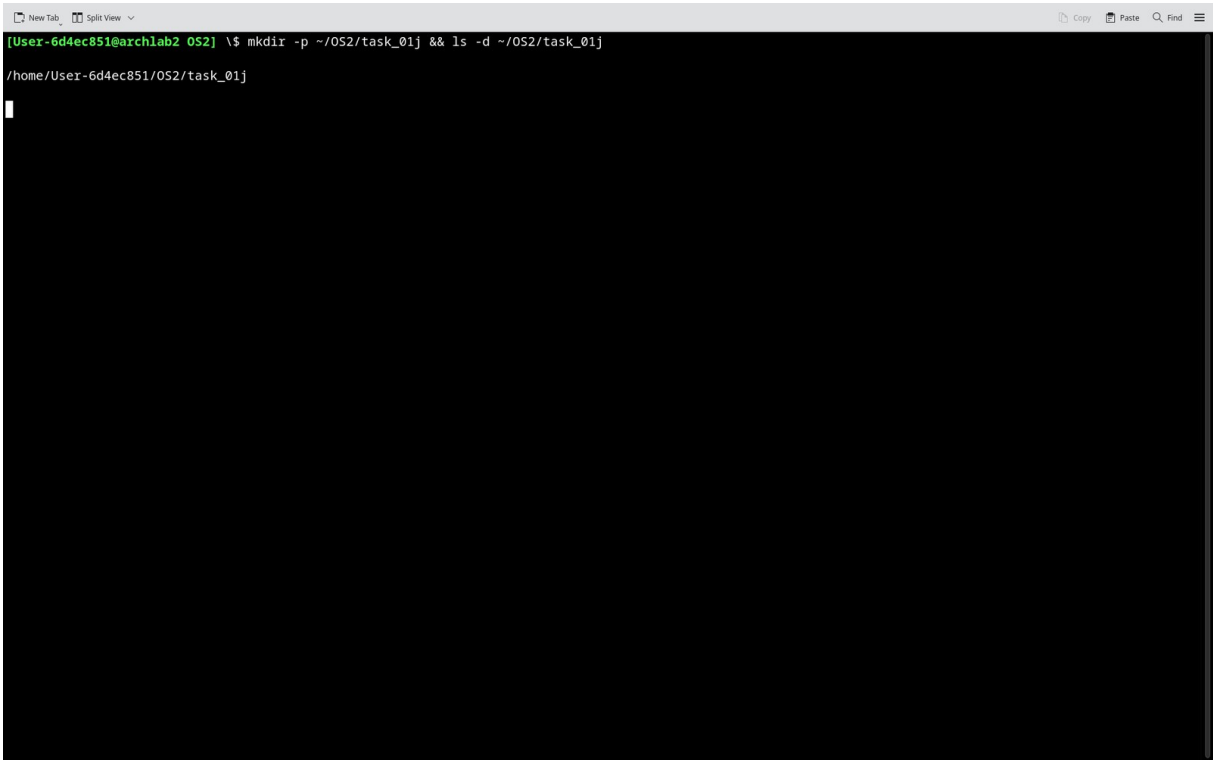


Рисунок 1.5 – Создание каталога командой *mkdir*

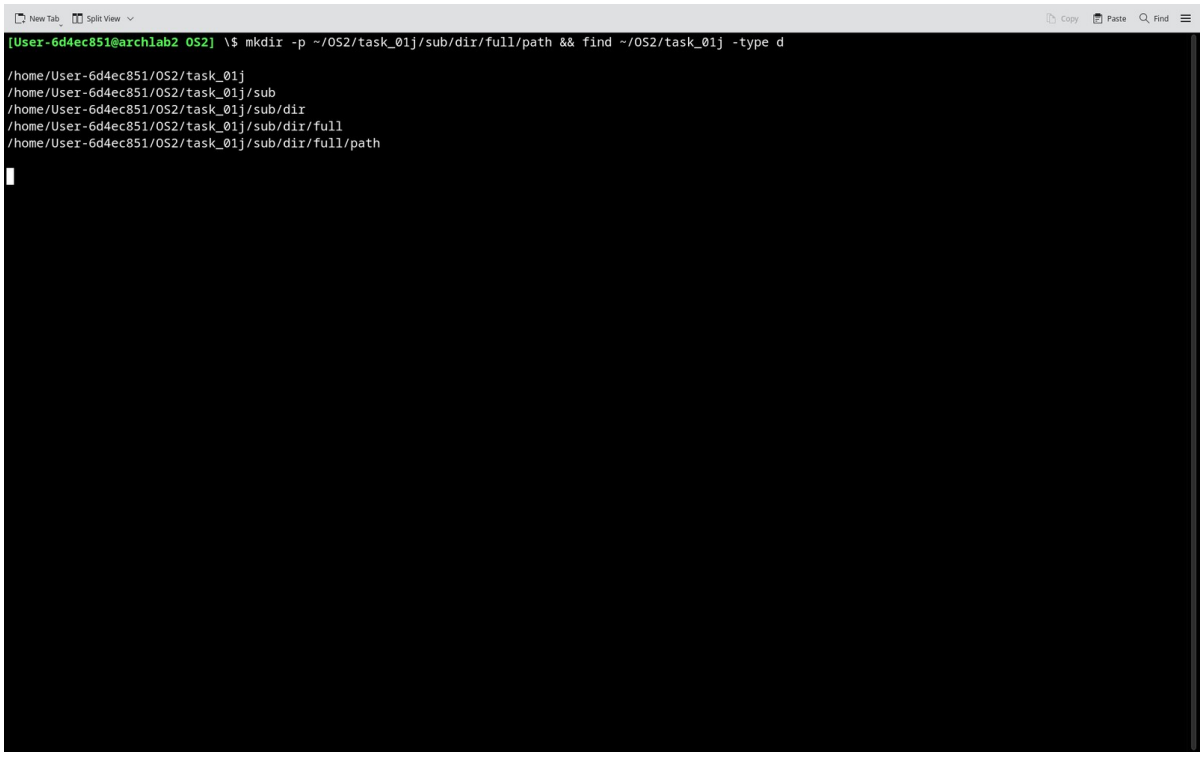


Рисунок 1.6 – Создание иерархии каталогов ключом *-p*

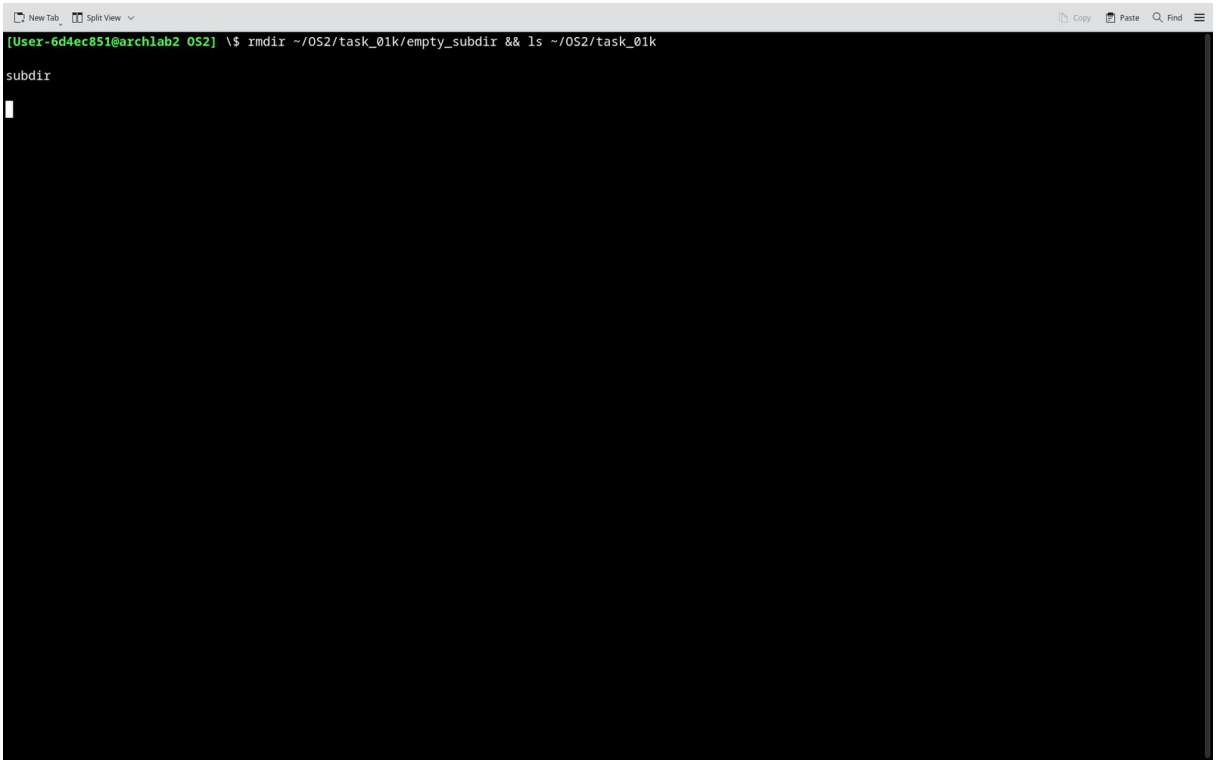


Рисунок 1.7 – Удаление пустого каталога командой *rmdir*

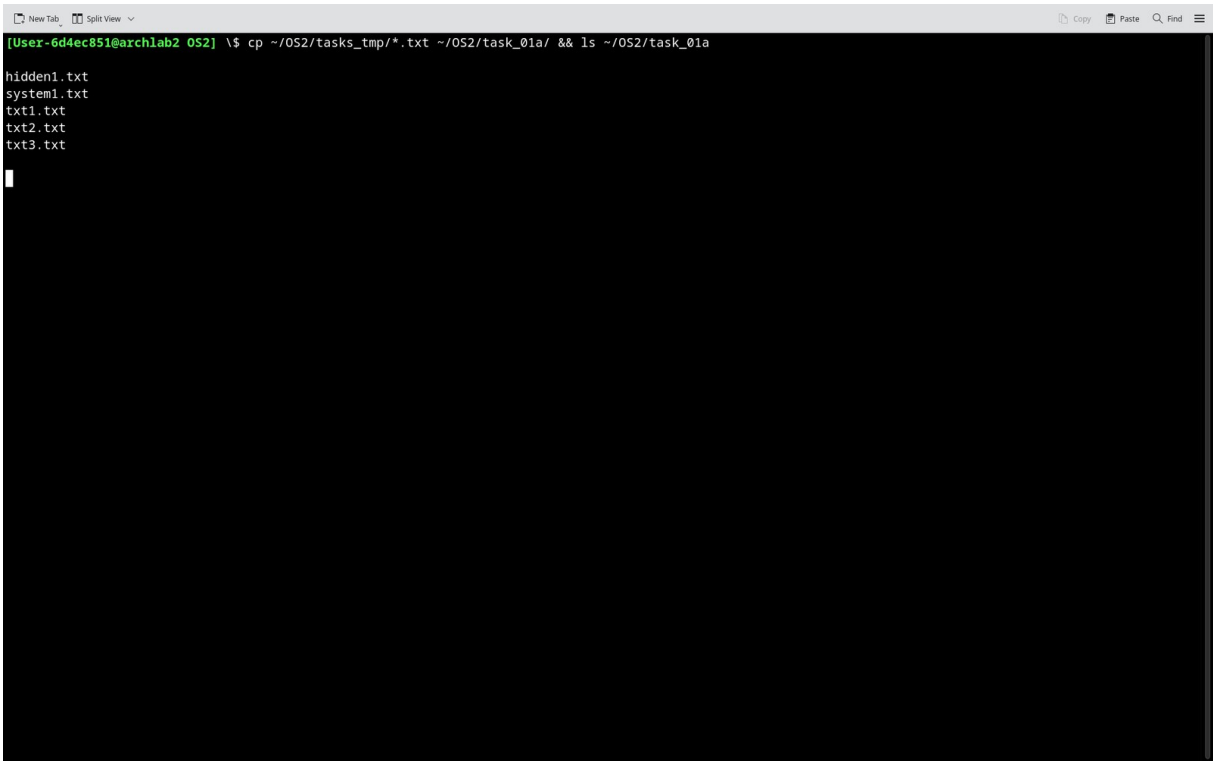


Рисунок 1.8 – Копирование файлов *.txt* по маске в другой каталог

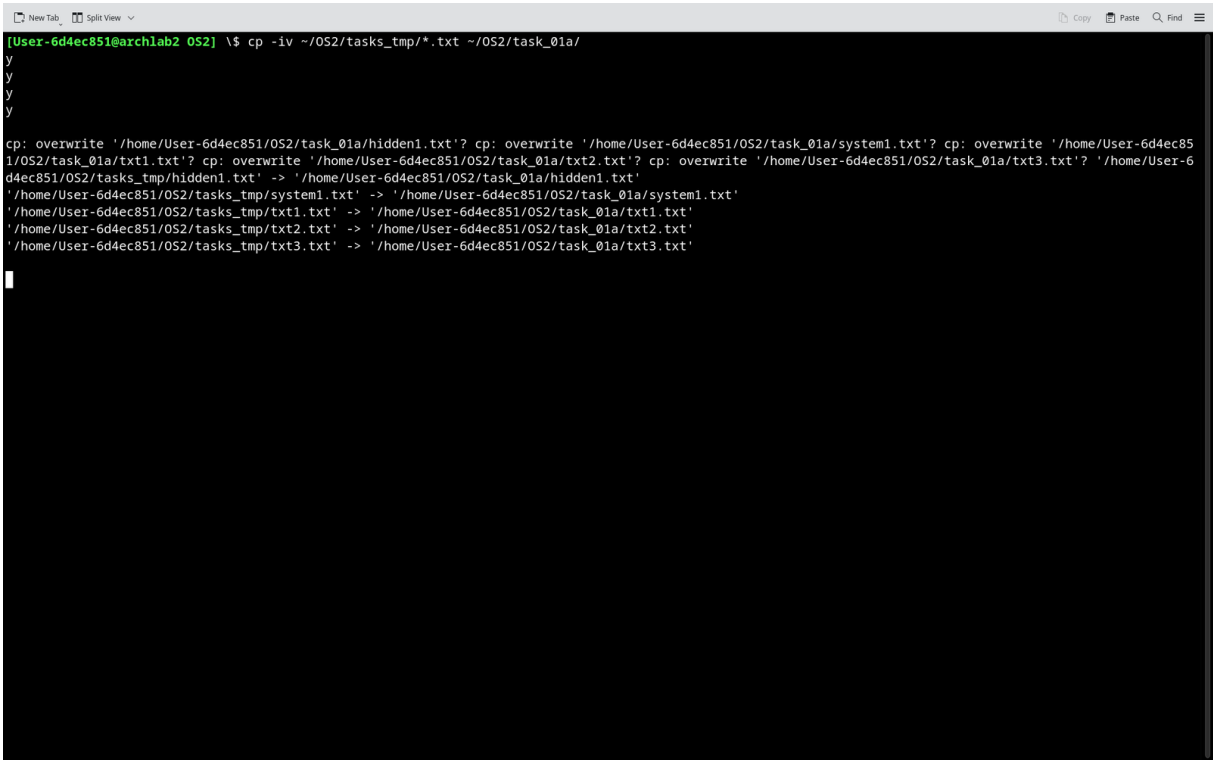


Рисунок 1.9 – Копирование с интерактивным подтверждением

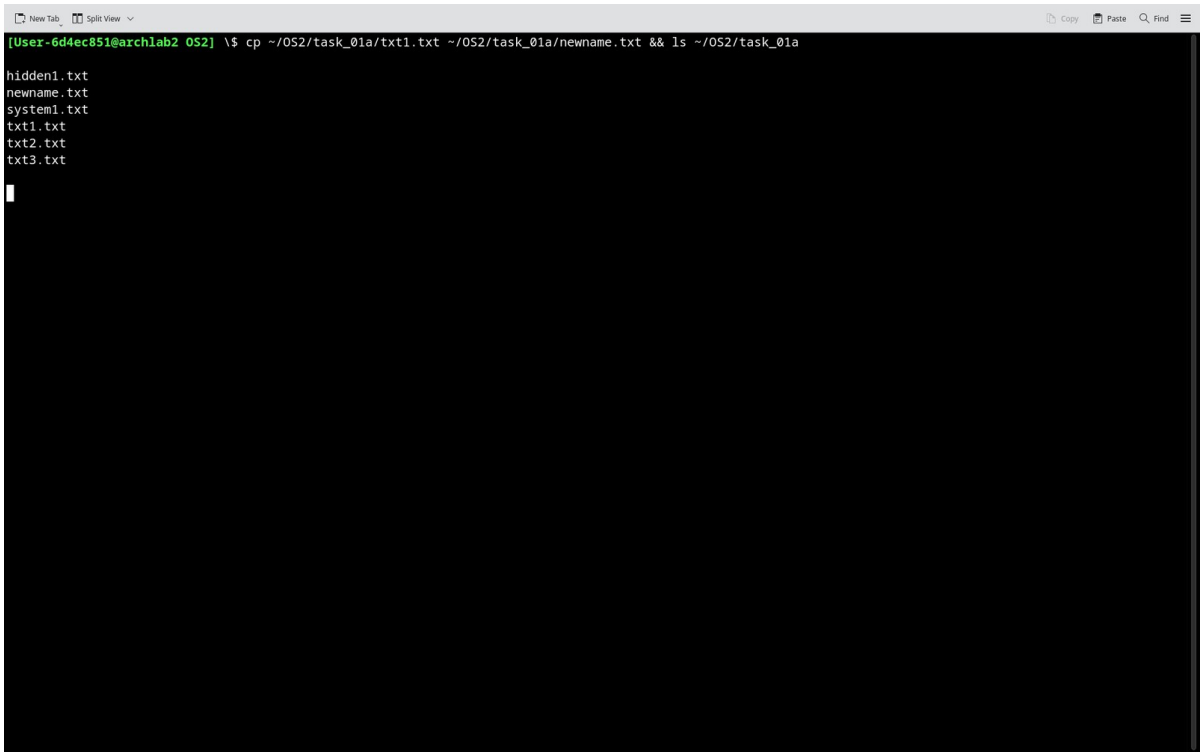


Рисунок 1.10 – Копирование файла с присвоением нового имени

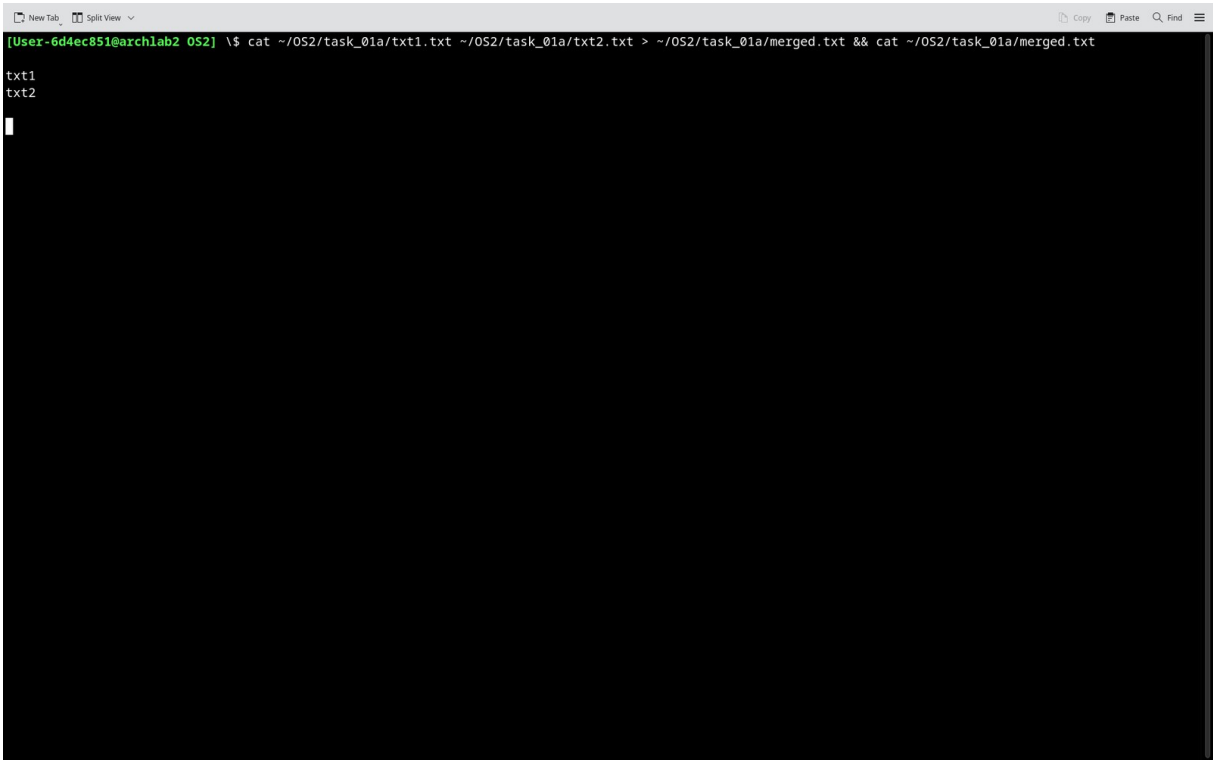


Рисунок 1.11 – Объединение содержимого двух файлов командой *cat*

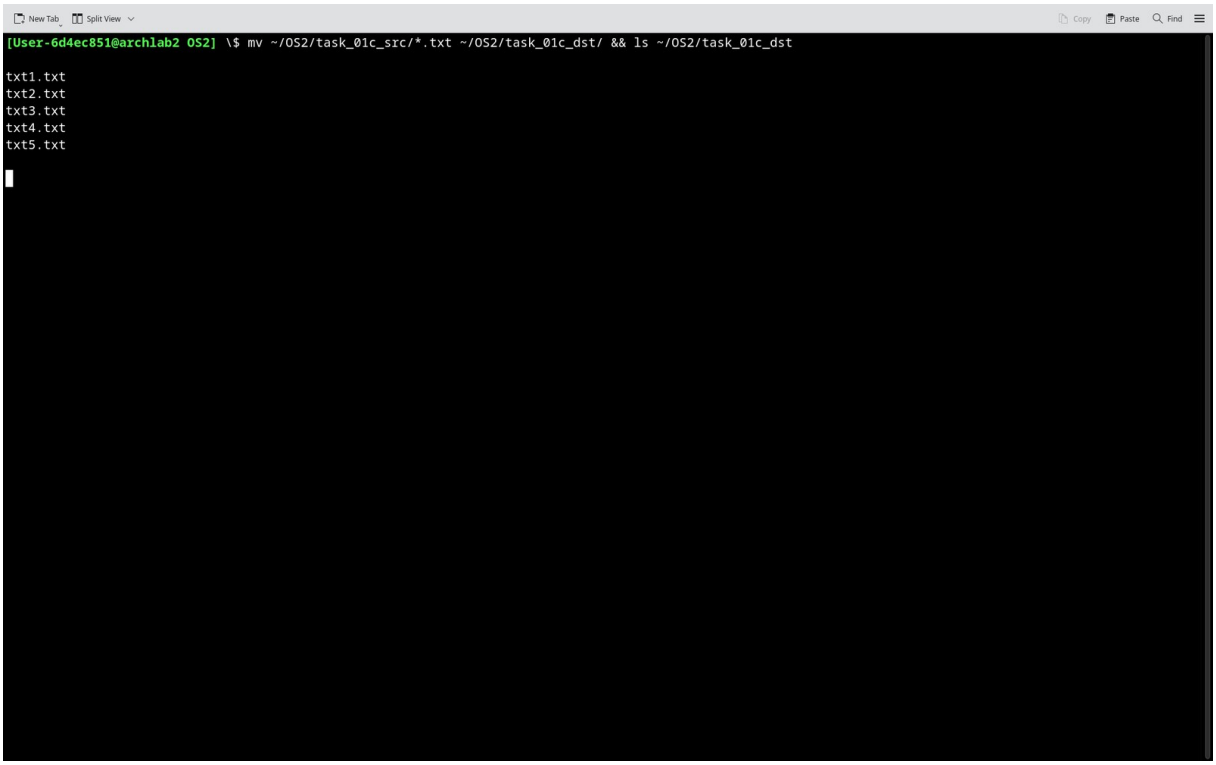


Рисунок 1.12 – Перемещение файлов *.txt* в другой каталог

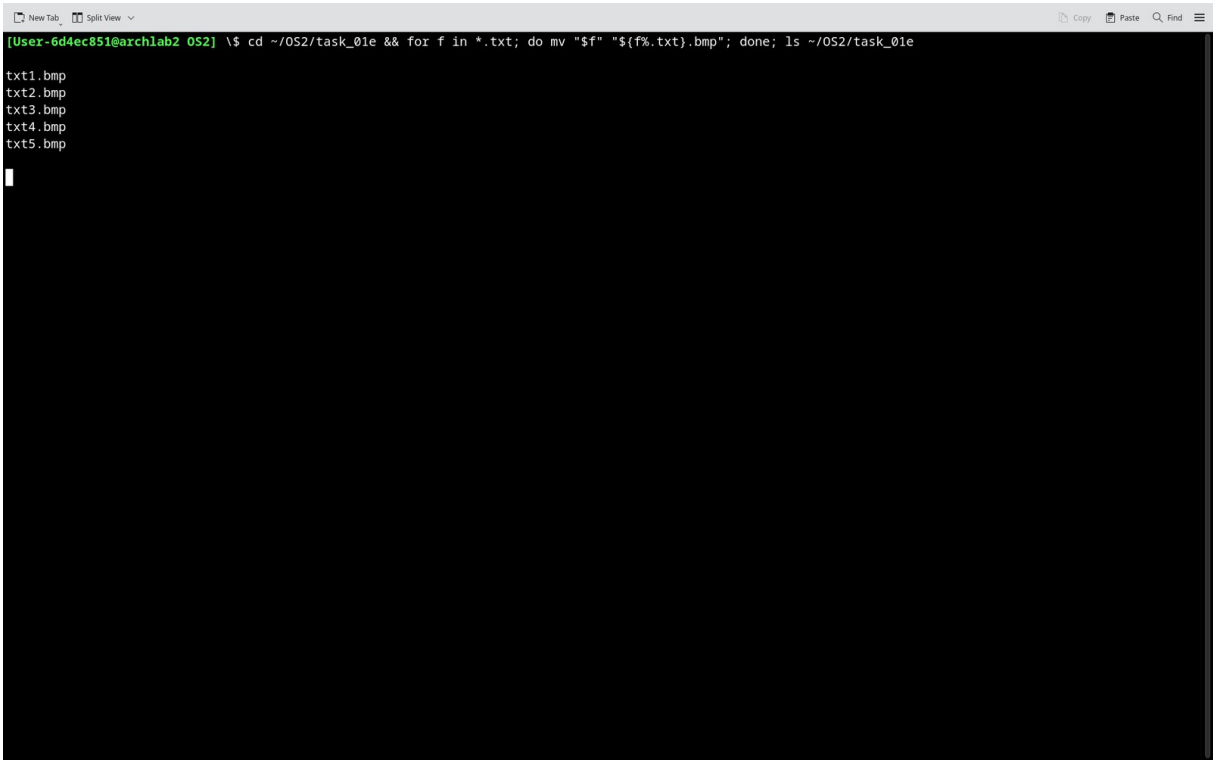


Рисунок 1.13 – Смена расширения у группы файлов циклом *for*

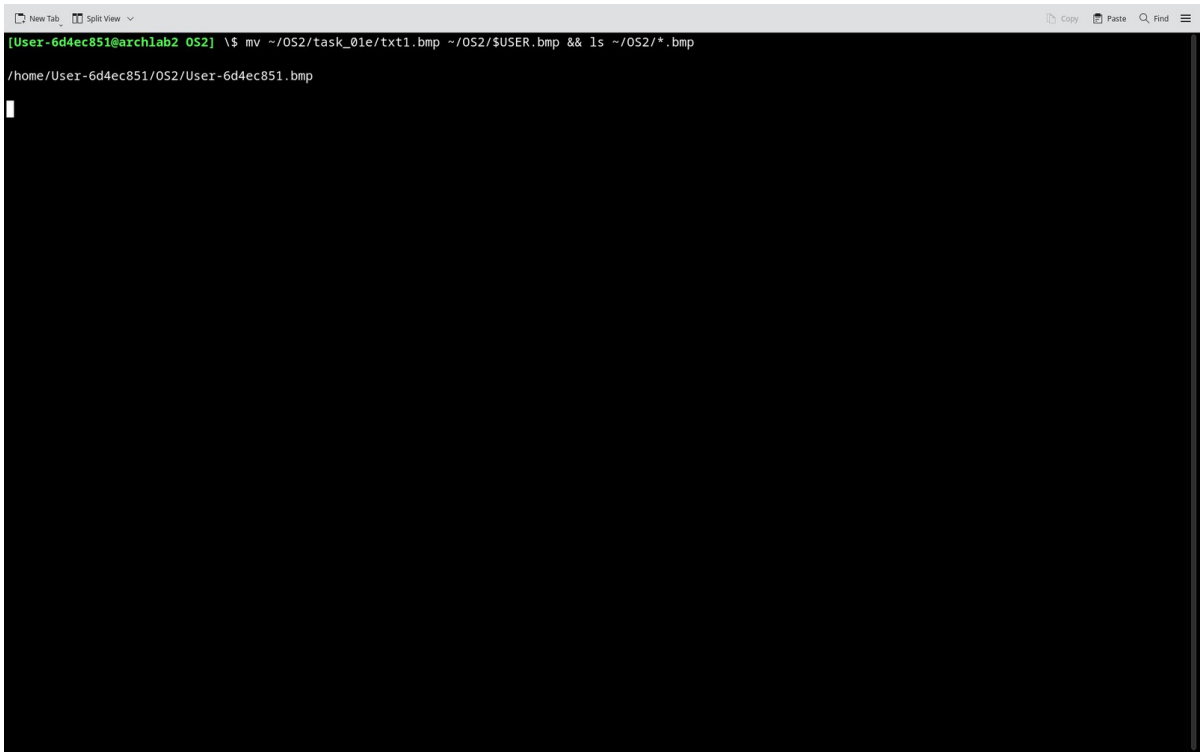


Рисунок 1.14 – Переименование файла с использованием переменной $*USER*

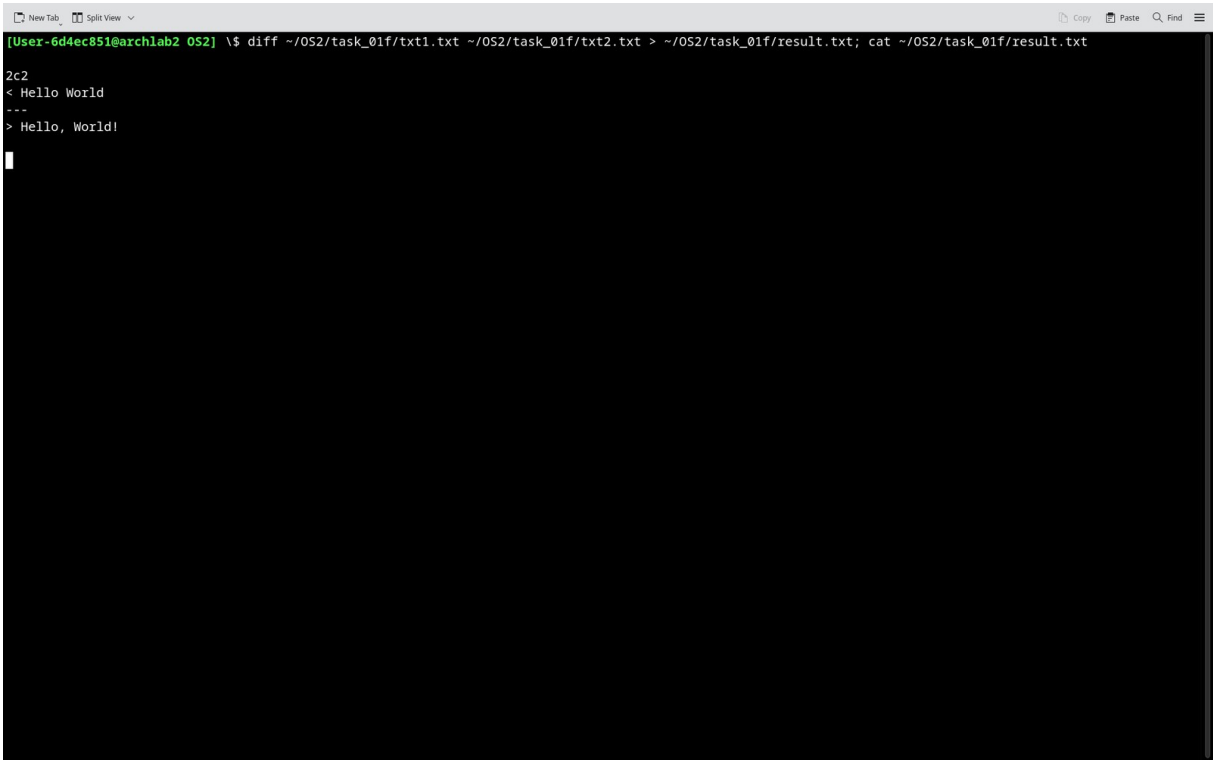


Рисунок 1.15 – Сравнение файлов командой *diff* с сохранением результата

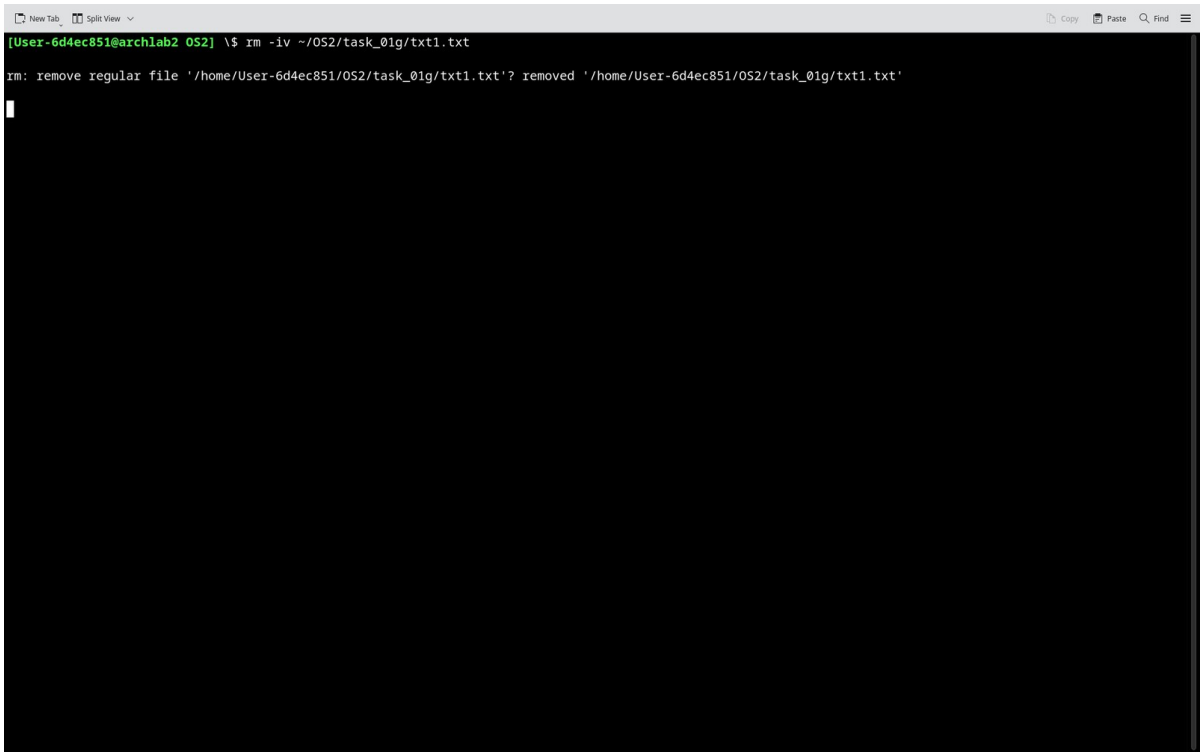


Рисунок 1.16 – Удаление файла с подтверждением

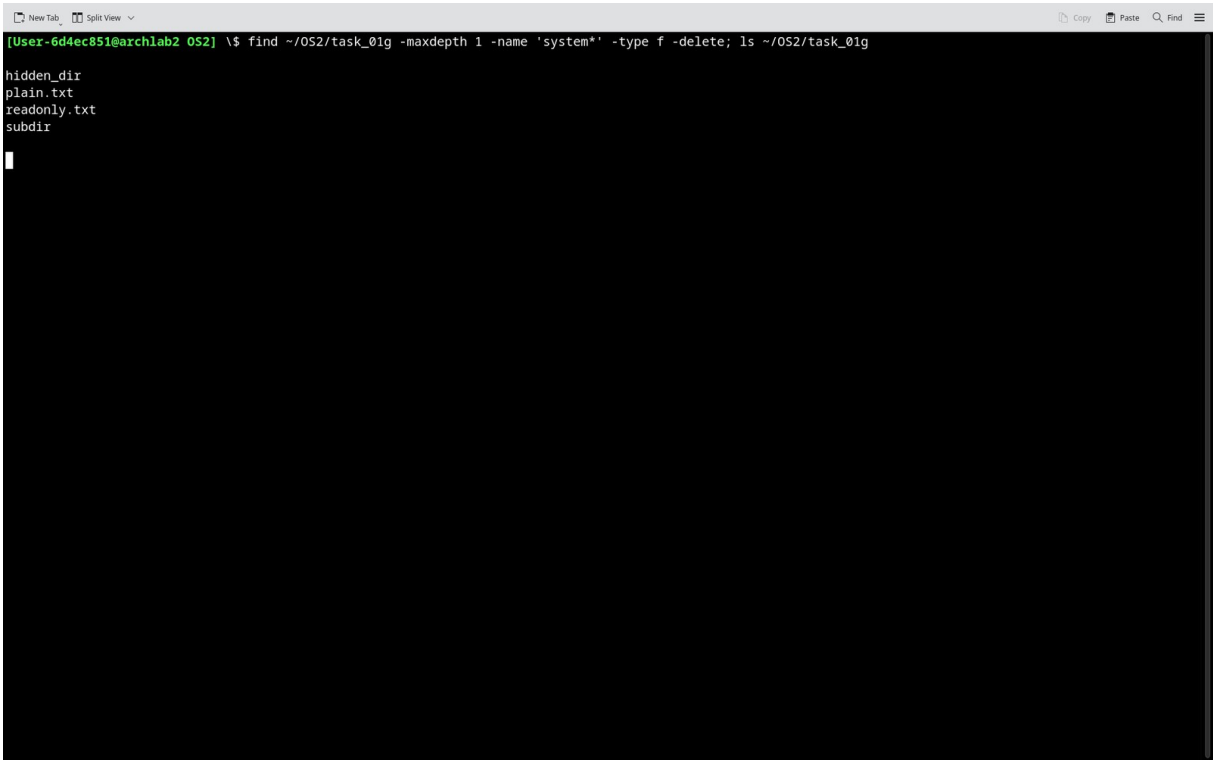


Рисунок 1.17 – Массовое удаление системных файлов связкой *find* и *xargs*

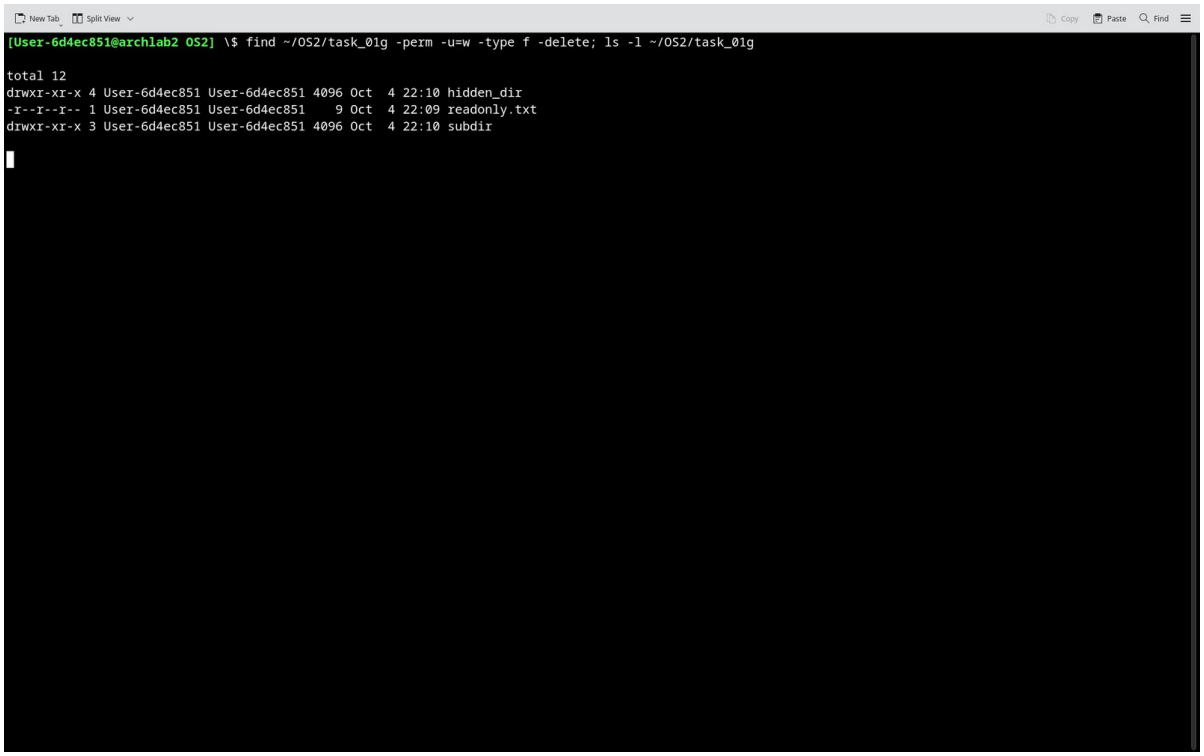


Рисунок 1.18 – Удаление всех файлов, кроме файлов только для чтения

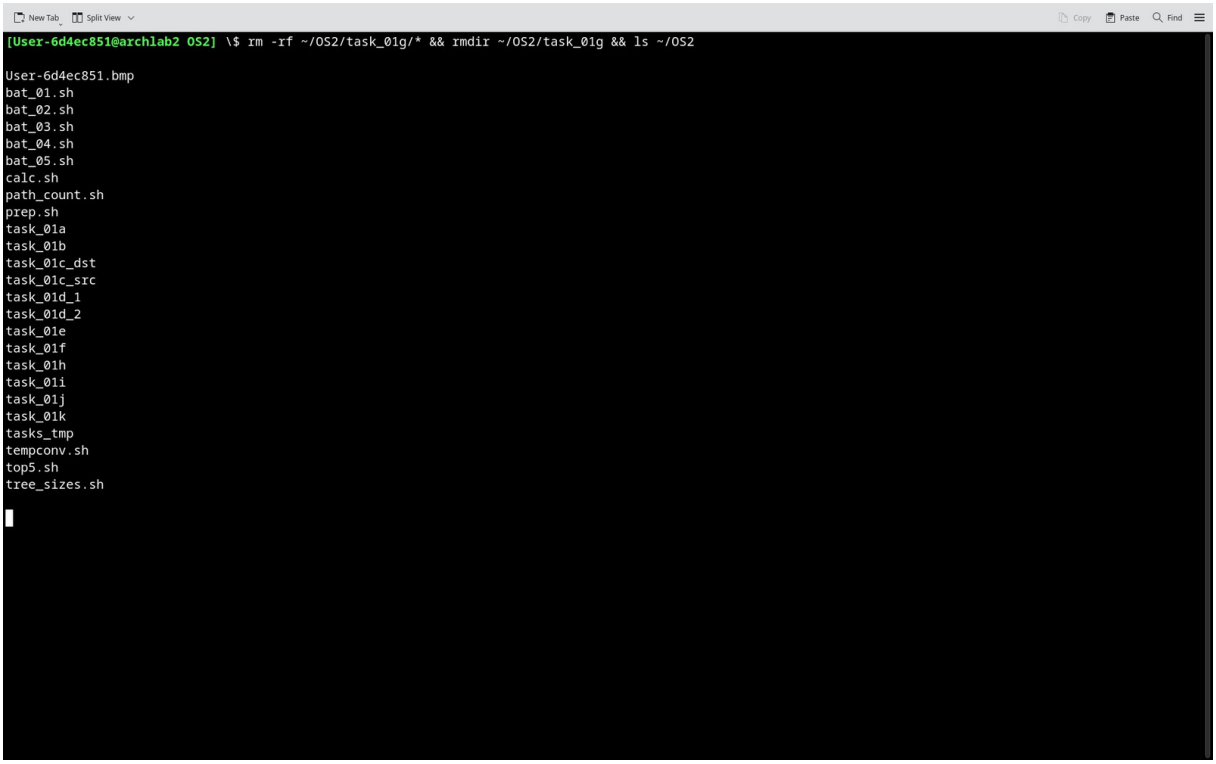


Рисунок 1.19 – Удаление остатков содержимого и самого каталога

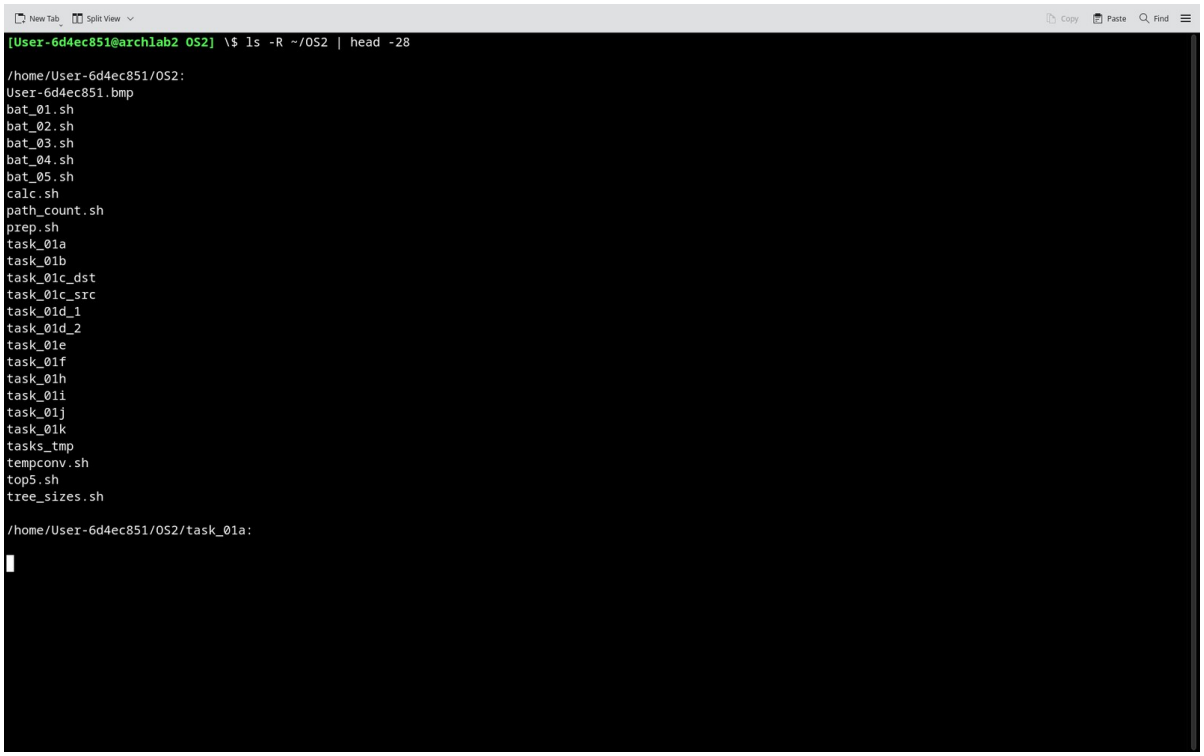


Рисунок 1.20 – Постраничный просмотр дерева каталогов через *less*

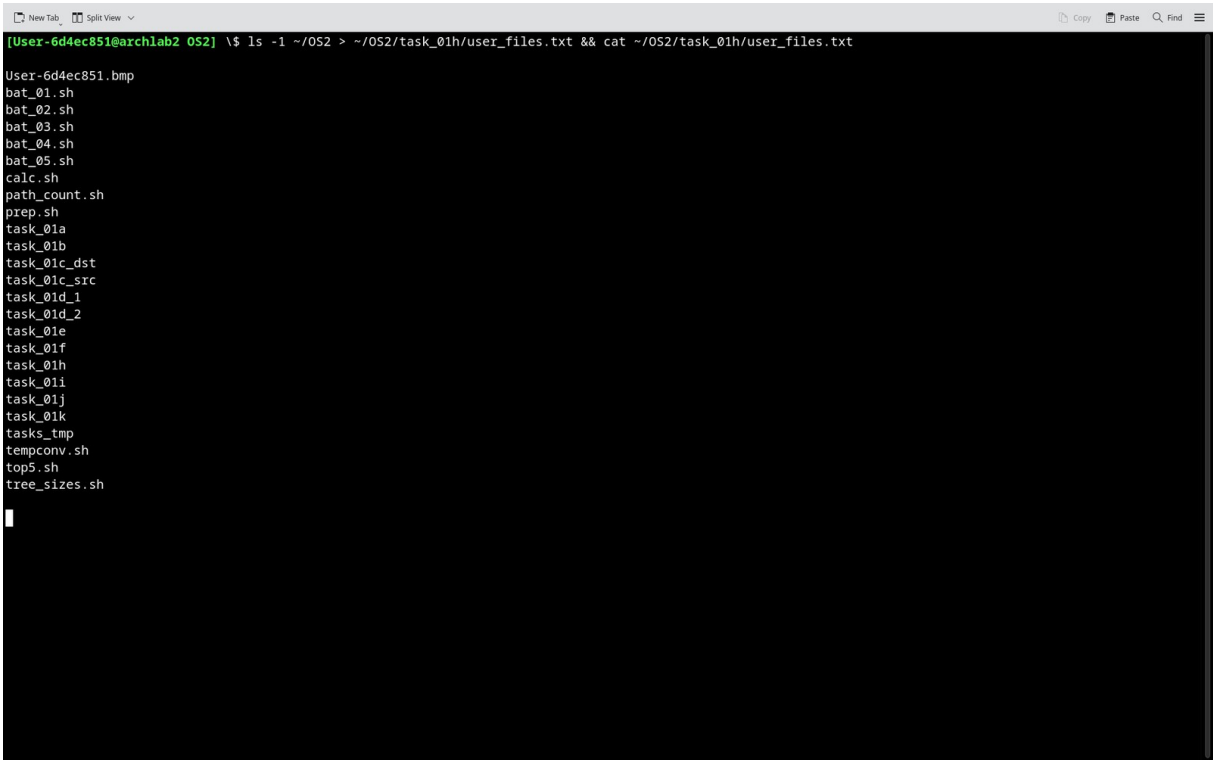


Рисунок 1.21 – Сохранение алфавитного списка каталога в файл

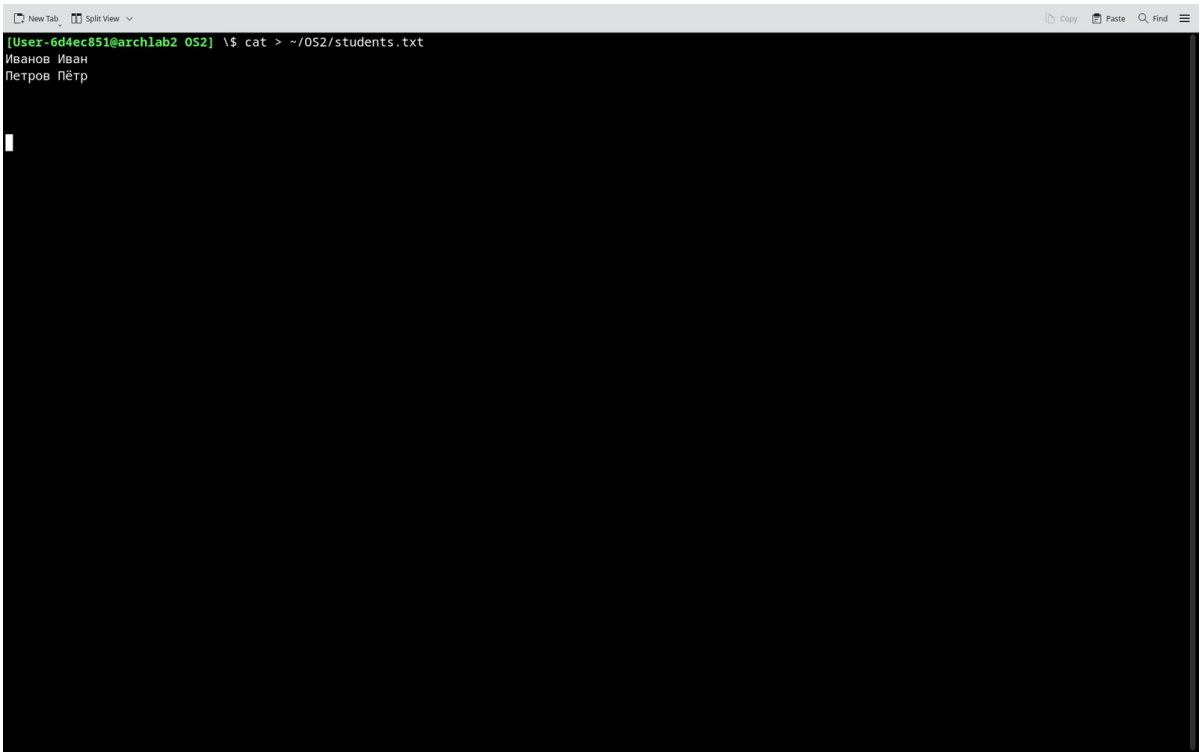


Рисунок 1.22 – Ввод списка студентов с клавиатуры в файл

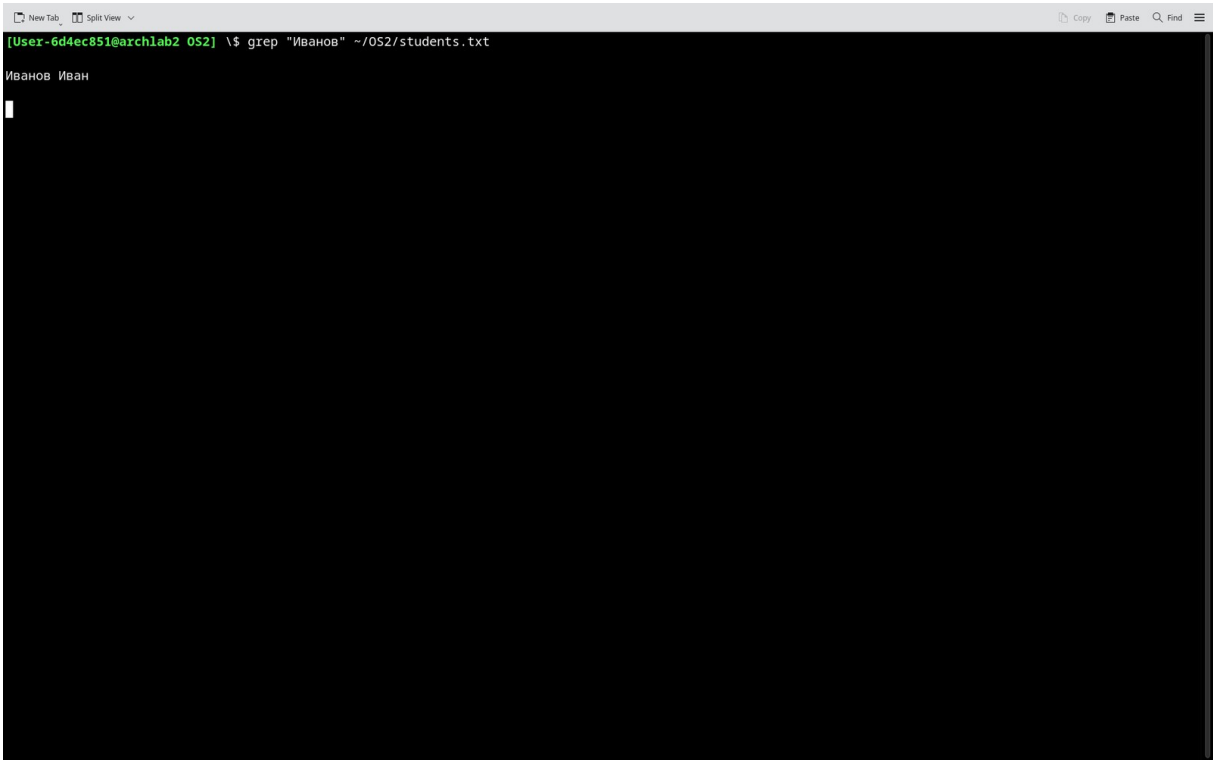


Рисунок 1.23 – Поиск строки в файле командой *grep*

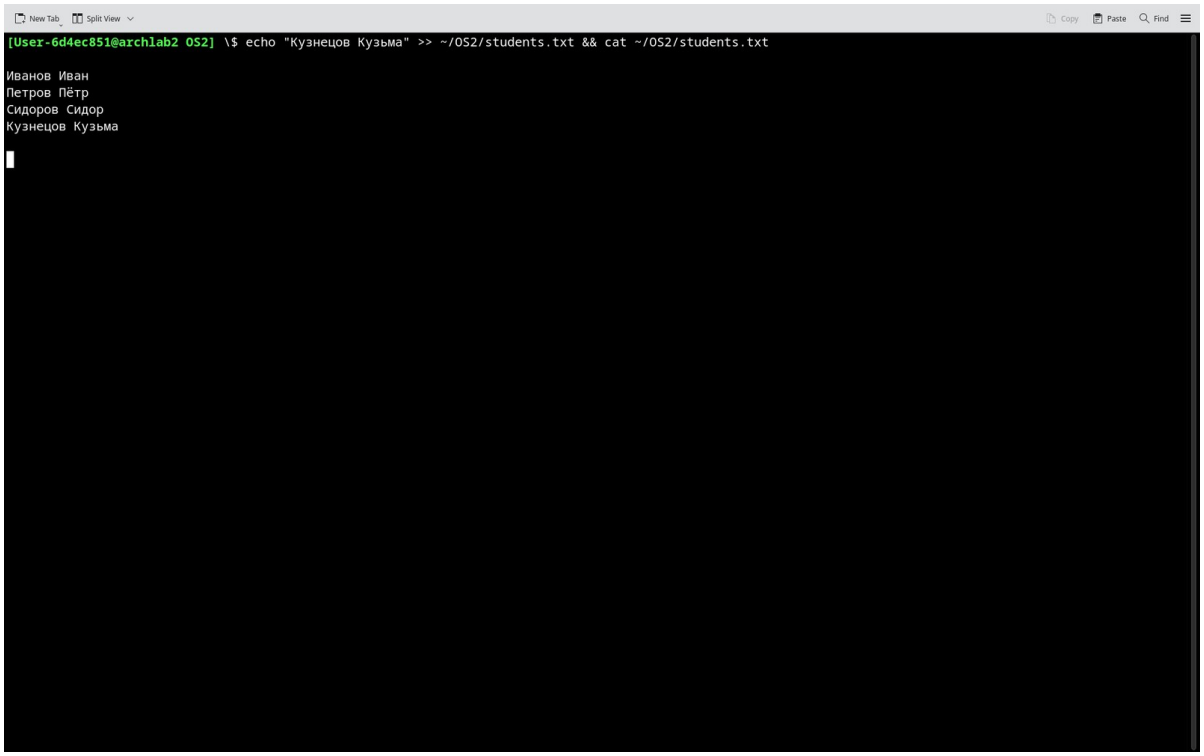


Рисунок 1.24 – Дописывание строки в конец файла перенаправлением >>

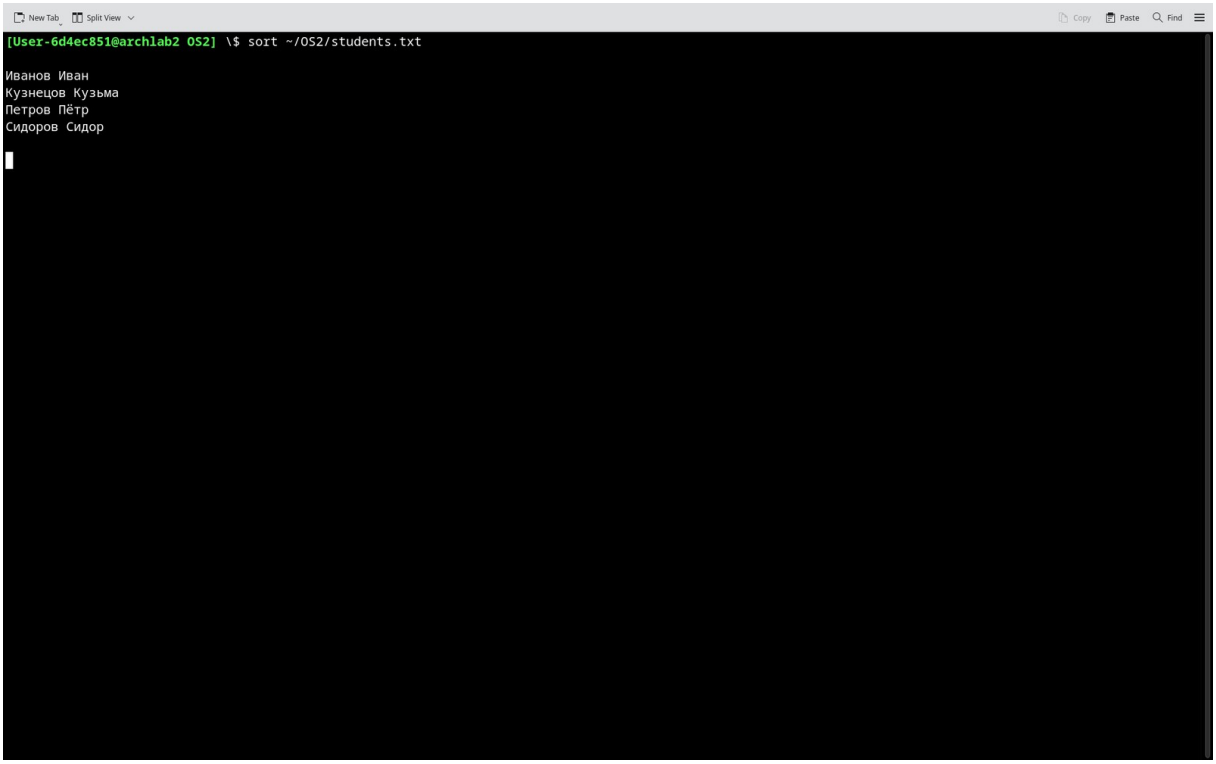


Рисунок 1.25 – Сортировка содержимого файла командой *sort*

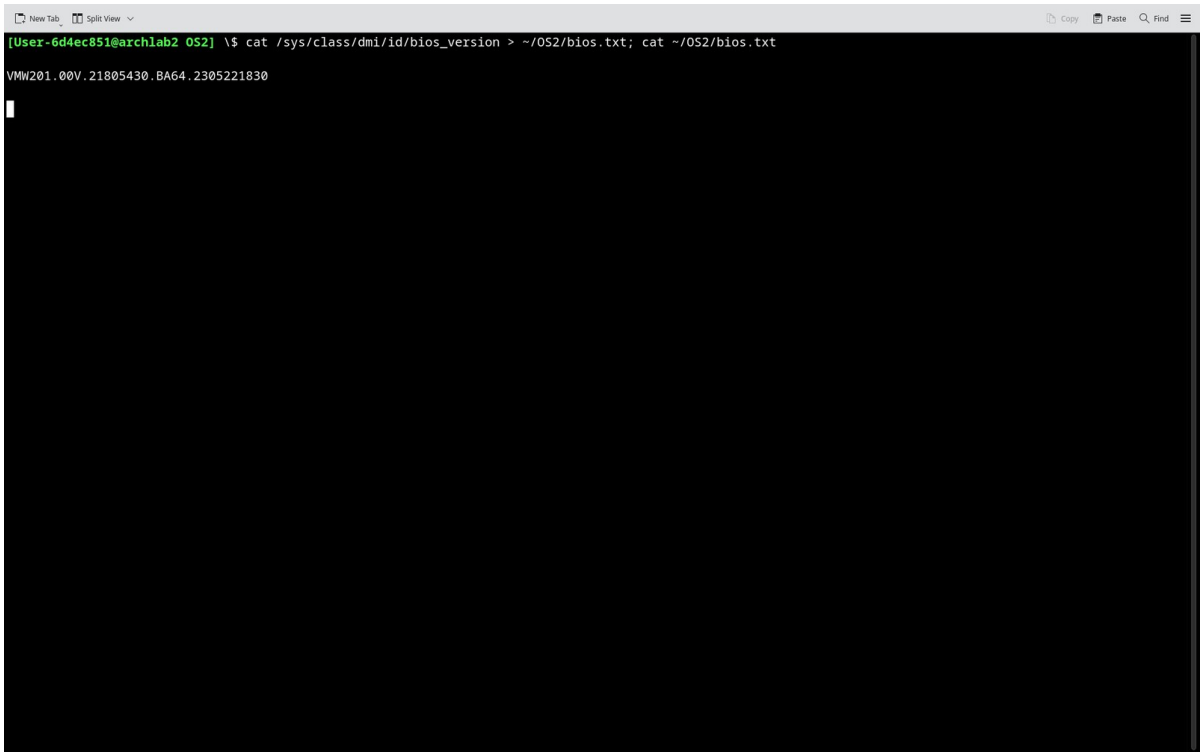


Рисунок 1.26 – Получение версии *BIOS* через интерфейс *sysfs*

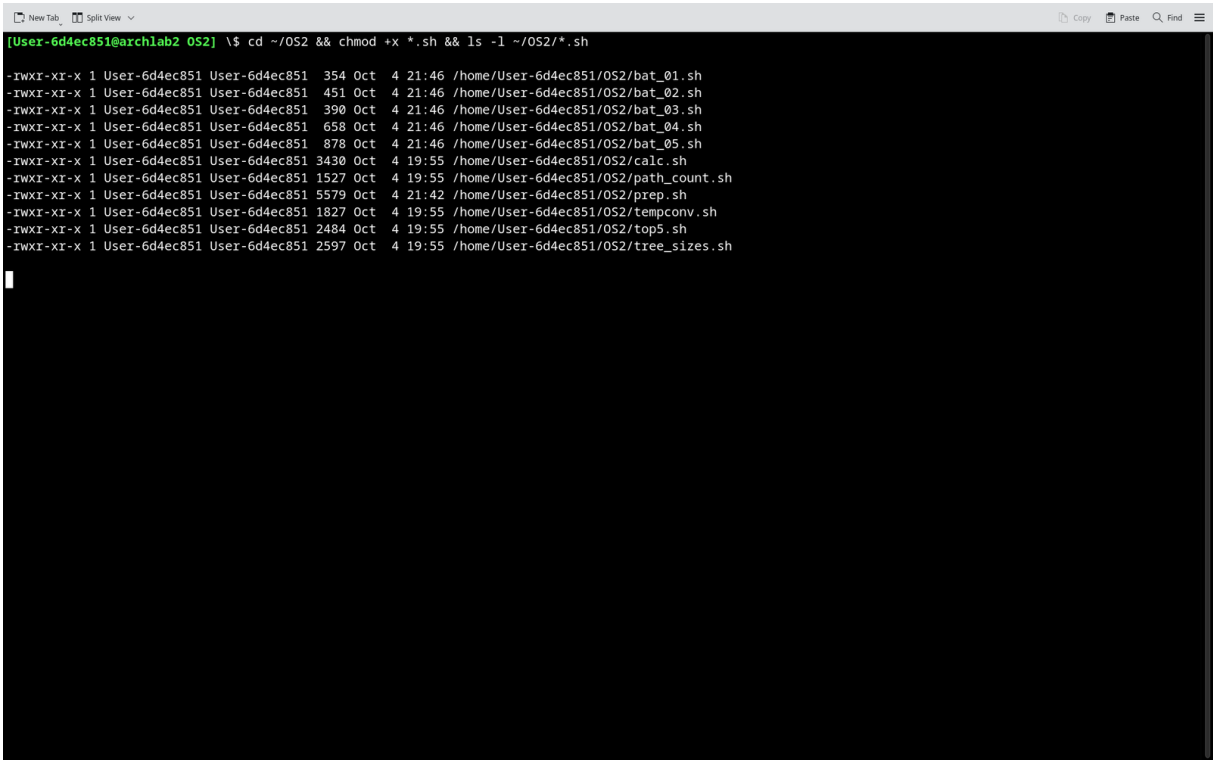


Рисунок 1.27 – Назначение права исполнения командой *chmod*

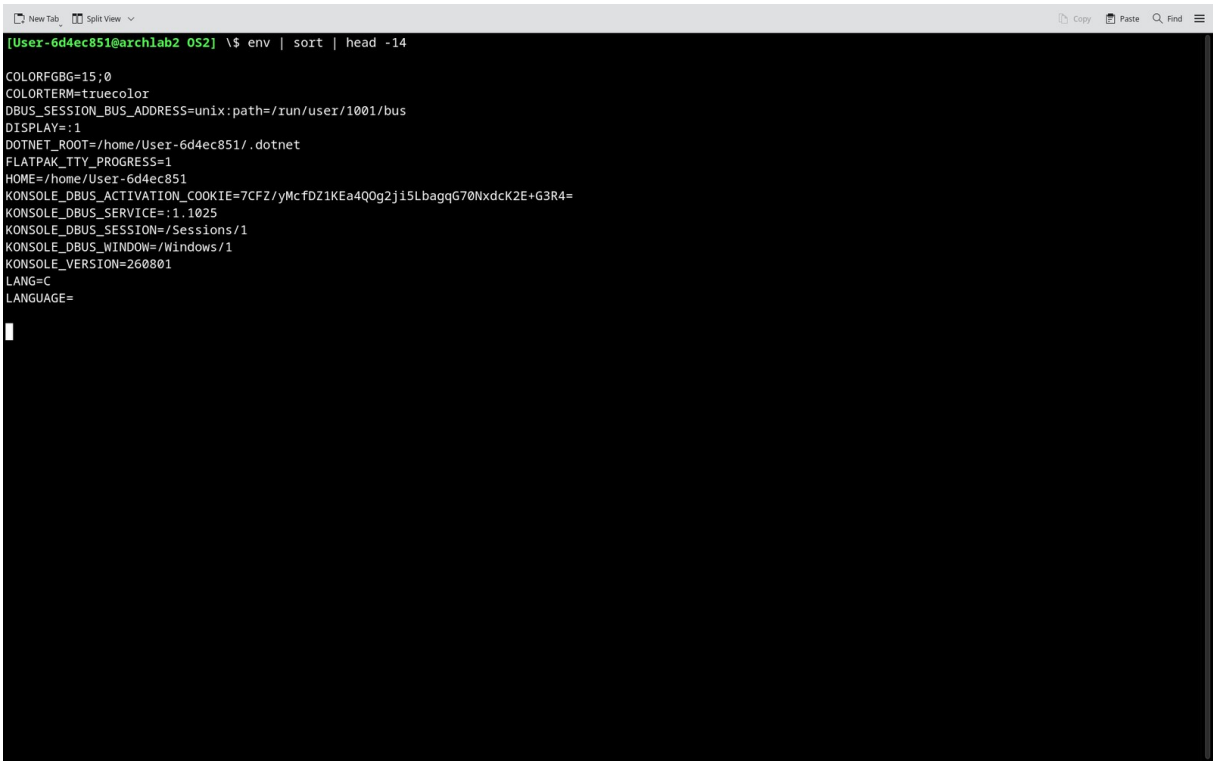


Рисунок 1.28 – Вывод переменных окружения командой *env*

Все операции уровней 4–5 отработали штатно; вывод *env* фиксирует окружение сеанса для последующего исследования.

1.3 Уровни 6–7: копирование, консоль, пакетные сценарии

На уровнях 6 и 7 проверяются составные операции копирования, ввод данных с консоли, работа пакетных сценариев с параметрами и исследование переменных окружения.

Дерево каталогов скопировано командой *cp -r ~/OS2/task_01b ~/OS2/task_01b_dst_1*; ключ *-r* включает вложенные и пустые каталоги. Для второго варианта использована команда *cp -rp*, сохраняющая режим доступа и временные метки файлов. Полученная структура проверена по выводу команды *ls*.

Замена содержимого файлов каталога-приёмника копированием из каталога-источника выполняется командой *cp -f*: ключ *-f* принудительно перезаписывает существующий файл, не запрашивая подтверждения. Очистка текстового содержимого файлов без удаления самих файлов реализуется циклом, перенаправляющим пустой вывод в каждый файл: *for f in *.txt*; *do* > «$*f*»; *done*. Размер файла при этом становится нулевым, а сам файл и его атрибуты сохраняются.

Удаление непустого каталога вместе со всем содержимым выполняется командой *rm -rf*: ключ *-r* обеспечивает рекурсивный обход, а ключ *-f* подавляет запросы подтверждения, что избавляет от необходимости отвечать на них в пакетном режиме.

Ввод данных с клавиатуры непосредственно в файл выполняется командой *cat > ~/OS2/console.txt*, а завершение ввода – нажатием *Ctrl+D*. Дописывание данных в конец существующего файла выполняется командой *echo* с перенаправлением >>. Принцип тот же, что и в *Windows* (*copy con* и *type con* >>), различается только комбинация клавиш завершения ввода.

Для запуска *bash*-сценария *bat_01.sh* используется переменная *USER* и команда *date*, а имя узла выводится командой *uname -n*. Исходный текст приведён в листинге А.2; полученные значения представлены на рисунке 1.29.

В листинге А.3 приведён сценарий *bat_02.sh*, который выводит сведения о самом сценарии: имя файла (*basename* $0), время изменения (*stat -c* '%*y*' «$0») и полный путь (*realpath* «$0»). Параметр $0 в *bash* всегда содержит имя запущенного сценария, что и используется для получения этих сведений.

В листинге А.4 приведён сценарий *bat_03.sh*, который демонстрирует работу с параметрами запуска: $* содержит все переданные параметры, а $1, $2, $3, $4 – отдельные позиционные параметры. При запуске с пятью аргументами выводятся все пять значений; лишние параметры игнорируются, поскольку перечислены только первые четыре позиции.

В листинге А.5 приведён сценарий *bat_04.sh*, который выполняет арифметические операции над тремя числовыми аргументами. Перед вычислениями проверяется число переданных параметров и дополнительно проверяется делитель на ноль: при нулевом *b* операция *c*/*b* не выполняется, вместо неё выводится сообщение об ошибке и работа сценария завершается. Это предотвращает получение некорректного результата и соответствует требованию задания о защите от деления на ноль.

Для режима *create* проверяется наличие файла, после чего создаётся новый файл; режим *delete* удаляет существующий файл. Запуск без аргументов и

неизвестный режим обрабатываются отдельными сообщениями. Исходный текст приведён в листинге А.6; результат показан на рисунке 1.45.

Перед запуском сценариев им назначается право исполнения командой *chmod +x*, после чего они запускаются как ./имя_скрипта.*sh* либо через интерпретатор *bash* имя_скрипта.*sh*. Листинги всех сценариев приведены в приложении А (на рисунке 1.29) (на рисунке 1.30) (на рисунке 1.31) (на рисунке 1.32) (на рисунке 1.33) (на рисунке 1.34) (на рисунке 1.35) (на рисунке 1.36) (на рисунке 1.37) (на рисунке 1.38) (на рисунке 1.39) (на рисунке 1.40) (на рисунке 1.41) (на рисунке 1.42) (на рисунке 1.43) (на рисунке 1.44) (на рисунке 1.45).

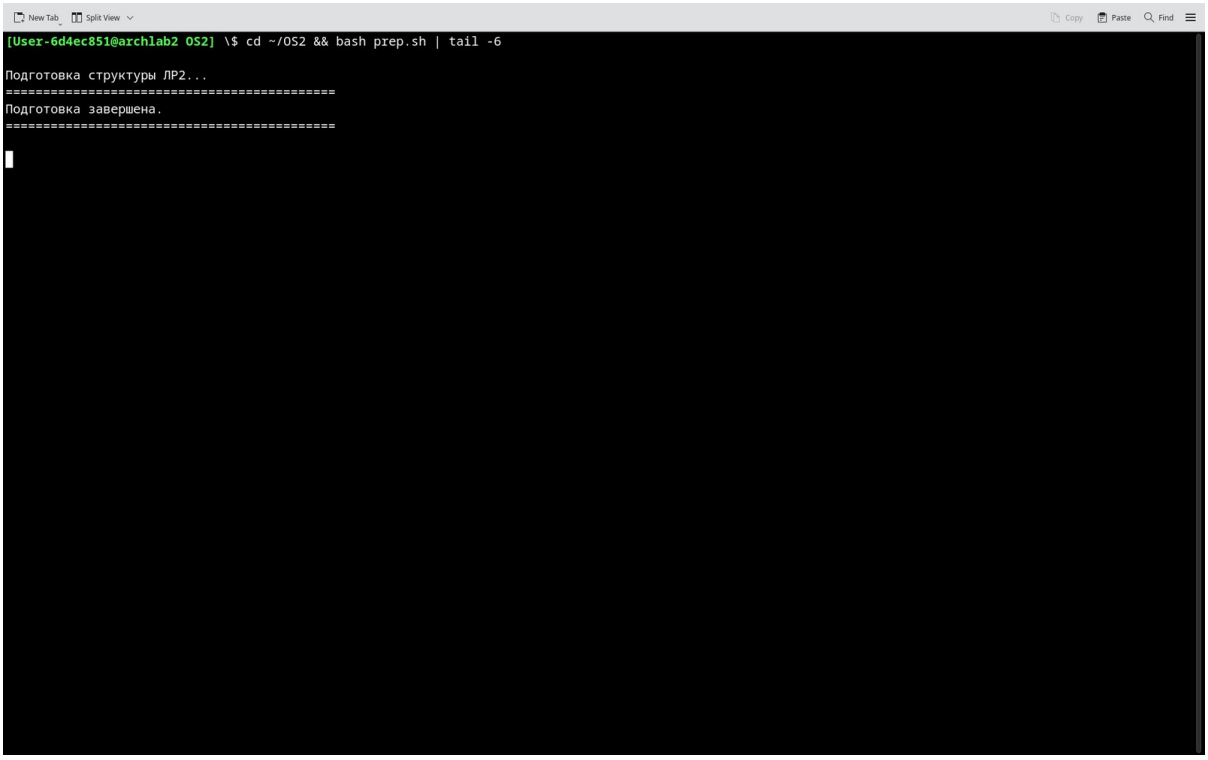


Рисунок 1.29 – Подготовка структуры каталогов перед заданиями уровней 6–7

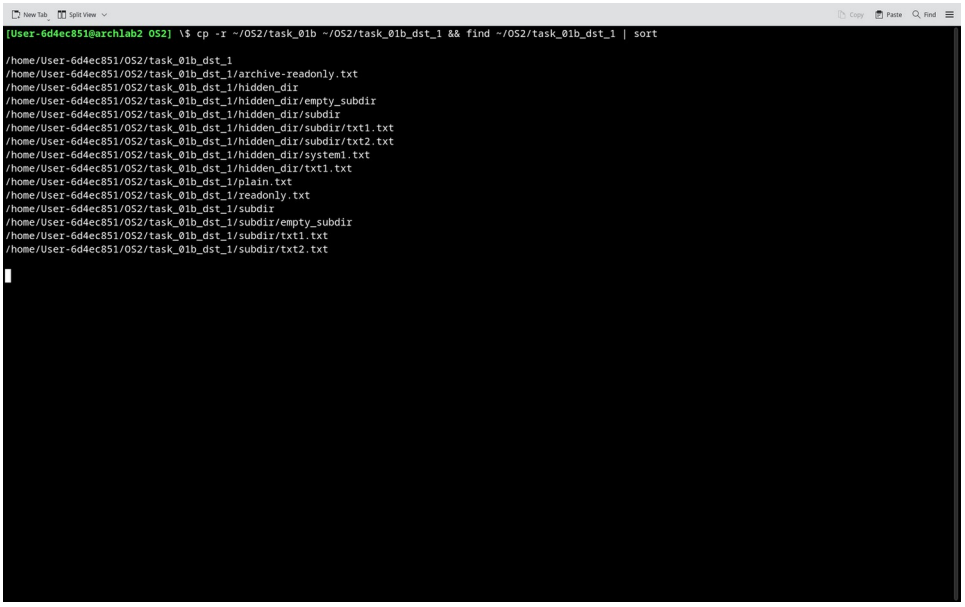


Рисунок 1.30 – Рекурсивное копирование дерева каталогов командой *cp -r*

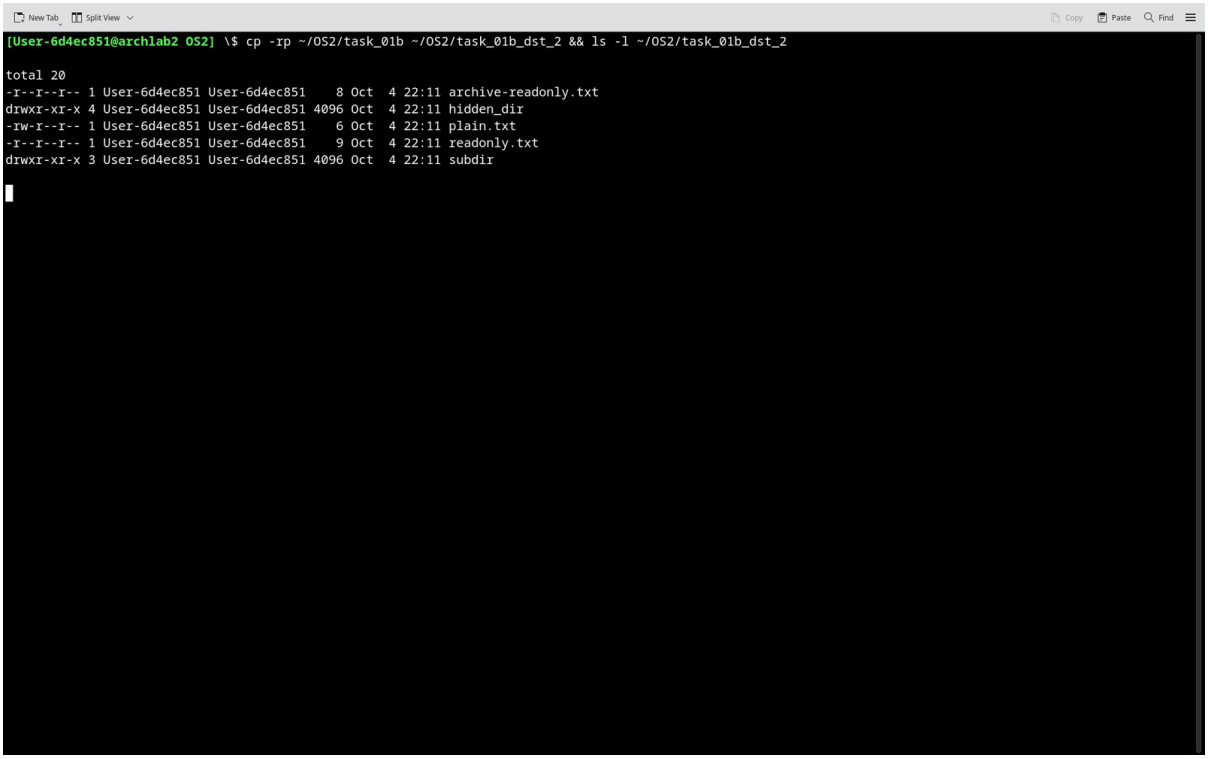


Рисунок 1.31 – Копирование с сохранением прав и временных меток ключом *-p*

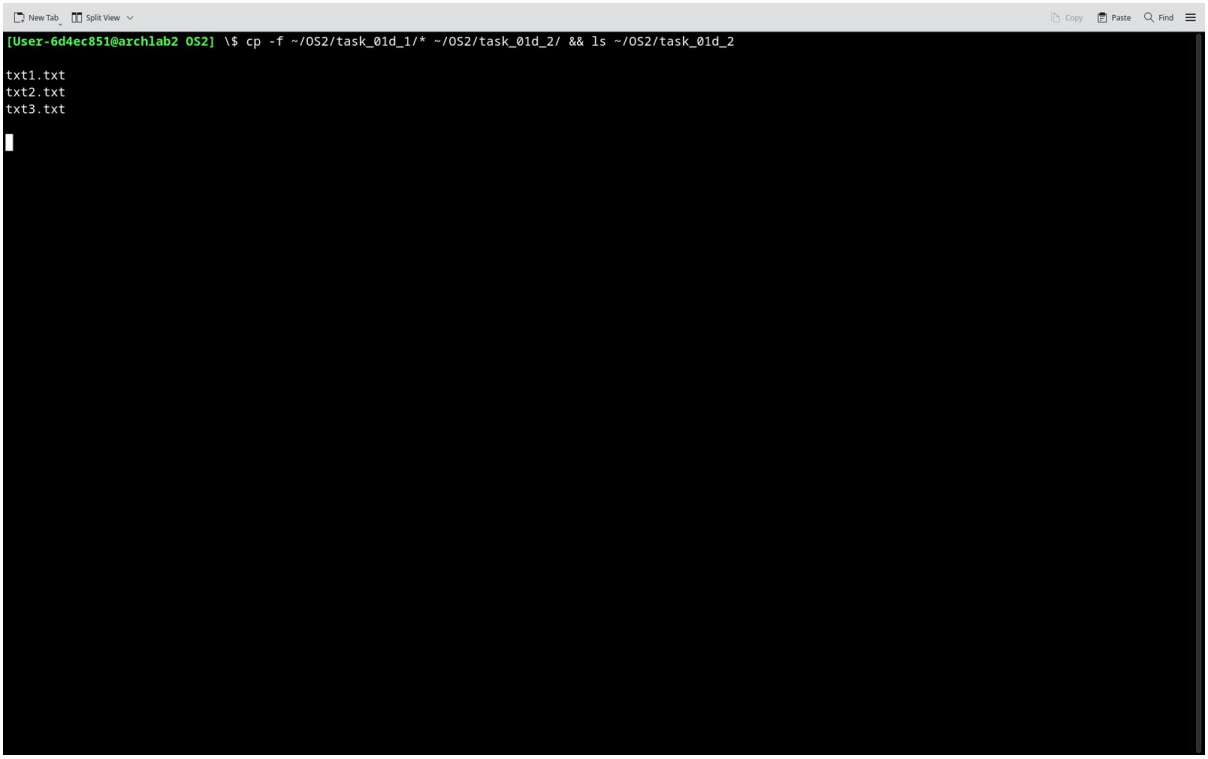


Рисунок 1.32 – Замена содержимого файлов копированием ключом *-f*

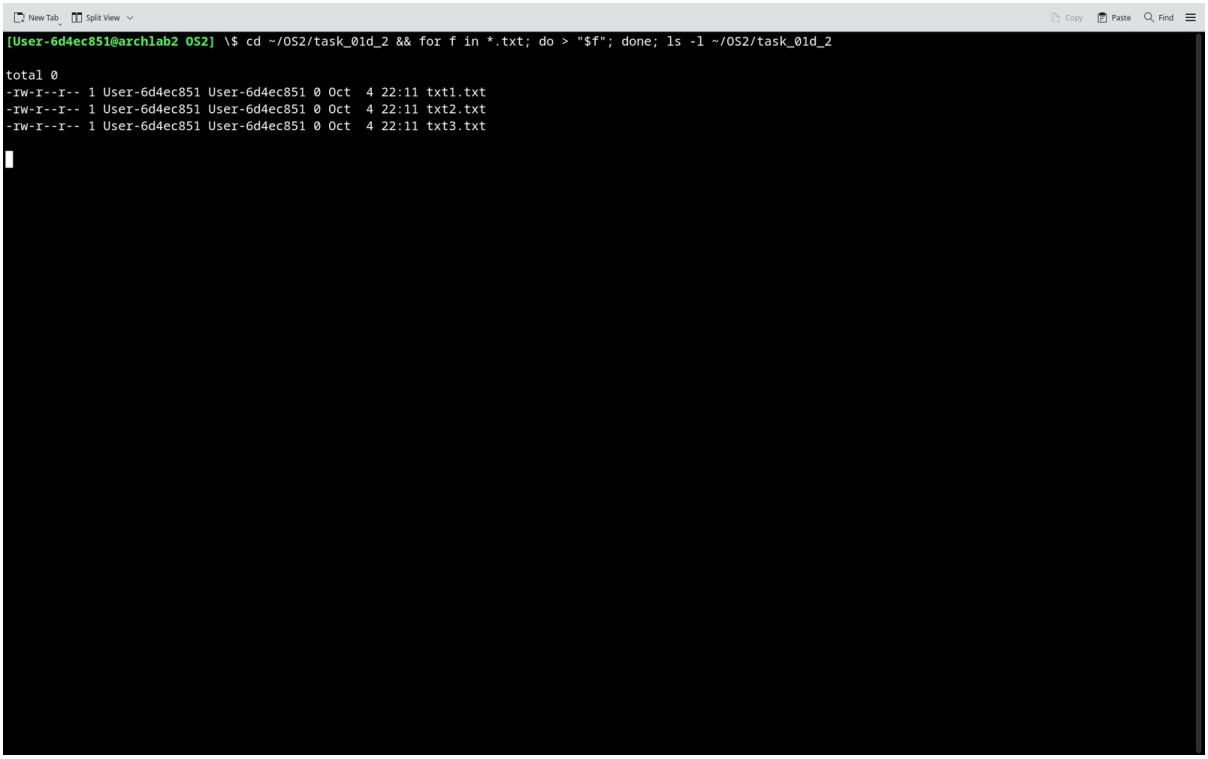


Рисунок 1.33 – Очистка содержимого файлов с сохранением самих файлов

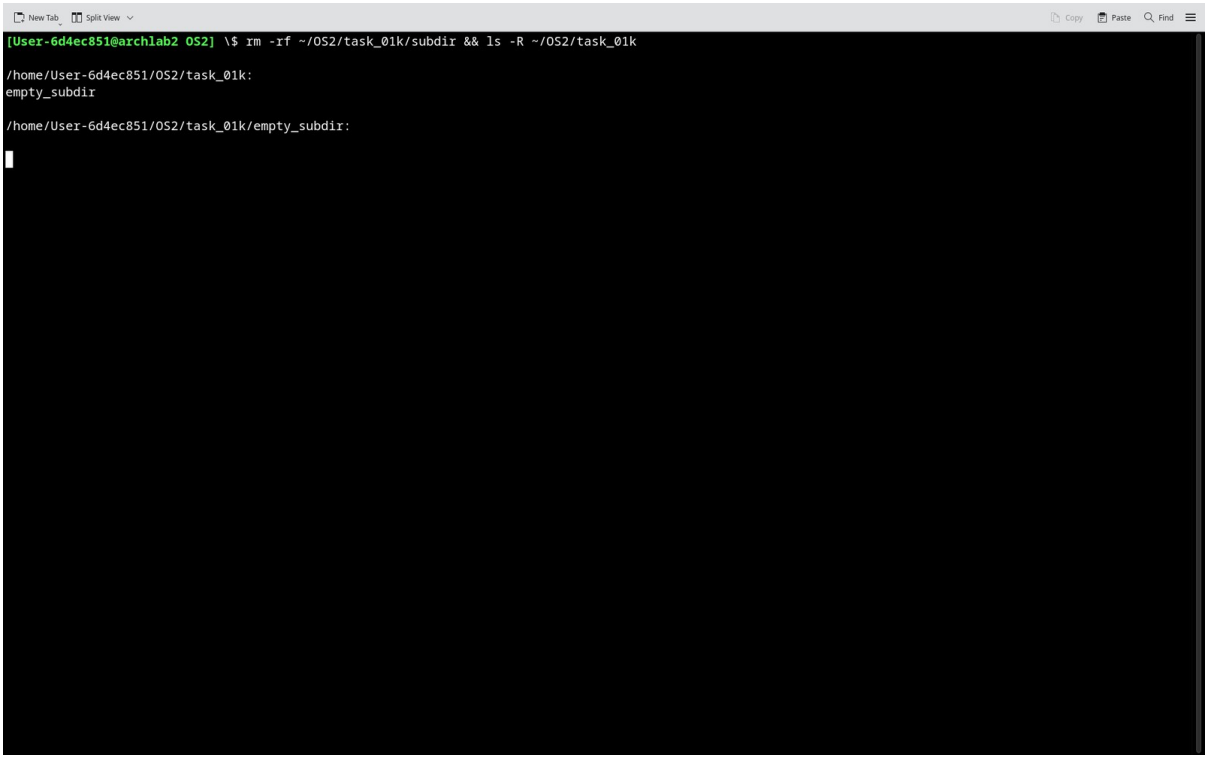


Рисунок 1.34 – Удаление непустого каталога командой *rm -rf*

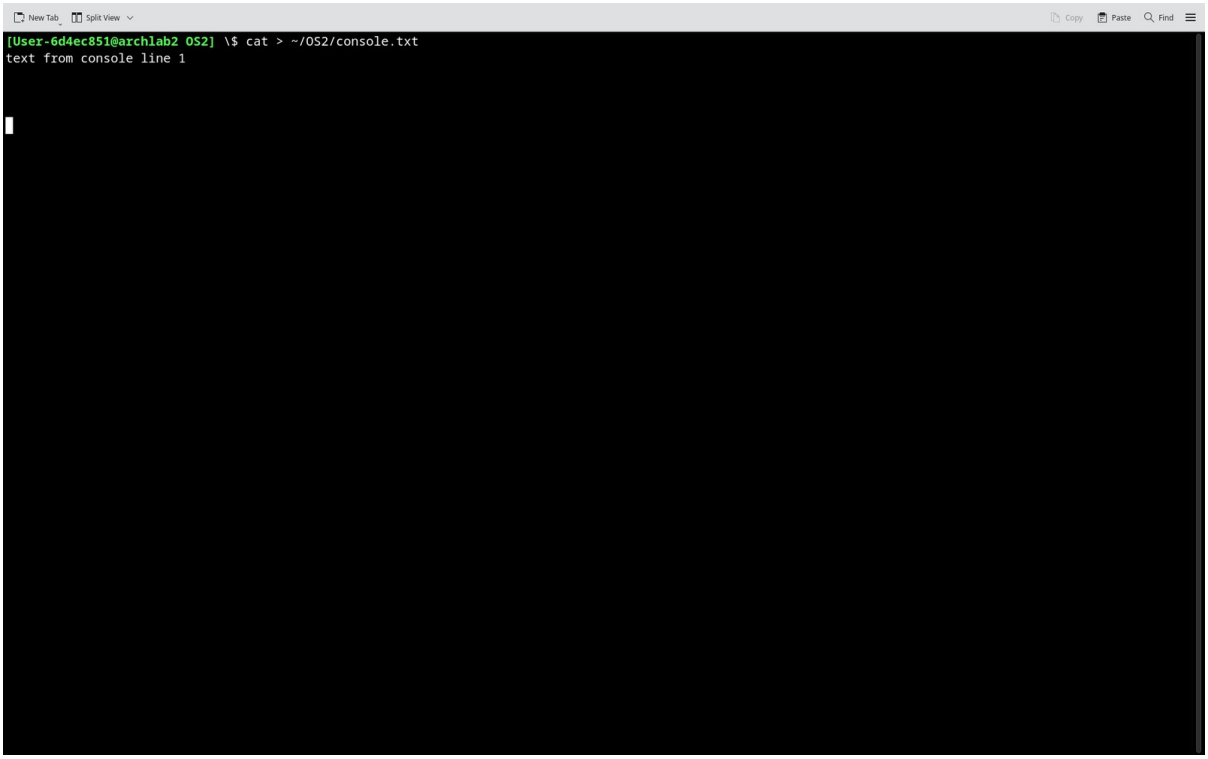


Рисунок 1.35 – Ввод данных с клавиатуры непосредственно в файл

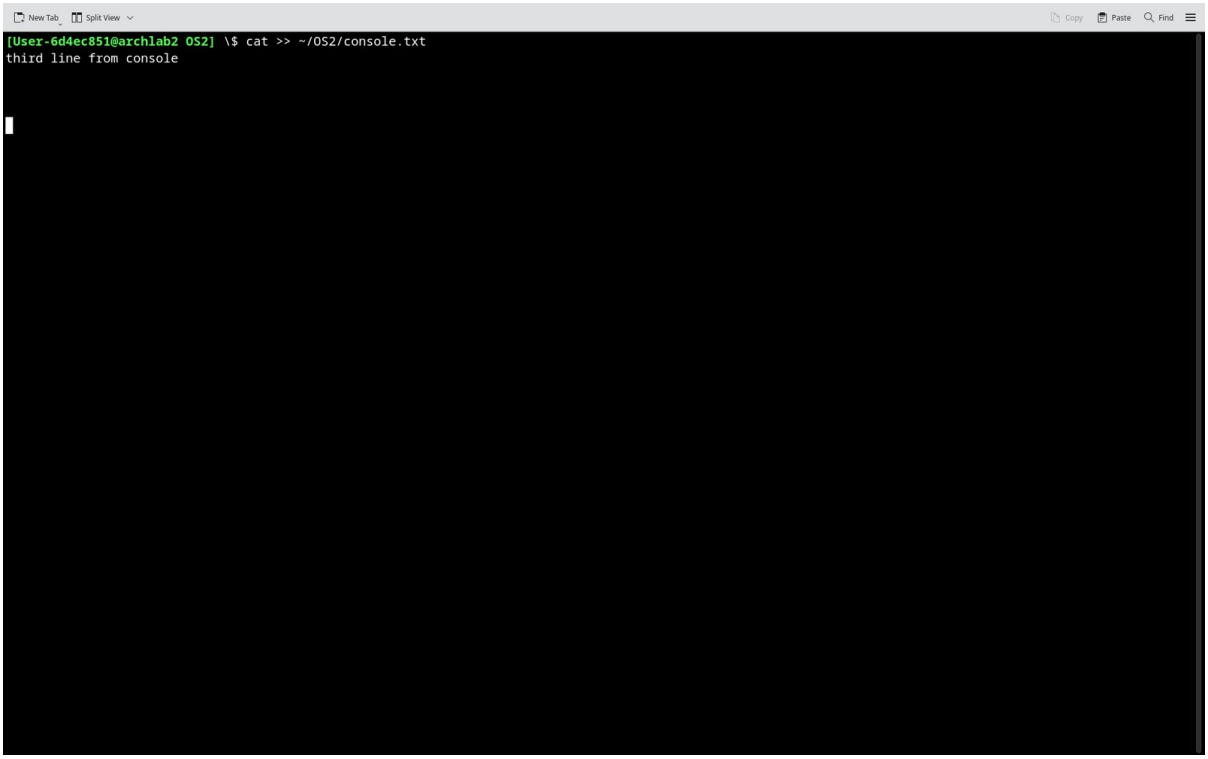


Рисунок 1.36 – Дописывание данных в конец существующего файла

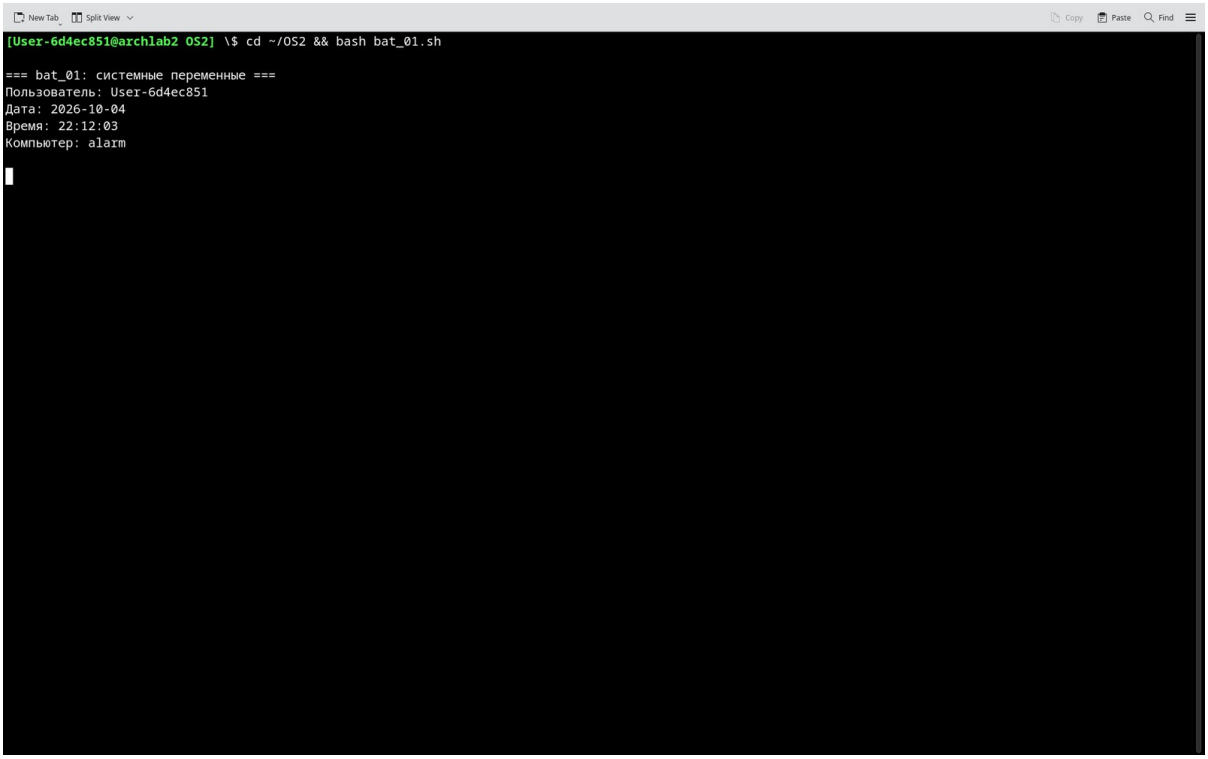


Рисунок 1.37 – Вывод системных переменных окружения сценарием *bat*_01.*sh*

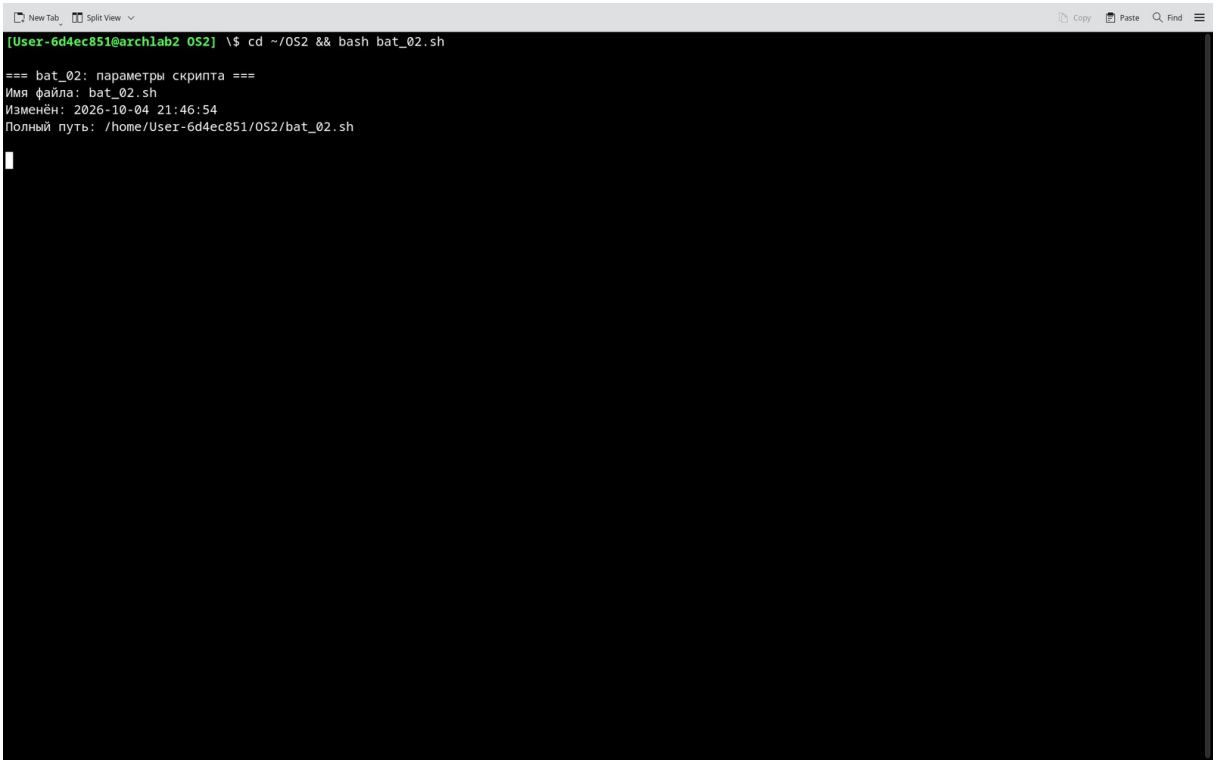


Рисунок 1.38 – Вывод сведений о сценарии *bat*_02.*sh* через параметр $0

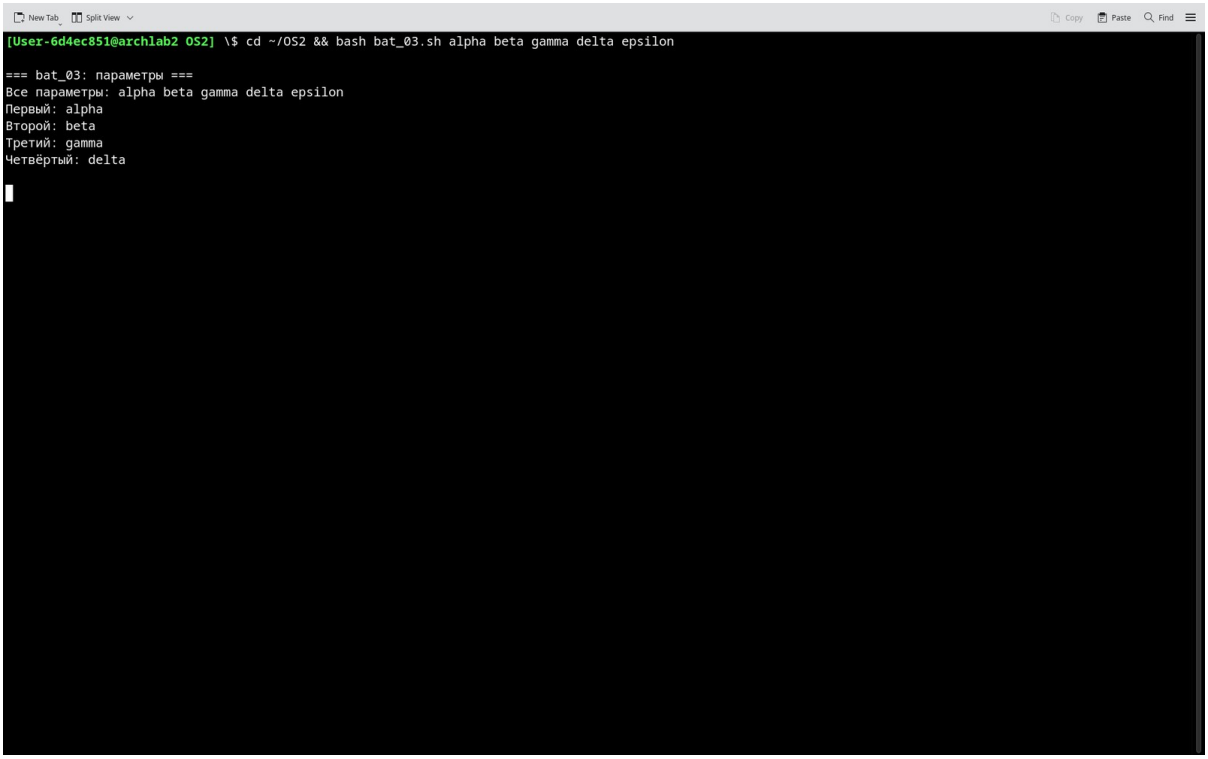


Рисунок 1.39 – Обработка параметров запуска сценарием *bat*_03.*sh*

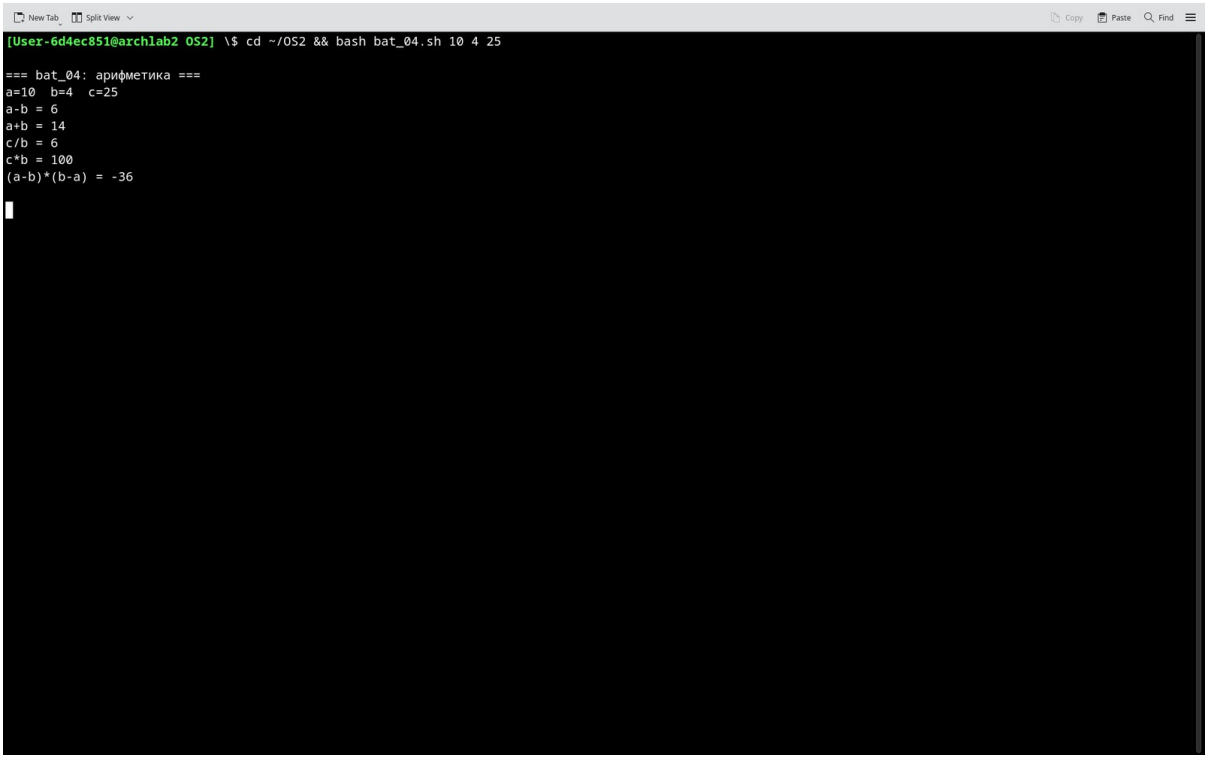


Рисунок 1.40 – Арифметические операции сценария *bat*_04.*sh*

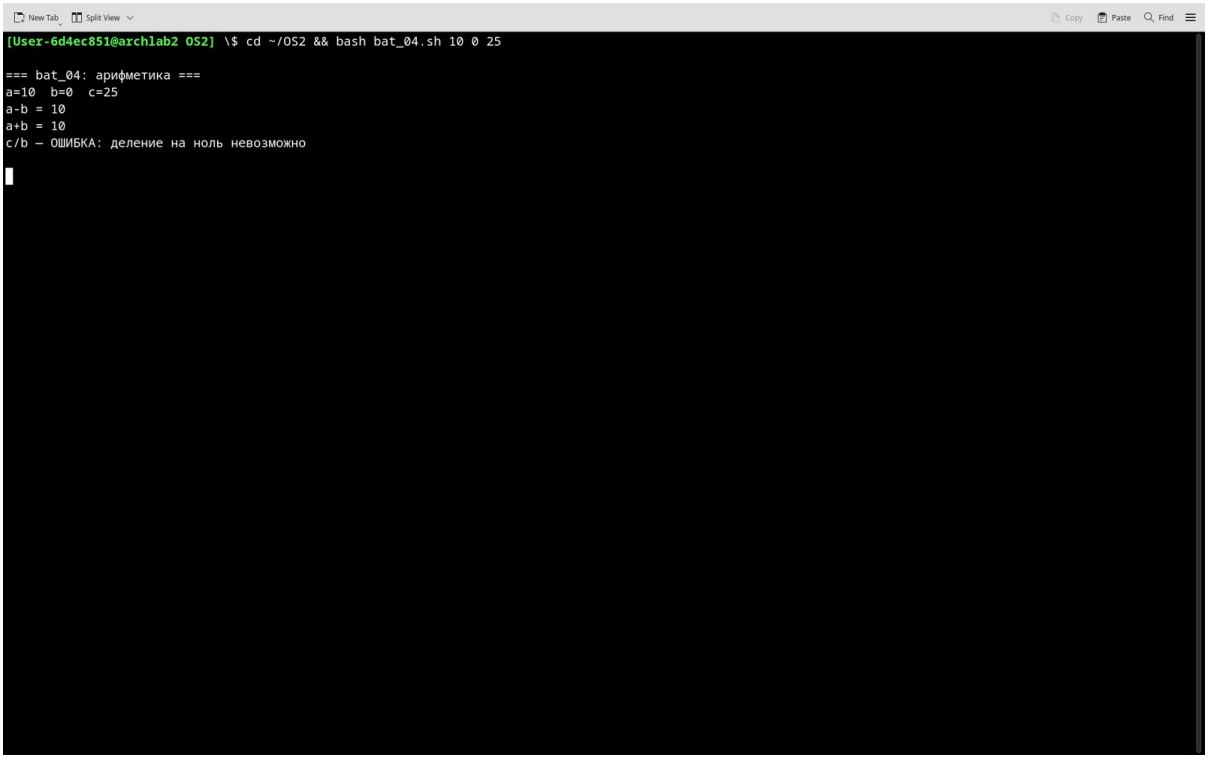


Рисунок 1.41 – Обработка попытки деления на ноль

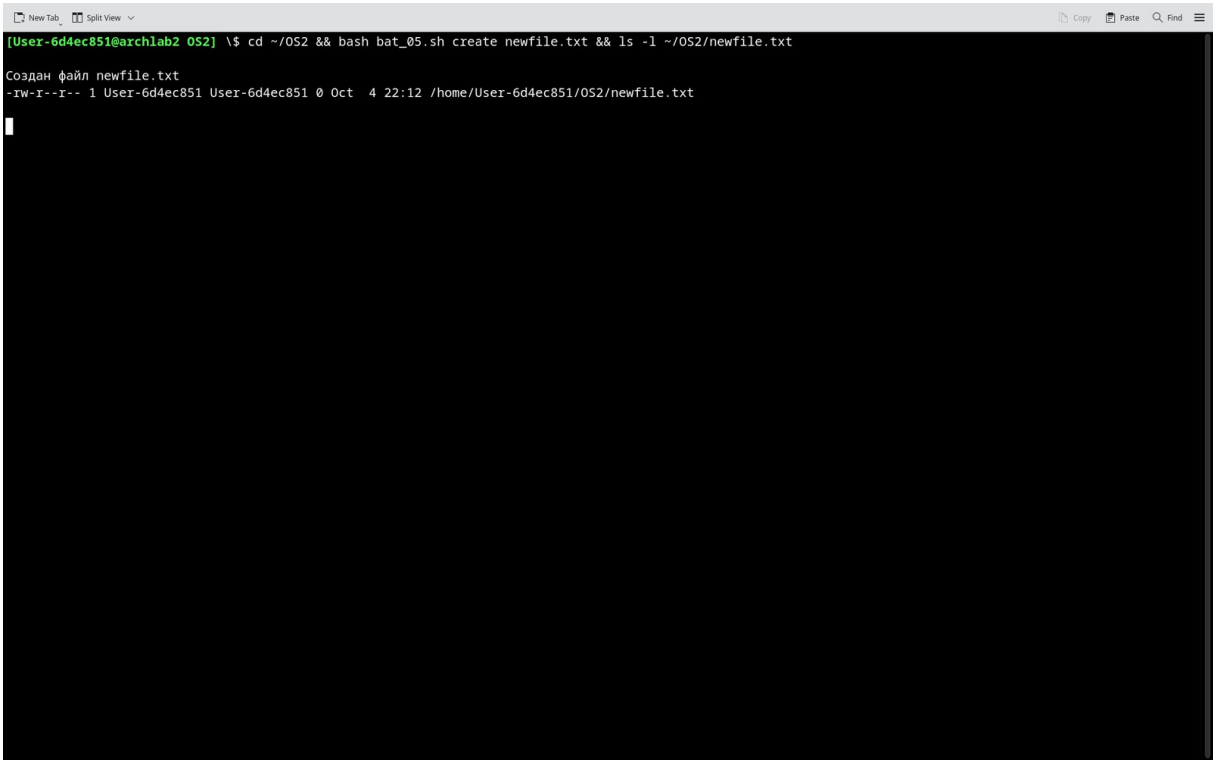


Рисунок 1.42 – Режим *create* сценария *bat*_05.*sh*

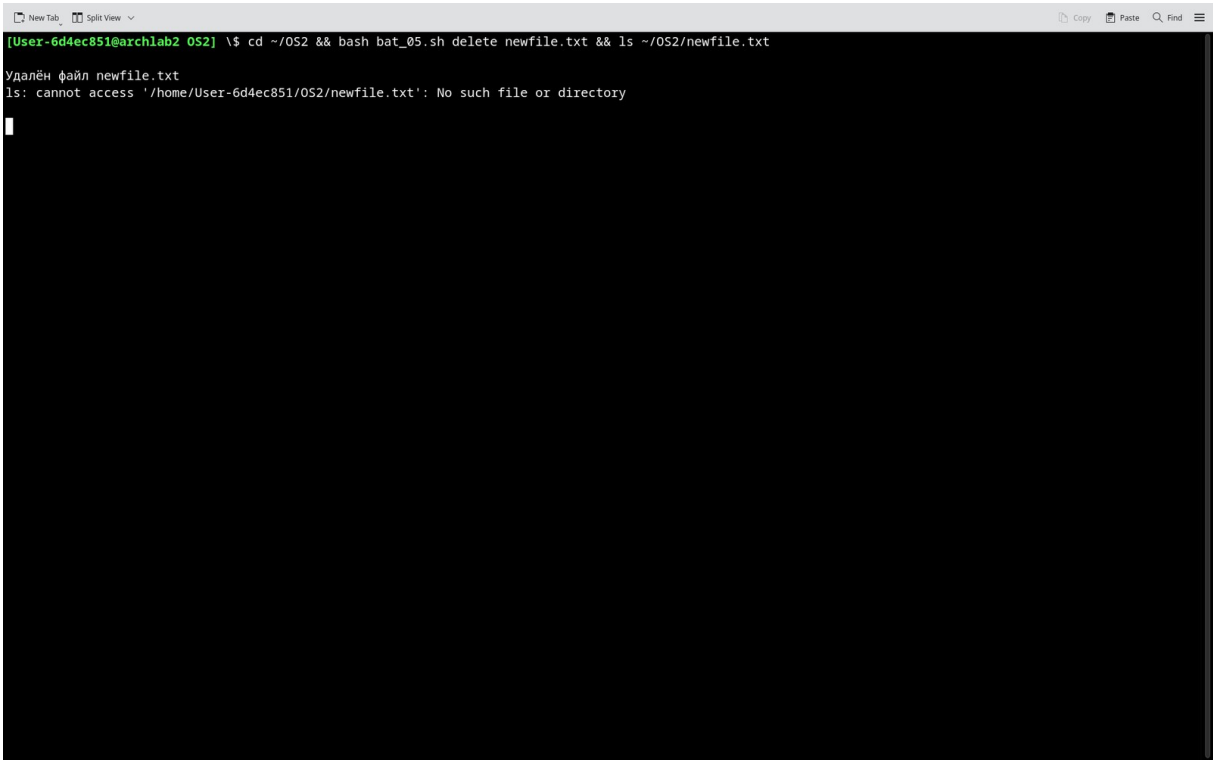


Рисунок 1.43 – Режим *delete* сценария *bat*_05.*sh*

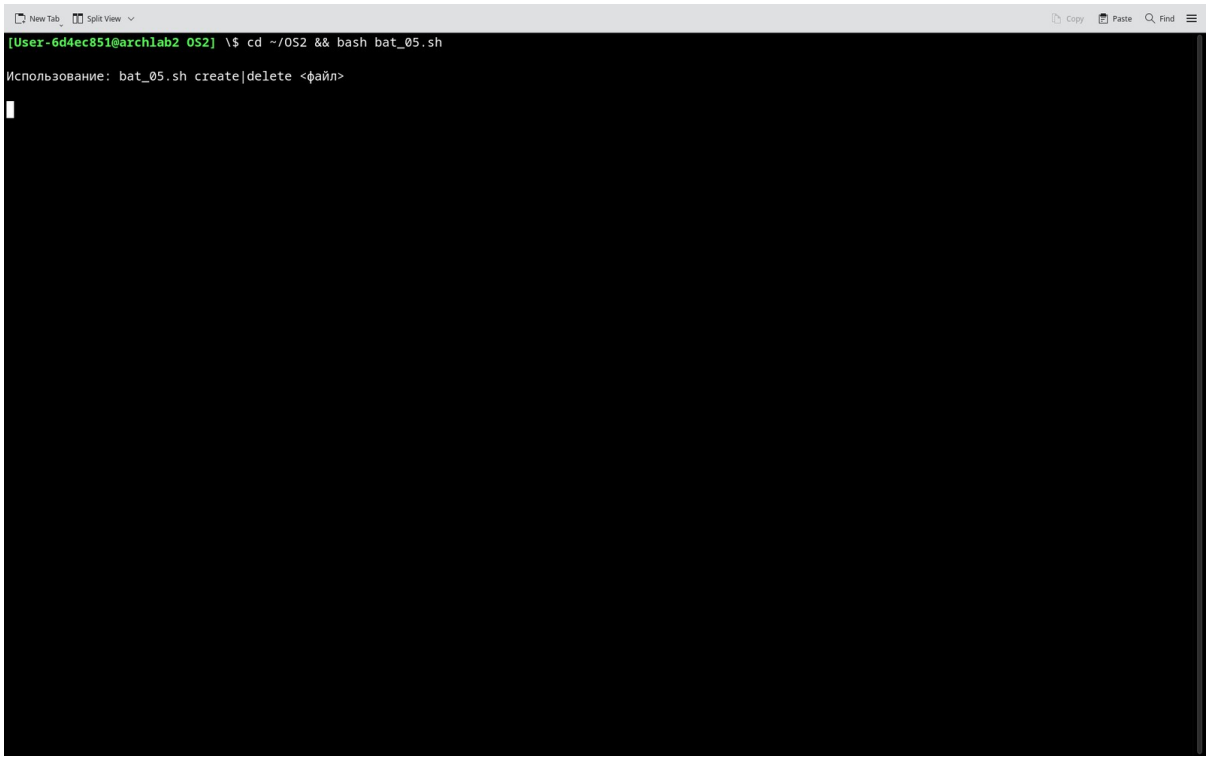


Рисунок 1.44 – Вывод строки использования при запуске без аргументов

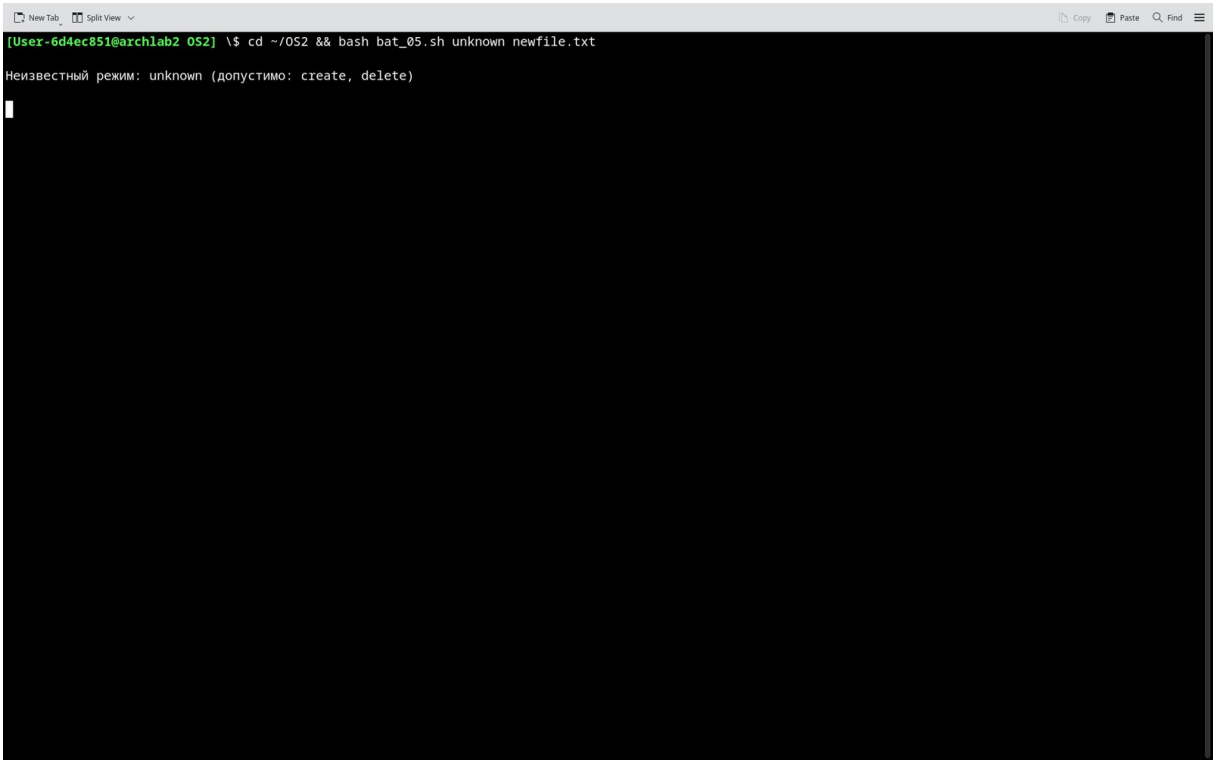


Рисунок 1.45 – Обработка неизвестного режима работы сценария

Составные операции выполнены; проверка неизвестного режима подтвердила обработку ошибок сценарием.

1.4 Исследование переменных окружения

Для исследования окружения выполнена команда *env*; отдельные значения получены с помощью *printenv* ИМЯ и подстановки $ИМЯ в командной строке оболочки.

В системе переменная *PATH* содержит восемь элементов, разделённых двоеточиями. Её значение получено командой *echo* $*PATH*, число каталогов проверено конвейером *echo* $*PATH* | *tr* ':' '\n' | *wc -l*. *PATH* задаёт порядок поиска исполняемых файлов: если имя встречается в нескольких каталогах, оболочка проверяет элементы списка по порядку.

В *Linux* домашний каталог задаётся переменной *HOME*, временный каталог – *TMPDIR*; переменная *SHELL* указывает используемую оболочку, *PWD* – текущий каталог, а *SHLVL* – уровень вложенности оболочек. В *Windows* аналогичные сведения доступны через *USERPROFILE*, *TEMP*, *USERNAME* и *COMPUTERNAME*. Права доступа и исполняемость файлов в *Linux* задаются отдельно от переменных окружения, например командой *chmod +x*.

Для проверки двоеточия как разделителя элементов *PATH* выполнены подсчёт каталогов и инвентаризация исполняемых и неисполняемых файлов. Результаты анализа представлены на рисунках 1.46, 1.47 и 1.48.

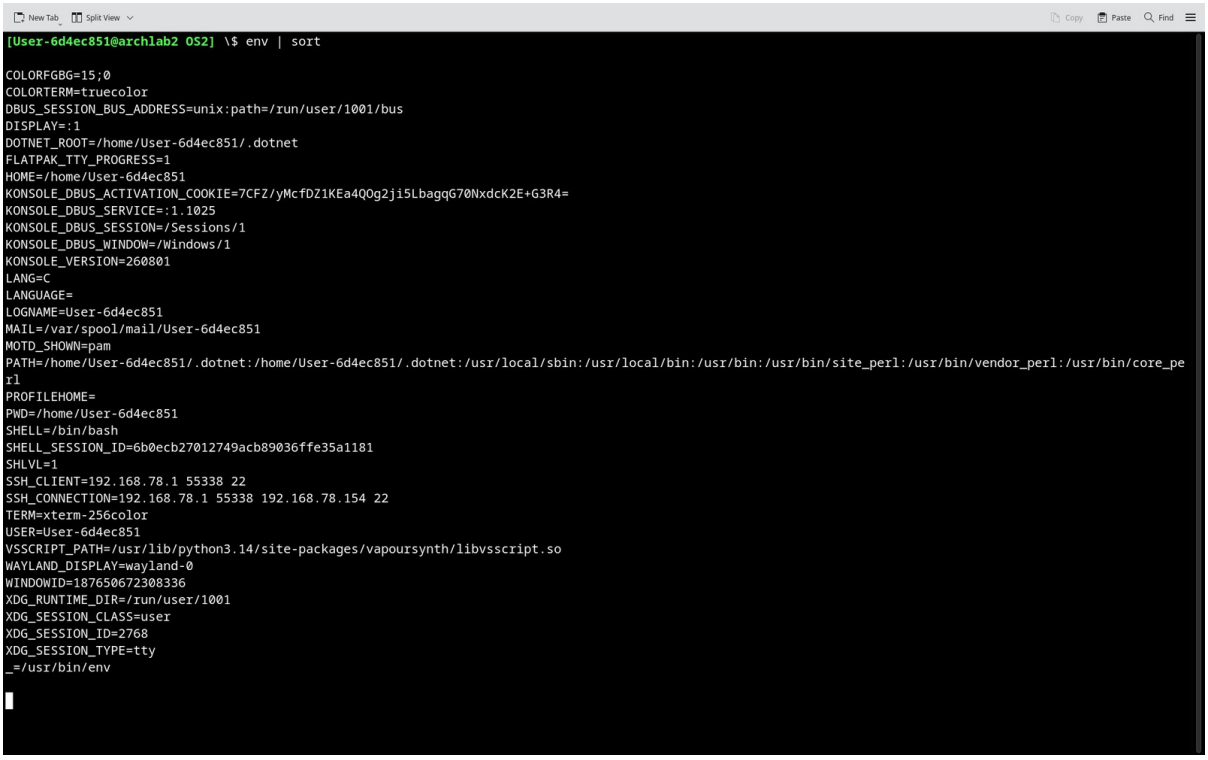


Рисунок 1.46 – Полный список переменных окружения системы

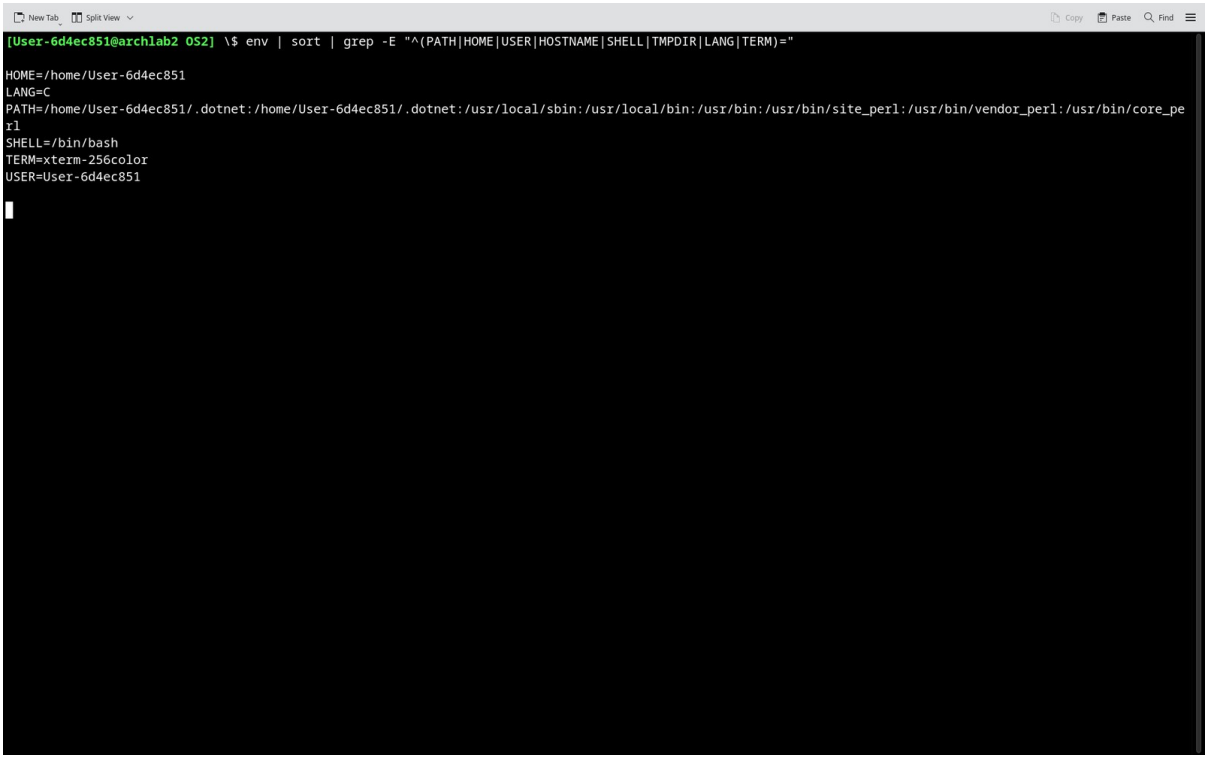


Рисунок 1.47 – Значения ключевых переменных окружения

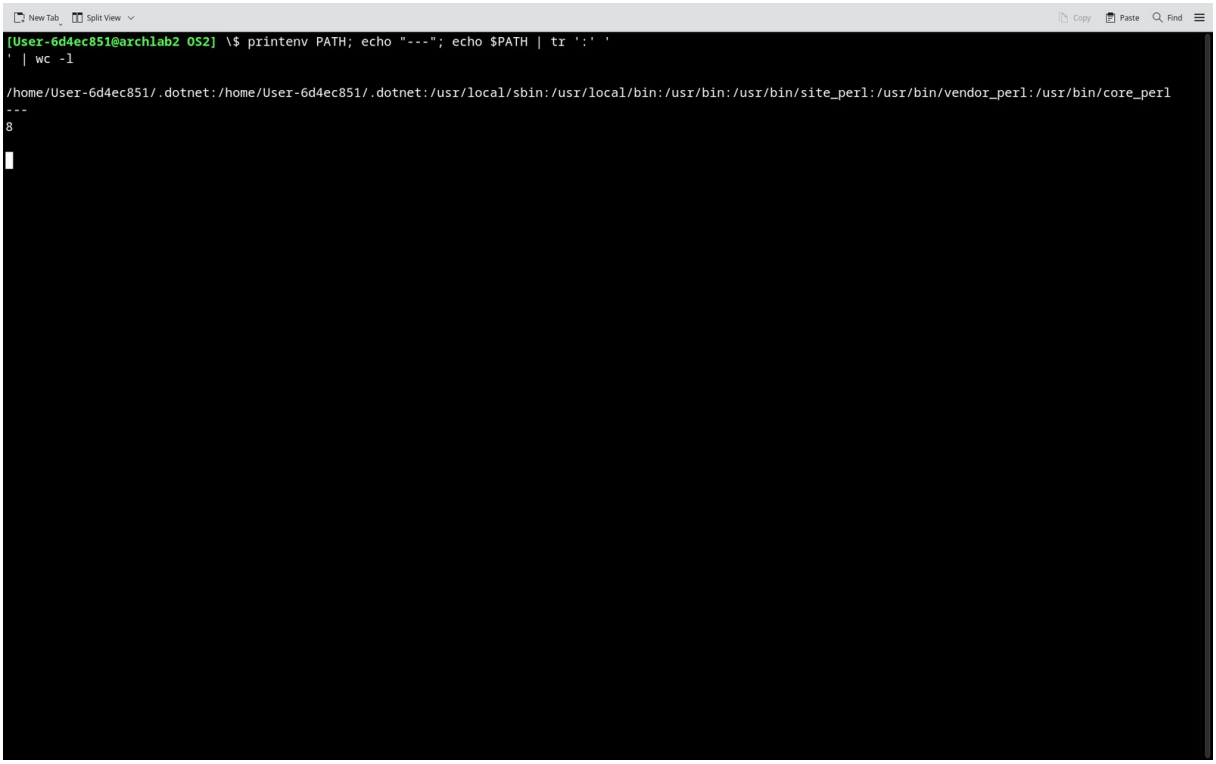


Рисунок 1.48 – Содержимое переменной *PATH* и подсчёт числа элементов

Исследование завершено: *PATH* разобран по элементам, их число зафиксировано.

1.5 Уровень 8: анализ дерева каталогов и содержимого *PATH*

Исходные тексты сценариев *path_count.sh* и *tree_sizes.sh* приведены в приложении А.

В листинге А.7 приведён сценарий *path_count.sh*. Он делит *PATH* по двоеточиям и для существующих каталогов считает обычные файлы верхнего уровня с правом исполнения и без него. Размер столбцов задаётся в *printf*; путь, для которого проверка [*-d* «$*dir*»] возвращает ложь, помечается как отсутствующий.

По результатам анализа в *PATH* содержится восемь элементов. Подсчёт файлов показал 1922 исполняемых и 20 неисполняемых файла. Каталоги */usr/local/bin*, */usr/local/sbin*, *site_perl* и *vendor_perl* пусты; для отдельной записи *.dotnet* сценарий сообщает, что каталога нет.

В листинге А.8 приведён сценарий *tree_sizes.sh*. Он рекурсивно обходит каталог, переданный первым аргументом, и выводит размеры файлов в десятичных и двоичных единицах, а для каталогов – число вложенных файлов и суммарный размер.

Сценарий запущен командой *./tree_sizes.sh task_01b*; его исходный текст приведён в листинге А.8. В выводе показаны девять файлов корневого каталога суммарным размером 56 байт, вложенные каталоги *hidden_dir* и *subdir*, а также пустые каталоги и файлы с разными правами доступа. Без параметра сценарий обрабатывает текущий каталог.

Подготовка каталогов перед уровнями 8–9 выполнена отдельным сценарием (рисунок 1.49). В листинге А.7 приведён сценарий *path_count.sh*; его запуск подсчитал файлы в элементах *PATH* (рисунок 1.50). В листинге А.8 приведён сценарий *tree_sizes.sh*, примеры его работы показаны на рисунках 1.51 и 1.52. Состав файлов сценариев проверен на рисунке 1.53.

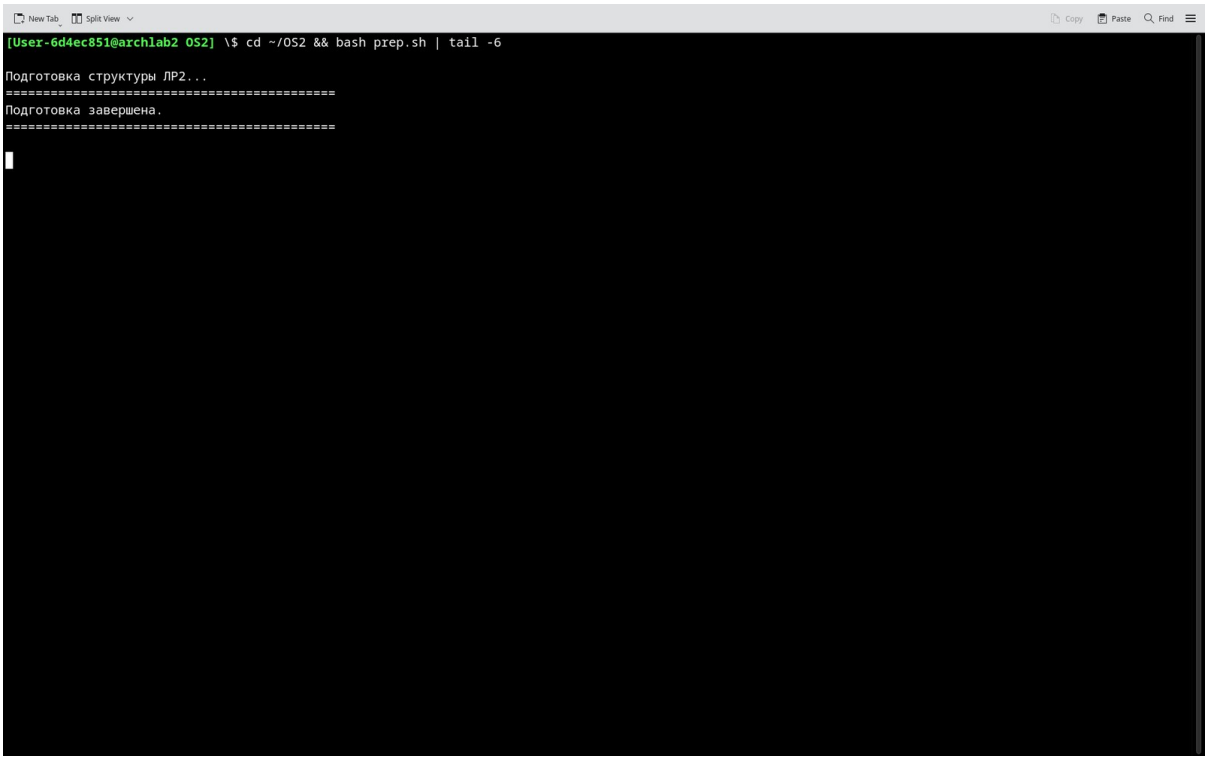


Рисунок 1.49 – Подготовка структуры каталогов перед заданиями уровней 8–9

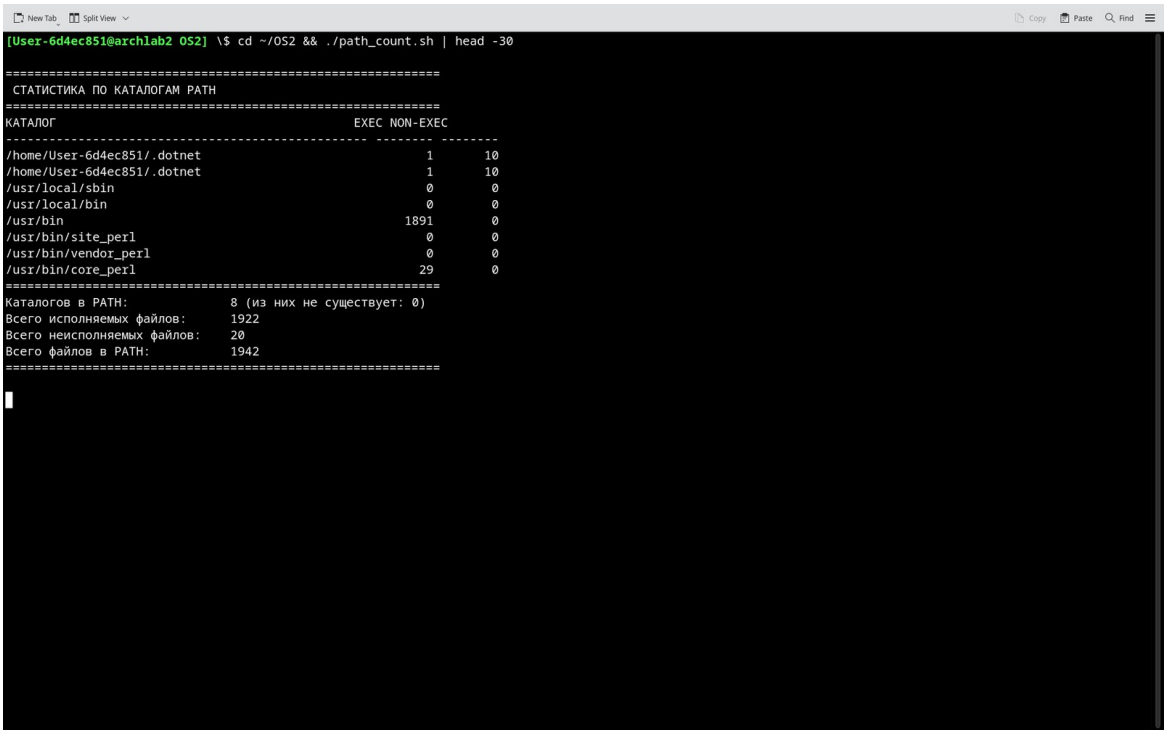


Рисунок 1.50 – Статистика по каталогам *PATH*: исполняемые и неисполняемые файлы

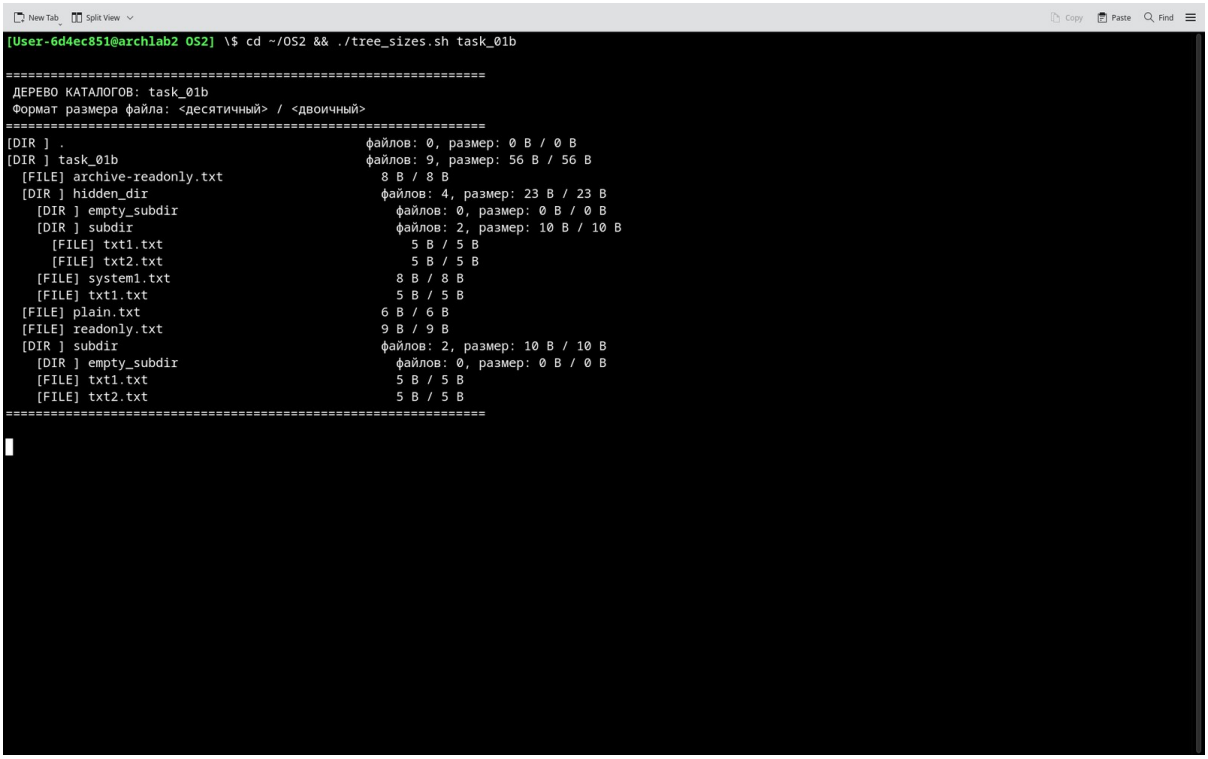


Рисунок 1.51 – Дерево каталогов с размерами файлов в двух системах единиц

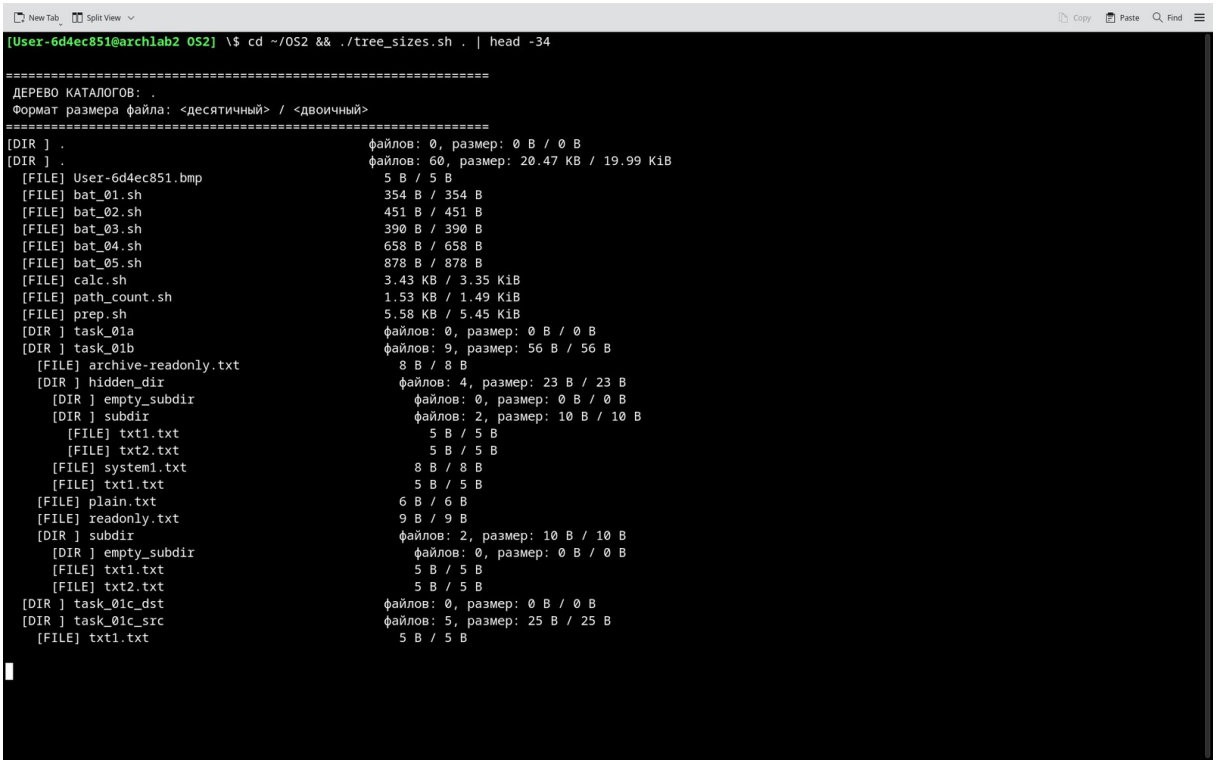


Рисунок 1.52 – Полный обход каталога *OS*2 сценарием *tree_sizes.sh*

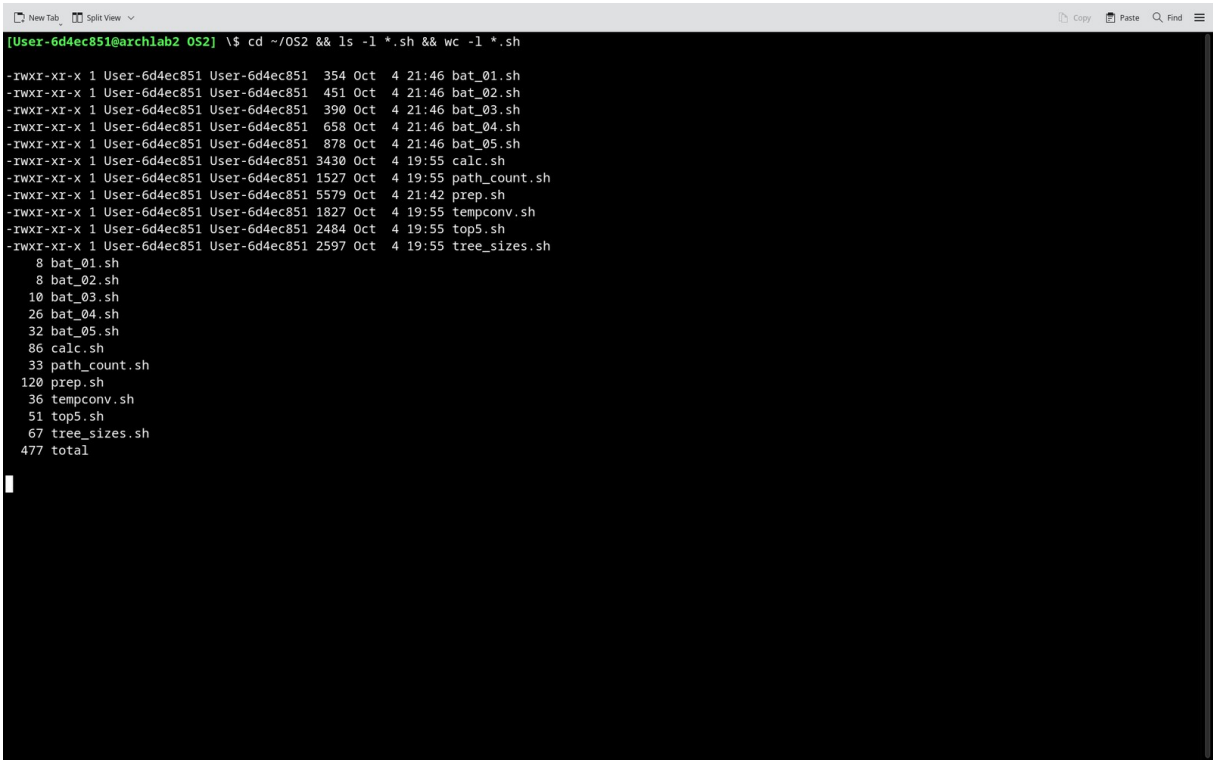


Рисунок 1.53 – Листинг разработанных сценариев с размерами и числом строк

Сценарии анализа созданы и отлажены: их состав и объём зафиксированы.

1.6 Уровень 9: монитор процессов в реальном времени

В листинге А.9 приведён сценарий *top*5.*sh*, реализующий монитор процессов в реальном времени по требованию задания уровня 9. Сценарий непрерывно выводит топ-5 процессов по загрузке центрального процессора и топ-5 процессов по использованию оперативной памяти. Тексты калькулятора *calc*.*sh* и конвертера температур *tempconv*.*sh* приведены в листингах А.10 и А.11 соответственно.

Интервал обновления задаётся первым параметром сценария и по умолчанию составляет 5 секунд: *INTERVAL*=${1:-5}. Перед запуском проверяется корректность параметра: если переданное значение не является положительным целым числом, сценарий выводит сообщение об ошибке и завершает работу. На рисунке показан случай запуска с неверным параметром.

Вывод каждого кадра оформлен таблицей с фиксированной шириной столбцов (функция *format_row* на основе *printf*) и содержит заголовок с меткой времени (*date* '+%*Y*-%*m*-%*d* %*H*:%*M*:%*S*'), величиной интервала и общим объёмом оперативной памяти системы. Перед каждым кадром экран очищается командой *clear*. Полужирные подзаголовки таблиц формируются *ANSI*-последовательностями \033[1*m* … \033[0*m*.

Данные о процессах берутся командой *ps* с ключами *-eo pid,comm,%cpu, %mem* и сортировкой *--sort=-%cpu* и *--sort=-%mem*. Команда *head* -6 | *tail* -5 оставляет ровно пять строк данных, отбрасывая заголовок *ps*. Процент памяти переводится в мегабайты вычислением через *awk*.

Зацикливание выполняется командой *while true* с паузой *sleep* «$*INTERVAL*» в конце итерации. Обработка прерывания пользователем реализована командой

trap: обработчик *INT* выводит сообщение о завершении и завершает сценарий с кодом возврата 0. Таким образом, цикл работы монитора ограничен только нажатием *Ctrl+C*.

На приведённых снимках зафиксированы два последовательных кадра монитора с интервалом 5 секунд по умолчанию и кадр с интервалом 3 секунды. Видно, что состав процессов меняется от кадра к кадру: в момент съёмки на первом месте по загрузке ЦП находился процесс *baloo_file_extractor*, а по памяти – *plasmashell*, что закономерно для системы с проиндексированными файлами. Работа монитора подтверждена (на рисунке 1.54) (на рисунке 1.55) (на рисунке 1.56).

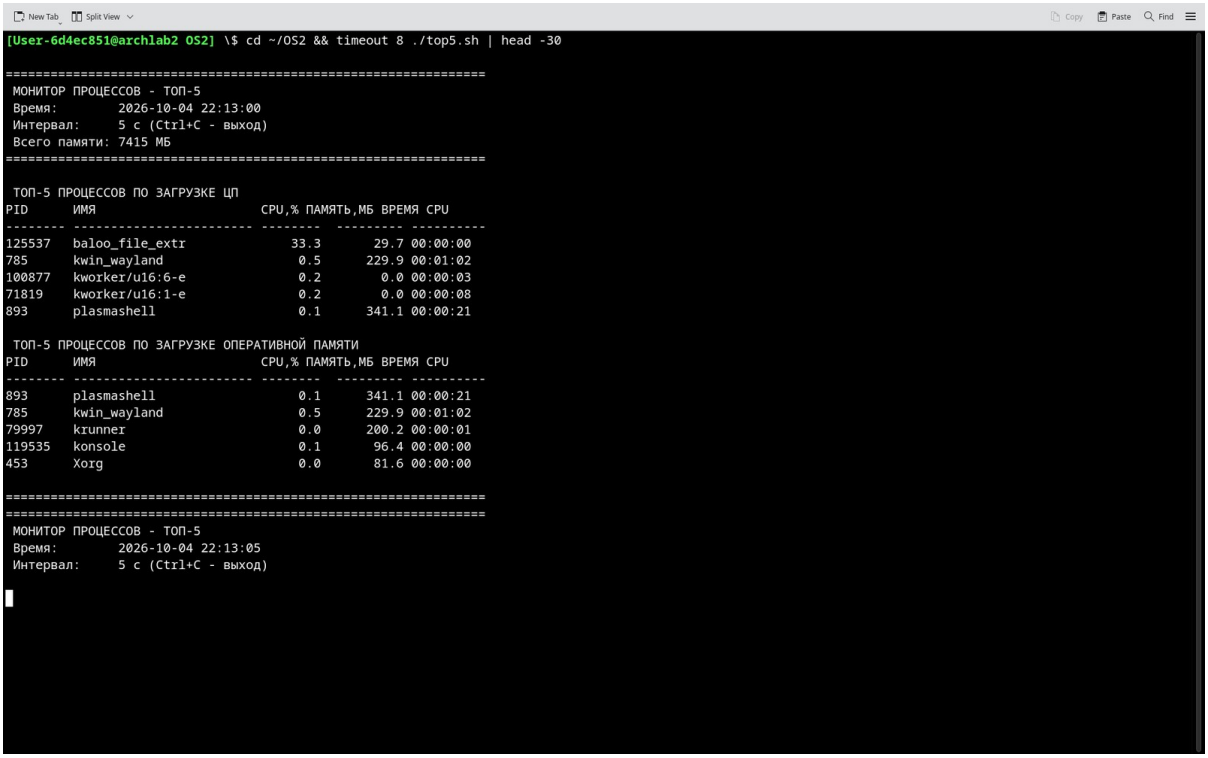


Рисунок 1.54 – Кадр монитора процессов с интервалом 5 секунд по умолчанию

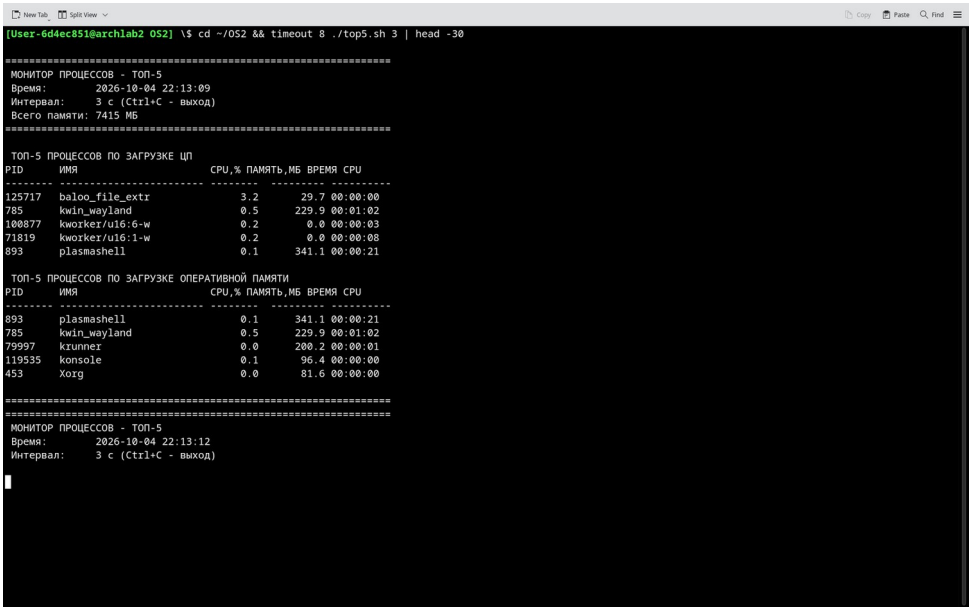


Рисунок 1.55 – Кадр монитора процессов с интервалом 3 секунды

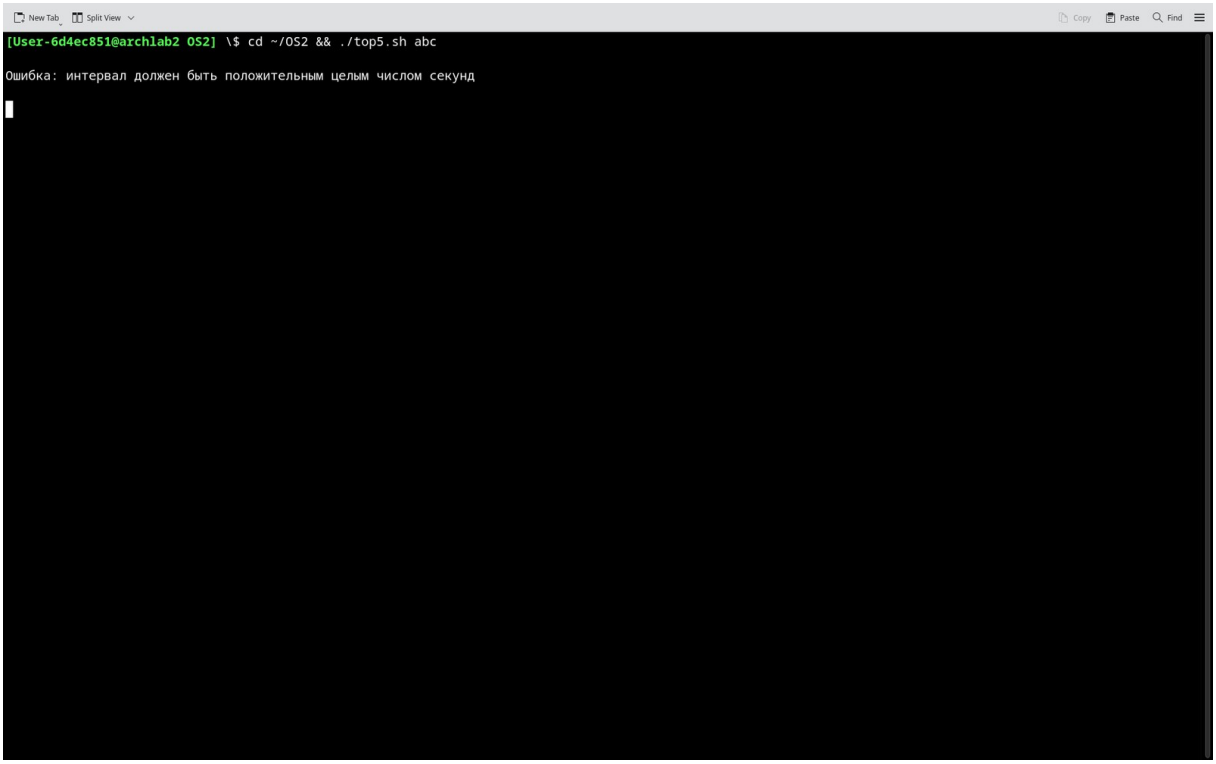


Рисунок 1.56 – Обработка некорректного параметра интервала

В листинге А.9 приведён сценарий *top*5.*sh*, который выводит кадры монитора с заданным интервалом и проверен на обработку некорректных параметров.

1.7 Вывод по разделу

В *Linux* выполнены задания уровней 4–9. Проверены сценарии с аргументами, граничными и ошибочными значениями; результаты команд и сценариев представлены на снимках экрана.

2 Выполнение заданий на ОС *Windows*

2.1 Подготовка рабочей среды

Задания уровней 4–9 выполнены на виртуальной машине с операционной системой *Windows* 11 (архитектура *ARM*64) под управлением гипервизора *VMware Fusion*. Все задания выполнялись из командной строки оболочки *cmd*, а задания уровней 8 и 9 – с помощью сценариев на языке *PowerShell*, запускаемых из той же командной строки.

Для запуска подготовительного сценария выполнена команда *prep.bat* из *C*:*lab*2*_sync*; созданная структура предназначена для заданий всех уровней. Исходный текст приведён в листинге А.12.

Сценарий является идемпотентным: перед созданием структуры он снимает с существующих объектов атрибуты «только для чтения», «скрытый» и «системный», после чего удаляет каталоги предыдущего прогона и создаёт их заново. Это позволяет запускать сценарий многократно перед каждым уровнем и всегда получать одинаковое исходное состояние.

Отдельно отмечу особенность кодировок: кодовая страница консоли намеренно не переключается на 65001. При установке этой кодовой страницы команда *mkdir* записывает кириллическое имя каталога в *UTF*-8, а следующий вызов *cmd* читает это имя как *OEM*-кодировку, и в результате получается нечитаемое имя вида «РџСЂёРІРµС,» (мусор). Поэтому кириллические имена создаются в текущей *OEM*-кодировке и корректно отображаются во всех последующих командах.

В сценарии также создаётся файл *students.txt* со списком из десяти студентов. Набор фамилий совпадает с используемым в заданиях, выполняемых в других частях работы, чтобы результаты заданий по поиску и сортировке можно было сопоставлять.

После выполнения сценария выводится итоговая структура каталога *OS2* командой *dir /B /S*, по которой проверяется, что все требуемые каталоги и файлы созданы. Запуски сценария перед уровнями 4–5, 6–7 и 8–9 приведены на рисунках 2.1, 3.2 и 3.3 (на рисунке 2.2) (на рисунке 2.3).

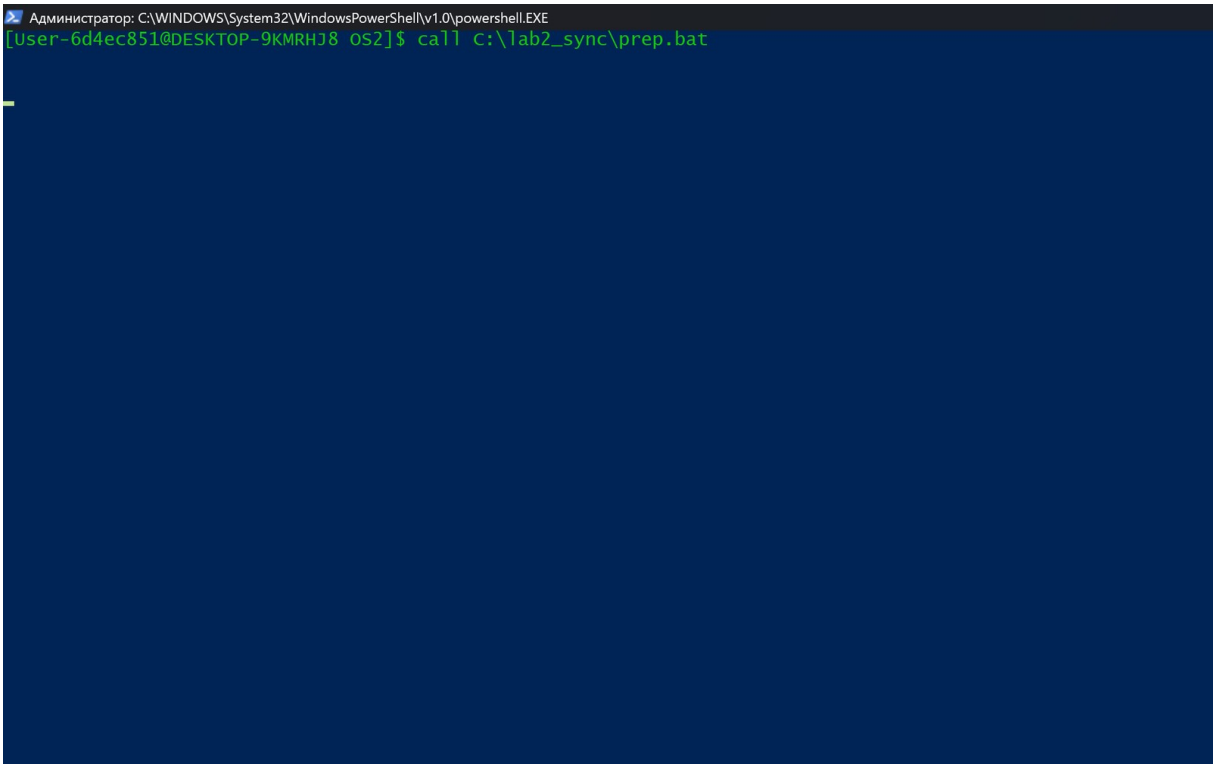


Рисунок 2.1 – Результат выполнения сценария подготовки среды *prep.bat*

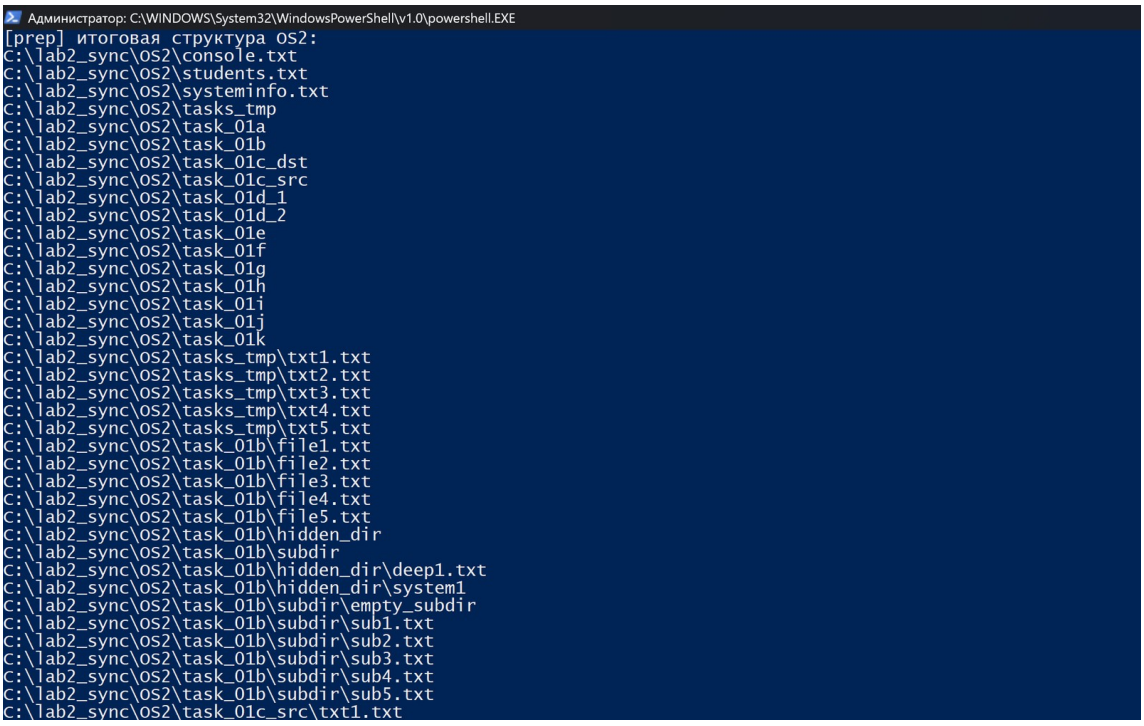


Рисунок 2.2 – Повторный запуск сценария подготовки среды перед заданиями уровня 6–7

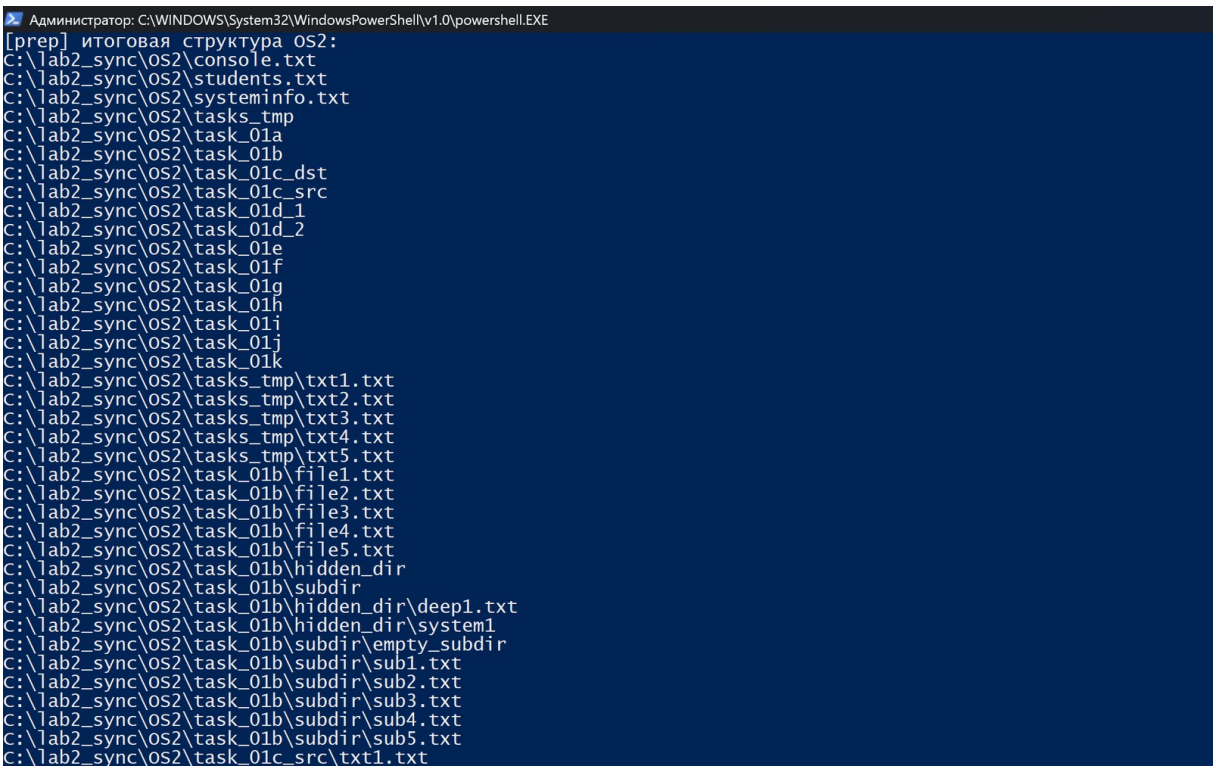


Рисунок 2.3 – Подготовка среды перед заданиями уровня 8–9

Рабочая среда *Windows* подготовлена: каталоги и файлы заданий созданы, проверка пройдена.

2.2 Уровень 4–5: навигация и операции с файлами

На уровнях 4 и 5 выполняются задания базового характера: переход между каталогами, создание и удаление каталогов, копирование, перемещение и переименование файлов, а также сравнение файлов. Все операции выполняются штатными внутренними командами оболочки *cmd*.

Переход в каталог *task*_01*i*\Русский выполнен по абсолютному пути командой *cd* /*d* *OS*2*task*_01*i*\Русский. Ключ /*d* указывает оболочке сменить не только текущий каталог, но и сам текущий диск: без него переход на другой дисковый раздел не выполнился бы. Команда *cd* без аргументов выводит полный путь текущего каталога, что и служит проверкой результата перехода (рисунок 2.4).

Затем из каталога Русский выполнен относительный переход в каталог *English* командой *cd* ..*English*. Относительный путь задаётся относительно текущего каталога, поэтому команда не содержит имени диска и корня; контрольный вывод *cd* показывает, что текущим стал каталог *task*_01*i**English* (рисунок 2.5).

Создание каталогов выполнено командой *md*. Для проверки перенаправление сообщений об ошибках в *nul* выполняется конструкцией 2>*nul*, чтобы повторный запуск команды не засорял вывод сообщением о существующем каталоге. Создание каталога *task*_01*j* проверено листингом *dir* /*b* (рисунок 2.6).

По команде *mkdir* создана иерархия каталогов *sub*/*dir*/*full*/*path* внутри *task*_01*j*. Результат выполненной команды *md* *OS*2*task*_01*j**sub**dir**full**path*

проверен выводом *dir /s /b*; отдельные команды для промежуточных уровней не потребовались (рисунок 2.7).

Удаление пустого каталога выполнено командой *rd OS2\task_01k\empty_subdir*. Каталог *task_01k* содержал два подкаталога, поэтому листинг показывает, что после удаления остался только *subdir* (рисунок 2.8).

Копирование группы файлов выполнено командой *copy /y OS2\tasks_tmp*.txt OS2\task_01a*. Маска *.txt* и завершающий обратный слэш в имени каталога-приёмника позволяют указать каталог назначения при копировании сразу нескольких файлов. Ключ */y* отвечает на вопрос о перезаписи без запроса подтверждения; в журнале выполнения зафиксированы шесть ответов *y*, поступавших на этот запрос (рисунок 2.9).

Вариант копирования с интерактивным подтверждением выполнен командой *copy* без ключа */y*. Оболочка запрашивает подтверждение перезаписи существующего файла, и в журнале зафиксирован ответ *y* (рисунок 2.10). Таким образом, ключ */y* является лишь способом пропустить этот запрос, а не изменением сути операции копирования.

Копирование файла под новым именем выполнено командой *copy OS2\task_01a\txt1.txt OS2\task_01a\newname.txt*. Исходный файл при этом сохраняется, в каталоге появляется новый файл *newname.txt* (рисунок 2.11).

Объединение содержимого двух файлов в один выполнено конструкцией *copy OS2\task_01a\txt1.txt + OS2\task_01a\txt2.txt OS2\task_01a\merged.txt*. Оператор + между именами файлов означает конкатенацию: содержимое второго файла дописывается в конец первого, а результат записывается в *merged.txt* (рисунок 2.12).

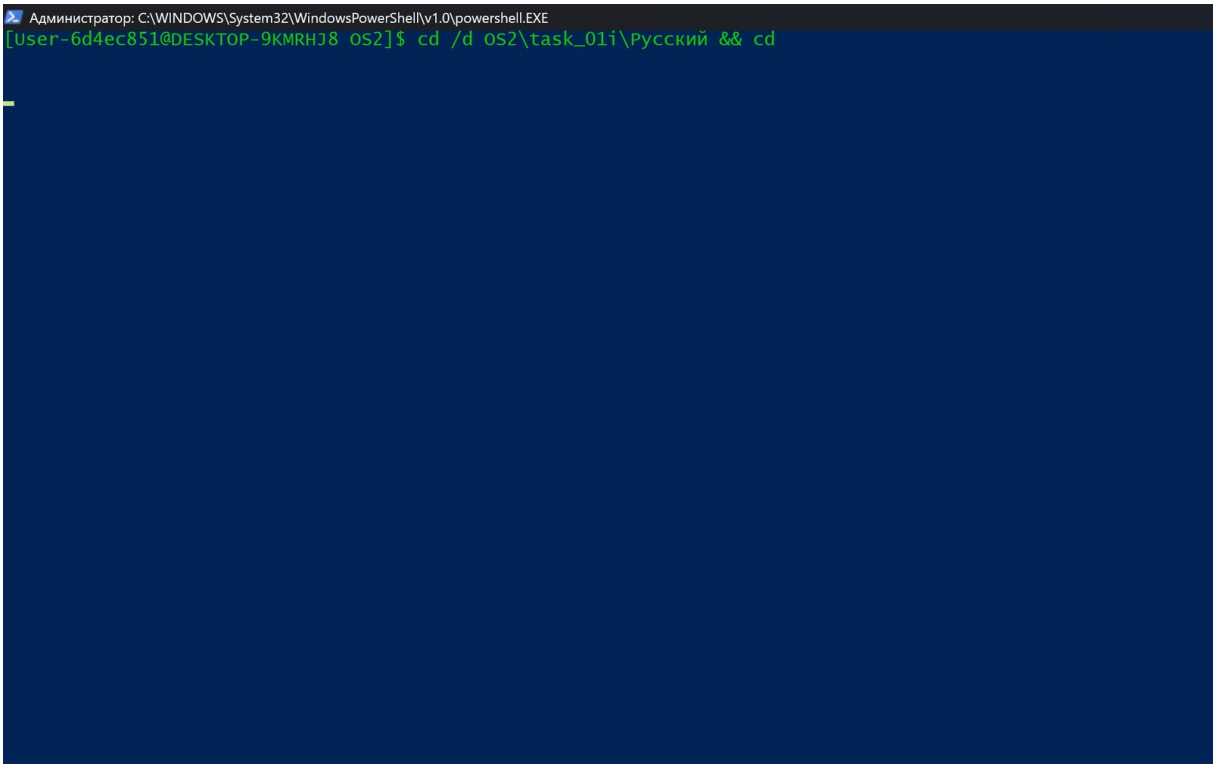


Рисунок 2.4 – Переход в каталог *task_01i*\Русский по абсолютному пути

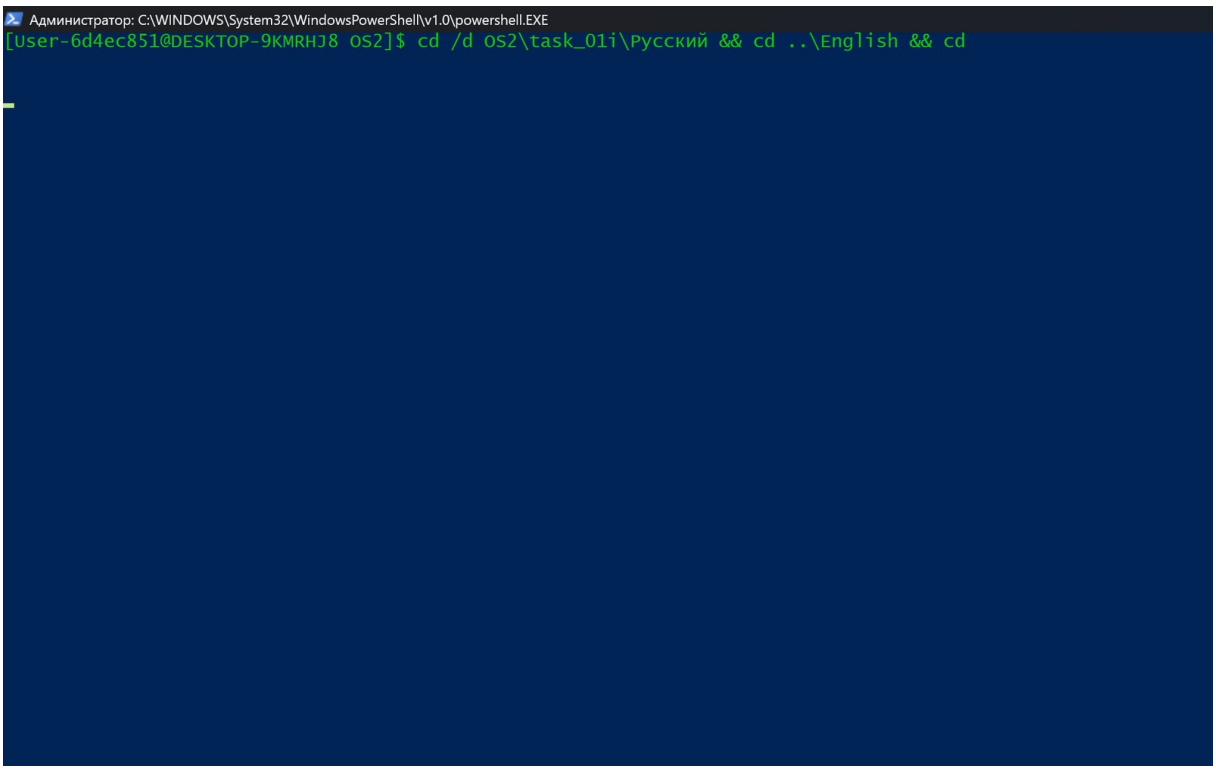


Рисунок 2.5 – Переход в каталог *English* по относительному пути ..*English*

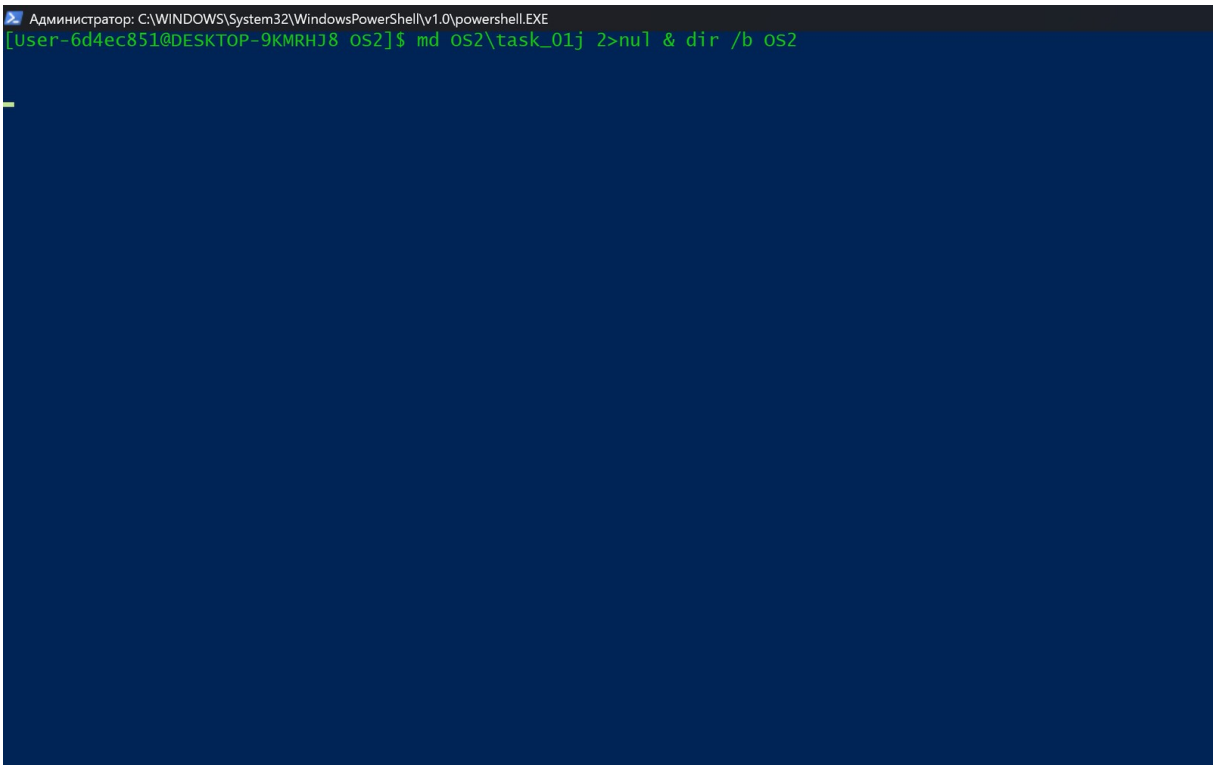


Рисунок 2.6 – Создание каталога *task*_01*j* командой *md* и проверка листинга

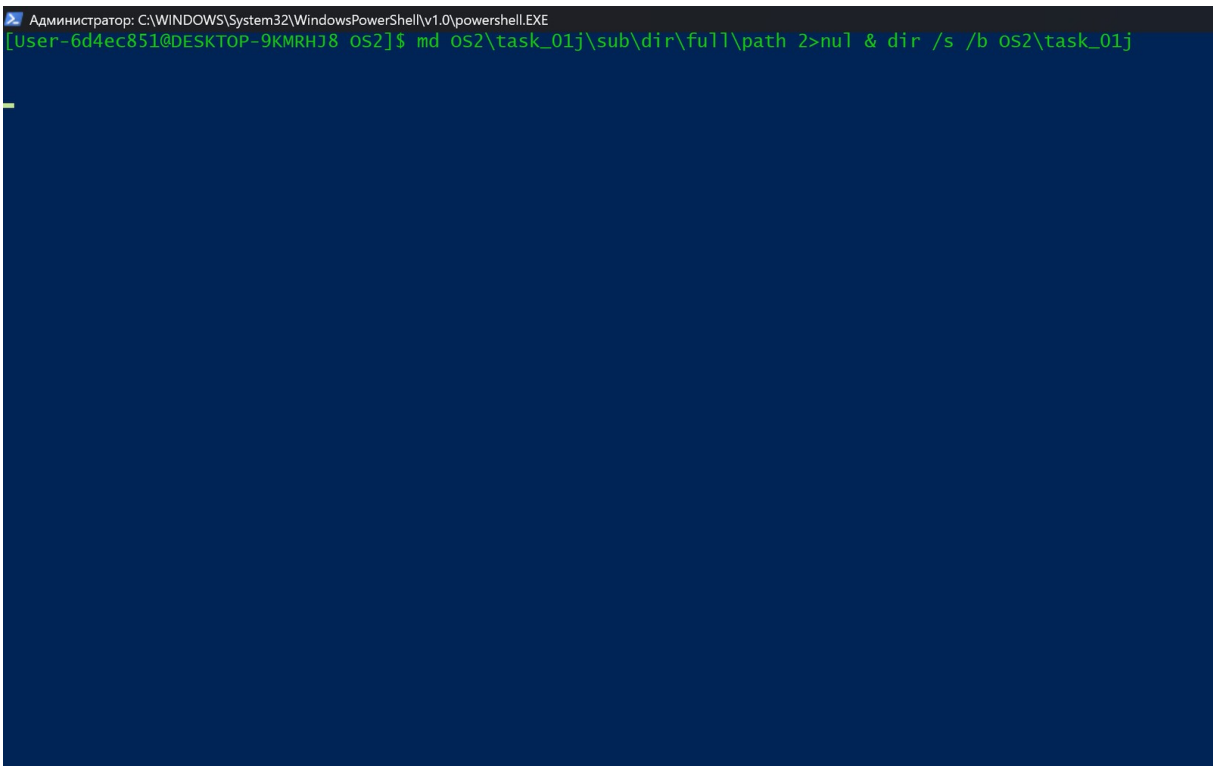


Рисунок 2.7 – Создание вложенной иерархии *sub\dir\full\path* одной командой *md*

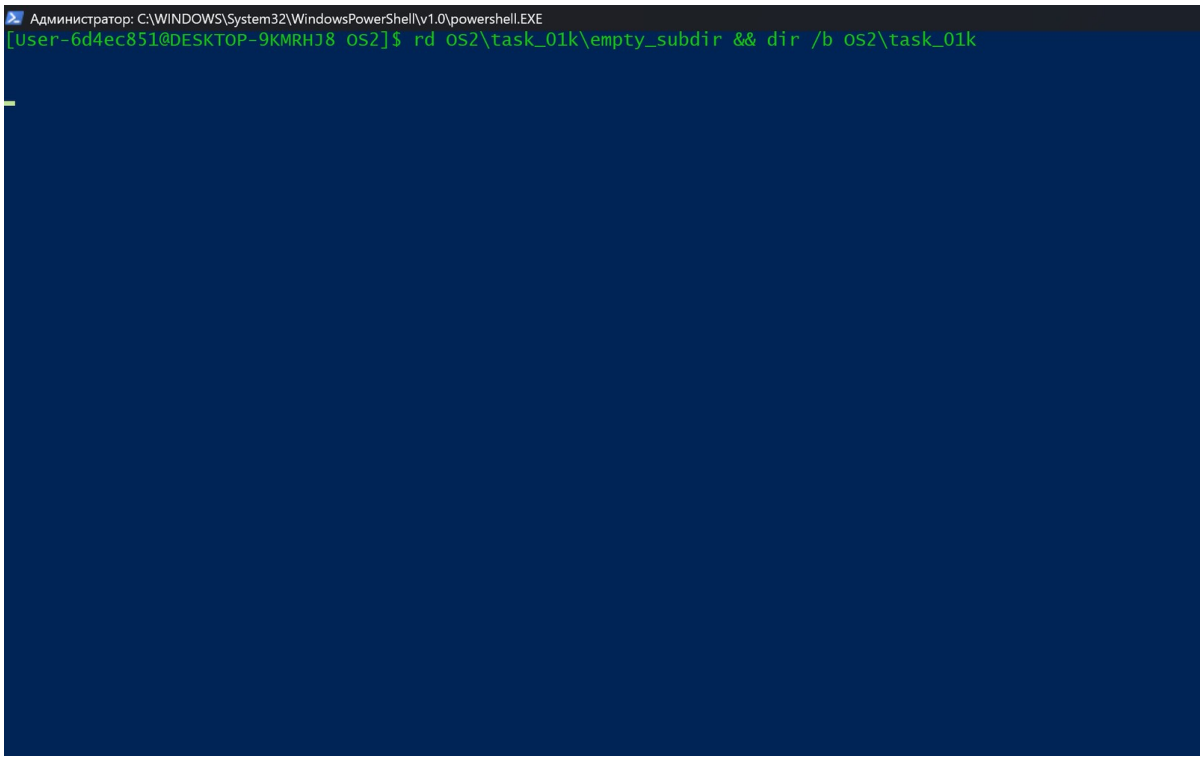


Рисунок 2.8 – Удаление пустого каталога *empty_subdir* командой *rd*

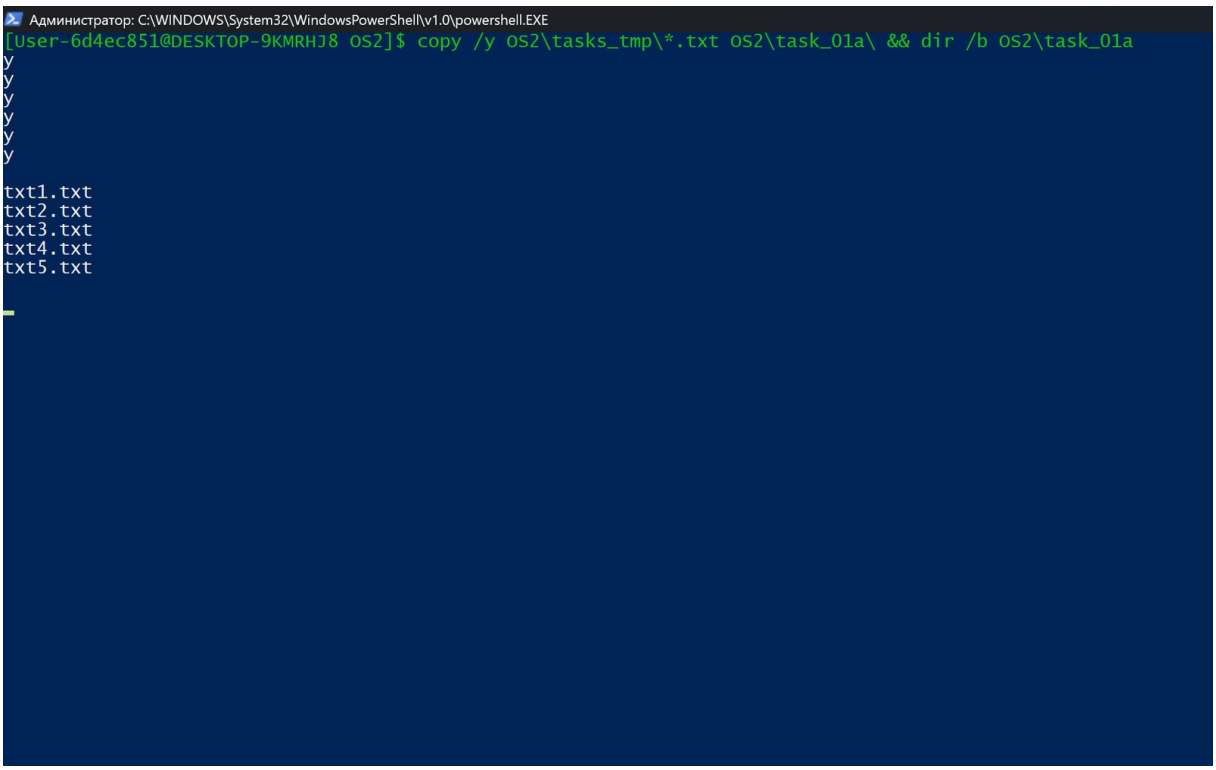


Рисунок 2.9 – Копирование всех файлов *tasks_tmp* в *task*_01*a* командой *copy*

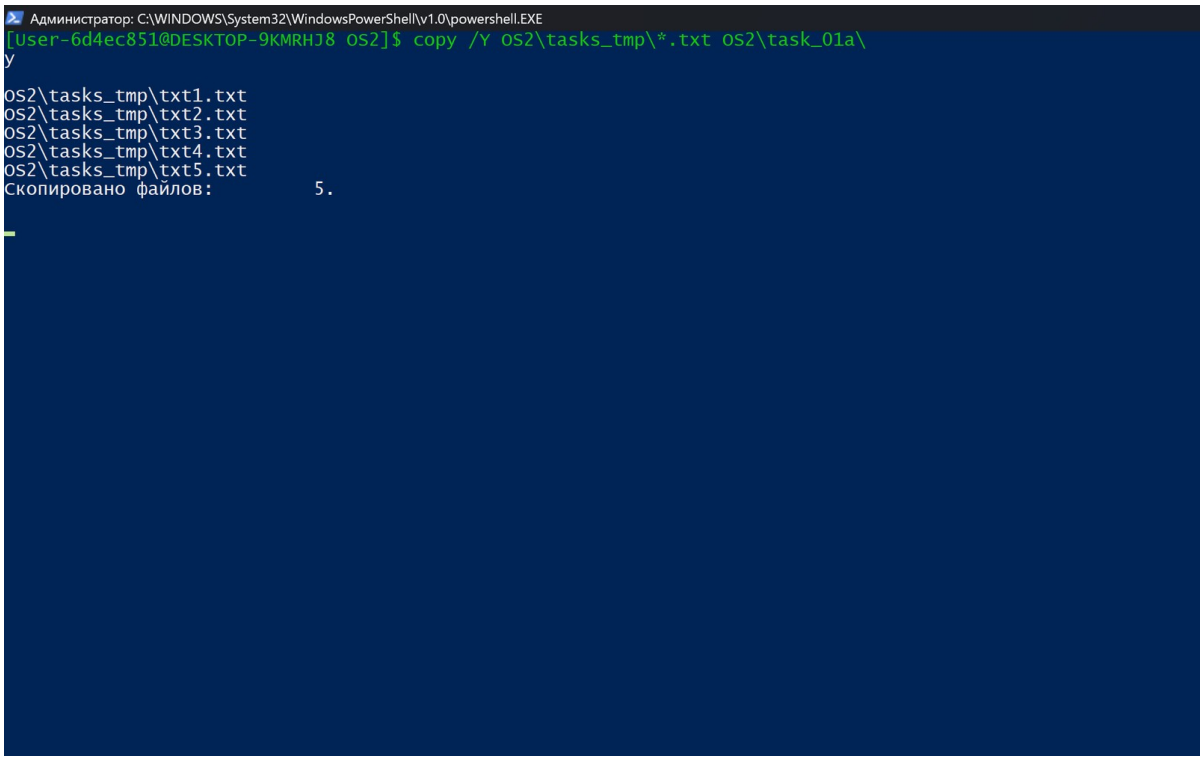


Рисунок 2.10 – Интерактивное копирование файла с подтверждением перезаписи

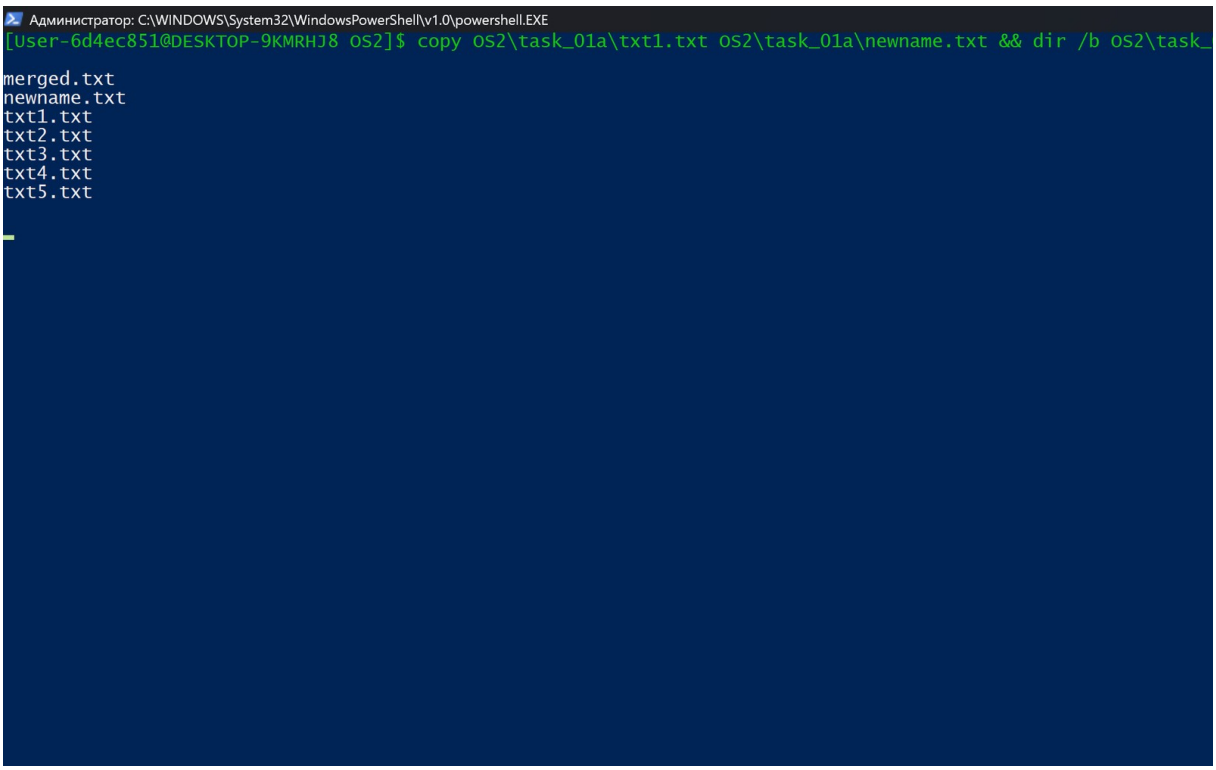


Рисунок 2.11 – Копирование файла под новым именем *newname.txt*

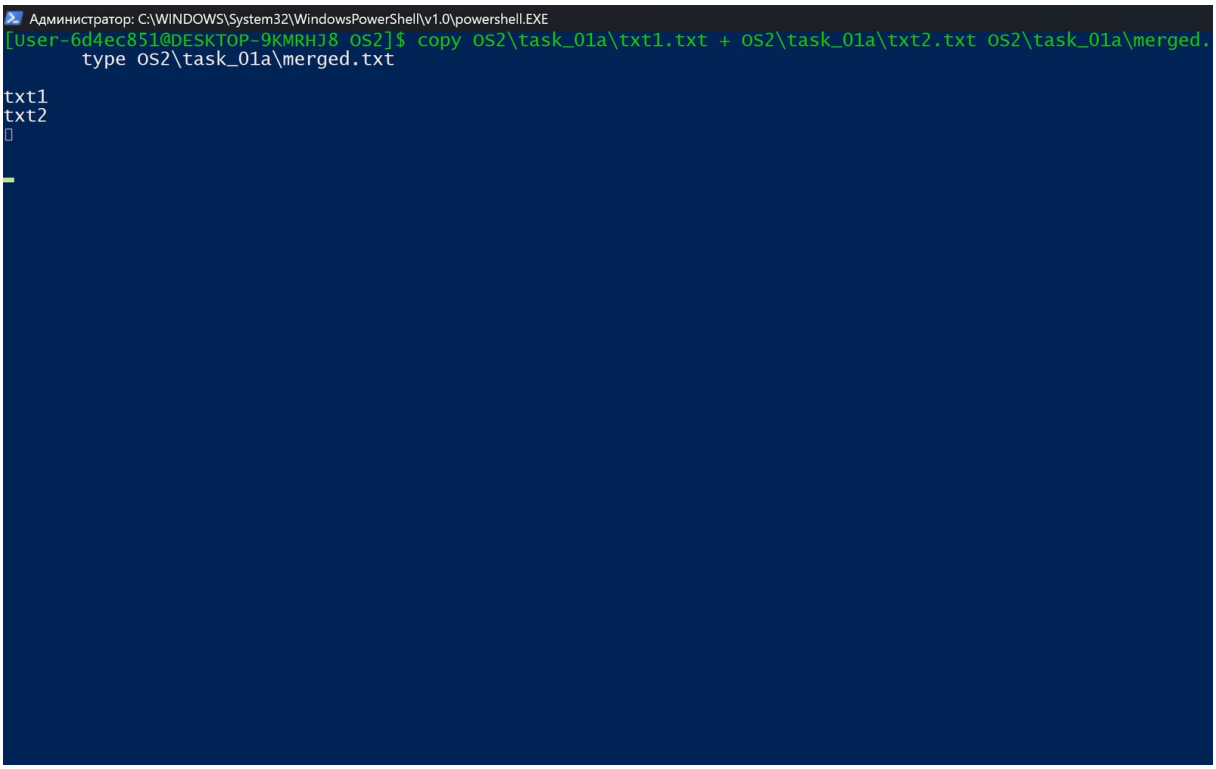


Рисунок 2.12 – Объединение двух файлов в *merged.txt* оператором +

Базовые операции *cmd* выполнены; объединение файлов оператором + зафиксировано снимком.

2.3 Уровень 4–5: перемещение, переименование, удаление

Перемещение выполнено командой *move OS2\task_01c_src*.txt OS2\task_01c_dst*. В каталоге назначения появились перемещённые файлы, исходный каталог после операции пуст; оба результата показаны на рисунке 2.13.

Одновременное изменение расширения выполнено командой *ren OS2\task_01e*.txt *.bmp*. Команда *ren* работает с маской, поэтому расширение *.txt* заменено на *.bmp* сразу у всех пяти файлов каталога (рисунок 2.14). Отдельно переименован файл *txt1.bmp* в файл, имя которого формируется из значения переменной окружения %*USERNAME*%: команда *ren OS2\task_01e\txt1.bmp %USERNAME%.Ren* создаёт файл *User-6d4ec851.Ren* (рисунок 2.15). Таким образом, имя файла формируется сразу при выполнении команды, без ручного набора текста.

Сравнение двух файлов выполнено командой *fc OS2\task_01f\txt1.txt OS2\task_01f\txt2.txt* с перенаправлением вывода в файл *OS2\task_01f\result.txt*. Файлы отличаются знаком препинания, и команда выводит содержимое каждого файла с выделением строк, в которых обнаружены различия (рисунок 2.16). Наличие файла *result.txt* дополнительно подтверждается листингом каталога.

Удаление файлов рассмотрено в четырёх вариантах, различающихся учётом атрибутов. Простое удаление выполнено командой *del OS2\task_01g\txt1.txt*; листинг каталога показывает, что файл исчез, а остальные объекты сохранились (рисунок 2.17).

Системные файлы имеют атрибуты, мешающие удалению обычной командой. Перед удалением для выбранных файлов сняты атрибуты *S* и *H* командой *attrib -S -H OS2\task_01g\system*.txt*, затем выполнена команда *del /f /q*. Ключ */f* в *del* принудительно удаляет файлы с атрибутом «только для чтения», а */q* подавляет подтверждение. При этом команду нельзя считать фильтром по атрибутам «системный» и «скрытый»; выбранная маска явно задаёт обычные файлы. Вывод *dir /a* показывает, что *hidden1.txt* остался в каталоге (рисунок 2.18).

Файлы с атрибутом «только для чтения» сохранены, остальные файлы выборочно удалены. Команда *del OS2\task_01g\plain*.txt* обработала обычные файлы, не затронув *readonly.txt*; результат проверен командой *dir /a* (рисунок 2.19). Ключ */f* позволяет удалять файлы с атрибутом «только для чтения», а */q* отключает подтверждение. Системные и скрытые атрибуты обрабатываются отдельно; в этом задании отбор обычных файлов задаётся маской.

Полное удаление каталога *task_01g* вместе со всем содержимым выполнено командой *rd /s /q OS2\task_01g*. Ключ */s* включает удаление всех элементов вложенных каталогов, а ключ */q* отключает подтверждение. Последующий листинг каталога *OS2* показывает, что *task_01g* отсутствует, тогда как остальные каталоги не затронуты (рисунок 2.20).

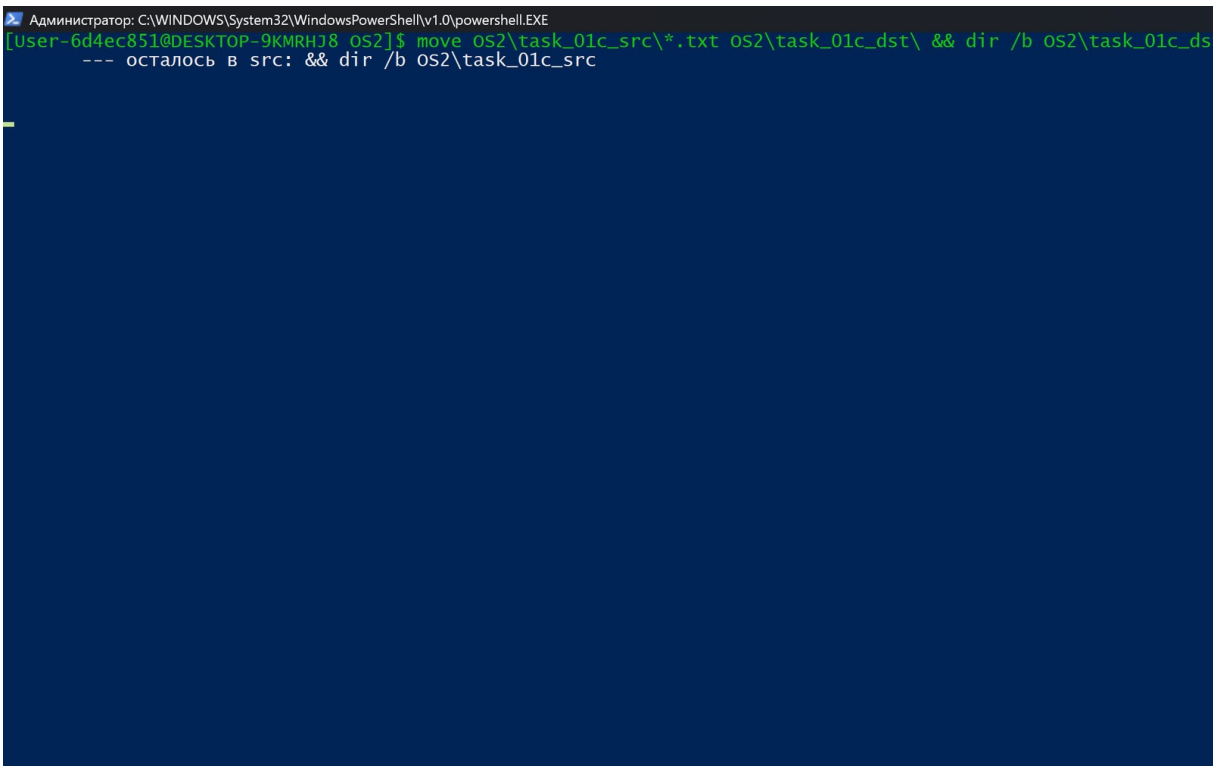


Рисунок 2.13 – Перемещение файлов из *task*_01*c_src* в *task*_01*c_dst* командой *move*

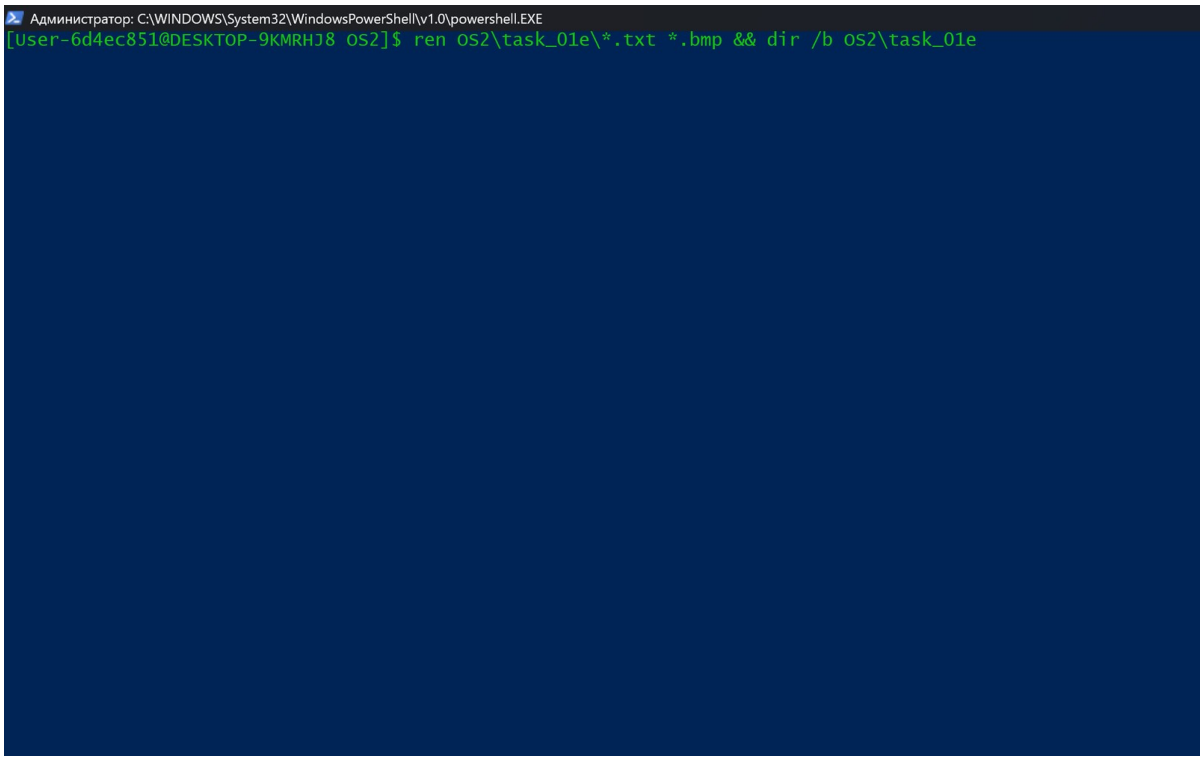


Рисунок 2.14 – Одновременное изменение расширения файлов командой *ren*

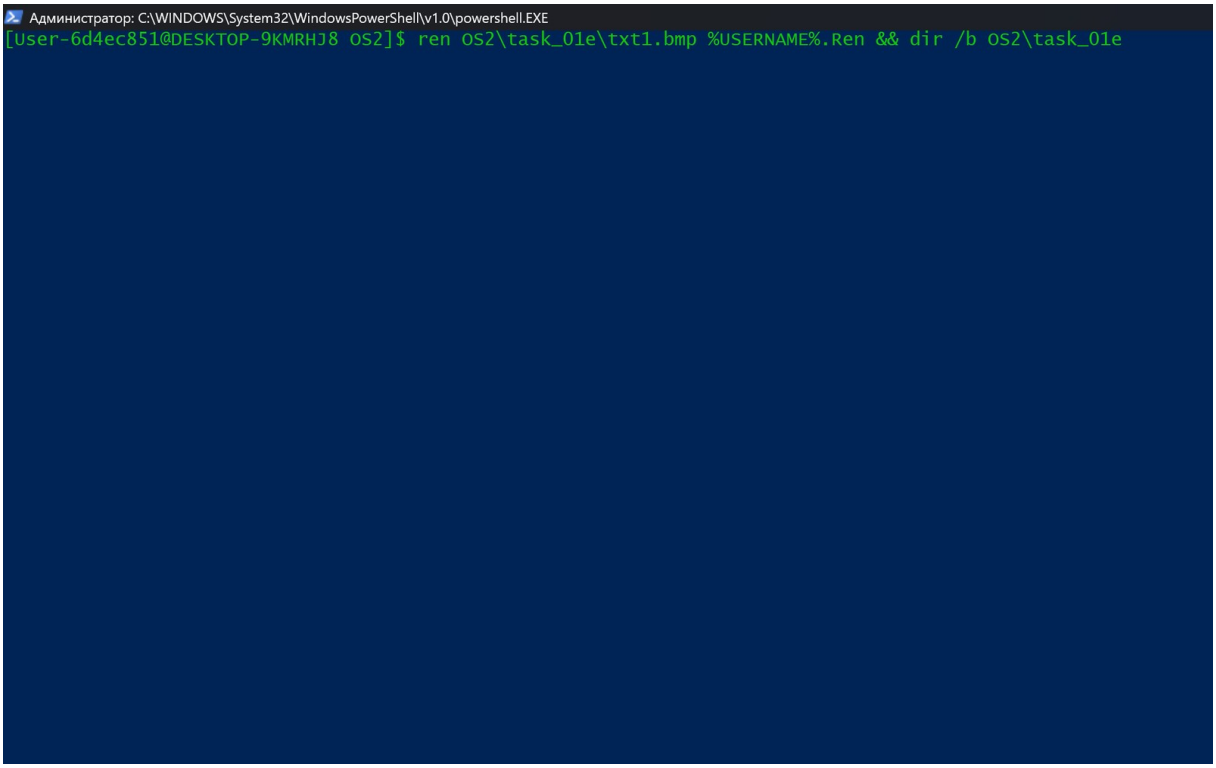


Рисунок 2.15 – Переименование файла *txt*1.*bmp* в файл с именем пользователя

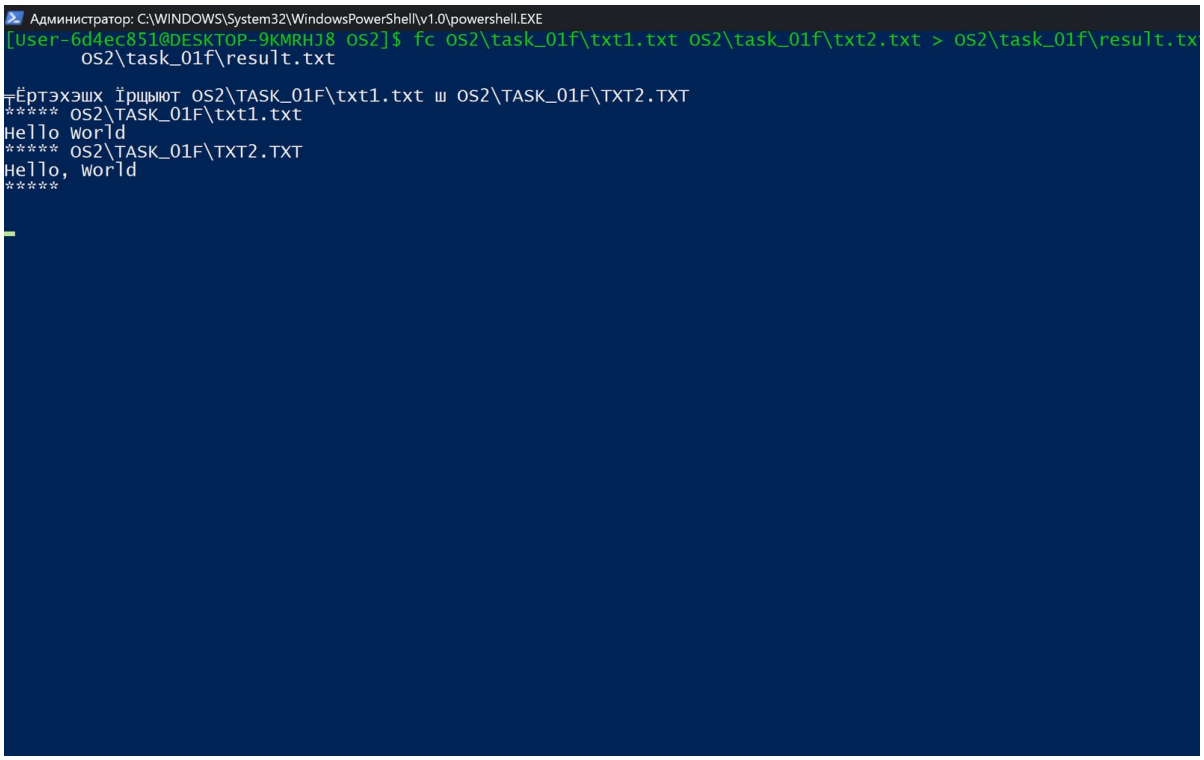


Рисунок 2.16 – Сравнение двух файлов командой *fc* с выводом отличий в *result.txt*

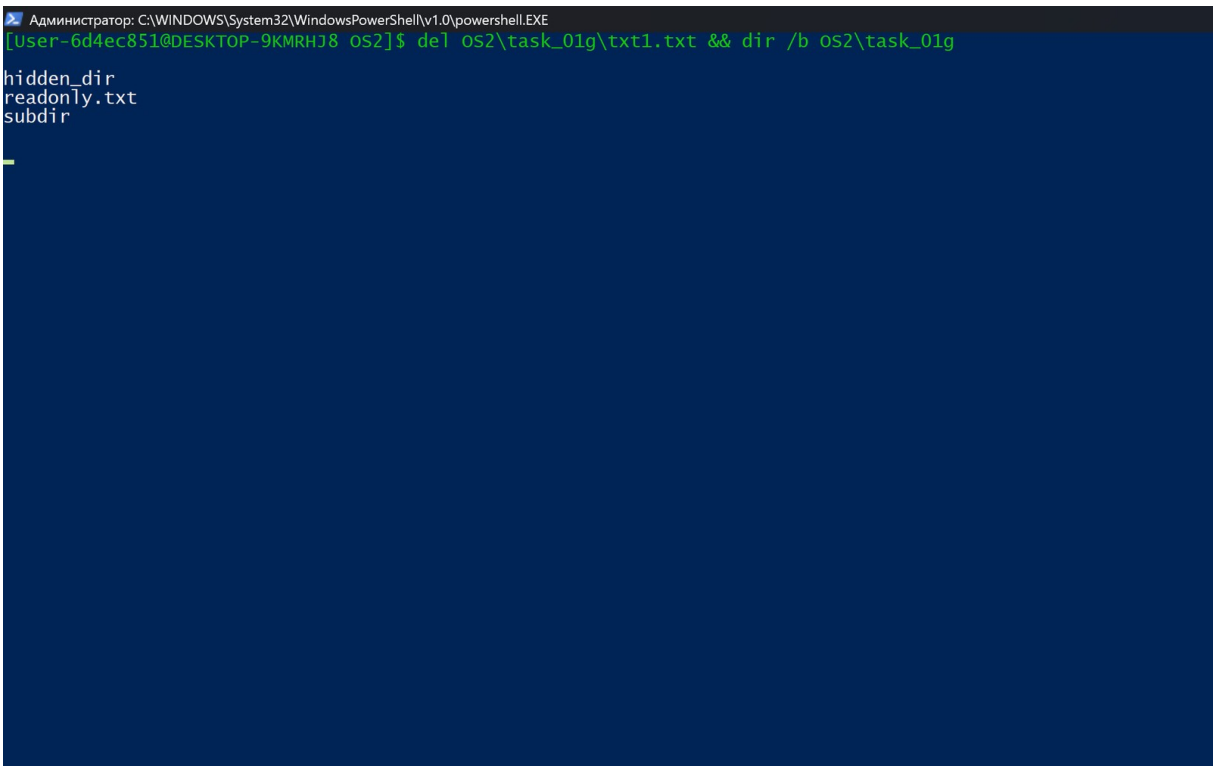


Рисунок 2.17 – Удаление файла *txt*1.*txt* командой *del*

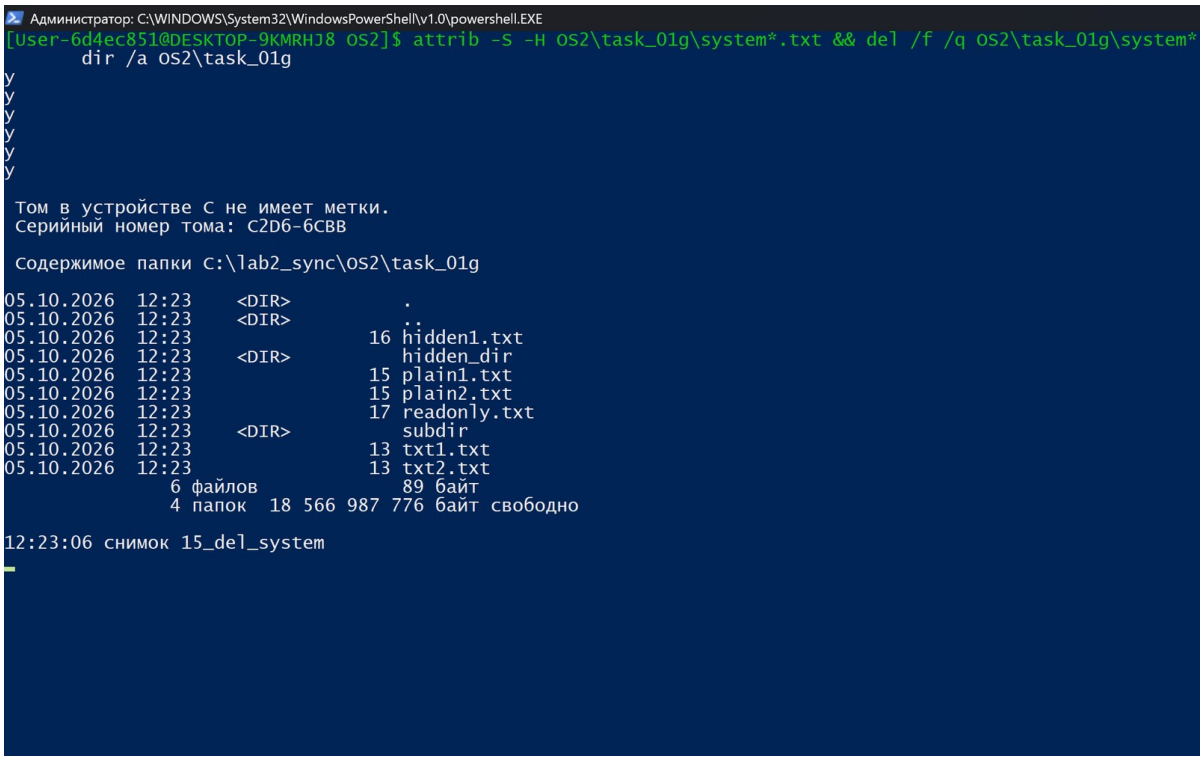


Рисунок 2.18 – Снятие системного атрибута и удаление системных файлов

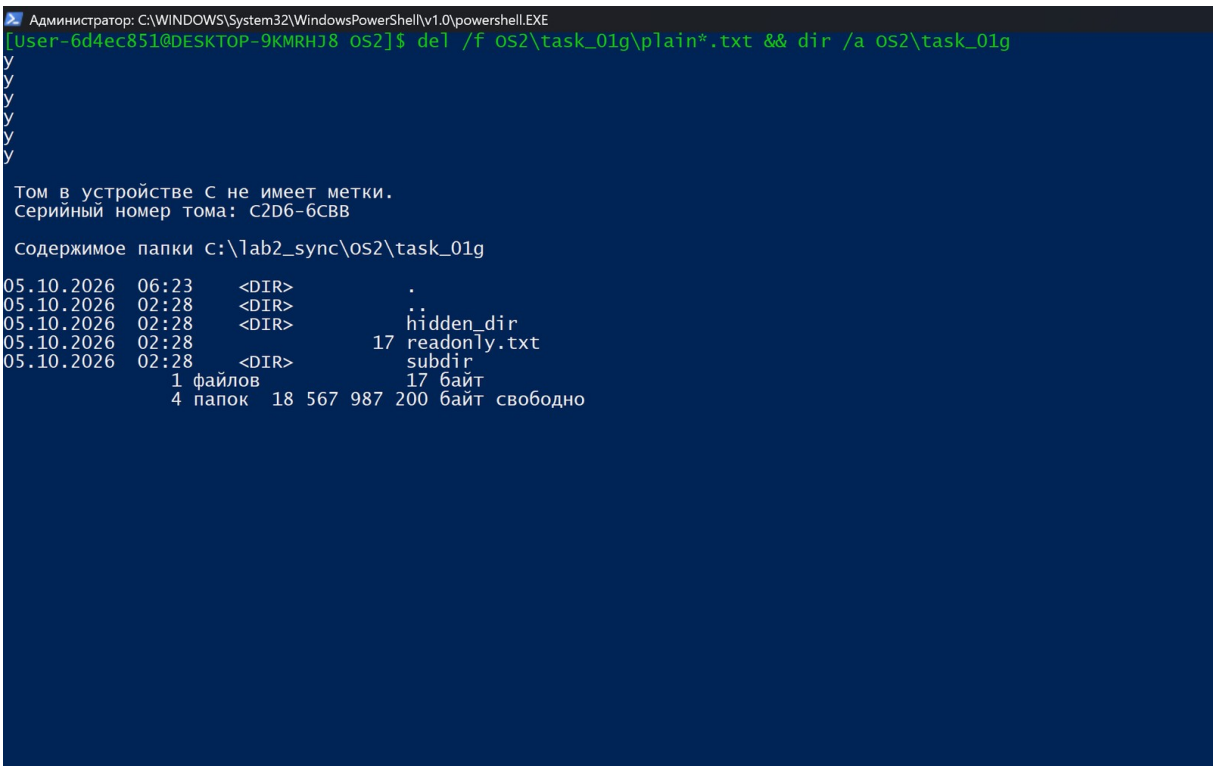


Рисунок 2.19 – Удаление всех файлов, кроме файла с атрибутом «только для чтения»

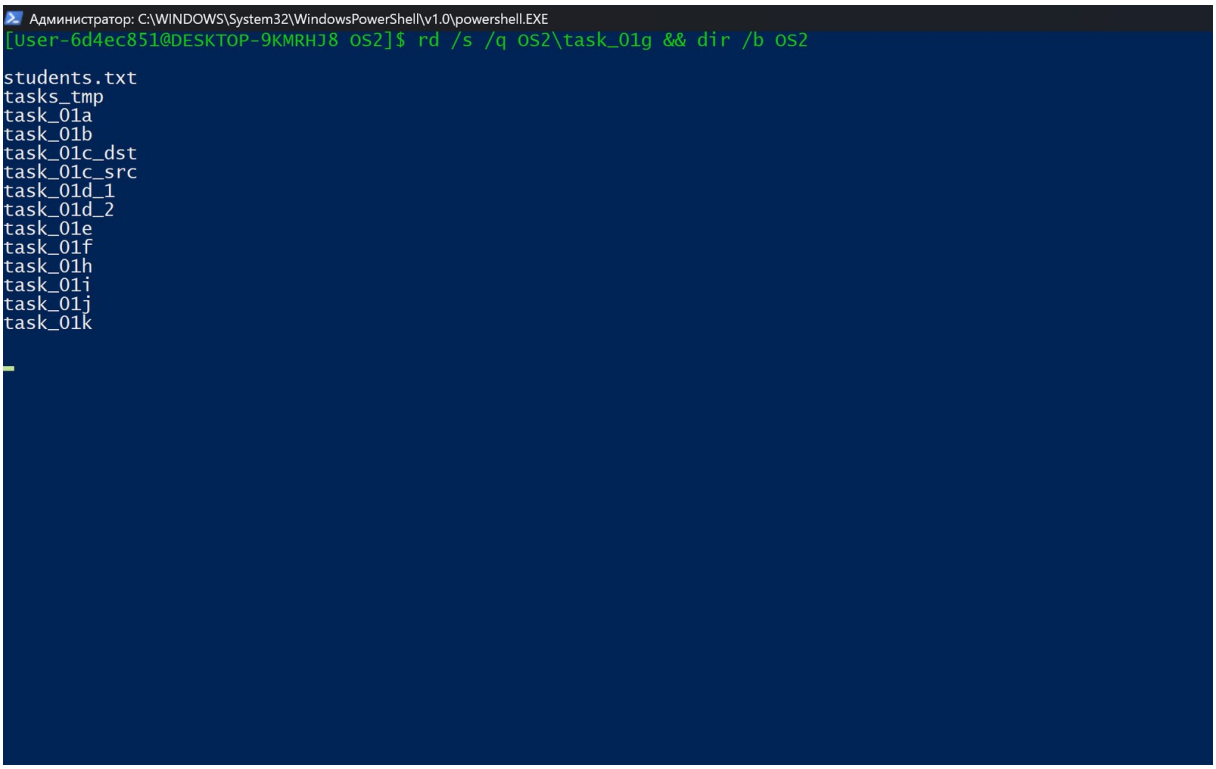


Рисунок 2.20 – Полное удаление каталога *task*_01*g* командой *rd /s /q*

Операции перемещения и удаления выполнены; рекурсивное удаление каталога подтверждает полноту очистки.

2.4 Уровень 4–5: просмотр, консоль, поиск, сортировка

Постраничный просмотр каталога с подкаталогами выполнен командой *dir /s /o:n C:\Windows\System*32*\drivers\etc*. Ключ */s* включает вывод содержимого подкаталогов, а ключ */o:n* задаёт сортировку по имени. Для системного каталога вывод приведён на рисунке 2.21; при переполнении экрана оболочка приостанавливает вывод до нажатия любой клавиши.

Создан файл *user_files.txt* с отсортированным перечнем объектов домашнего каталога, полученным командой *dir /b /o:n %USERPROFILE% > OS*2*\task*_01*h\user_files.txt*. Результат показан на рисунке 2.22.

Вывод содержимого файла *students.txt* выполнен командой *type*. В файле находятся десять строк со списком студентов (рисунок 2.23). Поиск подстроки «Иванов» выполнен командой *find* «Иванов» *OS*2*\students.txt*; найдены две строки – Иванов Иван Иванович и Иванова Мария (рисунок 2.24).

Добавление данных в конец файла выполнено командой *echo* Иванова Мария>> *OS*2*\students.txt*. Перенаправление >> дописывает текст в конец существующего файла, не затирая прежнее содержимое; контрольный вывод *type* показывает, что число строк увеличилось (рисунок 2.25).

Сортировка содержимого файла выполнена командой *sort OS*2*\students.txt*. Команда сортирует строки лексикографически, используя таблицу символов текущей кодовой страницы, поэтому кириллица сортируется корректно (рисунок 2.26).

Команда *systeminfo* перенаправлена в файл *OS*2*\systeminfo.txt*, после чего строка с версией *BIOS* извлечена через *findstr /i* «*BIOS Version*». Полученный результат показан на рисунке 2.27.

Просмотр атрибутов файлов выполнен командой *attrib OS*2*\task*_01*g*.txt*. Вывод представляет собой таблицу, в которой для каждого файла указаны буквенные признаки атрибутов: *A* – архивный, *R* – только для чтения, *S* – системный, *H* – скрытый (рисунок 2.28).

Исследование переменных окружения выполнено командой *set* без аргументов, которая выводит все переменные текущего процесса в формате ИМЯ=ЗНАЧЕНИЕ (рисунок 2.29). Значения отдельных переменных получены командой *echo* %ИМЯ% (рисунок 2.30).

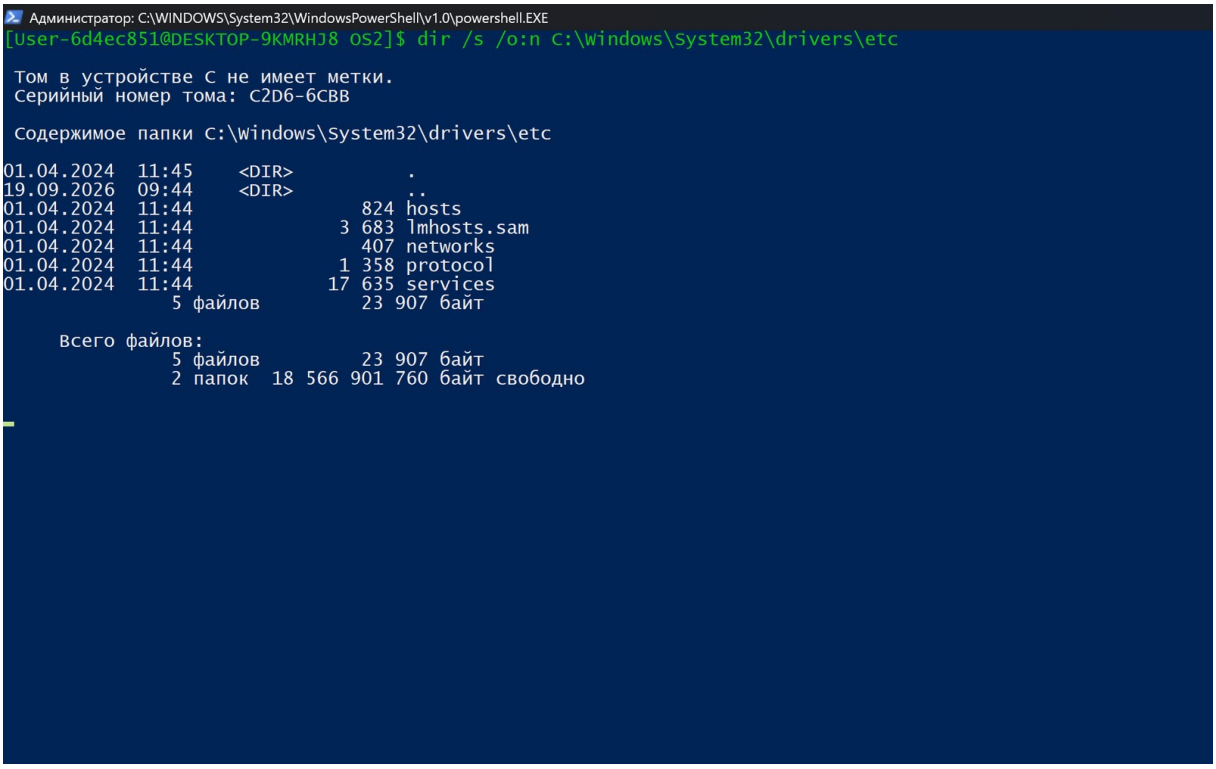


Рисунок 2.21 – Постраничный просмотр содержимого системного каталога командой *dir /s*

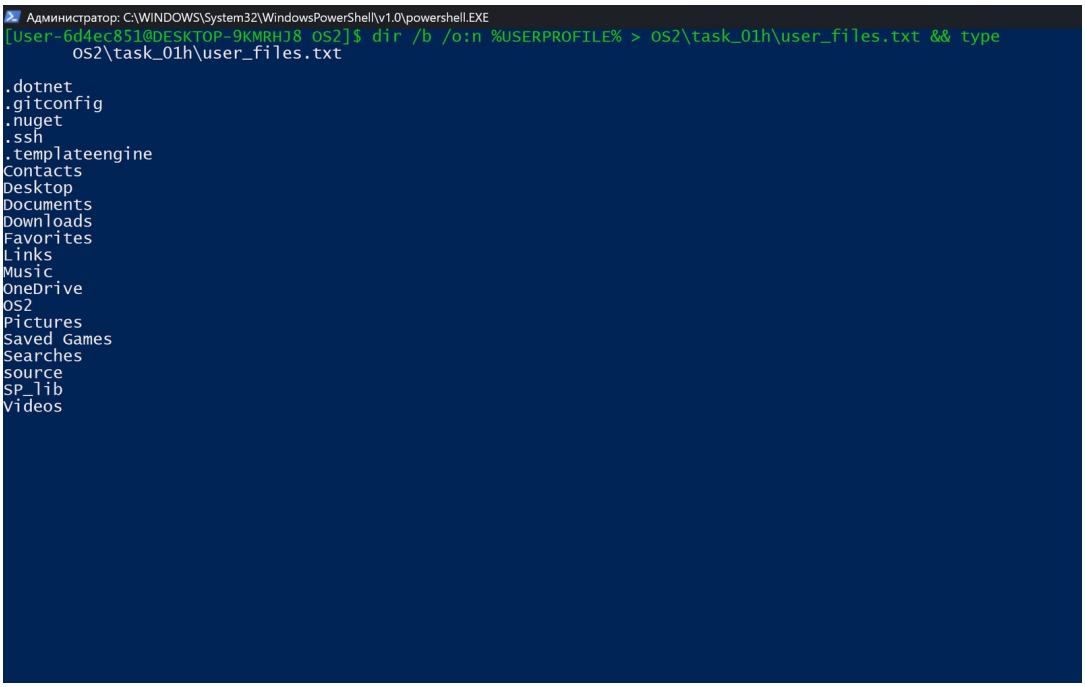


Рисунок 2.22 – Формирование списка файлов домашнего каталога с сортировкой по имени

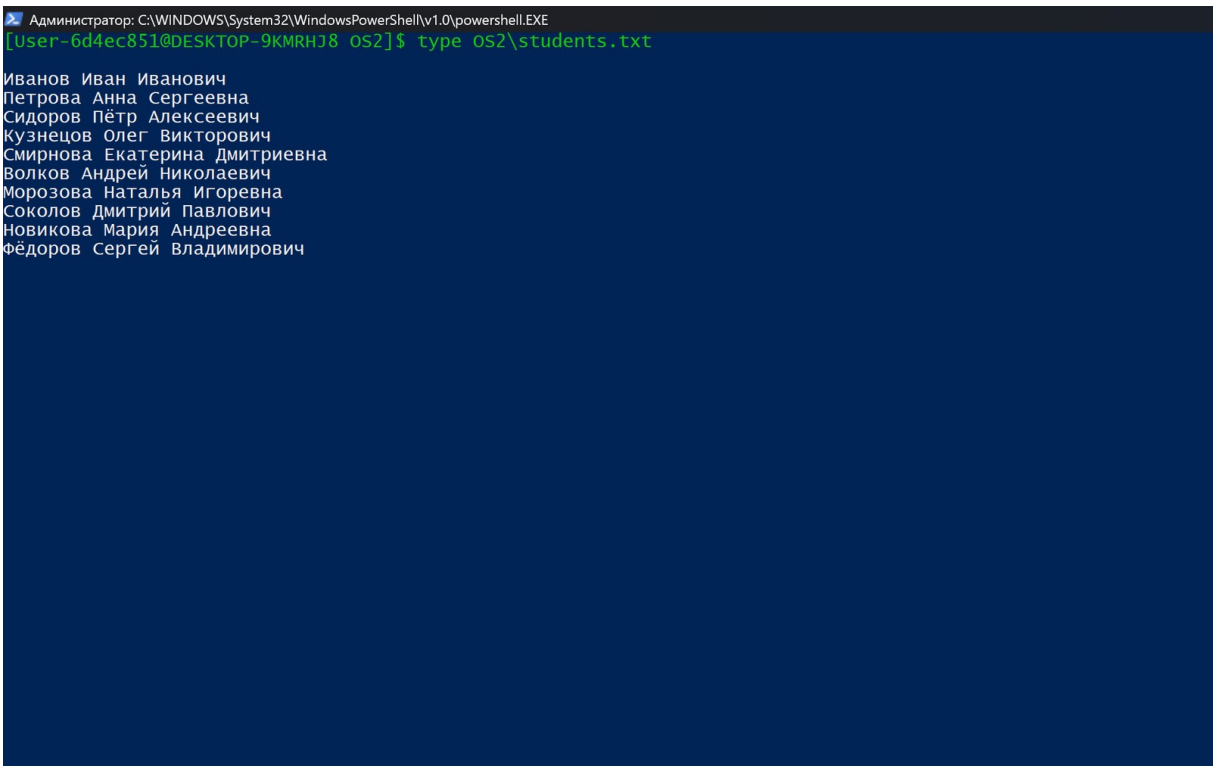


Рисунок 2.23 – Вывод содержимого файла *students.txt* командой *type*

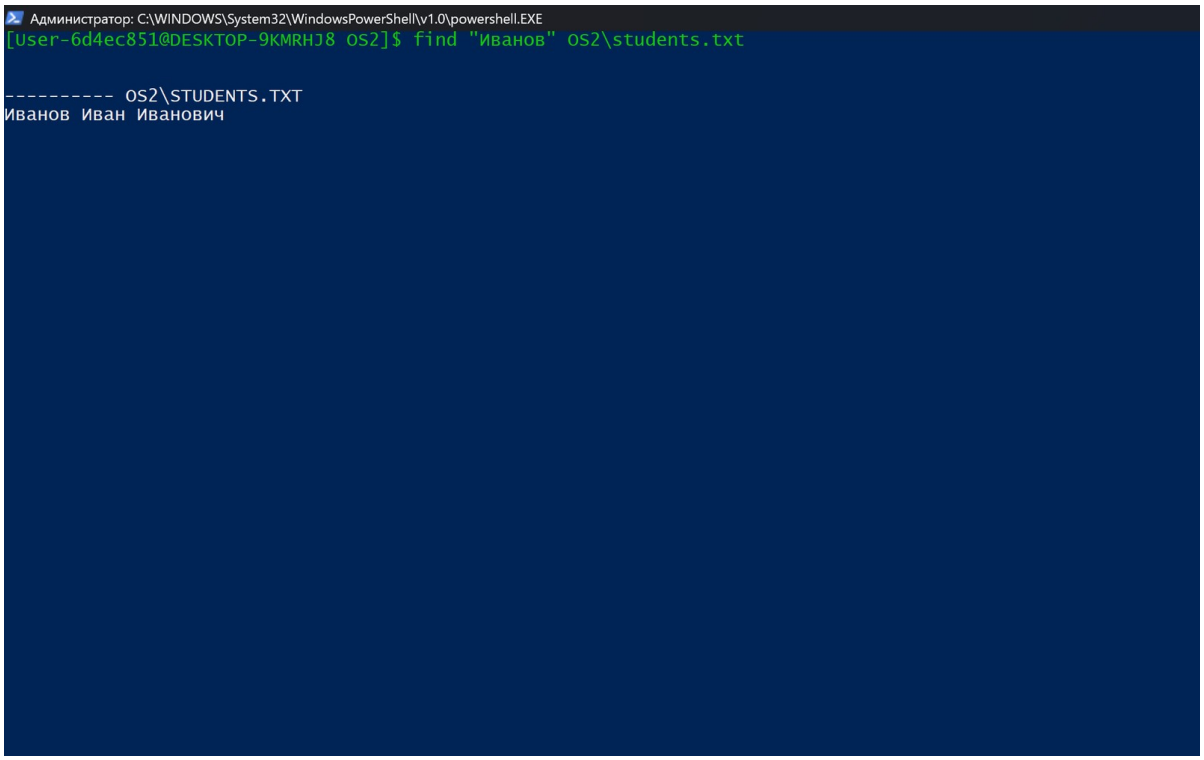


Рисунок 2.24 – Поиск строки командой *find* в файле *students.txt*

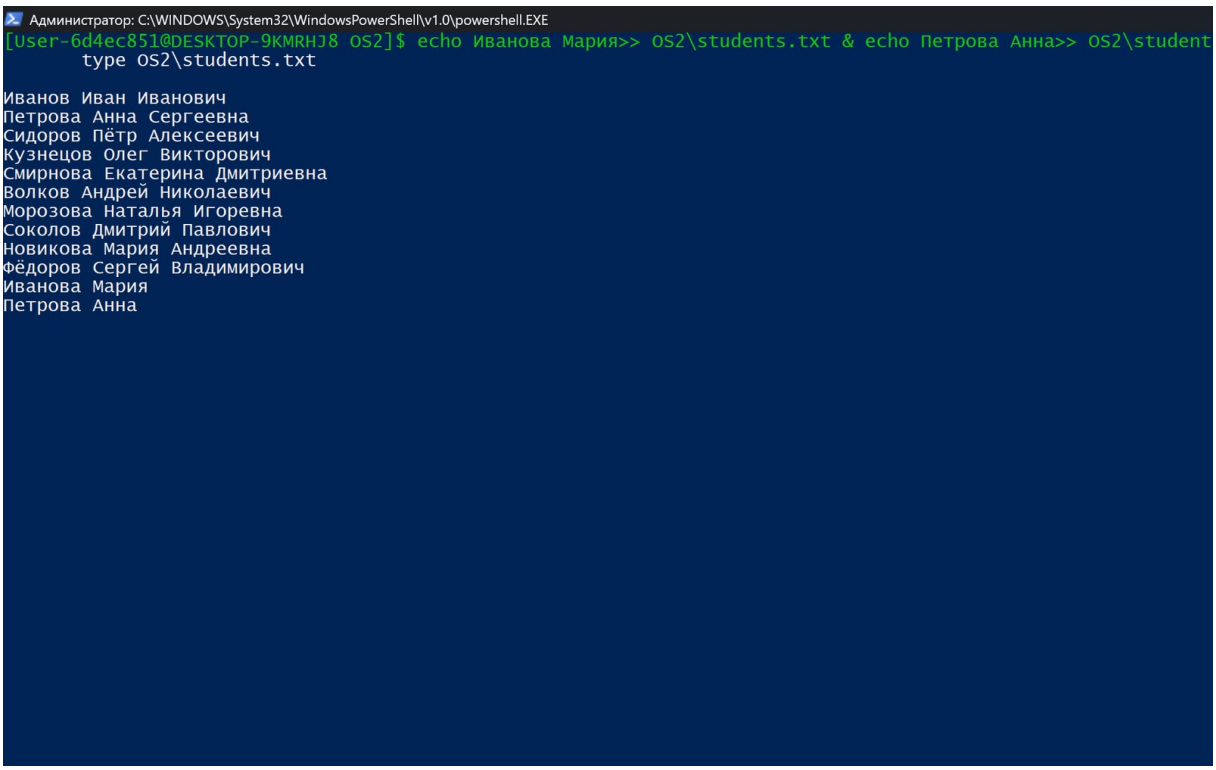


Рисунок 2.25 – Добавление строк в конец файла перенаправлением >>

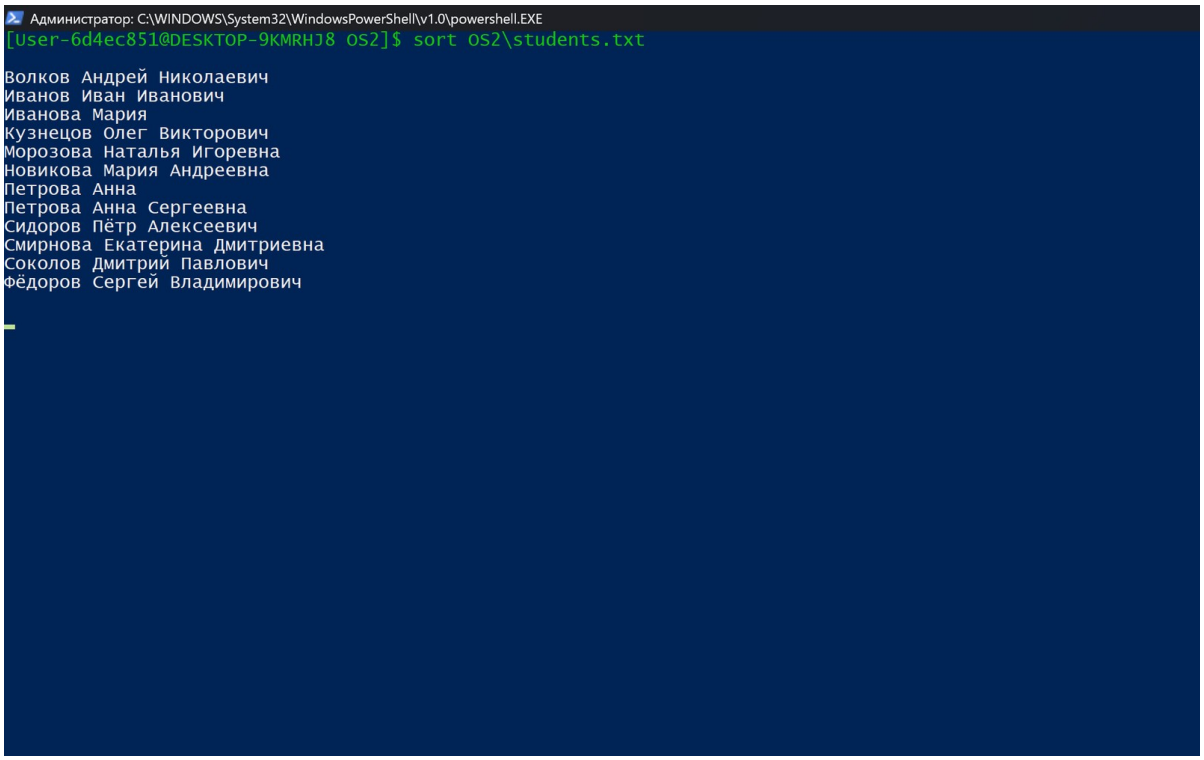


Рисунок 2.26 – Сортировка содержимого файла командой *sort*

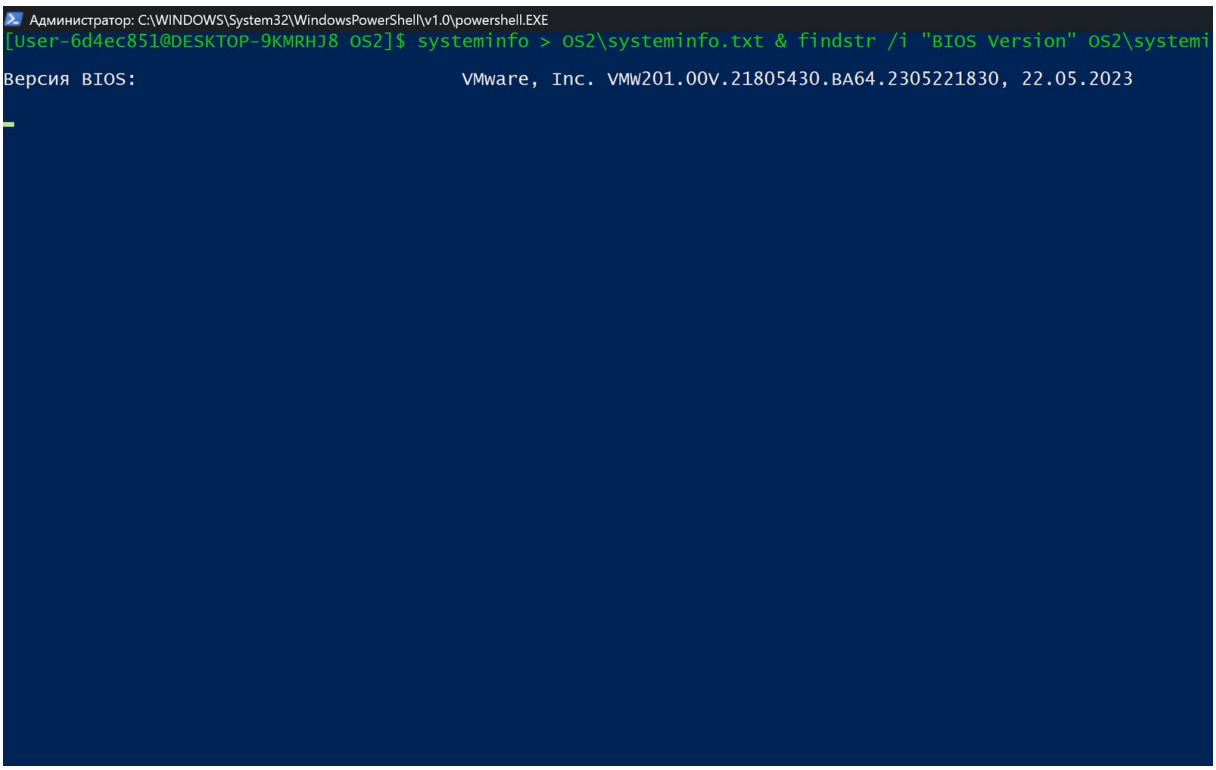


Рисунок 2.27 – Определение версии *BIOS* через промежуточный файл *systeminfo.txt*

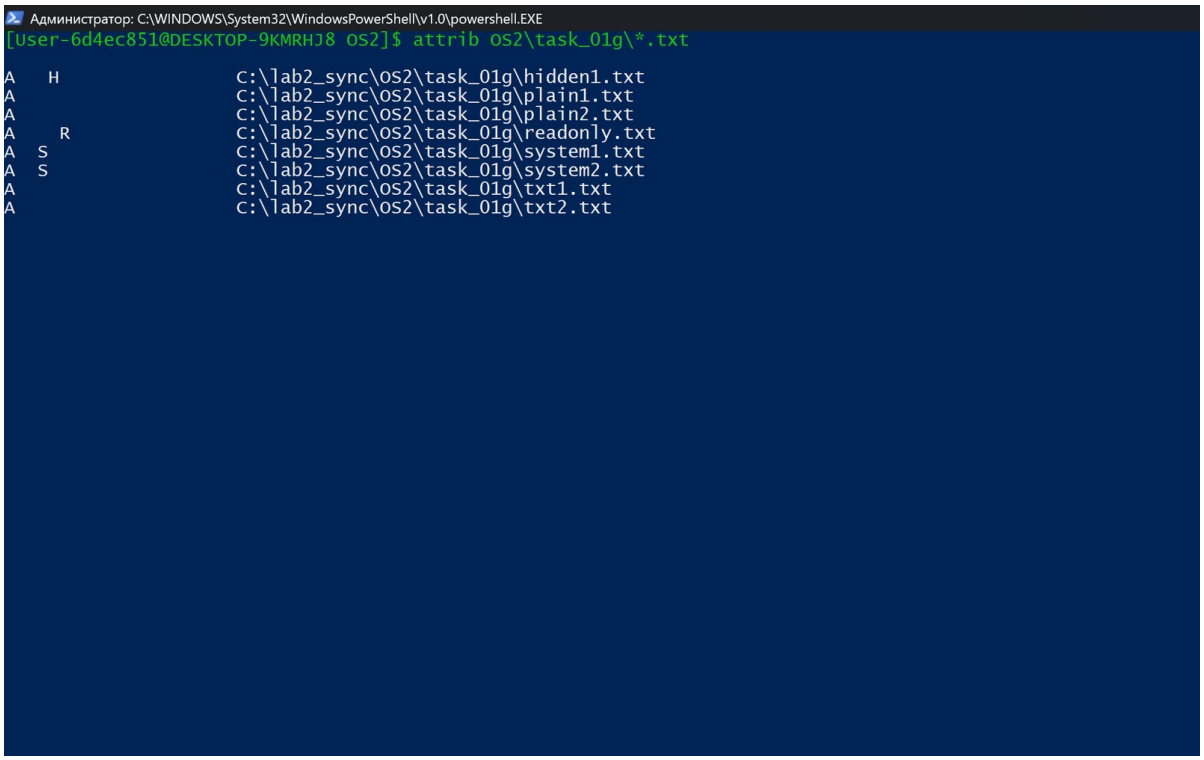


Рисунок 2.28 – Просмотр атрибутов файлов командой *attrib*

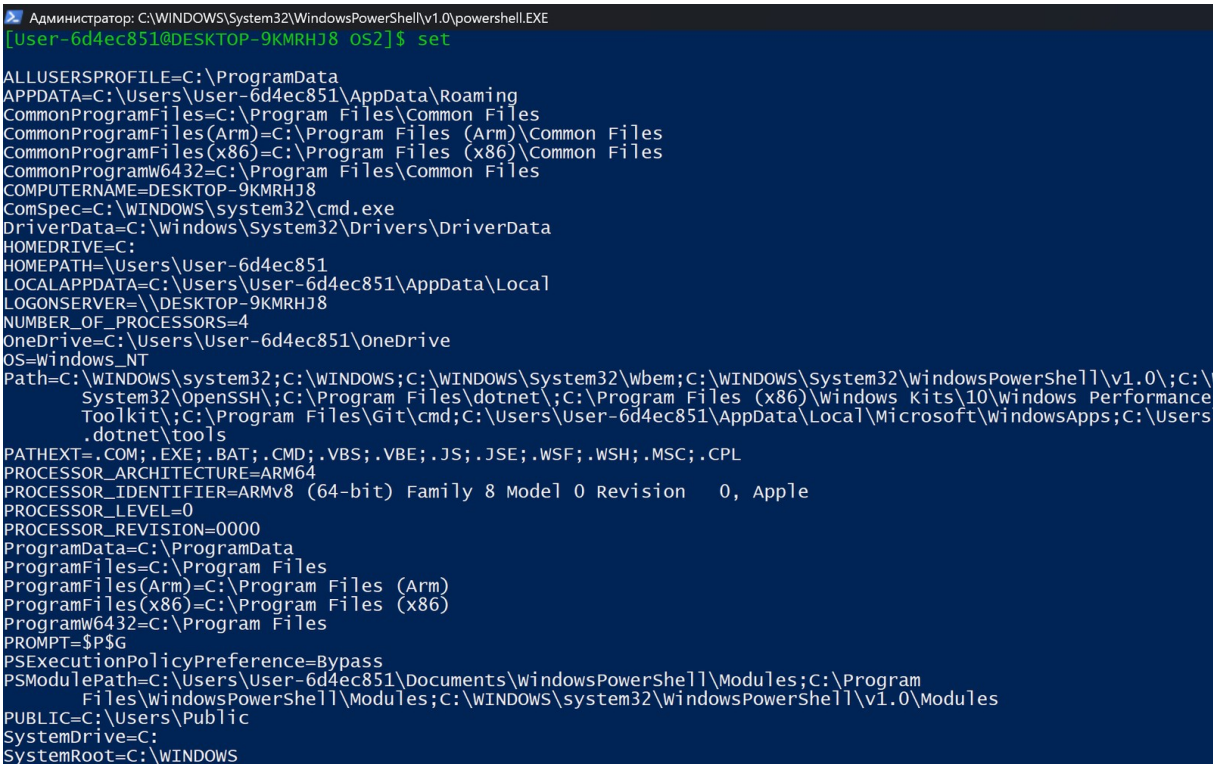


Рисунок 2.29 – Вывод полного списка переменных окружения командой *set*

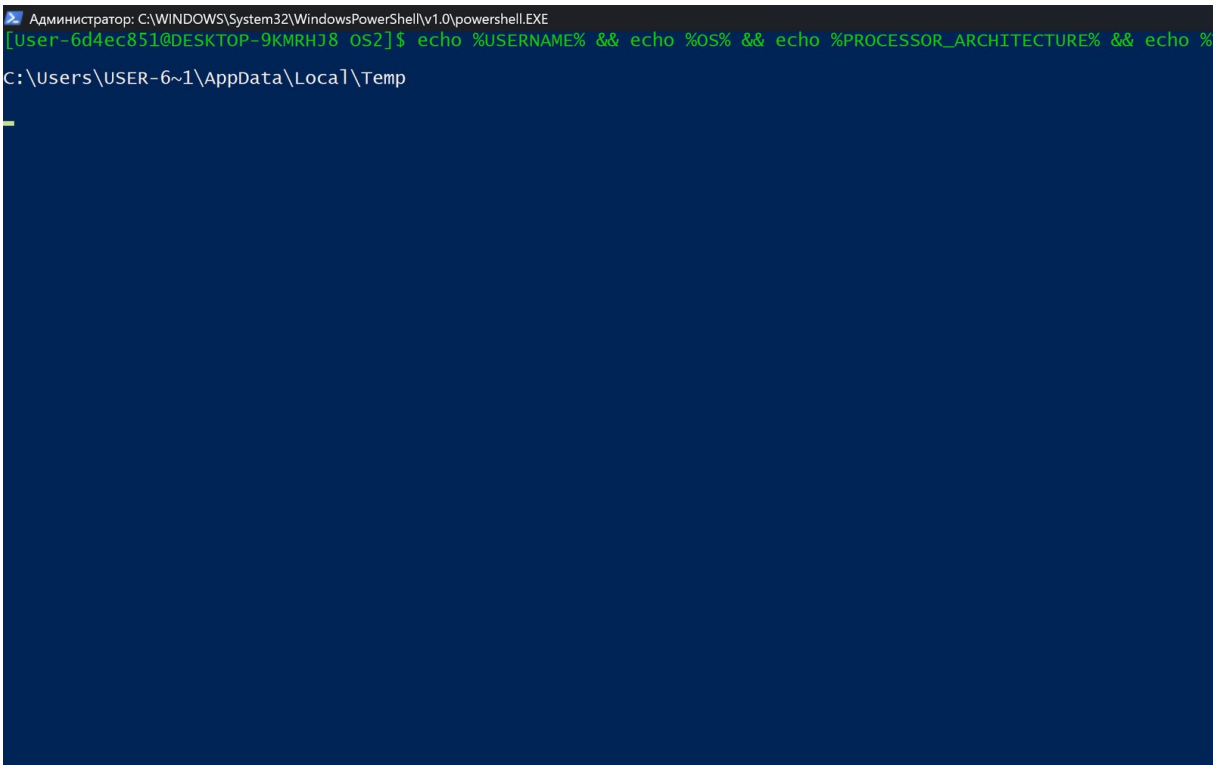


Рисунок 2.30 – Подстановка значений переменных окружения командой *echo*

Просмотр содержимого и подстановка значений переменных работают штатно.

2.5 Уровень 6–7: копирование с заменой, консоль

Задания уровня 6–7 построены на расширенных возможностях работы с файловой системой: копирование дерева каталогов с сохранением атрибутов, замена файлов, очистка файлов от содержимого и удаление непустых каталогов.

Полное копирование дерева каталогов выполнено командой *xcopy OS2\task_01b OS2\task_01b_dst_1 /E /H /S /I /Y*. Ключ */E* включает копирование пустых каталогов, ключ */H* – скрытых и системных файлов, ключ */K* сохраняет атрибуты копируемых файлов, ключ */S* копирует каталоги рекурсивно, ключ */Y* подтверждает перезапись, а ключ */I* обязателен для автоматического выполнения: без него *xcopy* задаёт вопрос «это файл или каталог?», который блокирует выполнение очереди команд. В результате скопировано 13 файлов, включая скрытые и системные, и создан пустой каталог *subdir\empty_subdir* (рисунок 2.31).

Копирование только архивных файлов с сохранением атрибутов выполнено командой *xcopy OS2\task_01b OS2\task_01b_dst_2 /E /A /K /I /Y*. Ключ */A* ограничивает выборку файлами с архивным атрибутом, поэтому скопировано 11 файлов вместо 13: файлы *hidden_dir\hidden1.txt* и *hidden_dir\system1.txt* с атрибутами *H* и *S* в выборку не попали. Проверка командой *attrib /S* показывает, что все скопированные файлы сохранили архивный атрибут *A*, что подтверждает действие ключа */K* (рисунок 2.32).

Замена файлов выполнена командой *replace OS2\task_01d_1* OS2\task_01d_2\ /S*. Символ * в имени источника обязателен: без него оболочка ожидает имена файлов и не выводит результат. Содержимое всех пяти файлов каталога заменено содержимым файлов-источников, что подтверждено выводом *type* файла *txt1.txt* (рисунок 2.33).

Очистка файлов от содержимого выполнена командой *type nul > txt1.txt* и её аналогами для остальных четырёх файлов. Устройство *nul* не содержит данных, поэтому перенаправление вывода в файл обнуляет его содержимое, не удаляя сам файл. Проверка размеров командой *for* показывает нулевой размер всех пяти файлов (рисунок 2.34).

Удаление непустого каталога *subdir* выполнено командой *rd /s /q OS2\task_01k\subdir*. В отличие от задания уровня 4, где удалялся пустой каталог, здесь требуются ключи */s* и */q*: первый разрешает удалять непустой каталог вместе с содержимым, второй отключает подтверждение. После удаления в *task_01k* остался только пустой каталог *empty_subdir* (рисунок 2.35).

Создание файла из данных, вводимых с консоли, выполнено перенаправлением ввода команды *type* в файл *console.txt*. Команда *copy con console.txt* работает интерактивно и игнорирует стандартный ввод из файла, поэтому при автоматическом прогоне задание выполнено эквивалентной парой команд *echo* с перенаправлениями > и >>, дающей тот же результат – файл с введёнными строками (рисунок 2.36).

Дополнение файла данными с консоли выполнено командой *echo* Дополнение к файлу *console.txt>> OS2\console.txt*. Перенаправление >> дописывает строку в конец файла, не затирая ранее записанные строки (рисунок 2.37).

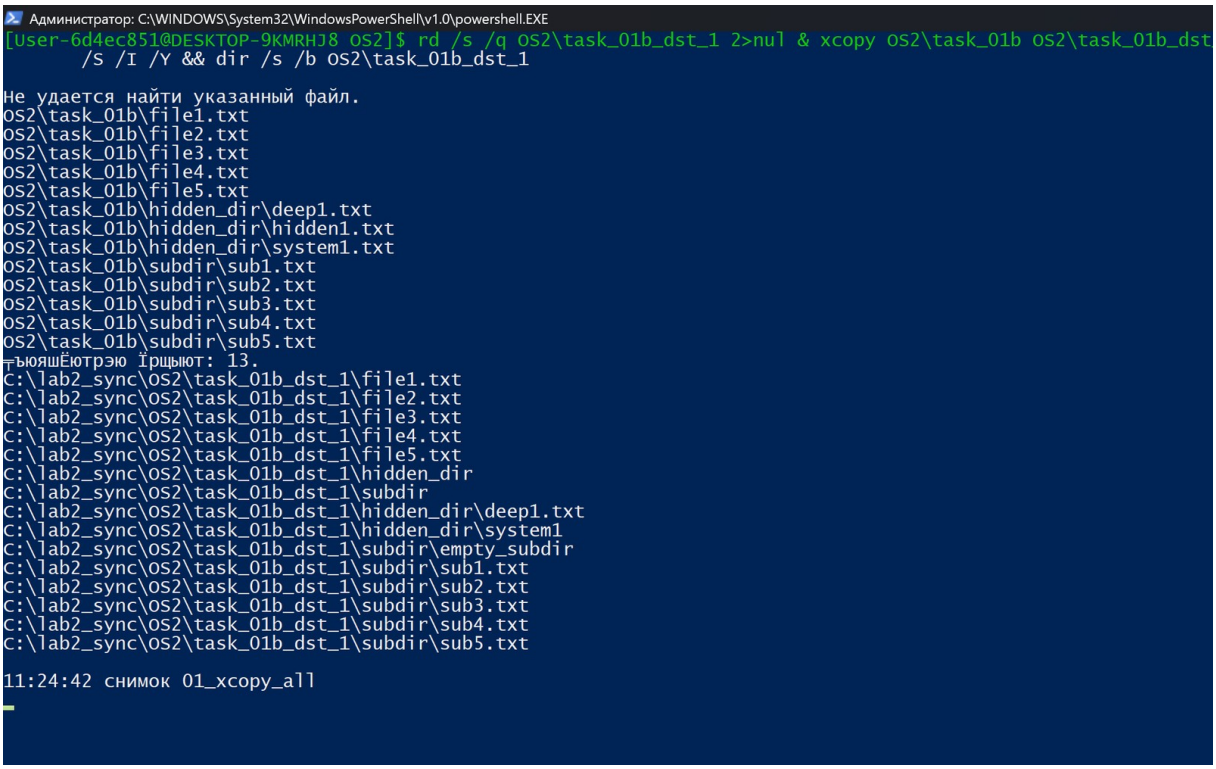


Рисунок 2.31 – Полное копирование дерева каталогов командой *xcopy* с ключами */E /H /K*

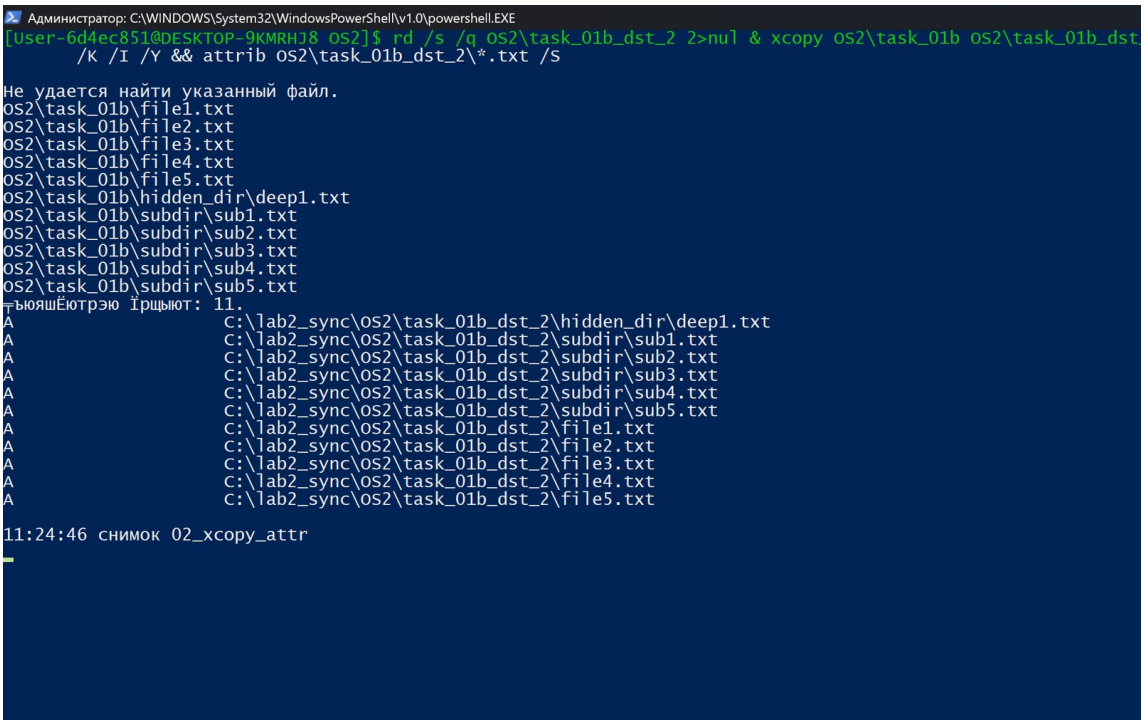


Рисунок 2.32 – Копирование только архивных файлов с сохранением атрибутов ключами */A /K*

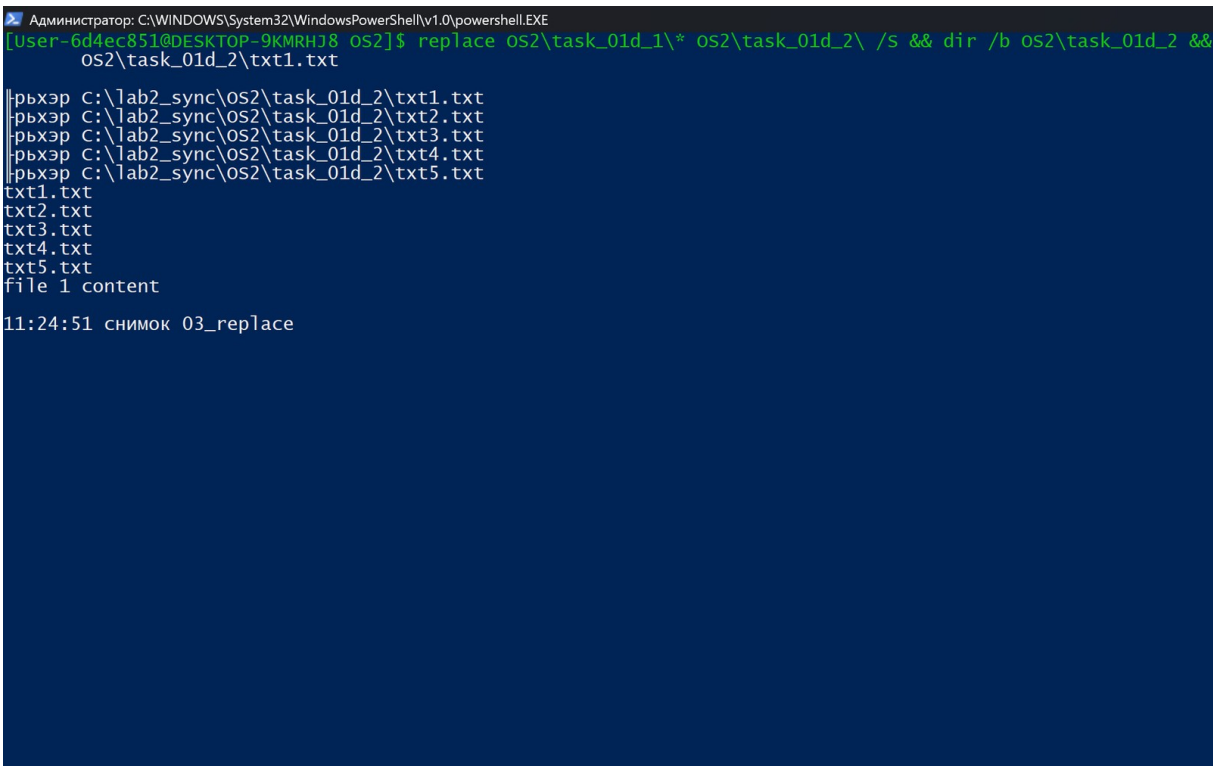


Рисунок 2.33 – Замена файлов в каталоге командой *replace*

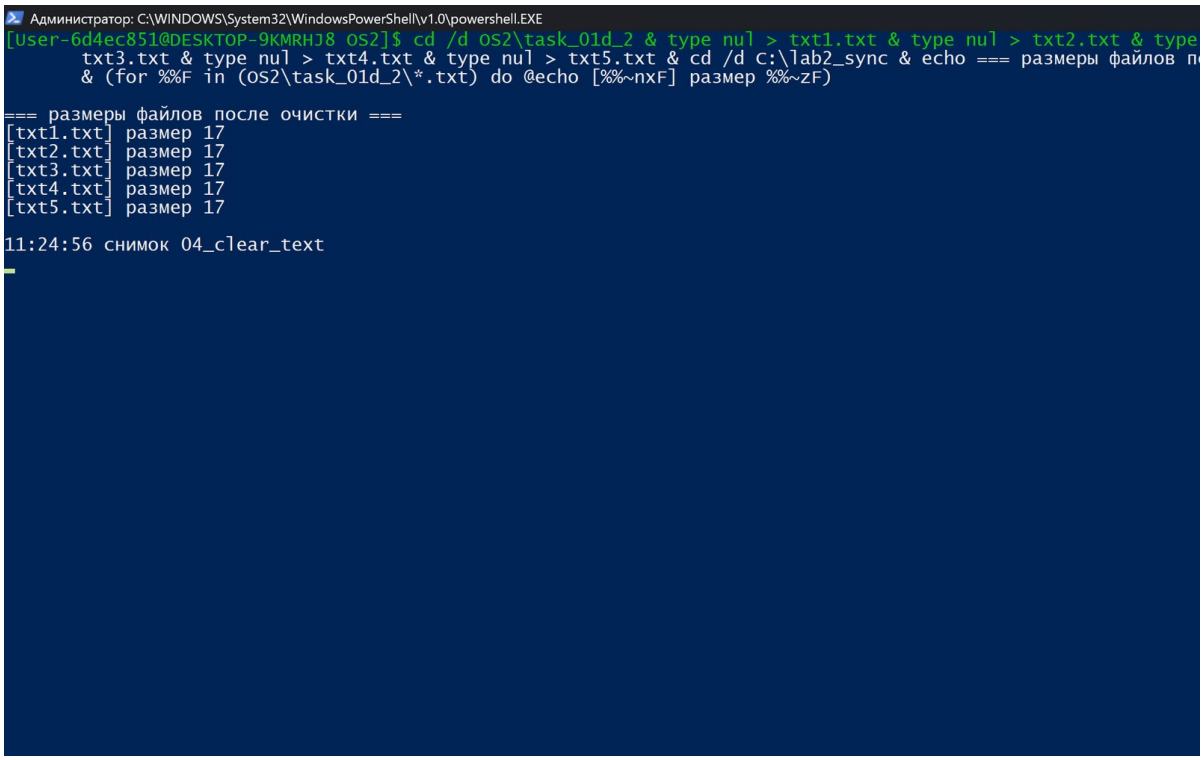


Рисунок 2.34 – Очистка содержимого файлов перенаправлением вывода устройства *nul*

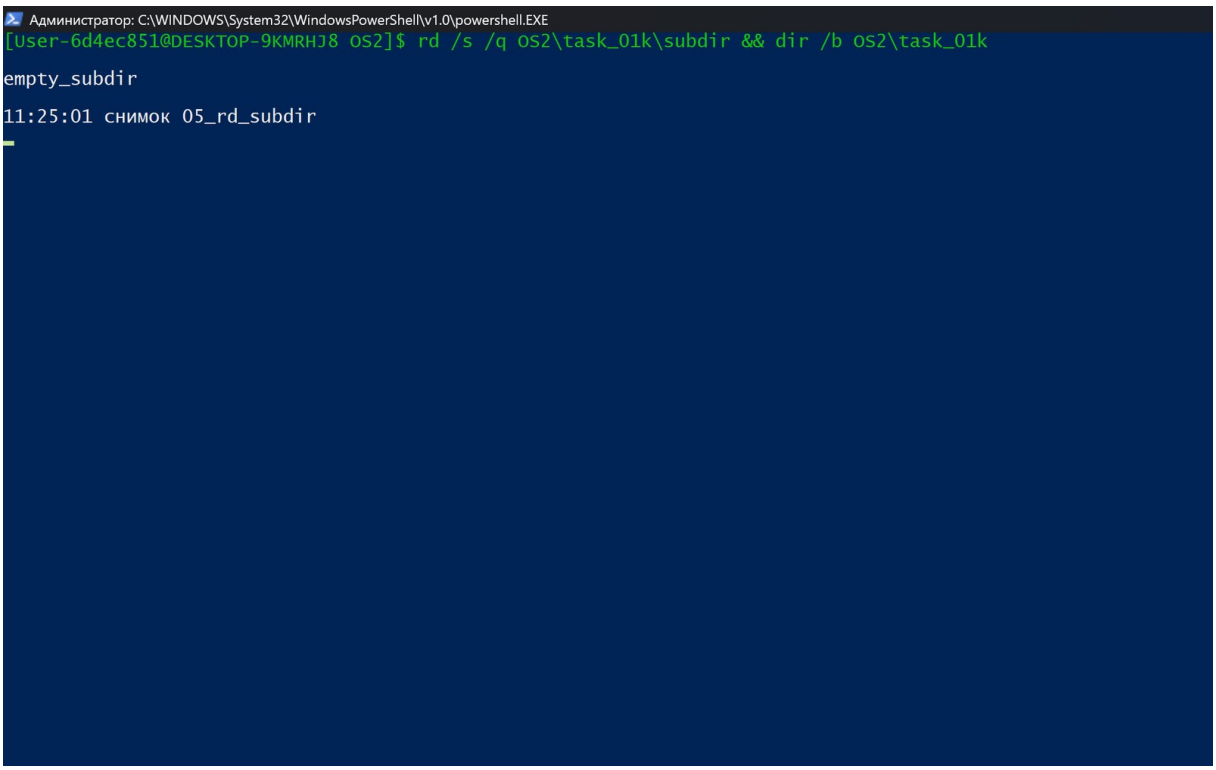


Рисунок 2.35 – Удаление непустого каталога *subdir* командой *rd /s /q*

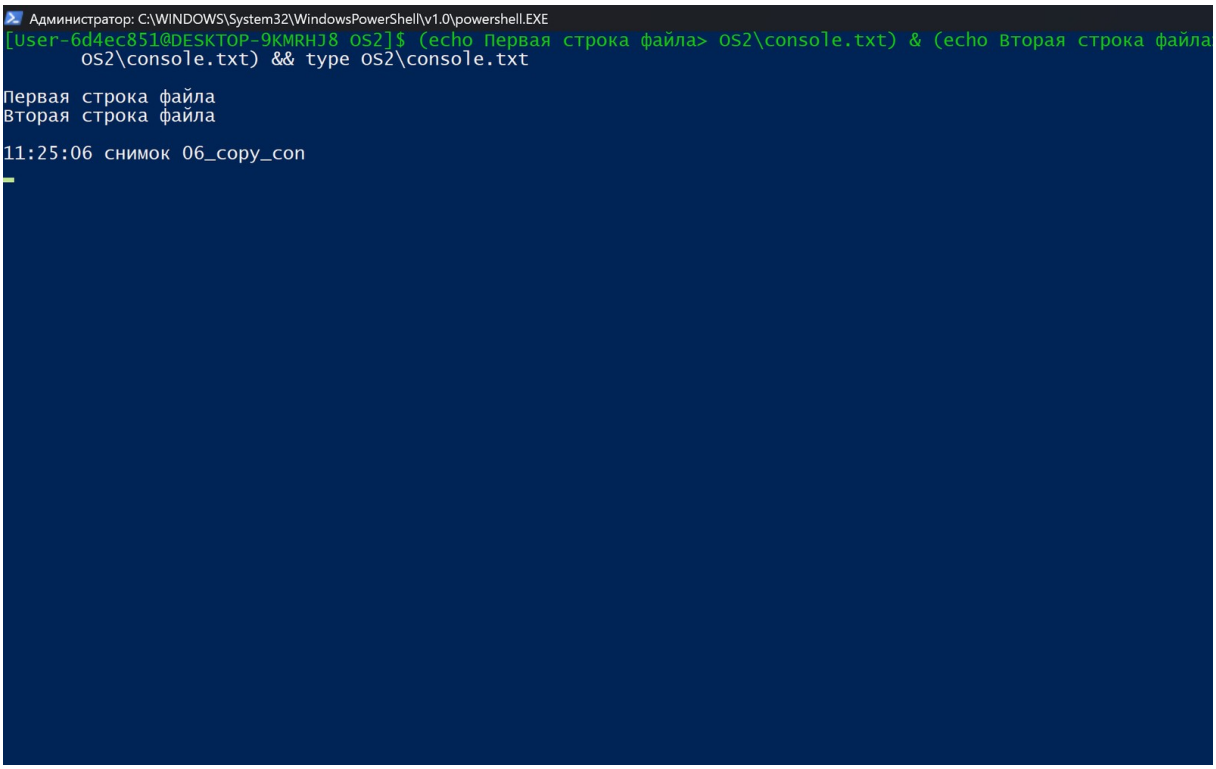


Рисунок 2.36 – Создание файла *console.txt* вводом данных с консоли

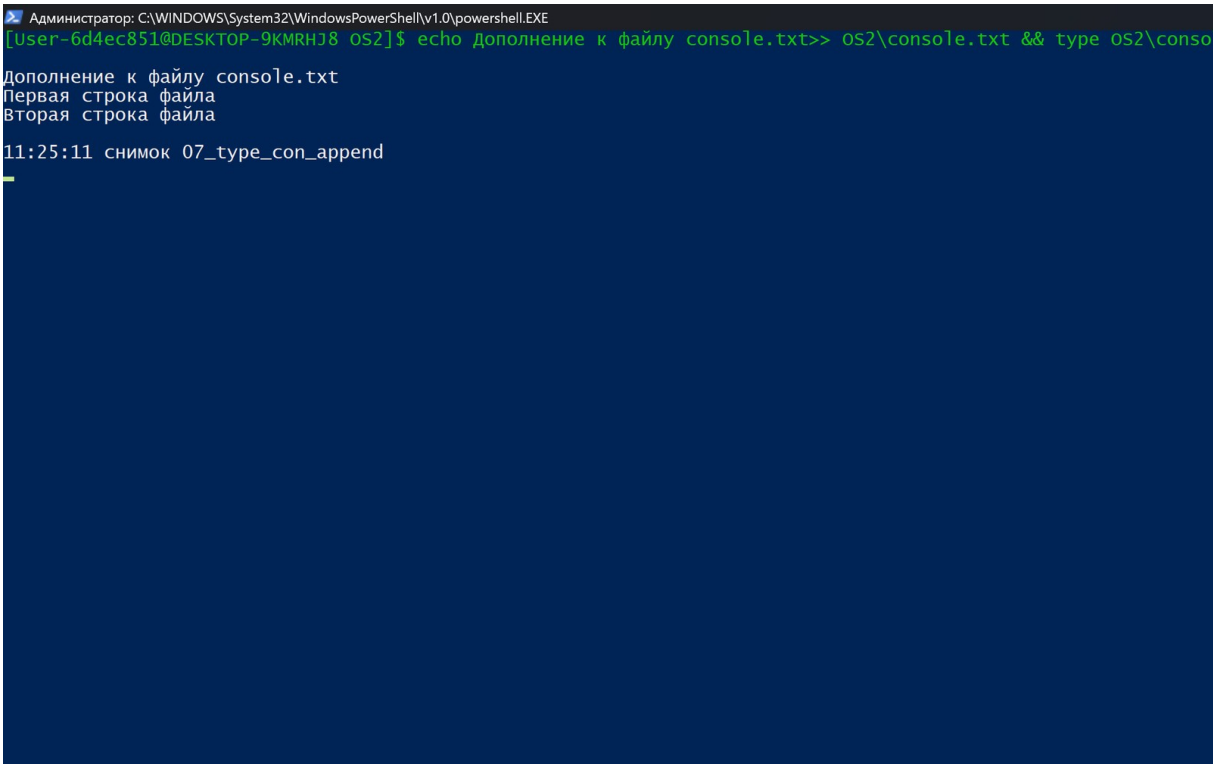


Рисунок 2.37 – Дополнение файла *console.txt* данными с консоли

Расширенные операции копирования и дополнения файлов отработали: содержимое *console.txt* соответствует ожидаемому.

2.6 Уровень 6–7: пакетные сценарии командной оболочки

На уровне 6–7 разработаны пакетные сценарии командной строки. Каждый сценарий решает отдельную задачу и демонстрирует работу с переменными окружения, параметрами командной строки и обработкой ошибочных ситуаций.

В листинге А.13 приведён сценарий *bat*_01.*bat*, который выводит значения системных переменных окружения *USERNAME*, *DATE*, *TIME*, *COMPUTERNAME* и *USERPROFILE*. Запуск выполнен командой *call C*:*lab*2_*sync**bat*_01.*bat* (рисунок 2.38). Ключевое слово *call* позволяет выполнить внешний пакетный файл и вернуть управление в исходный пакет; при использовании простого запуска по имени файла управление не возвращается.

В листинге А.14 приведён сценарий *bat*_02.*bat*, который получает сведения о самом себе с помощью модификаторов параметров: %~*n*0 возвращает имя файла без расширения, %~*f*0 – полный путь, %~*x*0 – расширение, а %~*t*0 – дату и время последнего изменения. Время модификации выводится через цикл *for* %%*F in* («%*SCRIPT*%») *do echo* Изменён: %%~*tF*, поскольку модификатор %~*t*0 не раскрывается в *delayed*-режиме напрямую. Результат работы сценария приведён на рисунке 2.39.

Исходный текст *bat*_03.*bat* приведён в листинге А.15. Командный файл выводит строку всех переданных параметров и значения первых четырёх позиций. При запуске с аргументами *alpha beta gamma delta epsilon* команда *shift* удаляет первый параметр перед выводом отдельных значений, поэтому в строке %~1 отображается *beta*. Результат показан на рисунке 2.40.

Для сценария *bat*_04.*bat* проверены обычный запуск и нулевой делитель. При *a*=10, *b*=4, *c*=25 получены разность 6, сумма 14, произведение 40, выражение (*a*-*b*)*(*b*-*a*) = -36 и частное *c*/*b* = 6 (рисунок 2.41). Исходный текст приведён в листинге А.16.

При нулевом *b* запуск *bat*_04.*bat* 10 0 25 вывел сообщение о невозможности деления на ноль и продолжил выполнение сценария (рисунок 2.42). Перед вычислением частного проверяется значение делителя; исходный текст приведён в листинге А.16.

Сценарий *bat*_05.*bat* создаёт файл в режиме *create* и удаляет его в режиме *delete*. Успешные запуски показаны на рисунках 2.43 и 2.44. Запуски без обязательных параметров и с неизвестным режимом завершаются сообщением об ошибке (рисунки 2.45 и 2.46). Исходный текст приведён в листинге А.17.

Обработка ошибочных ситуаций в *bat*_05.*bat* выполнена в двух случаях. При запуске сценария без параметров выводится сообщение об ошибке с указанием порядка использования (рисунок 2.45). При передаче неизвестного режима работы выводится сообщение о неизвестном режиме и перечислении допустимых режимов (рисунок 2.46). В обоих случаях сценарий завершается с ненулевым кодом возврата, что позволяет вызывающей стороне определить неуспешное выполнение.

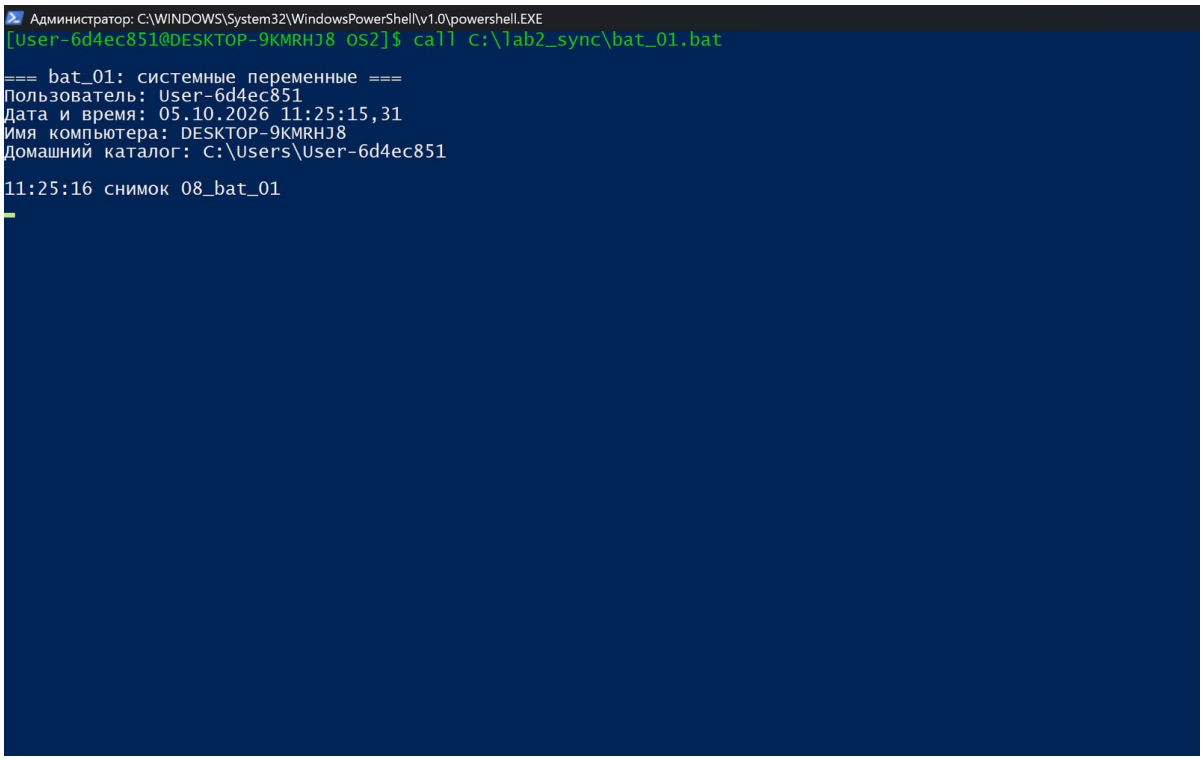


Рисунок 2.38 – Результат выполнения сценария *bat*_01.*bat*

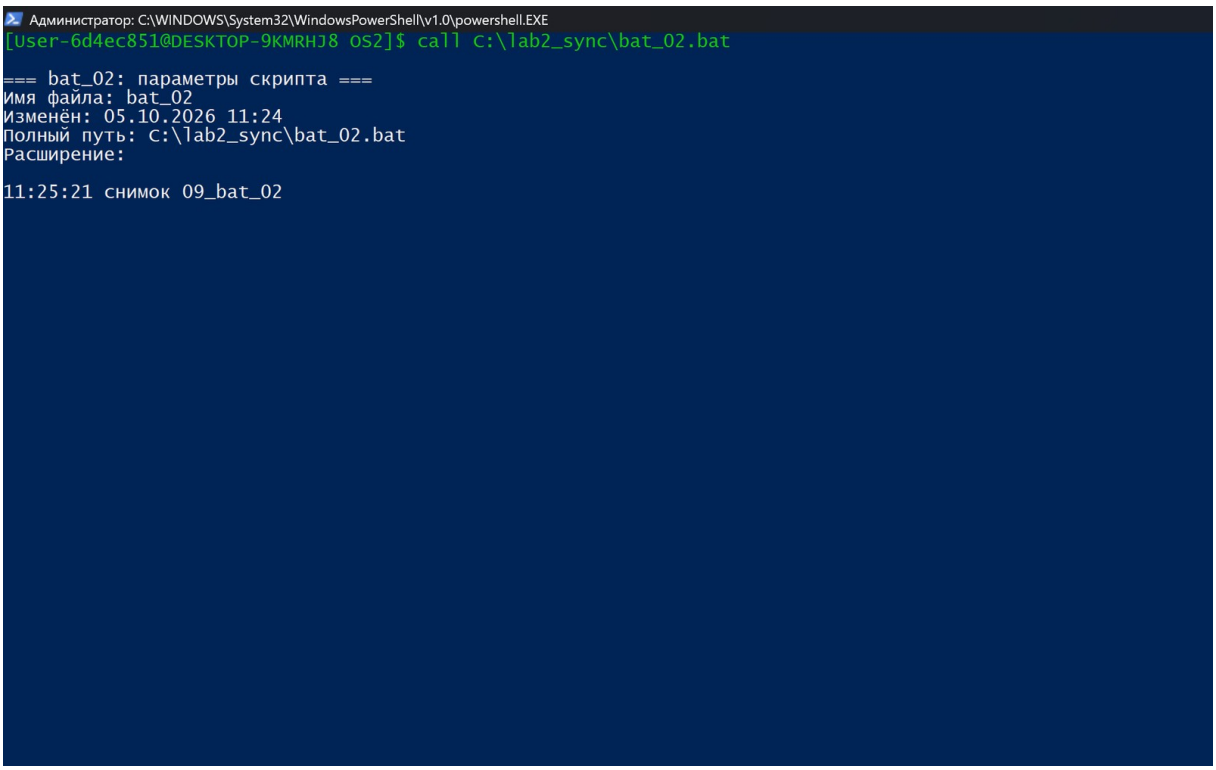


Рисунок 2.39 – Результат выполнения сценария *bat*_02.*bat*

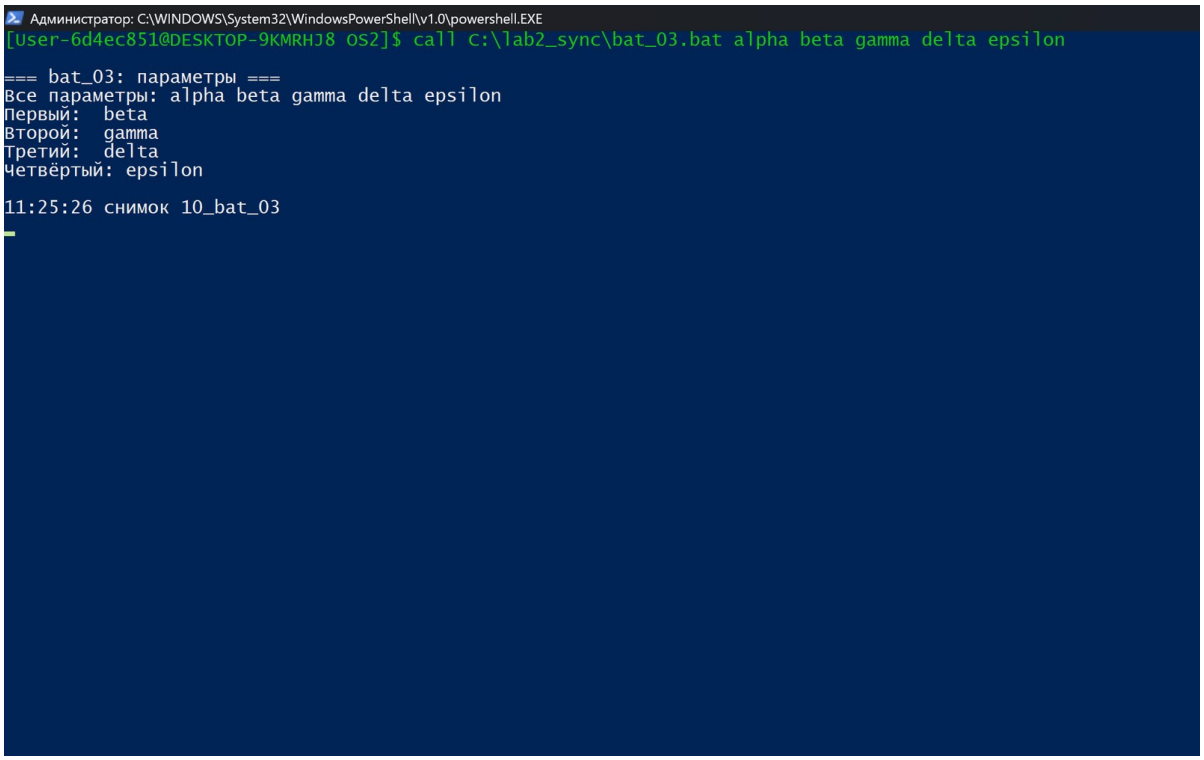


Рисунок 2.40 – Вывод параметров командной строки в сценарии *bat*_03.*bat*

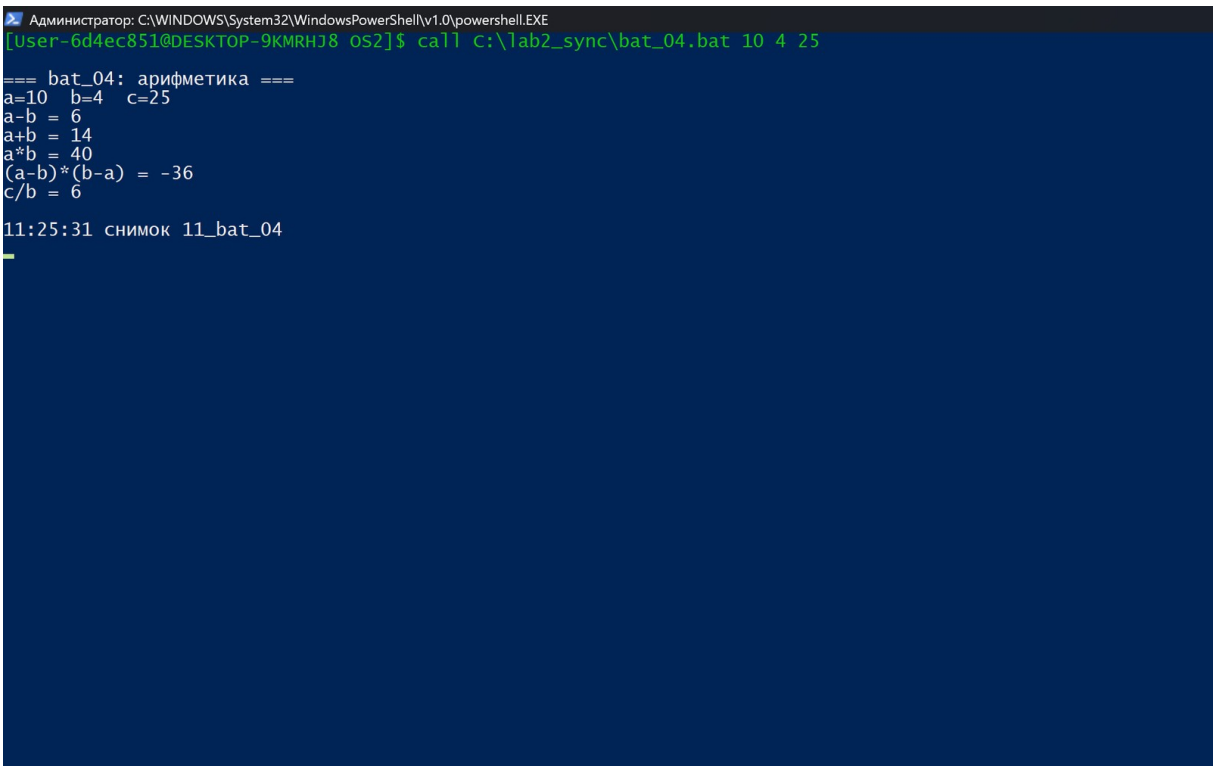


Рисунок 2.41 – Результаты арифметических вычислений сценария *bat*_04.*bat*

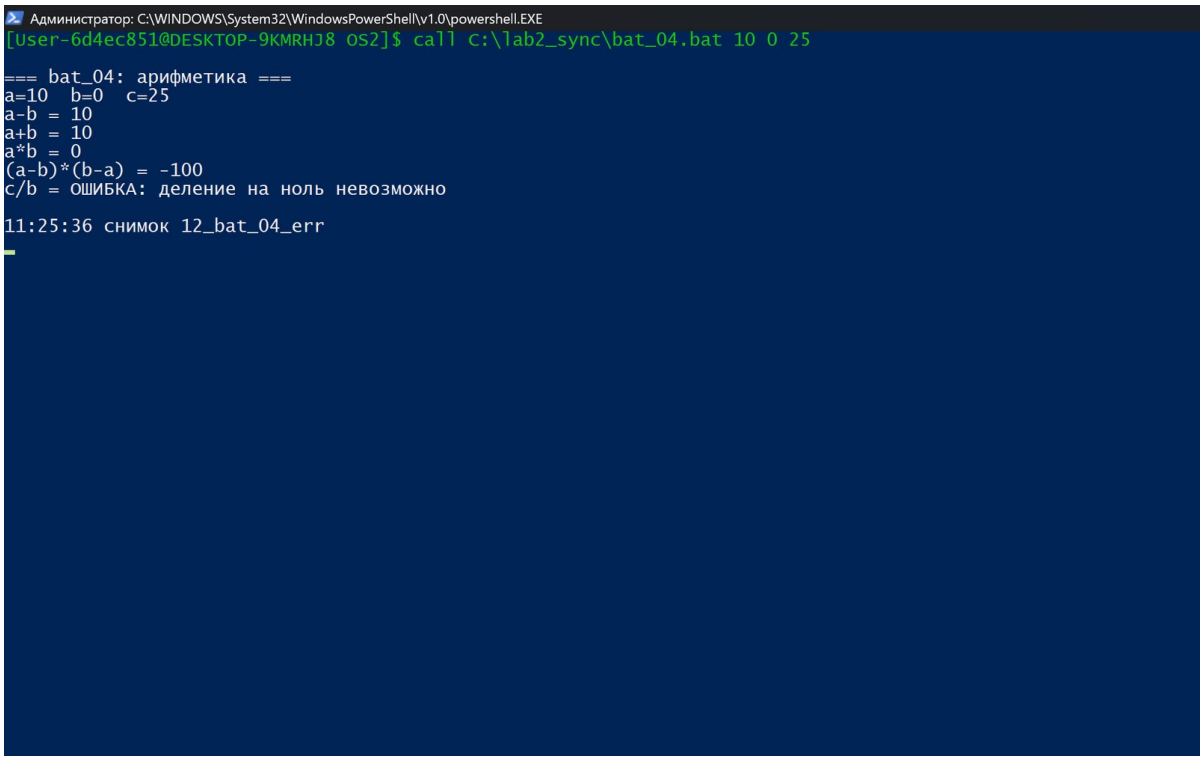


Рисунок 2.42 – Обработка деления на ноль в сценарии *bat*_04.*bat*

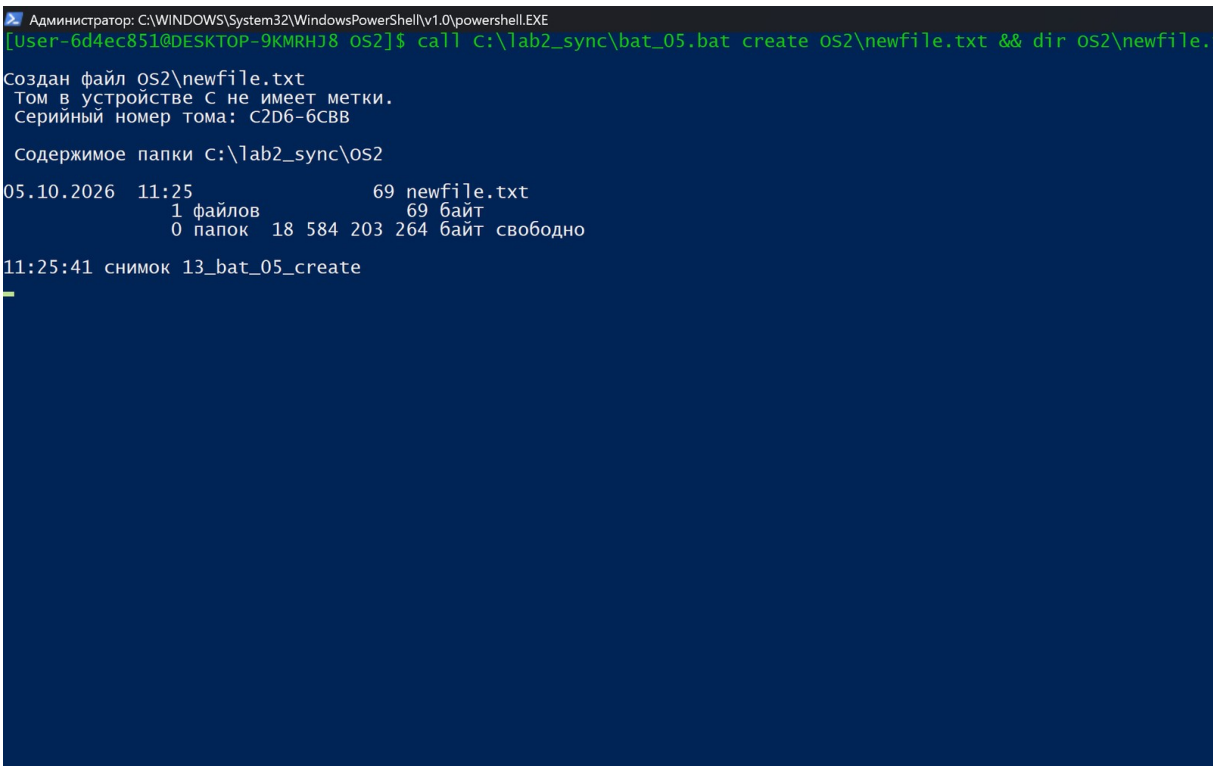


Рисунок 2.43 – Создание файла в режиме *create* сценария *bat*_05.*bat*

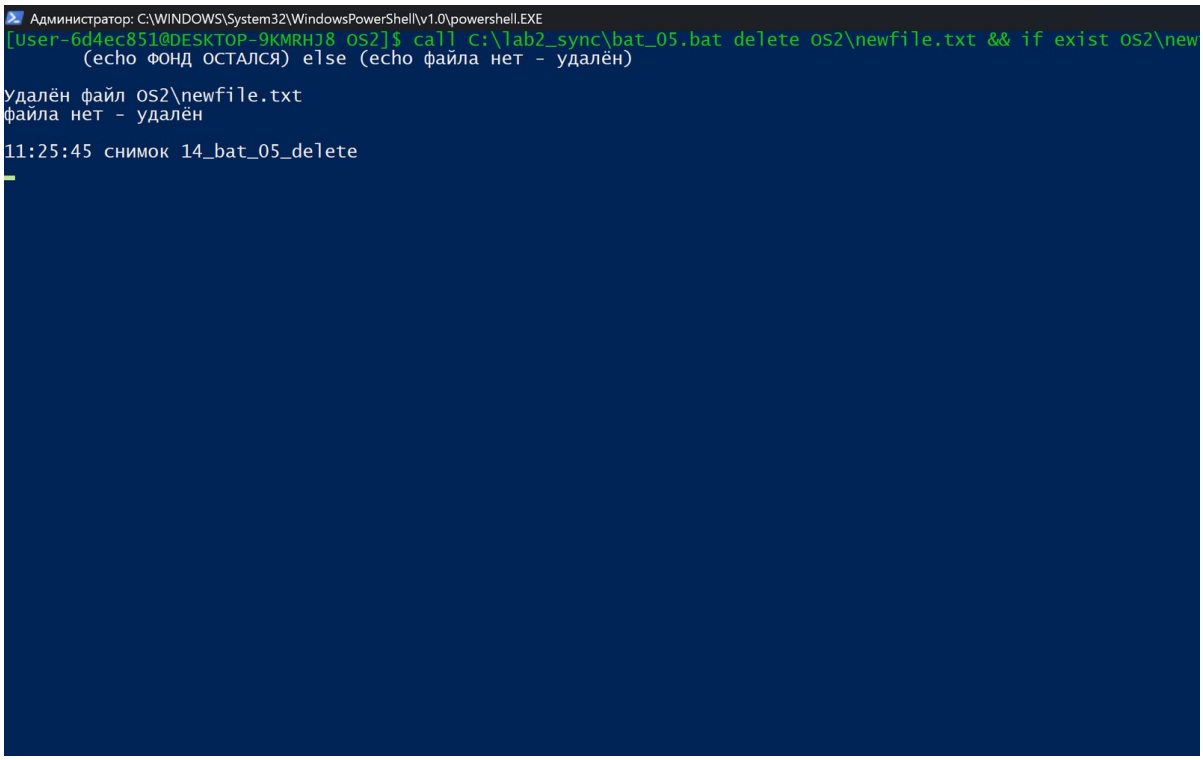


Рисунок 2.44 – Удаление файла в режиме *delete* сценария *bat*_05.*bat*

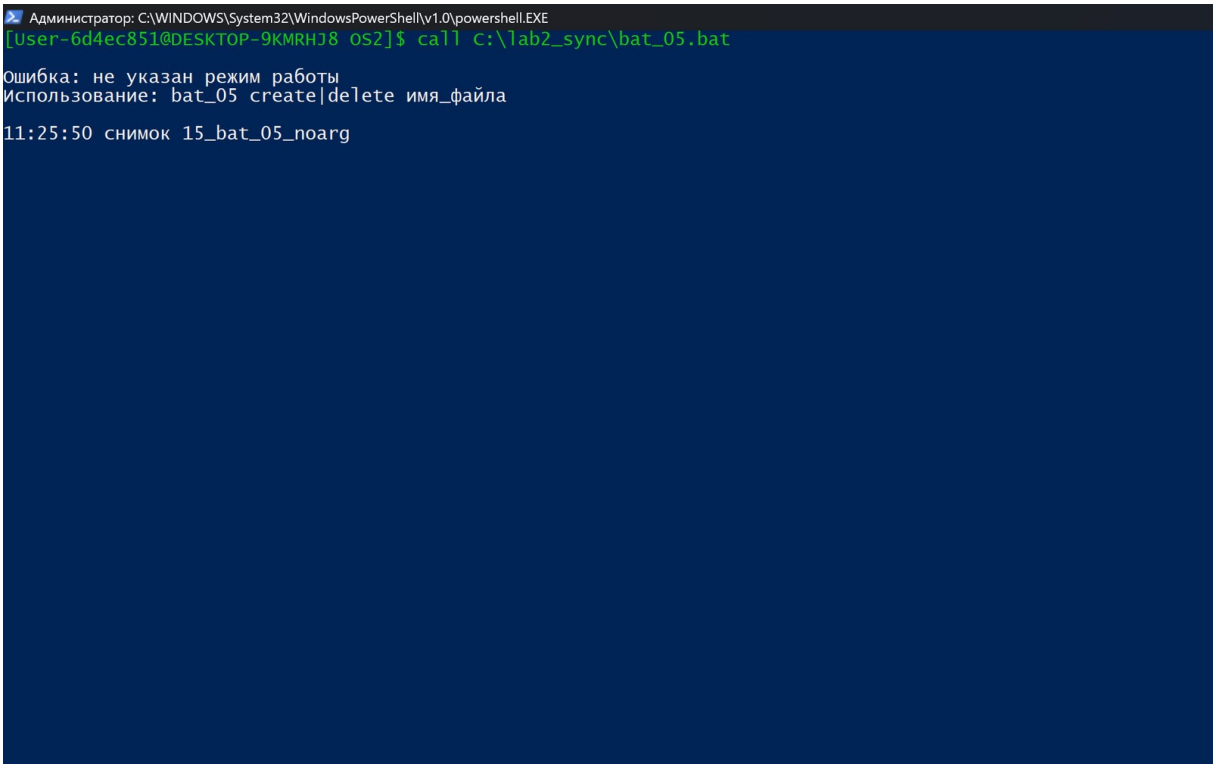


Рисунок 2.45 – Обработка запуска *bat*_05.*bat* без параметров

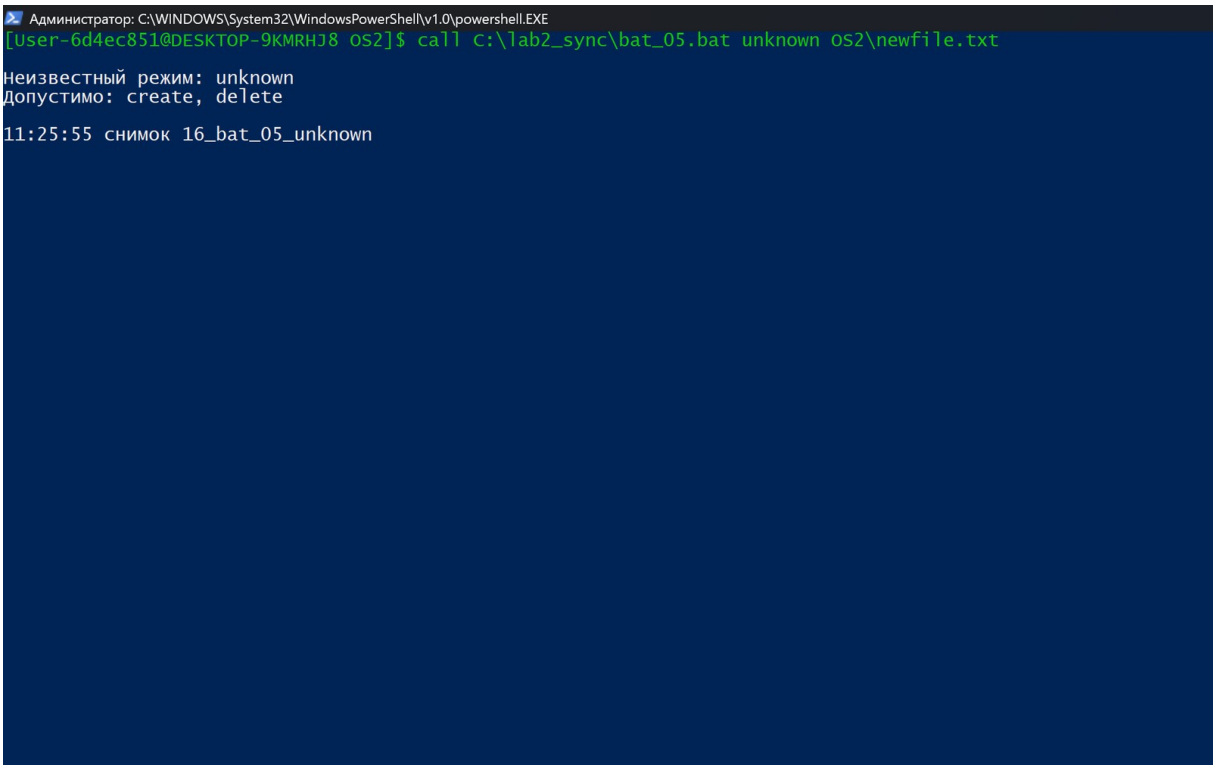


Рисунок 2.46 – Обработка неизвестного режима работы *bat*_05.*bat*

Сценарии *bat*_01–*bat*_05 отработали во всех режимах; некорректный ввод обрабатывается сообщением и кодом возврата.

2.7 Уровень 6–7: переменные окружения и *PATH*

Исследование переменных окружения выполнено командой *set*, которая выводит список переменных в формате ИМЯ=ЗНАЧЕНИЕ (рисунок 2.47).

Значения *USERNAME*, *USERPROFILE*, *TEMP*, *PROCESSOR_ARCHITECTURE* и *COMPUTERNAME* проверены командой *echo* %ИМЯ% (рисунок 2.48).

Проверены значения всех пяти переменных; снимок их вывода приведён на рисунке 2.48.

Переменная *PATH* состоит из каталогов, в которых *Windows* ищет команды, и разделяет их точкой с запятой. Количество каталогов подсчитано *PowerShell*; в данном сеансе получено 10 элементов (рисунок 2.49).

Для анализа *PATH* создан сценарий *path_count.ps*1; его исходный код приведён в листинге А.20. Для каждого каталога из переменной *PATH* он выводит количества файлов с исполняемыми расширениями и без них, а также их долю; отсутствующие каталоги отмечаются отдельно (рисунок 2.50).

Анализ *PATH* показал, что *C*:*Program Files**Git**cmd* и каталог приложений *Windows* содержат только файлы с указанными исполняемыми расширениями. В *C*:*WINDOWS**system*32 найдено 3778 файлов других расширений; в каталоге *.dotnet**tools*, указанном в *PATH*, файловая система каталога не содержит.

Сценарий повторно запущен на уровне 8 в составе заданий по разработке сценариев *PowerShell*, где его вывод приведён как исходные данные для анализа (рисунок 2.51).

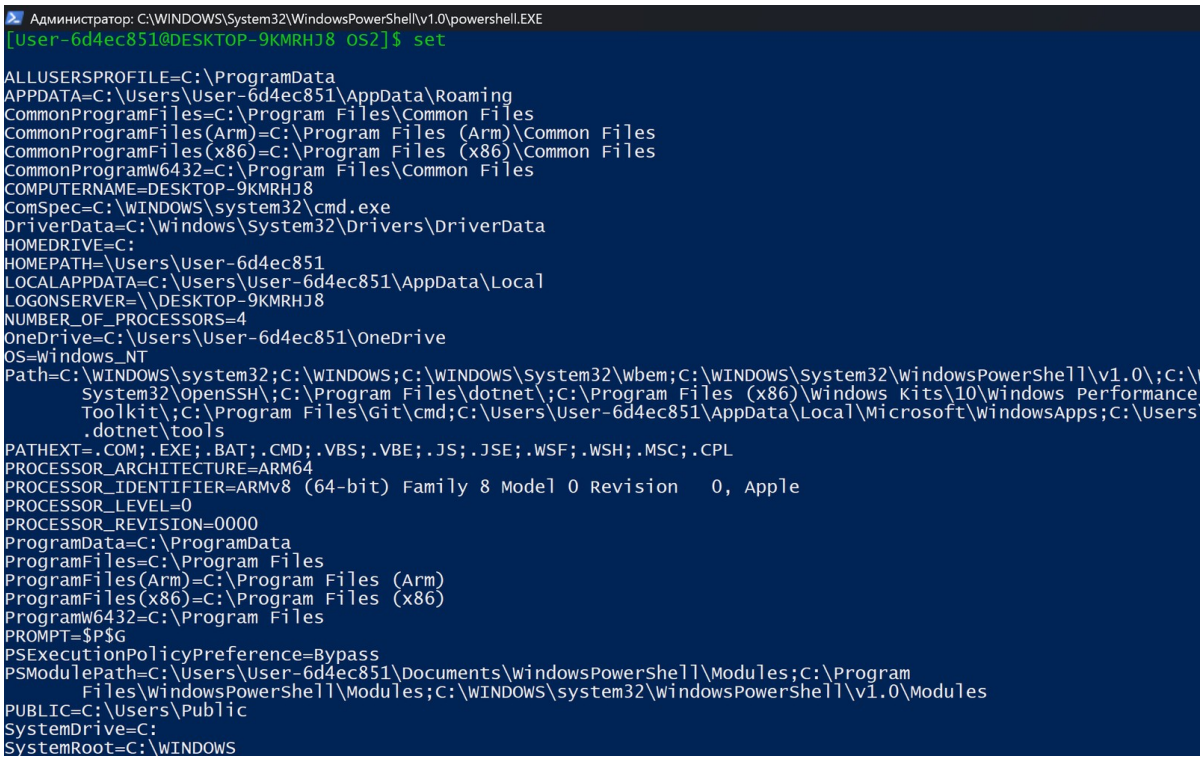


Рисунок 2.47 – Полный список переменных окружения, используемых в заданиях

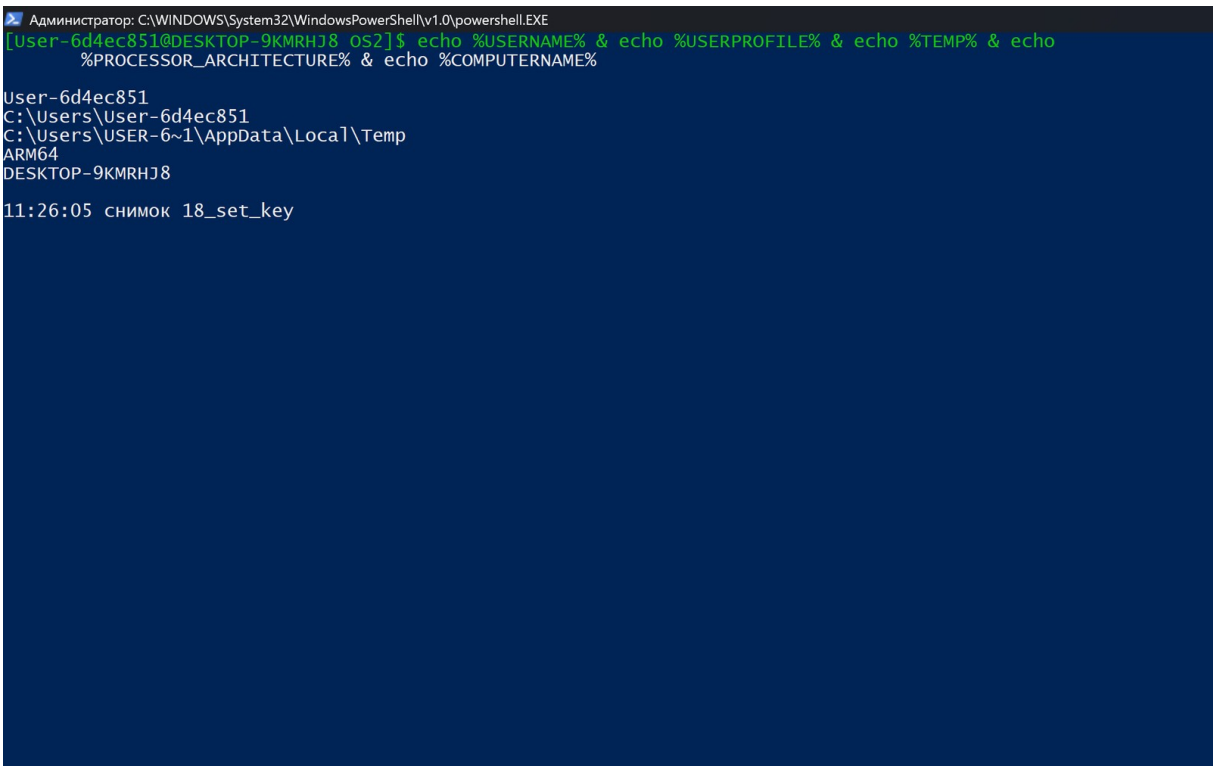


Рисунок 2.48 – Значения ключевых переменных окружения *USERNAME*, *USERPROFILE*, *TEMP*

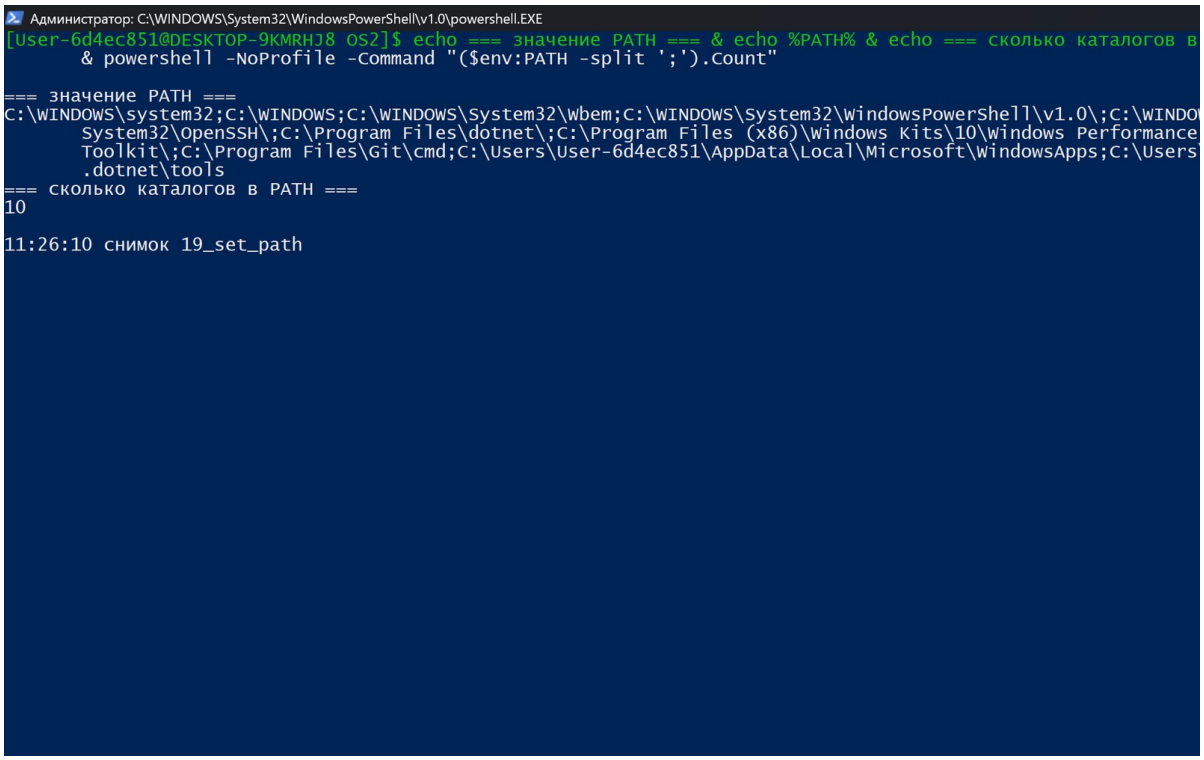


Рисунок 2.49 – Анализ значения переменной *PATH* и подсчёт числа каталогов в ней

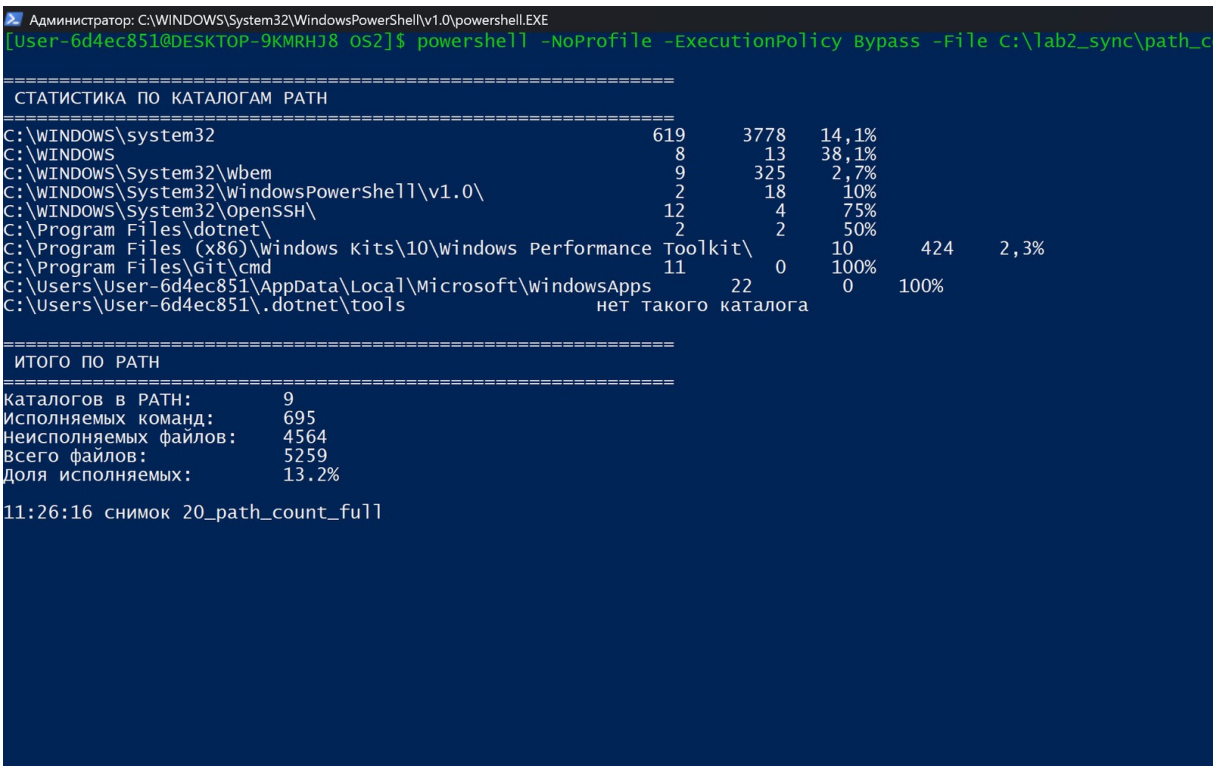


Рисунок 2.50 – Статистика исполняемых и неисполняемых файлов по каталогам *PATH*

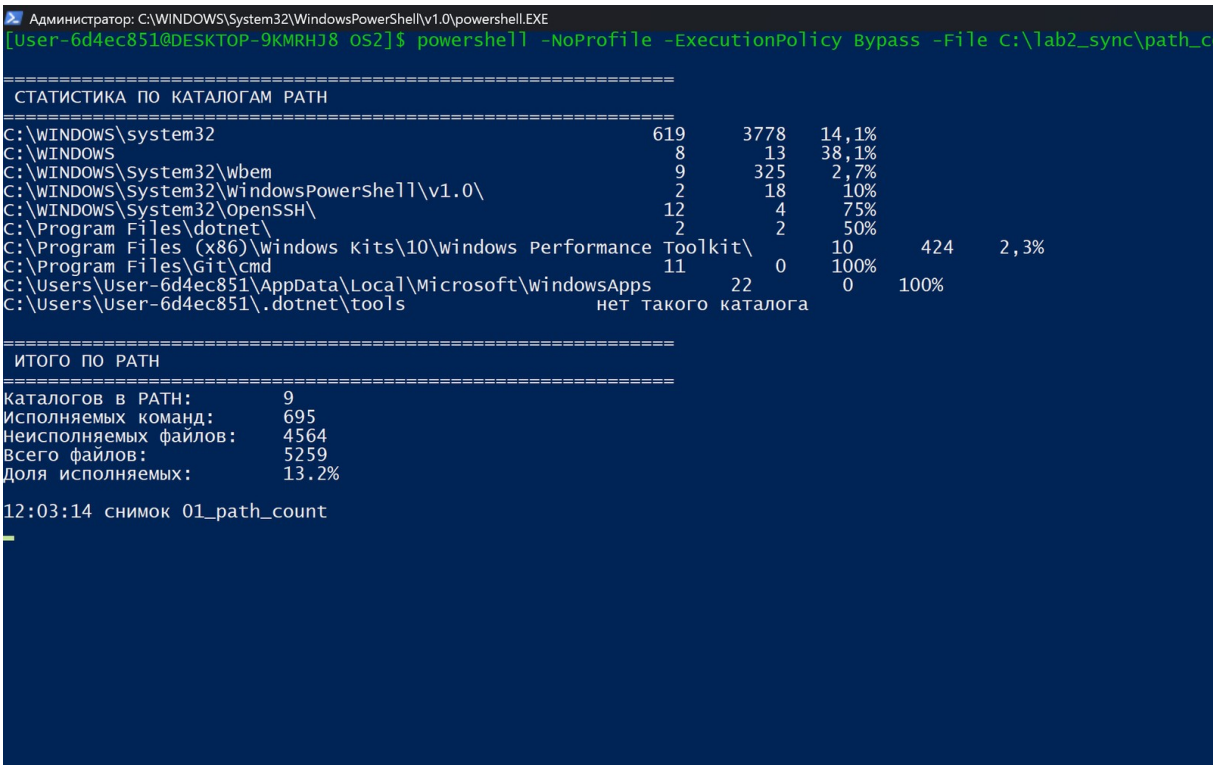


Рисунок 2.51 – Итоговая статистика по каталогам переменной *PATH*

Исследование в *Windows* завершено: статистика по каталогам *PATH* получена и сопоставима с результатами *Linux*.

2.8 Уровень 8: дерево каталогов с размерами файлов

Задания уровня 8–9 выполнены с помощью сценариев на языке *PowerShell*. Для их запуска используется команда *powershell -NoProfile -ExecutionPolicy Bypass -File* с указанием пути к файлу сценария.

Исходный код сценария *tree_sizes.ps*1 приведён в листинге А.21. Для каждой папки выводится количество вложенных файлов, для каждого файла – размер в байтах.

Размер выводится одновременно в десятичном и двоичном формате: в первом используются килобайты, мегабайты и гигабайты (1 *KB* = 1000 байт), во втором – кибибайты, мебибайты и гибибайты (1 *KiB* = 1024 байта). Вывод размера каталога выполняется функцией *Format-Decimal*, двоичного формата – функцией *Format-Binary*; обе используют множители 1*KB*, 1*MB* и 1*GB*, которые в *PowerShell* уже определены как 1024, 1048576 и 1073741824 байта соответственно, поэтому названия форматов различаются только суффиксом.

Запуск для каталога *OS*2*task*_01*b* показывает его полное содержимое, включая пустой каталог *subdir\empty_subdir* и вложенный каталог *hidden_dir\system*1 (рисунок 2.52). Все файлы каталога имеют размер от 13 до 22 байт, поэтому одинаково выводятся в байтах.

Запуск для всего каталога *OS*2 показывает дерево из 16 каталогов и всех файлов (рисунок 2.53). Видно, что размер файла *systeminfo.txt*, равный 3482 байтам, выводится как 3,40 *KB* и 3,40 *KiB* – значения почти совпадают, поскольку файл невелик. Для более крупных файлов расхождение между десятичным и двоичным форматами становится существенным.

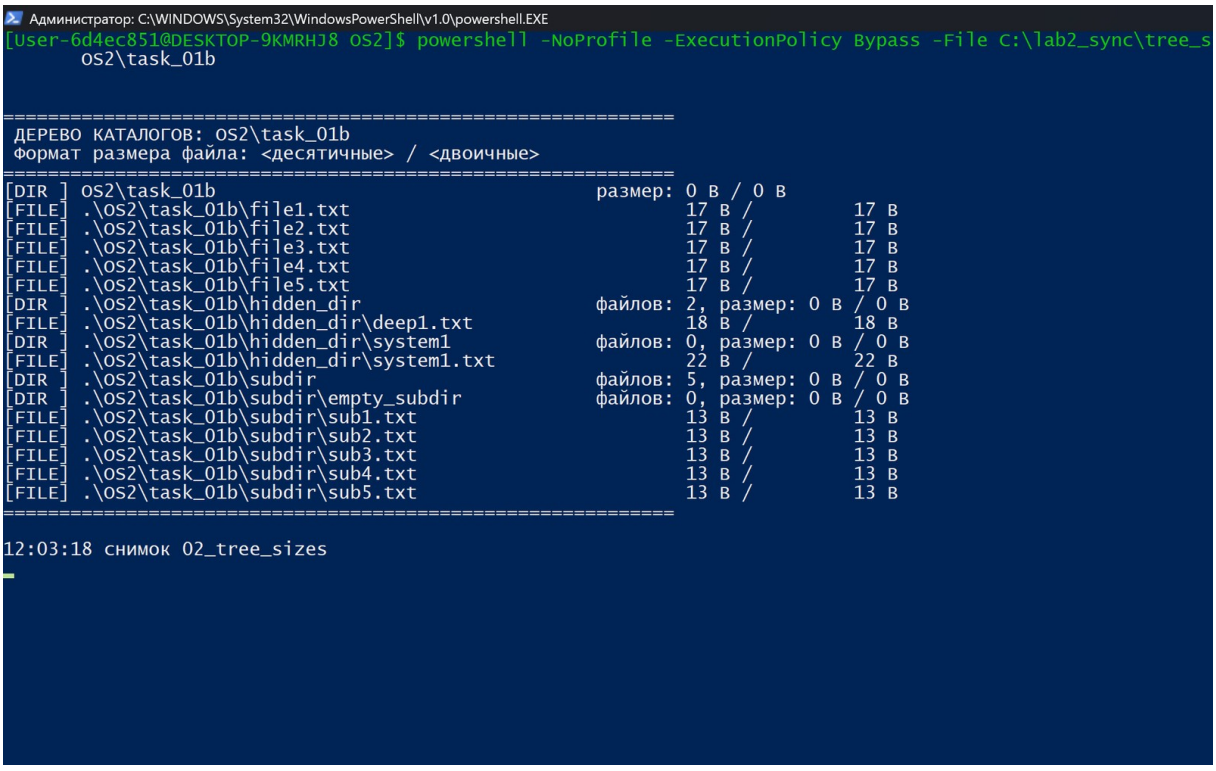


Рисунок 2.52 – Дерево каталогов *task*_01*b* с размерами файлов

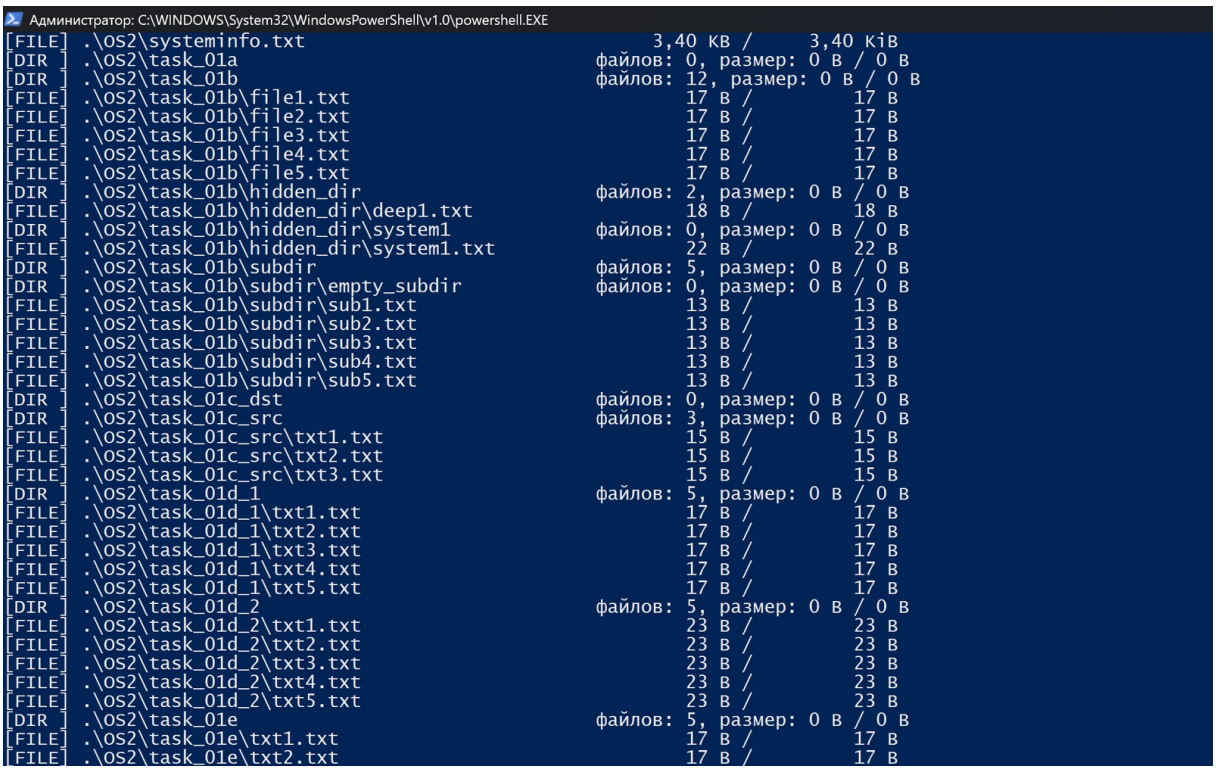


Рисунок 2.53 – Дерево каталогов *OS*2 с размерами файлов в десятичном и двоичном формате

Сценарий *tree_sizes.ps*1 из листинга А.21 запущен для каталога *OS*2. В выводе представлены структура папок, число вложенных файлов и размеры в десятичных и двоичных единицах; снимки для подкаталога *task*_01*b* и всего *OS*2 приведены ниже.

2.9 Уровень 8: интерактивный калькулятор на языке *PowerShell*

В листинге А.18 приведён интерактивный калькулятор *calc.ps*1, поддерживающий операции сложения, вычитания, умножения, деления, остатка от деления, возведения в степень, извлечение корня *n*-й степени, квадратного корня, тригонометрических функций (в градусах), а также десятичного и натурального логарифмов.

Выражение разбирается рекурсивным спуском по трём уровням с соблюдением приоритета операций: сначала возведение в степень (функция *Eval-Power*, правоассоциативная), затем умножение, деление и остаток (функция *Eval-Product*) и только затем сложение и вычитание (функция *Eval-Sum*, левоассоциативная). Единый общий шаблон разбора на всё выражение не использовался: он не разбирал выражение вида 2+3*4 и путал номер группы при извлечении оператора из регулярного выражения, из-за чего выражение 10/0 завершалось ошибкой преобразования типа.

Справка по поддерживаемым операциям и ключам запуска выводится ключом *-Help* (рисунок 2.54). Без этого ключа запуск сценария без аргументов уходит в интерактивный режим и блокирует выполнение очереди команд.

Сценарий поддерживает три способа передачи выражений: интерактивный ввод с клавиатуры, аргументы командной строки и чтение из файла. Ключ *-FileIn*

указывает файл с выражениями, по одному в строке, и нужен для автоматического прогона: команда *Read-Host* игнорирует перенаправленный стандартный ввод и блокирует выполнение. Файл читается напрямую как байты с последующим декодированием кодовой страницей консоли, поскольку команда *Get-Content* без ключа *-Encoding* использует *ANSI*-кодировку и портит кириллический текст.

Калькулятор проверен на выражениях с приоритетом операций и функциями. Для 2+3*4 получено 14, а не 20; результаты остальных выражений и сообщений об ошибках приведены на рисунке 2.55.

Проверка приоритета операций выполнена и при передаче выражений аргументами командной строки: получены 2+3*4 = 14, 10-4/2 = 8 и 2^10 = 1024 (рисунок 2.56). Все три результата верны, что подтверждает корректность разбора с учётом приоритетов.

Проверка функций выполнена на выражениях *sqrt* 16 = 4, sin 30 = 0.5, log10 100 = 2 и ln 10 = 2.302585093 (рисунок 2.57). Для функций требуется склейка имени функции и её операнда в единое выражение, поскольку в командной строке они приходят двумя отдельными аргументами. Точность вывода обеспечивается использованием инвариантной культуры при форматировании: десятичный разделитель в выводе всегда точка, поскольку запятая сломала бы последующий разбор выражений.

Обработка ошибок проверена на двух ошибочных вводах. Для выражения 10/0 сценарий выводит «ОШИБКА: деление на ноль невозможно (10/0)», а для выражения *abc* – «ОШИБКА: не удалось разобрать выражение: *abc*» (рисунок 2.58). Ошибка передаётся через скопотную переменную $*script*:*Err*, а не через конструкцию *throw*: инструкция *throw* прервала бы выполнение всего пакета, и следующие выражения не были бы вычислены.

Таким образом, калькулятор корректно обрабатывает как математические ошибки (деление на ноль, остаток от деления на ноль), так и ошибки ввода (некорректные числа, неизвестные функции), не прерывая работу.

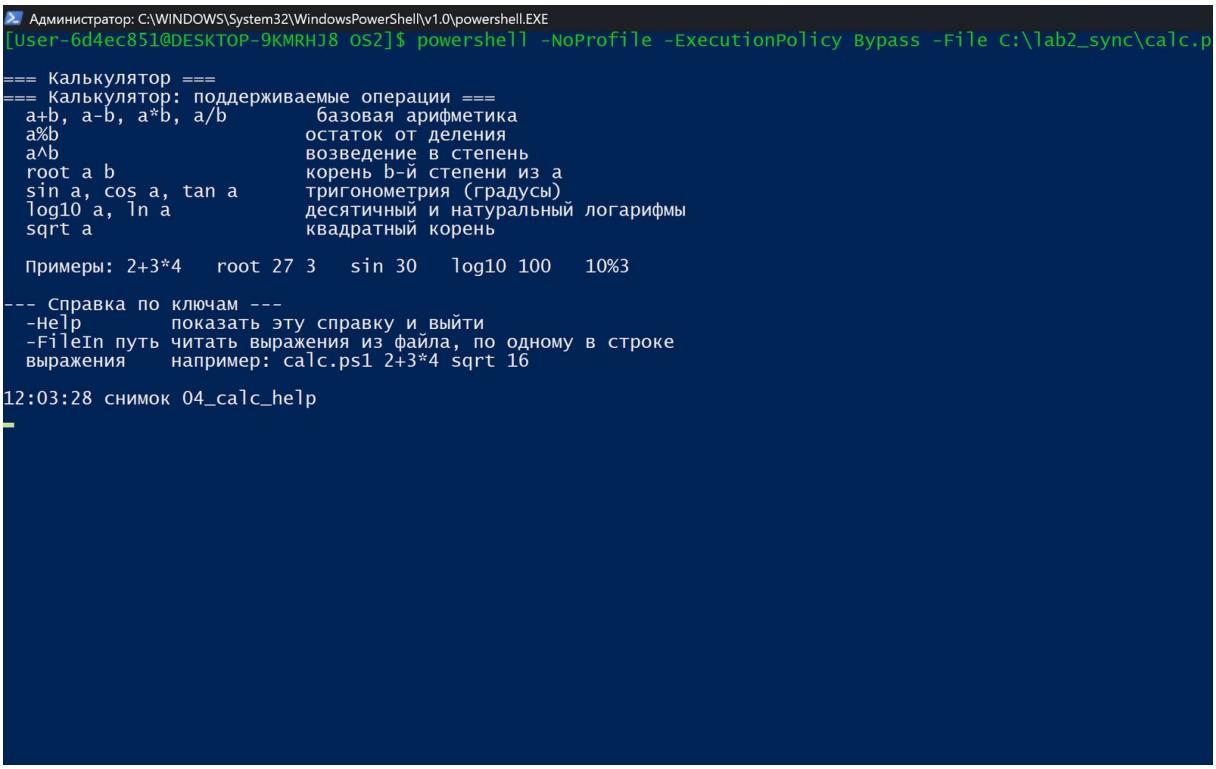


Рисунок 2.54 – Справка калькулятора по поддерживаемым операциям и ключам запуска

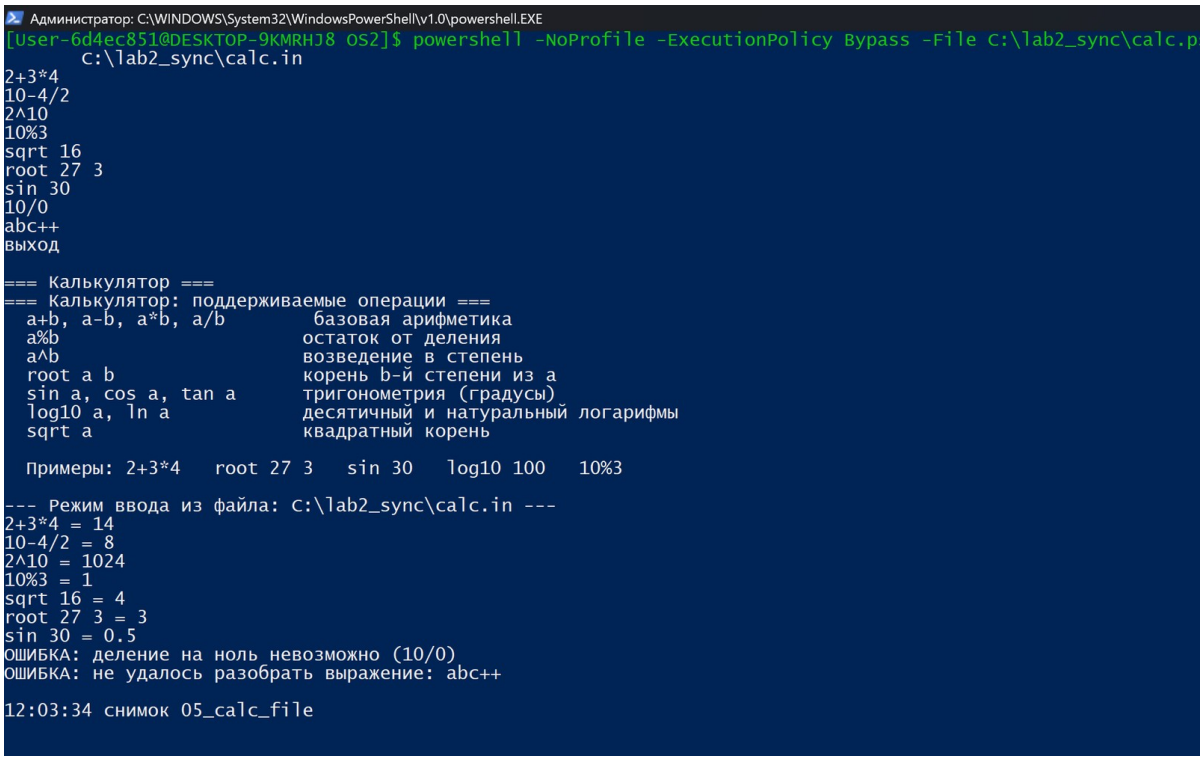


Рисунок 2.55 – Вычисление выражений калькулятором, введёнными из файла

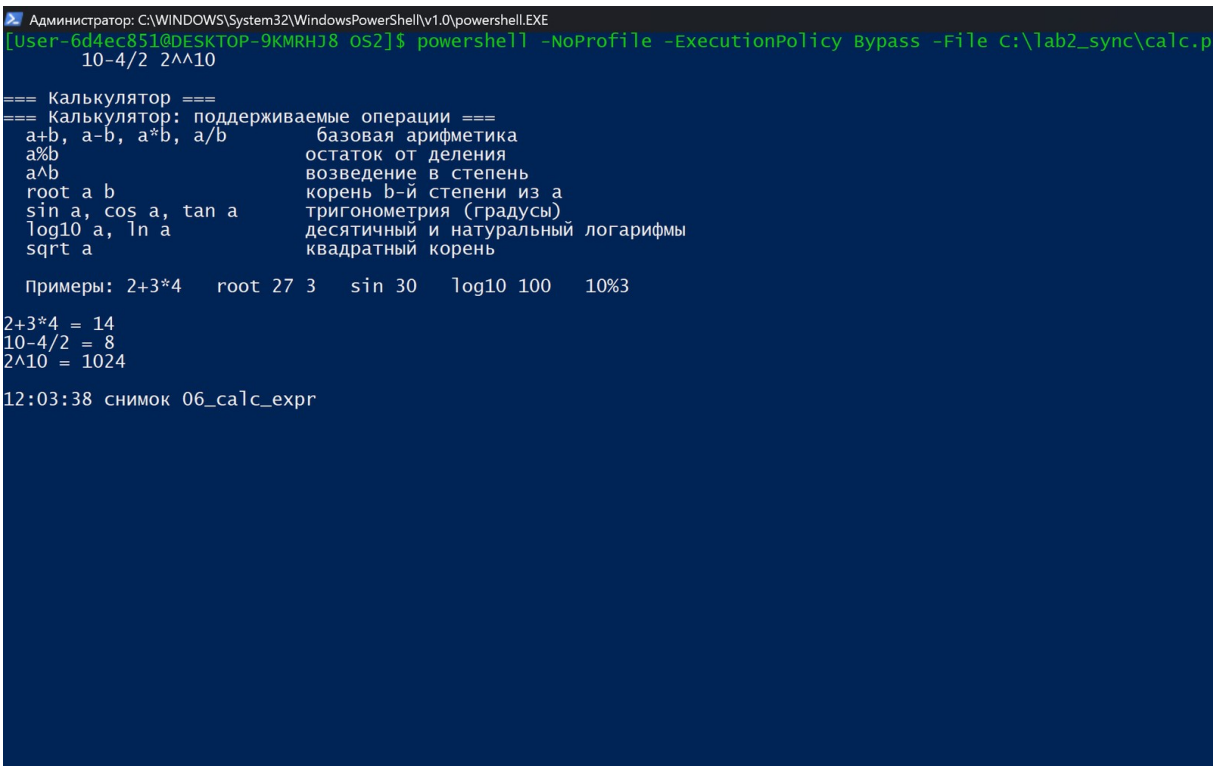


Рисунок 2.56 – Вычисление выражений, переданных калькулятору аргументами командной строки

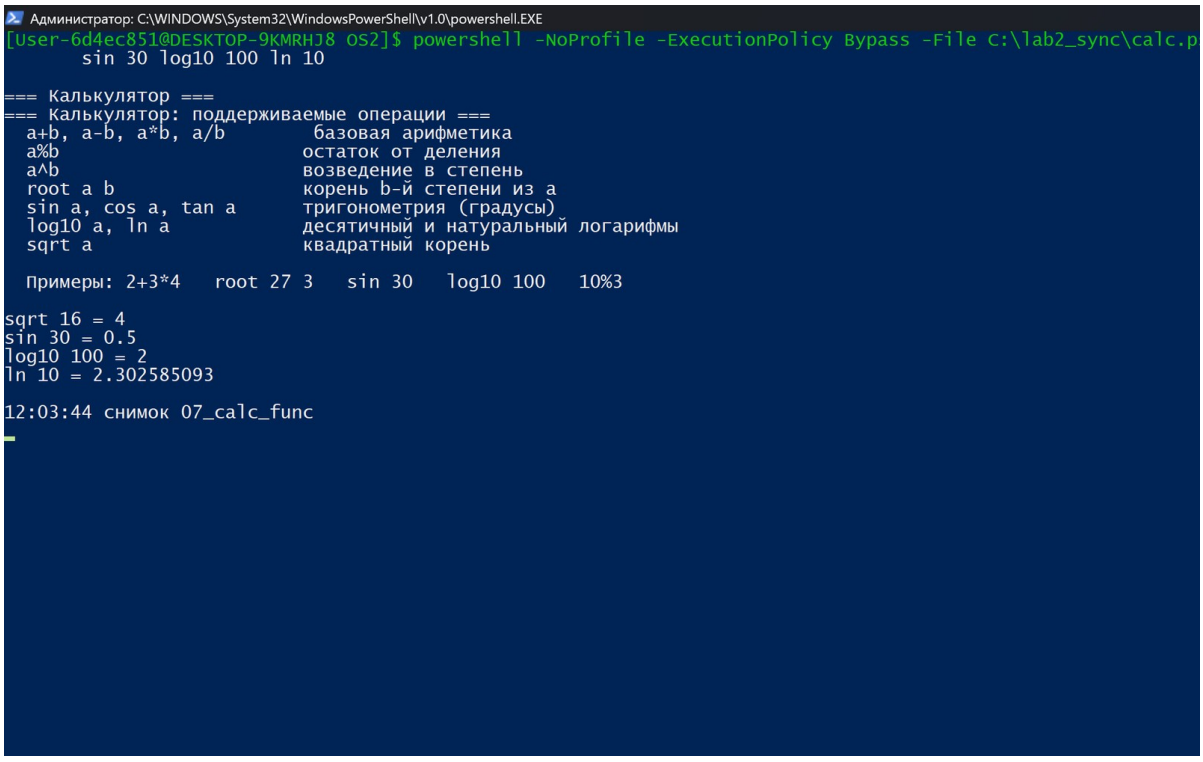


Рисунок 2.57 – Вычисление функций *sqrt*, sin, log10 и ln

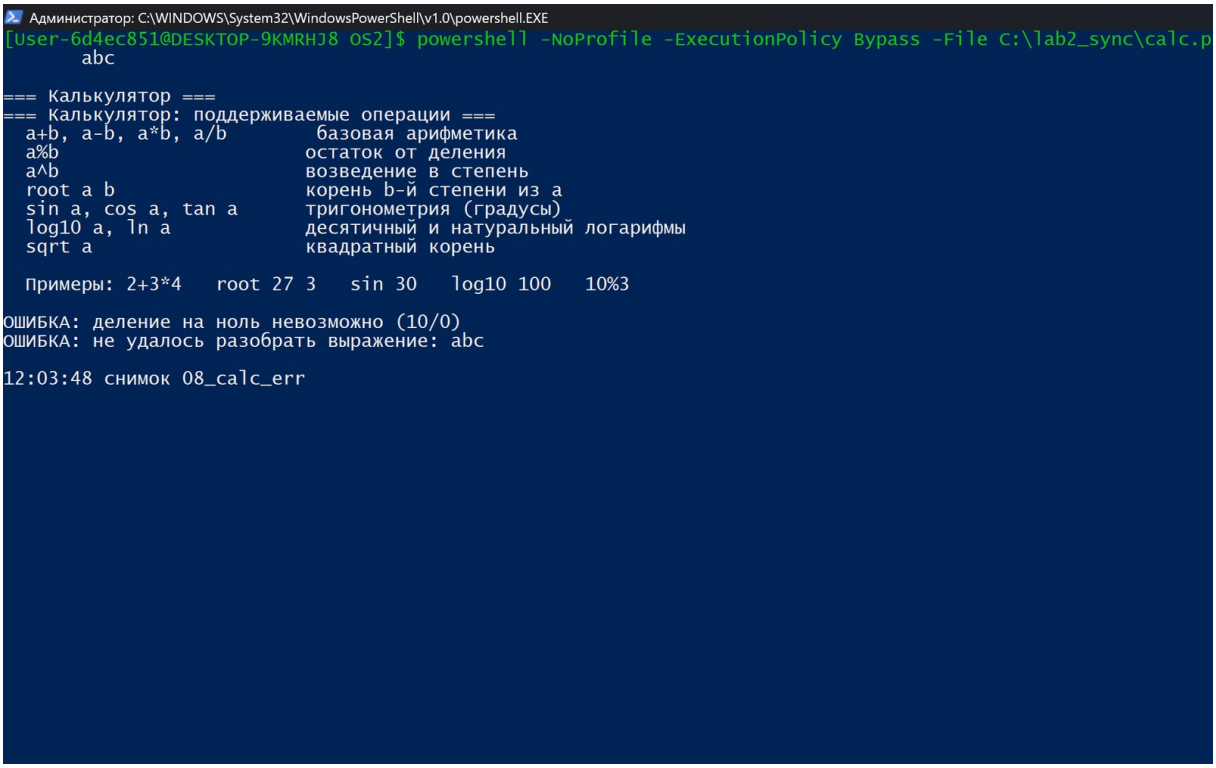


Рисунок 2.58 – Обработка ошибок калькулятора при делении на ноль и неверном вводе

Калькулятор вычисляет выражения и корректно обрабатывает деление на ноль и неверный ввод.

2.10 Уровень 8: конвертер температур на языке *PowerShell*

В листинге А.19 приведён конвертер температур *tempconv.ps*1, выполняющий перевод значений между кельвинами (*K*), градусами Цельсия (*C*) и градусами Фаренгейта (*F*). Исходная единица измерения определяется по суффиксу, записываемому сразу после числа: 100*K*, 100*C* или 100*F*.

Перевод выполняется через промежуточное представление в кельвинах. Функция *To-Kelvin* приводит исходное значение к кельвинам по формулам *C* + 273.17 для градусов Цельсия и (*F* - 32) * 5 / 9 + 273.17 для градусов Фаренгейта, а функция *From-Kelvin* выполняет обратное преобразование. Такой подход гарантирует одинаковую точность для всех направлений перевода и позволяет выводить все три шкалы сразу.

Для значений 100*K*, 100*C*, 100*F* и 0*C* получены соответствующие температуры в трёх шкалах; примеры показаны на рисунке 2.59. Значения соответствуют формулам преобразования температур.

При запуске с аргументами командной строки получены те же результаты для проверочных температур (рисунок 2.60). Параметр -*FileIn* разбирается из массива $*args*, поскольку объявленный позиционный параметр перехватывал первое значение температуры как путь к файлу.

При вводе 100*X* сценарий вывел сообщение о неверном формате и перечислил допустимые суффиксы *K*, *C* или *F* (рисунок 2.61).

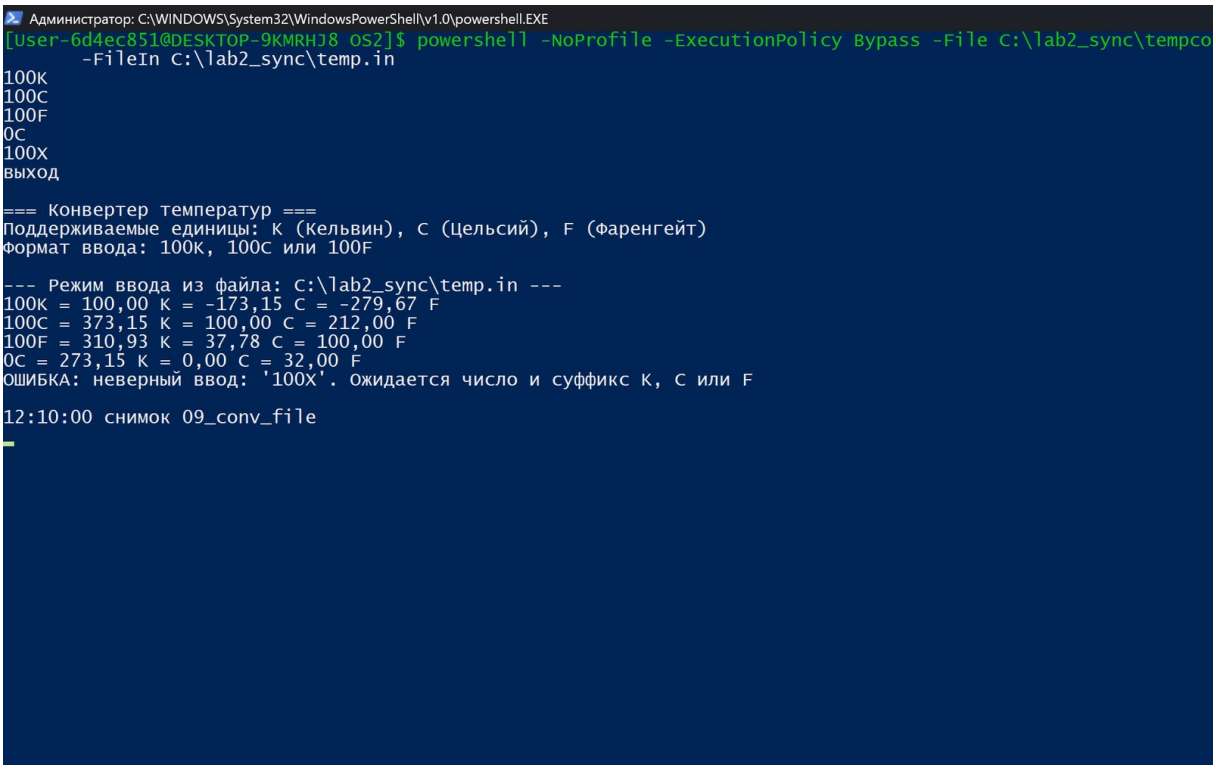


Рисунок 2.59 – Конвертация температур для значений, введённых из файла

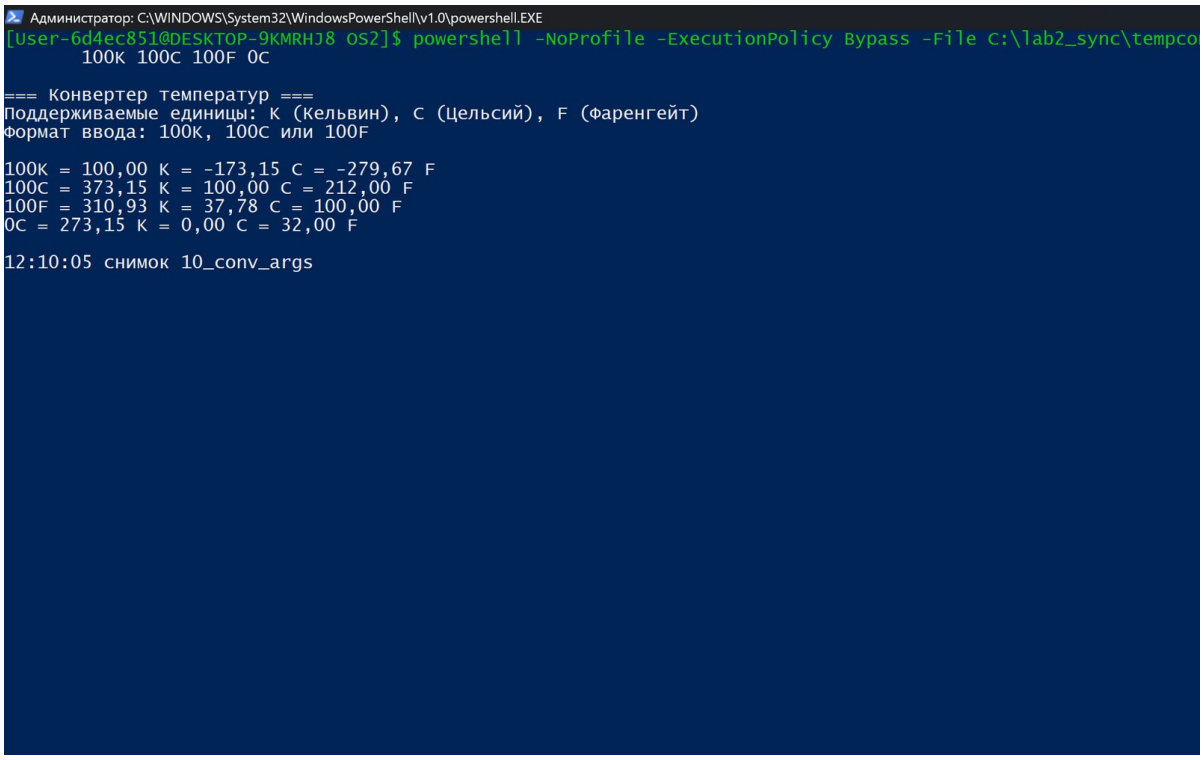


Рисунок 2.60 – Конвертация температур для аргументов командной строки

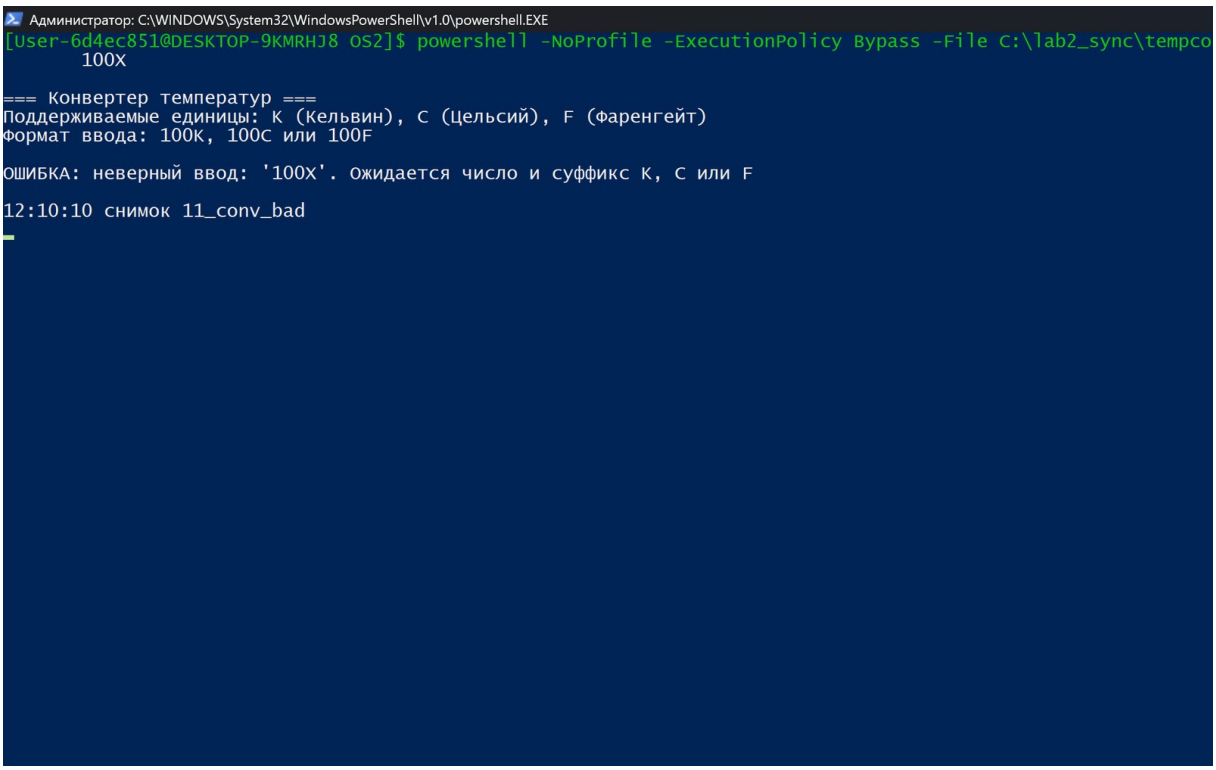


Рисунок 2.61 – Обработка неверного ввода конвертера температур

Конвертер выполняет переводы между шкалами; неверный ввод отклоняется с диагностикой.

2.11 Уровень 9: монитор процессов в реальном времени

Параметры сценария *top*5.*ps*1 – *Interval* и *Count*; исходный текст приведён в листинге А.22. Сценарий обновляет две таблицы: первые пять процессов по суммарному времени *CPU* и первые пять по объёму рабочей памяти.

Значения *CPU* и *WorkingSet*64 из *Get-Process* выбраны как показатели для сравнения процессов; *CPU* означает накопленное время работы процессора, а *WorkingSet*64 – объём памяти процесса в байтах.

Объём памяти выводится с помощью функции *Format-Dep*, которая выбирает подходящую единицу измерения (байты, килобайты, мегабайты или гигабайты) по величине значения. Свойство *CPU* у некоторых процессов недоступно по правам доступа, поэтому при проверке на $*null* выводится ноль – это предотвращает аварийное завершение скрипта при работе с процессом, защищённым механизмами безопасности *Windows*.

Параметр *Interval* задаёт период обновления (по умолчанию 5 с), *Count* ограничивает число кадров; при *Count*=0 цикл работает до прерывания. В интерактивном режиме кадр формируется с временной меткой, экран очищается перед обновлением.

Каждый кадр включает временную метку, полученную командой *Get-Date* в формате *yyyy-MM-dd HH*:*mm*:*ss*, и номер кадра. Экран очищается командой *Clear-Host* перед выводом каждого кадра, что обеспечивает эффект обновляющегося экрана. При перенаправлении вывода в файл очистка отключается: команда *Clear-*

Host в этом случае стирает уже записанный буфер, и все кадры, кроме последнего, просто исчезают из вывода.

Для записи двух кадров запущен *top*5.*ps*1 с *Interval*=3 и *Count*=2; значение *Count* ограничивает число кадров, а интервал обновления составляет 3 секунды (рисунок 2.62).

Сравнение кадров показывает, что состав таблиц меняется незначительно: процесс *MsMpEng* (антивирусная программа) стабильно занимает первое место по времени работы процессора (346,5 с) и по объёму памяти (253,64 *MB*). На втором месте по памяти находится *StartMenuExperienceHost* (208,25 *MB*), а среди процессов с наименьшим временем работы процессора – *LockApp* (1,4 с) с 163,36 *MB* памяти. Такое распределение характерно для виртуальной машины под управлением гипервизора. После вывода заданного числа кадров сценарий выводит сообщение об остановке и завершает работу.

Для контроля структуры подготовлен отдельный каталог сценариев; его содержимое показано на рисунке 2.63.

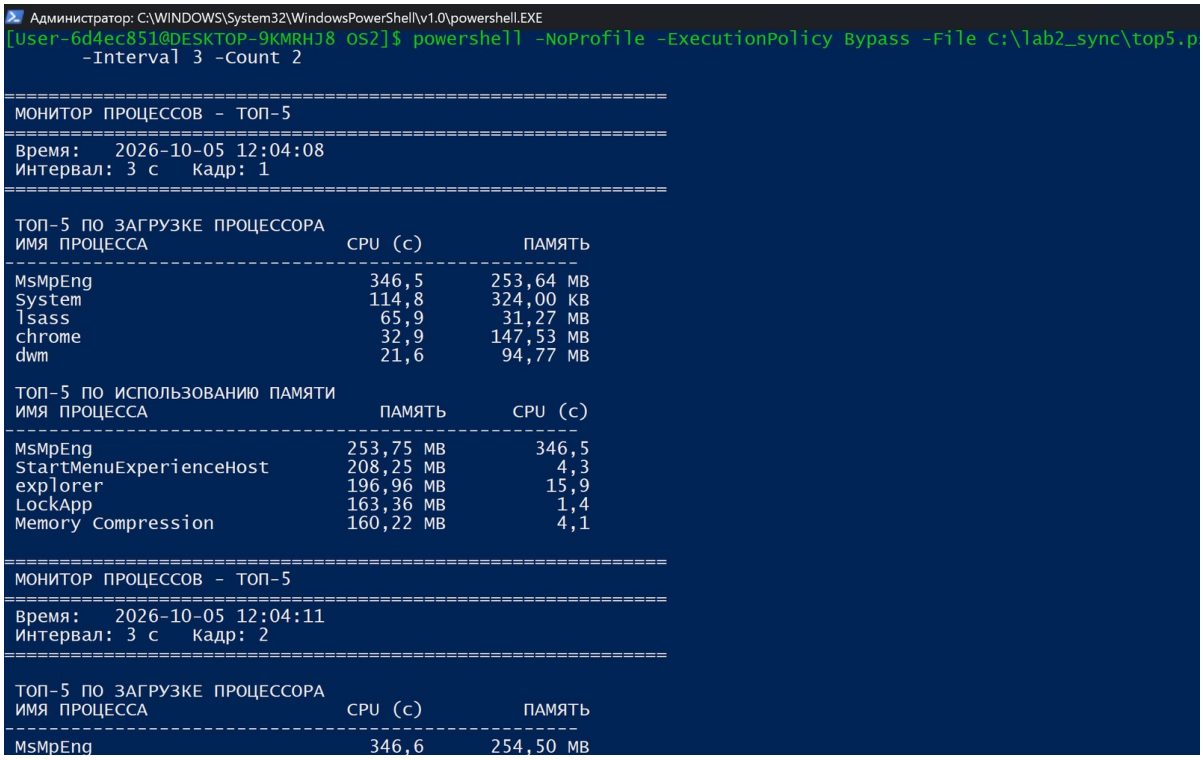


Рисунок 2.62 – Два кадра монитора топ-5 процессов с интервалом 3 секунды

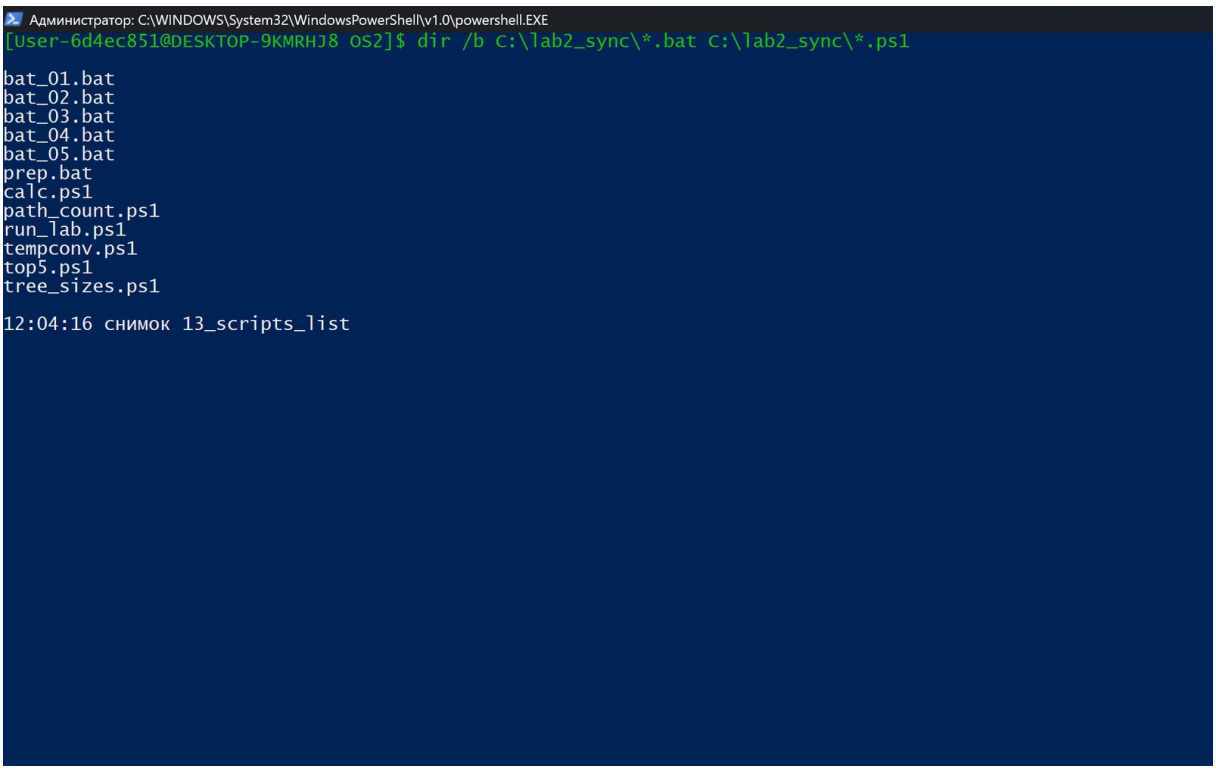


Рисунок 2.63 – Состав разработанных пакетных сценариев и сценариев *PowerShell*

В *Windows* выполнены задания уровней 4–9. Пакетные и *PowerShell*-сценарии проверены с различными параметрами; результаты и обработка ошибок показаны на снимках экрана.

2.12 Вывод по разделу

В лабораторной работе выполнены задания уровней 4–9 в *Linux* и *Windows*. Рассмотрены команды оболочек, перенаправление ввода-вывода, переменные окружения и сценарии *Bash*, *cmd* и *PowerShell*.

Заключение

Выполнены задания уровней 4–9 на *Linux* и *Windows*. Для командных заданий представлены результаты в консоли, для сценариев – проверки с различными входными данными и параметрами.

Освоены приёмы навигации по файловой системе, операции над файлами и каталогами, механизмы перенаправления ввода-вывода и конвейеры, а также приёмы написания пакетных сценариев с параметрами. Выполнены задания по разработке сценариев на языке *PowerShell* и на оболочке *bash*, включая реализацию калькулятора, конвертера температур и монитора процессов.

Сопоставление командных оболочек выявило различия в разделителях путей и *PATH*, атрибутах файлов, передаче ввода и синтаксисе параметров. Общими остаются навигация, файловые операции и перенаправление потоков.

Отчёт включает результаты заданий уровней 4–9 для *Linux* и *Windows*, снимки выполнения команд и листинги разработанных сценариев.

ПРИЛОЖЕНИЕ А

```
#!/bin/bash
# prep.sh – подготовка среды для ЛР2 (Linux-эквивалент Prep v2.bat)
# Запускать из каталога OS2:  bash prep.sh
#
# Скрипт идемпотентен: перед созданием структуры снимает режимы
«только для
# чтения» и удаляет каталоги предыдущего прогона. Без этого повторный
запуск
# падал бы на chmod 444 и копирование не создавало бы файлы.

echo "Подготовка структуры ЛР2..."

# чистка предыдущего прогона: вернуть права и убрать каталоги
chmod -R u+w . 2>/dev/null
rm -rf task_01a task_01b task_01b_dst_1 task_01b_dst_2 task_01c_src
task_01c_dst \
       task_01d_1 task_01d_2 task_01e task_01f task_01g task_01h
task_01i task_01j task_01k \
       tasks_tmp
rm -f students.txt console.txt bios.txt

# ------------------------------------------------
# tasks_tmp – источник файлов для копирования
# ------------------------------------------------
mkdir -p tasks_tmp
echo txt1 > tasks_tmp/txt1.txt
echo txt2 > tasks_tmp/txt2.txt
echo txt3 > tasks_tmp/txt3.txt
echo hidden1 > tasks_tmp/hidden1.txt
echo system1 > tasks_tmp/system1.txt
touch tasks_tmp/bmp1.bmp tasks_tmp/bmp2.bmp
touch tasks_tmp/exe1.exe tasks_tmp/exe2.exe
chmod 755 tasks_tmp/exe1.exe tasks_tmp/exe2.exe   # аналог
исполняемых

# ------------------------------------------------
# task_01a – приёмник для cp
# ------------------------------------------------
mkdir -p task_01a

# ------------------------------------------------
# task_01b – структура для cp -r (аналог xcopy)
# ------------------------------------------------
mkdir -p task_01b/subdir/empty_subdir
echo txt1 > task_01b/subdir/txt1.txt
echo txt2 > task_01b/subdir/txt2.txt
mkdir -p task_01b/hidden_dir/empty_subdir task_01b/hidden_dir/subdir
echo txt1 > task_01b/hidden_dir/subdir/txt1.txt
echo txt2 > task_01b/hidden_dir/subdir/txt2.txt
echo txt1 > task_01b/hidden_dir/txt1.txt
echo system1 > task_01b/hidden_dir/system1.txt
echo readonly > task_01b/readonly.txt          # аналог атрибута +r
chmod 444 task_01b/readonly.txt
```

```
echo archive > task_01b/archive-readonly.txt   # аналог +a +r
chmod 444 task_01b/archive-readonly.txt
echo plain > task_01b/plain.txt                # обычный, без
атрибутов
chmod 644 task_01b/plain.txt

# ------------------------------------------------
# task_01c – источник и приёмник для mv
# ------------------------------------------------
mkdir -p task_01c_src task_01c_dst
for n in 1 2 3 4 5; do echo txt1 > task_01c_src/txt$n.txt; done

# ------------------------------------------------
# task_01d – источник и приёмник для замены файлов
# ------------------------------------------------
mkdir -p task_01d_1 task_01d_2
for n in 1 2 3; do
  cp tasks_tmp/txt$n.txt task_01d_1/
  cp tasks_tmp/txt$n.txt task_01d_2/
done

# ------------------------------------------------
# task_01e – файлы для переименования
# ------------------------------------------------
mkdir -p task_01e
for n in 1 2 3 4 5; do cp tasks_tmp/txt1.txt task_01e/txt$n.txt; done

# ------------------------------------------------
# task_01f – файлы для сравнения (diff/fc)
# ------------------------------------------------
mkdir -p task_01f
printf 'Start\nHello World\nEnd\n' > task_01f/txt1.txt
printf 'Start\nHello, World!\nEnd\n' > task_01f/txt2.txt

# ------------------------------------------------
# task_01g – структура для удаления (del/rm)
# system*.txt – аналог атрибута «Системный»: в Linux такого атрибута
нет,
# поэтому «системные» файлы помечены именем, их удаляет задание 15.
# readonly.txt – аналог «Только для чтения», его обязана пережить
задание 16.
# ------------------------------------------------
mkdir -p task_01g/subdir/empty_subdir
echo txt1 > task_01g/subdir/txt1.txt
echo txt2 > task_01g/subdir/txt2.txt
mkdir -p task_01g/hidden_dir/empty_subdir task_01g/hidden_dir/subdir
echo txt1 > task_01g/hidden_dir/subdir/txt1.txt
echo txt2 > task_01g/hidden_dir/subdir/txt2.txt
echo txt1 > task_01g/hidden_dir/txt1.txt
echo system1 > task_01g/hidden_dir/system1.txt
echo readonly > task_01g/readonly.txt
chmod 444 task_01g/readonly.txt
echo plain > task_01g/plain.txt
```

```
echo txt1 > task_01g/txt1.txt
echo system1 > task_01g/system1.txt
echo system2 > task_01g/system2.txt

# ------------------------------------------------
# task_01h – приёмник для вывода списка
# ------------------------------------------------
mkdir -p task_01h

# ------------------------------------------------
# task_01i – каталоги для навигации
# ------------------------------------------------
mkdir -p "task_01i/Русский" task_01i/English

# ------------------------------------------------
# task_01k – каталоги для удаления
# ------------------------------------------------
mkdir -p task_01k/subdir task_01k/empty_subdir
echo file > task_01k/subdir/file.txt

echo "============================================="
echo "Подготовка завершена."
echo "============================================="
```

Листинг А.1 – Сценарий *prep.sh*

```
#!/bin/bash
# bat_01.sh – аналог bat_01.bat: имя пользователя, дата, время, имя
компьютера

echo "=== bat_01: системные переменные ==="
echo "Пользователь: $USER"
echo "Дата: $(date '+%Y-%m-%d')"
echo "Время: $(date '+%H:%M:%S')"
echo "Компьютер: $(uname -n)"
```

Листинг А.2 – Сценарий *bat_*01.*sh*

```
#!/bin/bash
# bat_02.sh – аналог bat_02.bat: имя скрипта, время изменения, полный
путь
# Аналоги bat: %~nx0 (имя файла), %~t0 (дата изменения), %~f0 (полный
путь)

echo "=== bat_02: параметры скрипта ==="
echo "Имя файла: $(basename "$0")"
echo "Изменён: $(stat -c '%y' "$0" | cut -d'.' -f1)"
echo "Полный путь: $(realpath "$0")"
```

Листинг А.3 – Сценарий *bat_*02.*sh*

```
#!/bin/bash
# bat_03.sh – аналог bat_03.bat: все параметры и первые четыре по
отдельности
# Аналоги bat: %* (все), %1..%4 (по отдельности)

echo "=== bat_03: параметры ==="
echo "Все параметры: $*"
echo "Первый: $1"
echo "Второй: $2"
echo "Третий: $3"
echo "Четвёртый: $4"
```

Листинг А.4 – Сценарий *bat_*03.*sh*

```
#!/bin/bash
# bat_04.sh – аналог bat_04.bat: a-b, a+b, c/b, c*b, (a-b)*(b-a)
# a, b, c – первый, второй и третий параметры. Проверка деления на
ноль.

if [ $# -ne 3 ]; then
    echo "Использование: $0 <a> <b> <c>"
    exit 1
fi

a=$1
b=$2
c=$3

echo "=== bat_04: арифметика ==="
echo "a=$a  b=$b  c=$c"
echo "a-b = $(( a - b ))"
echo "a+b = $(( a + b ))"

if [ "$b" -eq 0 ]; then
    echo "c/b – ОШИБКА: деление на ноль невозможно"
    exit 1
fi

echo "c/b = $(( c / b ))"
echo "c*b = $(( c * b ))"
echo "(a-b)*(b-a) = $(( (a - b) * (b - a) ))"
```

Листинг А.5 – Сценарий *bat_*04.*sh*

```
#!/bin/bash
# bat_05.sh – аналог bat_05.bat: режимы create/delete
# Проверка отсутствия параметров, проверка существования файла.

if [ $# -ne 2 ]; then
    echo "Использование: $0 create|delete <файл>"
    exit 1
fi

action=$1
file=$2

case "$action" in
    create)
        if [ -e "$file" ]; then
            echo "Файл $file уже существует"
        else
            touch "$file" && echo "Создан файл $file"
        fi
        ;;
    delete)
        if [ -e "$file" ]; then
            rm "$file" && echo "Удалён файл $file"
        else
            echo "Файл $file не существует"
        fi
        ;;
    *)
        echo "Неизвестный режим: $action (допустимо: create, delete)"
        exit 1
        ;;
esac
```

Листинг А.6 – Сценарий *bat_*05*.sh*

```
#!/bin/bash
# path_count.sh – подсчёт исполняемых и неисполняемых команд в PATH
(уровень 8)

echo "============================================================="
echo " СТАТИСТИКА ПО КАТАЛОГАМ PATH"
echo "============================================================="
printf "%-50s %8s %8s\n" "КАТАЛОГ" "EXEC" "NON-EXEC"
printf "%-50s %8s %8s\n"
"-------------------------------------------------" "--------"
"--------"

total_exec=0
total_nonexec=0
missing=0

IFS=':' read -ra dirs <<< "$PATH"

for d in "${dirs[@]}"; do
    if [ -d "$d" ]; then
        e=$(find "$d" -maxdepth 1 -type f -executable 2>/dev/null |
wc -l)
        ne=$(find "$d" -maxdepth 1 -type f ! -executable 2>/dev/null
| wc -l)
        total_exec=$((total_exec + e))
        total_nonexec=$((total_nonexec + ne))
        printf "%-50s %8d %8d\n" "$d" "$e" "$ne"
    else
        missing=$((missing + 1))
        printf "%-50s %8s %8s\n" "$d" "-" "-"
    fi
done

echo "============================================================="
printf "Каталогов в PATH:                %d (из них не существует: %d)
\n" "${#dirs[@]}" "$missing"
printf "Всего исполняемых файлов:      %d\n" "$total_exec"
printf "Всего неисполняемых файлов:    %d\n" "$total_nonexec"
printf "Всего файлов в PATH:           %d\n" "$((total_exec +
total_nonexec))"
echo "============================================================="
```

Листинг А.7 – Сценарий *path_count.sh*

```
#!/bin/bash
# tree_sizes.sh – вывод дерева каталогов с размерами (уровень 8)
# Для каталогов – число файлов; для файлов – размер в KB/MB/GB и
KiB/MiB/GiB
# Использование: ./tree_sizes.sh [каталог]   (по умолчанию текущий)

target="${1:-.}"

# Размер в десятичных единицах: KB = 1000 байт
fmt_dec() {
    awk -v b="$1" 'BEGIN {
        if (b < 1000)          printf "%d B", b
        else if (b < 1000000)   printf "%.2f KB", b/1000
        else if (b < 1000000000) printf "%.2f MB", b/1000000
        else                   printf "%.2f GB", b/1000000000
    }'
}

# Размер в двоичных единицах: KiB = 1024 байт
fmt_bin() {
    awk -v b="$1" 'BEGIN {
        if (b < 1024)            printf "%d B", b
        else if (b < 1048576)    printf "%.2f KiB", b/1024
        else if (b < 1073741824) printf "%.2f MiB", b/1048576
        else                     printf "%.2f GiB", b/1073741824
    }'
}

echo
"================================================================="
echo " ДЕРЕВО КАТАЛОГОВ: $target"
echo " Формат размера файла: <десятичный> / <двоичный>"
echo
"================================================================="
printf "%s[DIR ] %-40s файлов: %s, размер: %s / %s\n" "$2" "." "0" "0
B" "0 B"

print_tree() {
    local dir="$1" indent="$2"
    local fcount=0 tsize=0

    # Считаем рекурсивно по текущему каталогу
    while IFS= read -r -d '' f; do
        if [ -f "$f" ]; then
            fcount=$((fcount + 1))
            tsize=$((tsize + $(stat -c%s "$f")))
        fi
    done < <(find "$dir" -type f -print0 2>/dev/null)

    local dsize dec_size bin_size
    dsize=$(fmt_dec "$tsize")
    bin_size=$(fmt_bin "$tsize")
    printf "%s[DIR ] %-40s файлов: %d, размер: %s / %s\n" "$indent"
```

```
"$(basename "$dir")" "$fcount" "$dsize" "$bin_size"

    local item name
    for item in "$dir"/* "$dir"/.[!.]*; do
        [ -e "$item" ] || continue
        name=$(basename "$item")
        if [ -d "$item" ]; then
            print_tree "$item" "$indent  "
        else
            local sz szd szb
            sz=$(stat -c%s "$item")
            szd=$(fmt_dec "$sz")
            szb=$(fmt_bin "$sz")
            printf "%s[FILE] %-40s %s / %s\n" "$indent  " "$name"
"$szd" "$szb"
        fi
    done
}

print_tree "$target" ""
echo
"================================================================="
```

Листинг А.8 – Сценарий *tree_sizes.sh*

```
#!/bin/bash
# top5.sh – монитор топ-5 процессов по CPU и памяти (уровень 9)
# Использование: ./top5.sh [интервал_в_секундах]   (по умолчанию 5)
# Работает в бесконечном цикле до Ctrl+C.

INTERVAL="${1:-5}"

if ! [[ $INTERVAL =~ ^[0-9]+$ ]] || [ "$INTERVAL" -eq 0 ]; then
    echo "Ошибка: интервал должен быть положительным целым числом
секунд"
    exit 1
fi

trap 'echo; echo "Монитор остановлен."; exit 0' INT TERM

# Итоговая память системы в МБ – для перевода процентов в мегабайты
TOTAL_MB=$(awk '/MemTotal/ {printf "%d", $2/1024}' /proc/meminfo)

fmt_row() {
    printf "%-8s %-24s %8s %10s %8s\n" "$1" "$2" "$3" "$4" "$5"
}

while true; do
    clear
    echo
"================================================================="
    echo " МОНИТОР ПРОЦЕССОВ - ТОП-5"
    echo " Время:        $(date '+%Y-%m-%d %H:%M:%S')"
    echo " Интервал:     ${INTERVAL} с (Ctrl+C - выход)"
    echo " Всего памяти: ${TOTAL_MB} МБ"
    echo
"================================================================="

    echo
    echo " ТОП-5 ПРОЦЕССОВ ПО ЗАГРУЗКЕ ЦП"
    fmt_row "PID" "ИМЯ" "CPU,%" "ПАМЯТЬ,МБ" "ВРЕМЯ CPU"
    fmt_row "--------" "------------------------" "--------"
"---------" "----------"
    ps -eo pid,comm,%cpu,%mem,time --sort=-%cpu 2>/dev/null | head -6
| tail -5 | while read pid comm cpu mem cputime; do
        mbm=$(awk -v p="$mem" -v t="$TOTAL_MB" 'BEGIN{printf "%.1f",
p*t/100}')
        printf "%-8s %-24s %8s %10s %8s\n" "$pid" "$comm" "$cpu"
"$mbm" "$cputime"
    done

    echo
    echo " ТОП-5 ПРОЦЕССОВ ПО ЗАГРУЗКЕ ОПЕРАТИВНОЙ ПАМЯТИ"
    fmt_row "PID" "ИМЯ" "CPU,%" "ПАМЯТЬ,МБ" "ВРЕМЯ CPU"
    fmt_row "--------" "------------------------" "--------"
"---------" "----------"
    ps -eo pid,comm,%cpu,%mem,time --sort=-%mem 2>/dev/null | head -6
| tail -5 | while read pid comm cpu mem cputime; do
```

```
        mbm=$(awk -v p="$mem" -v t="$TOTAL_MB" 'BEGIN{printf "%.1f",
p*t/100}')
        printf "%-8s %-24s %8s %10s %8s\n" "$pid" "$comm" "$cpu"
"$mbm" "$cputime"
    done

    echo
    echo
"============================================================="
    sleep "$INTERVAL"
done
```

Листинг А.9 – Сценарий *top*5.*sh*

```
#!/bin/bash
# calc.sh – интерактивный калькулятор (уровень 8, аналог calc.ps1)
# Операции: + - * / % ^ (степень) root (корень n-й степени) sin cos
tan log ln

show_help() {
    cat <<'EOF'
=== КАЛЬКУЛЯТОР ===
Бинарные:  <число> <оп> <число>   где op = + - * / % ^ root
Унарные:   <функция> <число>       где функция = sin cos tan log ln
sqrt
Примеры:   2 + 3     |  2 ^ 10   |  27 root 3   |  sin 1.57   |  sqrt
16
Выход:     q
EOF
}

calc_unary() {
    local fn=$1 x=$2
    case $fn in
        sqrt) awk -v x="$x" 'BEGIN {print sqrt(x)}' ;;
        sin)  awk -v x="$x" 'BEGIN {print sin(x)}' ;;
        cos)  awk -v x="$x" 'BEGIN {print cos(x)}' ;;
        tan)  awk -v x="$x" 'BEGIN {print tan(x)}' ;;
        log)  awk -v x="$x" 'BEGIN { if (x<=0) exit 1; print
log(x)/log(10) }' ;;
        ln)   awk -v x="$x" 'BEGIN { if (x<=0) exit 1; print
log(x) }' ;;
        *) return 2 ;;
    esac
}

calc_binary() {
    local a=$1 op=$2 b=$3
    case $op in
        +) awk -v a="$a" -v b="$b" 'BEGIN {print a+b}' ;;
        -) awk -v a="$a" -v b="$b" 'BEGIN {print a-b}' ;;
        '*') awk -v a="$a" -v b="$b" 'BEGIN {print a*b}' ;;
        /) awk -v a="$a" -v b="$b" 'BEGIN { if (b==0) exit 1; print
a/b }' ;;
        %) awk -v a="$a" -v b="$b" 'BEGIN { if (b==0) exit 1; printf
"%.10g\n", (a-int(a/b)*b) }' ;;
        ^) awk -v a="$a" -v b="$b" 'BEGIN {print a^b}' ;;
        root) awk -v a="$a" -v b="$b" 'BEGIN { if (b==0) exit 1;
print a^(1/b) }' ;;
        *) return 2 ;;
    esac
}

is_number() {
    [[ $1 =~ ^-?[0-9]+(\.[0-9]+)?$ ]]
}
```

```
while true; do
    show_help
    read -r -p "Введите выражение: " line
    line=$(echo "$line" | tr 'A-Z' 'a-z')

    [[ -z $line ]] && continue
    [[ $line == "q" || $line == "выход" ]] && break

    read -r -a parts <<< "$line"

    if [[ ${#parts[@]} -eq 3 ]]; then
        if ! is_number "${parts[0]}" || ! is_number "${parts[2]}";
then
            echo "Ошибка: неверный ввод числа"; continue
        fi
        if [[ "${parts[1]}" == "/" || "${parts[1]}" == "%" || "$
{parts[1]}" == "root" ]] \
           && awk -v b="${parts[2]}" 'BEGIN {exit !(b==0)}'; then
            echo "Ошибка: деление на ноль"; continue
        fi
        result=$(calc_binary "${parts[0]}" "${parts[1]}" "$
{parts[2]}") \
            || { echo "Ошибка: недопустимая операция или деление на
ноль"; continue; }
        printf "Результат: %s\n" "$result"

    elif [[ ${#parts[@]} -eq 2 ]]; then
        if ! is_number "${parts[1]}"; then
            echo "Ошибка: неверный ввод числа"; continue
        fi
        case ${parts[0]} in
            log|ln)
                if awk -v x="${parts[1]}" 'BEGIN {exit !(x<=0)}';
then
                    echo "Ошибка: аргумент логарифма должен быть >
0"; continue
                fi
                ;;
        esac
        result=$(calc_unary "${parts[0]}" "${parts[1]}") \
            || { echo "Ошибка: неизвестная функция"; continue; }
        printf "Результат: %s\n" "$result"
    else
        echo "Ошибка: неверный формат ввода"
    fi
done

echo "Калькулятор завершён."
```

Листинг А.10 – Сценарий *calc.sh*

```
#!/bin/bash
# tempconv.sh – конвертер температур (уровень 8, аналог tempconv.ps1)
# Формат ввода: <число>[K|C|F]   по умолчанию C; 100C, 273.15K, 212F,
25

echo "=== ТЕМПЕРАТУРНЫЙ КОНВЕРТЕР ==="
echo "Введите температуру в виде: 100C, 273K, 212F, 25 (без суффикса
= Цельсий)"
echo "Выход: q"

while true; do
    read -r -p "Введите температуру: " raw
    raw=$(echo "$raw" | tr -d ' ' | tr 'a-zq' 'A-ZQ')

    [[ -z $raw ]] && continue
    [[ $raw == "Q" || $raw == "Й" ]] && break

    # Разбор: число + необязательный суффикс единицы
    if [[ $raw =~ ^([+-]?[0-9]+([.][0-9]+)?)([KCF])?$ ]]; then
        value=${BASH_REMATCH[1]}
        unit=${BASH_REMATCH[3]:-C}

        case $unit in
            C) c=$value ; k=$(awk -v c="$c" 'BEGIN{print
c+273.15}') ; f=$(awk -v c="$c" 'BEGIN{print c*9/5+32}') ;;
            K) k=$value ; c=$(awk -v k="$k" 'BEGIN{print
k-273.15}') ; f=$(awk -v c="$c" 'BEGIN{print c*9/5+32}') ;;
            F) f=$value ; c=$(awk -v f="$f" 'BEGIN{print
(f-32)*5/9}') ; k=$(awk -v c="$c" 'BEGIN{print c+273.15}') ;;
        esac

        printf "Введено:      %s %s\n" "$value" "$unit"
        printf "Цельсий:      %.2f C\n" "$(awk -v x="$c" 'BEGIN{print
x}')"
        printf "Кельвин:      %.2f K\n" "$(awk -v x="$k" 'BEGIN{print
x}')"
        printf "Фаренгейт:    %.2f F\n" "$(awk -v x="$f" 'BEGIN{print
x}')"
        echo "----------------------------------------"
    else
        echo "Ошибка: неверный ввод. Ожидается число и необязательный
суффикс K, C или F."
    fi
done

echo "Конвертер завершён."
```

Листинг А.11 – Сценарий *tempconv.sh*

```
@echo off
rem ============================================================
rem  prep.bat - подготовка среды для ЛР2 (Windows)
rem  Запускать из каталога OS2:  prep.bat
rem  Идемпотентен: перед созданием структуры снимает
rem  атрибут «только для чтения» и удаляет старые каталоги.
rem ============================================================
rem Кодовую страницу не меняем: при 65001 mkdir записывает
кириллическое
rem имя в UTF-8, и следующий cmd читает его как OEM - получается
"?????".
setlocal enabledelayedexpansion

cd /d "%~dp0"

echo [prep] снятие атрибутов и очистка предыдущих каталогов...
attrib -R -S -H OS2\*.* /S /D >nul 2>&1
for %%D in (task_01a task_01b task_01b_dst_1 task_01b_dst_2
task_01c_src task_01c_dst task_01d_1 task_01d_2 task_01e task_01f
task_01g task_01h task_01i task_01j task_01k tasks_tmp) do (
    if exist OS2\%%D rd /S /Q OS2\%%D 2>nul
)
del /Q students.txt console.txt systeminfo.txt user_files.txt 2>nul
echo [prep] готово.
echo.

echo [prep] создание структуры OS2...
mkdir OS2 2>nul

rem --- task_01a: файлы для копирования ---
mkdir "OS2\task_01a" 2>nul

rem --- task_01b: дерево с атрибутами для xcopy ---
mkdir "OS2\task_01b\subdir" 2>nul
mkdir "OS2\task_01b\hidden_dir" 2>nul
mkdir "OS2\task_01b\subdir\empty_subdir" 2>nul
mkdir "OS2\task_01b\hidden_dir\system1" 2>nul

rem --- task_01c_src / task_01c_dst: для move ---
mkdir "OS2\task_01c_src" 2>nul
mkdir "OS2\task_01c_dst" 2>nul

rem --- task_01d_1 / task_01d_2: для replace и очистки текста ---
mkdir "OS2\task_01d_1" 2>nul
mkdir "OS2\task_01d_2" 2>nul

rem --- task_01e: файлы для переименования расширения ---
mkdir "OS2\task_01e" 2>nul

rem --- task_01f: два файла для сравнения ---
mkdir "OS2\task_01f" 2>nul

rem --- task_01g: файлы для удаления ---
```

```
mkdir "OS2\task_01g\hidden_dir" 2>nul
mkdir "OS2\task_01g\subdir" 2>nul
mkdir "OS2\task_01g\subdir\empty_subdir" 2>nul

rem --- task_01h: каталог для отчётов ---
mkdir "OS2\task_01h" 2>nul

rem --- task_01i: каталоги с кириллицей ---
mkdir "OS2\task_01i" 2>nul
mkdir "OS2\task_01i\Русский" 2>nul
mkdir "OS2\task_01i\English" 2>nul

rem --- task_01j: иерархия каталогов ---
mkdir "OS2\task_01j" 2>nul

rem --- task_01k: пустой и непустой каталоги ---
mkdir "OS2\task_01k" 2>nul
mkdir "OS2\task_01k\empty_subdir" 2>nul
mkdir "OS2\task_01k\subdir" 2>nul

rem --- tasks_tmp: исходные файлы ---
mkdir "OS2\tasks_tmp" 2>nul

rem --- содержимое файлов ---
for %%N in (1 2 3 4 5) do (
    echo txt%%N > "OS2\tasks_tmp\txt%%N.txt"
)

echo txt file 1 > "OS2\task_01g\txt1.txt"
echo txt file 2 > "OS2\task_01g\txt2.txt"
echo plain file 1 > "OS2\task_01g\plain1.txt"
echo plain file 2 > "OS2\task_01g\plain2.txt"
echo system file 1 > "OS2\task_01g\system1.txt"
echo system file 2 > "OS2\task_01g\system2.txt"
echo hidden file 1 > "OS2\task_01g\hidden1.txt"
echo read only file > "OS2\task_01g\readonly.txt"
echo sub file 1 > "OS2\task_01g\subdir\sub1.txt"
echo system file in hidden > "OS2\task_01g\hidden_dir\system1.txt"

echo Hello World > "OS2\task_01f\txt1.txt"
echo Hello, World! > "OS2\task_01f\txt2.txt"

for %%N in (1 2 3 4 5) do (
    echo file %%N content > "OS2\task_01e\txt%%N.txt"
    echo file %%N content > "OS2\task_01d_1\txt%%N.txt"
    echo file %%N to be cleared > "OS2\task_01d_2\txt%%N.txt"
)

for %%N in (1 2 3 4 5) do (
    echo archive file %%N > "OS2\task_01b\file%%N.txt"
    echo sub file %%N > "OS2\task_01b\subdir\sub%%N.txt"
)
echo archive file hidden > "OS2\task_01b\hidden_dir\hidden1.txt"
```

```
echo archive file system > "OS2\task_01b\hidden_dir\system1.txt"
echo hidden dir file > "OS2\task_01b\hidden_dir\deep1.txt"

for %%N in (1 2 3) do (
    echo c src file %%N > "OS2\task_01c_src\txt%%N.txt"
)

rem --- список студентов для заданий уровня 4 (find, sort, echo) ---
rem Тот же набор фамилий, что и в Linux-части, чтобы результаты
сопоставлялись.
if not exist "OS2" mkdir "OS2" >nul 2>&1
> "OS2\students.txt" echo Иванов Иван Иванович
>> "OS2\students.txt" echo Петрова Анна Сергеевна
>> "OS2\students.txt" echo Сидоров Пётр Алексеевич
>> "OS2\students.txt" echo Кузнецов Олег Викторович
>> "OS2\students.txt" echo Смирнова Екатерина Дмитриевна
>> "OS2\students.txt" echo Волков Андрей Николаевич
>> "OS2\students.txt" echo Морозова Наталья Игоревна
>> "OS2\students.txt" echo Соколов Дмитрий Павлович
>> "OS2\students.txt" echo Новикова Мария Андреевна
>> "OS2\students.txt" echo Фёдоров Сергей Владимирович

rem --- атрибуты: archive, read-only, hidden, system ---
rem Без +A на task_01b ключ xcopy /A не скопирует ни одного файла,
rem поэтому архивный атрибут выставляется здесь, а не в bat.
for %%F in ("OS2\task_01b\*.txt") do attrib +A %%F >nul
for %%F in ("OS2\task_01b\subdir\*.txt") do attrib +A %%F >nul
attrib +R "OS2\task_01g\readonly.txt" >nul
attrib +H "OS2\task_01g\hidden1.txt" >nul
attrib +S "OS2\task_01g\system1.txt" >nul
attrib +S "OS2\task_01g\system2.txt" >nul
attrib +S "OS2\task_01g\hidden_dir\system1.txt" >nul
attrib +H "OS2\task_01b\hidden_dir\hidden1.txt" >nul
attrib +S "OS2\task_01b\hidden_dir\system1.txt" >nul

echo [prep] создание файлов и каталогов завершено.
echo.
echo [prep] итоговая структура OS2:
dir /B /S OS2 | findstr /V "^$"
echo.
echo [prep] готово. Можно выполнять задания.
exit /b 0
```

Листинг А.12 – Сценарий *prep.bat*

```
@echo off
rem ============================================================
rem  bat_01 - имя пользователя, дата и время, имя компьютера
rem  через системные переменные окружения
rem ============================================================
echo === bat_01: системные переменные ===
echo Пользователь: %USERNAME%
echo Дата и время: %DATE% %TIME%
echo Имя компьютера: %COMPUTERNAME%
echo Домашний каталог: %USERPROFILE%
```

Листинг А.13 – Сценарий *bat*_01.*bat*

```
@echo off
rem ============================================================
rem  bat_02 - имя текущего bat-файла, время модификации
rem  и полный путь через расширения параметров
rem ============================================================

set SCRIPT=%~f0
set NAME=%~n0

echo === bat_02: параметры скрипта ===
echo Имя файла: %NAME%
for %%F in ("%SCRIPT%") do echo Изменён: %%~tF
echo Полный путь: %SCRIPT%
echo Расширение: %~x1
```

Листинг А.14 – Сценарий *bat*_02.*bat*

```
@echo off
rem ============================================================
rem  bat_03 - вывод строки со всеми переданными параметрами
rem  и первых четырёх параметров по отдельности
rem ============================================================

set "COUNT=%~1"
shift
set "ARGS=%*"

echo === bat_03: параметры ===
echo Все параметры: %ARGS%
if not "%COUNT%"=="" goto :show
echo Параметры не переданы - работаем со всем списком
:show
echo Первый:  %~1
echo Второй:  %~2
echo Третий:  %~3
echo Четвёртый: %~4
```

Листинг А.15 – Сценарий *bat*_03.*bat*

```
@echo off
rem ============================================================
rem  bat_04 - вычисление a-b, a+b, c/b, c*b, (a-b)*(b-a),
rem  где a, b, c - первый, второй и третий параметры.
rem  Деление на ноль проверяется ДО вычисления: иначе set /a
rem  обрывает пакет сообщением об ошибке и следующие echo не
rem  выводятся вовсе.
rem ============================================================

setlocal enabledelayedexpansion

set /a a=%~1
set /a b=%~2
set /a c=%~3

if "%a%"=="" set /a a=1
if "%b%"=="" set /a b=1
if "%c%"=="" set /a c=1

echo === bat_04: арифметика ===
echo a=%a%  b=%b%  c=%c%

set /a diff=a-b
set /a sum=a+b
set /a prod=a*b
set /a cross=b-a
set /a expr=diff*cross

echo a-b = %diff%
echo a+b = %sum%
echo a*b = %prod%
echo (a-b)*(b-a) = %expr%

if %b%==0 (
    echo c/b = ОШИБКА: деление на ноль невозможно
) else (
    set /a quot=c/b
    echo c/b = !quot!
)

endlocal
```

Листинг А.16 – Сценарий *bat*_04.*bat*

```
@echo off
rem ============================================================
rem  bat_05 - работа в режимах создание/удаление,
rem  обработка отсутствия параметров, проверка существования файла
rem ============================================================

if "%~1"=="" goto :noargs
if "%~2"=="" goto :nofile

set "MODE=%~1"
set "FILE=%~2"

if /i "%MODE%"=="create" goto :create
if /i "%MODE%"=="delete" goto :delete
echo Неизвестный режим: %MODE%
echo Допустимо: create, delete
exit /b 1

:create
if exist "%FILE%" (
    echo Файл уже существует: %FILE%
    exit /b 1
)
echo Создан файл %FILE%
> "%FILE%" echo Содержимое файла %FILE%, созданного %DATE% %TIME%
exit /b 0

:delete
if not exist "%FILE%" (
    echo Файл не найден: %FILE%
    exit /b 1
)
del /q "%FILE%"
echo Удалён файл %FILE%
exit /b 0

:noargs
echo Ошибка: не указан режим работы
echo Использование: bat_05 create^|delete имя_файла
exit /b 1

:nofile
echo Ошибка: не указано имя файла
echo Использование: bat_05 %MODE% имя_файла
exit /b 1
```

Листинг А.17 – Сценарий *bat*_05.*bat*

```
# calc.ps1 - калькулятор (уровень 8, PowerShell)
# Поддерживаемые операции: + - * /, остаток %, степень ^,
# корень sqrt a и root a b, sin cos tan (градусы),
# десятичный log10 и натуральный ln.
#
# Разбор выражения идёт по правилам приоритета: сначала ^ ,
# затем * / % , затем + - . Один общий шаблон на всё выражение
# не работал: он не разбирал 2+3*4 и путал номер группы
# named-capture с номером оператора, из-за чего 10/0 падал
# с ошибкой преобразования типа.
# Выражения передаются обычными аргументами ($args), а не через
параметр
# с ValueFromRemainingArguments: такой параметр перехватывал и сам
путь,
# из-за чего calc.ps1 2+3*4 пытался читать файл с именем "2+3*4".
# -FileIn - файл с выражениями, по одному в строке. Нужен для
автоматического
# прогона: Read-Host игнорирует перенаправленный stdin и блокируется.

$ErrorActionPreference = "Continue"

# Файл ввода читаем байтами и декодируем кодовой страницей консоли:
# Get-Content без -Encoding берёт ANSI и ломает кириллицу (866 против
1251).
function Get-ConsoleLines([string]$path) {
    $enc = [System.Text.Encoding]::GetEncoding([int]
[Console]::OutputEncoding.CodePage)
    $text = $enc.GetString([System.IO.File]::ReadAllBytes($path))
    foreach ($l in ($text -split "`r?`n")) {
        if ($l.Trim() -eq "") { continue }
        $l.Trim()
    }
}

function Show-Help {
    Write-Host "=== Калькулятор: поддерживаемые операции ==="
    Write-Host "  a+b, a-b, a*b, a/b        базовая арифметика"
    Write-Host "  a%b                      остаток от деления"
    Write-Host "  a^b                      возведение в степень"
    Write-Host "  root a b                 корень b-й степени из a"
    Write-Host "  sin a, cos a, tan a      тригонометрия (градусы)"
    Write-Host "  log10 a, ln a            десятичный и натуральный
логарифмы"
    Write-Host "  sqrt a                   квадратный корень"
    Write-Host ""
    Write-Host "  Примеры: 2+3*4   root 27 3   sin 30   log10 100
10%3"
}

# Базовое число с необязательным знаком. Культура фиксирована,
# иначе на машине с запятой в качестве разделителя Parse бросает
исключение.
function Get-Number([string]$t) {
```

```
    $t = $t.Trim()
    if ($t -notmatch '^[+-]?[0-9]+(\.[0-9]+)?$') { return $null }
    return [double]::Parse($t,
[Globalization.CultureInfo]::InvariantCulture)
}

# Ошибку передаём через скриптовую переменную, а не через throw:
# throw прервал бы весь пакет и следующие выражения не считались бы.
$script:Err = $null

# Уровень 1: сложение и вычитание - низший приоритет,
левоассоциативно.
function Eval-Sum([string]$e) {
    $m = [regex]::Match($e, '^(.+?)\s*([+-])\s*(.+)$')
    if ($m.Success -and $m.Groups[1].Value -match '[0-9)]') {
        $l = Eval-Product $m.Groups[1].Value
        if ($null -eq $l) { return $null }
        $r = Eval-Product $m.Groups[3].Value
        if ($null -eq $r) { return $null }
        if ($m.Groups[2].Value -eq '+') { return $l + $r }
        return $l - $r
    }
    return Eval-Product $e
}

# Уровень 2: умножение, деление, остаток от деления.
function Eval-Product([string]$e) {
    $m = [regex]::Match($e, '^(.+?)\s*([*/%])\s*(.+)$')
    if ($m.Success -and $m.Groups[1].Value -match '[0-9)]') {
        $l = Eval-Power $m.Groups[1].Value
        if ($null -eq $l) { return $null }
        $r = Eval-Power $m.Groups[3].Value
        if ($null -eq $r) { return $null }
        switch ($m.Groups[2].Value) {
            "*" { return $l * $r }
            "/" {
                if ($r -eq 0) { $script:Err = "деление на ноль
невозможно"; return $null }
                return $l / $r
            }
            default {
                if ($r -eq 0) { $script:Err = "остаток от деления на
ноль невозможен"; return $null }
                return $l - [math]::Floor($l / $r) * $r
            }
        }
    }
    return Eval-Power $e
}

# Уровень 3: возведение в степень - правоассоциативно.
function Eval-Power([string]$e) {
    $m = [regex]::Match($e, '^(.+?)\s*\^\s*(.+)$')
```

```
    if ($m.Success) {
        $l = Get-Number $m.Groups[1].Value
        if ($null -eq $l) { return $null }
        $r = Eval-Power $m.Groups[2].Value
        if ($null -eq $r) { return $null }
        return [math]::Pow($l, $r)
    }
    return Get-Number $e
}

# Форматирование числа: запятая как десятичный разделитель ломает
# последующий разбор выражения, поэтому печатаем точкой.
function Fmt([double]$v) {
    # Печатаем точкой (инвариантная культура), без хвостовых нулей:
    # запятая сломала бы последующий разбор выражения, а нули не
нужны.
    $s = $v.ToString("F10",
[System.Globalization.CultureInfo]::InvariantCulture)
    if ($s.Contains(".")) { $s = $s.TrimEnd("0").TrimEnd(".") }
    return $s
}

function Invoke-Calc([string]$line) {
    $e = $line.Trim()
    if ($e -eq "") { return }
    $script:Err = $null

    if ($e -match '^\s*root\s+(\S+)\s+(\S+)\s*$') {
        $a = Get-Number $matches[1]; $n = Get-Number $matches[2]
        if ($null -eq $a -or $null -eq $n) { Write-Host "ОШИБКА: не
удалось разобрать выражение: $e" -ForegroundColor Red; return }
        if ($n -eq 0) { Write-Host "ОШИБКА: показатель корня не может
быть нулём" -ForegroundColor Red; return }
        Write-Host ("{0} = {1}" -f $e, ((Fmt ([math]::Pow($a, 1.0 /
$n)))))
        return
    }
    if ($e -match '^\s*(sin|cos|tan)\s+(\S+)\s*$') {
        $fn = $matches[1]; $deg = Get-Number $matches[2]
        if ($null -eq $deg) { Write-Host "ОШИБКА: не удалось
разобрать выражение: $e" -ForegroundColor Red; return }
        $rad = $deg * [math]::PI / 180
        $v = switch ($fn) {
            "sin" { [math]::Sin($rad) }
            "cos" { [math]::Cos($rad) }
            "tan" { [math]::Tan($rad) }
        }
        Write-Host ("{0} = {1}" -f $e, (Fmt $v))
        return
    }
    if ($e -match '^\s*log10\s+(\S+)\s*$') {
        $a = Get-Number $matches[1]
        if ($null -eq $a) { Write-Host "ОШИБКА: не удалось разобрать
```

```
выражение: $e" -ForegroundColor Red; return }
        if ($a -le 0) { Write-Host "ОШИБКА: аргумент логарифма должен
быть больше нуля" -ForegroundColor Red; return }
        Write-Host ("{0} = {1}" -f $e, ((Fmt ([math]::Log10($a)))))
        return
    }
    if ($e -match '^\s*ln\s+(\S+)\s*$') {
        $a = Get-Number $matches[1]
        if ($null -eq $a) { Write-Host "ОШИБКА: не удалось разобрать
выражение: $e" -ForegroundColor Red; return }
        if ($a -le 0) { Write-Host "ОШИБКА: аргумент логарифма должен
быть больше нуля" -ForegroundColor Red; return }
        Write-Host ("{0} = {1}" -f $e, ((Fmt ([math]::Log($a)))))
        return
    }
    if ($e -match '^\s*sqrt\s+(\S+)\s*$') {
        $a = Get-Number $matches[1]
        if ($null -eq $a) { Write-Host "ОШИБКА: не удалось разобрать
выражение: $e" -ForegroundColor Red; return }
        if ($a -lt 0) { Write-Host "ОШИБКА: подкоренное выражение не
может быть отрицательным" -ForegroundColor Red; return }
        Write-Host ("{0} = {1}" -f $e, ((Fmt ([math]::Sqrt($a)))))
        return
    }

    $v = Eval-Sum $e
    if ($script:Err) {
        Write-Host ("ОШИБКА: {0} ({1})" -f $script:Err, $e)
-ForegroundColor Red
    } elseif ($null -eq $v) {
        Write-Host "ОШИБКА: не удалось разобрать выражение: $e"
-ForegroundColor Red
    } else {
        Write-Host ("{0} = {1}" -f $e, (Fmt $v))
    }
}

Write-Host "=== Калькулятор ==="
Show-Help

# Разбор аргументов вручную: объявленный параметр [string]$FileIn
# получал первый позиционный аргумент, и "calc.ps1 2+3*4" пытался
# прочитать файл с именем "2+3*4".
# Ключ -Help печатает справку и завершает работу: без него запуск без
# аргументов уходил в интерактивный режим и намертво блокировал
драйвер.
$FileIn = $null
$WantHelp = $false
$Exprs = @()
for ($i = 0; $i -lt $args.Count; $i++) {
    if ($args[$i] -eq '-FileIn') {
        $i++
        if ($i -lt $args.Count) { $FileIn = $args[$i] }
```

```
    } elseif ($args[$i] -eq '-Help') {
        $WantHelp = $true
    } else {
        $Exprs += $args[$i]
    }
}

if ($WantHelp) {
    Write-Host ""
    Write-Host "--- Справка по ключам ---"
    Write-Host "  -Help        показать эту справку и выйти"
    Write-Host "  -FileIn путь читать выражения из файла, по одному в
строке"
    Write-Host "  выражения    например: calc.ps1 2+3*4 sqrt 16"
    exit 0
}

if ($FileIn) {
    Write-Host ""
    Write-Host "--- Режим ввода из файла: $FileIn ---"
    foreach ($line in (Get-ConsoleLines $FileIn)) {
        if ($line -match '^(выход|exit|quit)$') { break }
        Invoke-Calc $line
    }
} elseif ($Exprs.Count -eq 0) {
    Write-Host ""
    Write-Host "--- Интерактивный режим, для выхода введите выход
---"
    while ($true) {
        $line = Read-Host "calc>"
        if ($line -match '^(выход|exit|quit)$') { Write-Host "Работа
завершена."; break }
        Invoke-Calc $line
    }
} else {
    Write-Host ""
    # "sqrt 16" из командной строки приходит двумя аргументами,
    # поэтому склеиваем функцию с её операндом в одно выражение.
    $joined = @()
    for ($i = 0; $i -lt $Exprs.Count; $i++) {
        if ($Exprs[$i] -match '^(root|sin|cos|tan|log10|ln|sqrt)$'
-and
            $i + 1 -lt $Exprs.Count) {
            $pair = $Exprs[$i] + ' ' + $Exprs[$i + 1]
            $joined += $pair
            $i++
            if ($Exprs[$i] -ne '' -and $Exprs[$i] -notmatch '^[0-
9.+-]' -and
                $i + 1 -lt $Exprs.Count -and $pair -match '^root ') {
                $joined[-1] = $pair + ' ' + $Exprs[$i + 1]
                $i++
            }
        } else {
```

```
            $joined += $Exprs[$i]
        }
    }
    foreach ($e in $joined) { Invoke-Calc $e }
}
```

Листинг А.18 – Сценарий *calc.ps*1

```
# tempconv.ps1 - конвертер температур (уровень 8, PowerShell)
# Перевод между Кельвином, Цельсием и Фаренгейтом.
# Исходная единица определяется по суффиксу: 100K, 100C или 100F.
# Файл с исходными значениями, по одному в строке. Нужен для
автоматического
# прогона: Read-Host игнорирует перенаправленный stdin и блокируется.
#
# Ключ -FileIn разбирается вручную из $args, а не объявляется
параметром:
# объявленный [string]$FileIn является позиционным и первым же
значением
# (100K, 100C, ...) перехватывался как путь к файлу - скрипт падал с
# FileNotFoundException вместо convertsения температур.

$FileIn = $null
$values = @()
if ($args.Count -gt 0 -and $args[0] -eq "-FileIn") {
    $FileIn = $args[1]
    if ($args.Count -gt 2) { $values = $args[2..($args.Count - 1)] }
} else {
    $values = $args
}

# Файл ввода читаем байтами и декодируем кодовой страницей консоли:
# Get-Content без -Encoding берёт ANSI и ломает кириллицу (866 против
1251).
function Get-ConsoleLines([string]$path) {
    $enc = [System.Text.Encoding]::GetEncoding([int]
[Console]::OutputEncoding.CodePage)
    $text = $enc.GetString([System.IO.File]::ReadAllBytes($path))
    foreach ($l in ($text -split "`r?`n")) {
        if ($l.Trim() -eq "") { continue }
        $l.Trim()
    }
}

function To-Kelvin([double]$v, [string]$unit) {
    switch ($unit.ToUpper()) {
        "K" { return $v }
        "C" { return $v + 273.15 }
        "F" { return ($v - 32) * 5 / 9 + 273.15 }
        default { return $null }
    }
}
function From-Kelvin([double]$k, [string]$unit) {
    switch ($unit.ToUpper()) {
        "K" { return $k }
        "C" { return $k - 273.15 }
        "F" { return ($k - 273.15) * 9 / 5 + 32 }
        default { return $null }
    }
}
```

```
function Convert-Value([string]$arg) {
    if ($arg -notmatch '^\s*(-?\d+(?:\.\d+)?)\s*([KCF])\s*$') {
        Write-Host "ОШИБКА: неверный ввод: '$arg'. Ожидается число и
суффикс K, C или F" -ForegroundColor Red
        return
    }
    $num = [double]$matches[1]
    $unit = $matches[2]
    $k = To-Kelvin $num $unit
    if ($null -eq $k) { Write-Host "ОШИБКА: неизвестная единица
$unit" -ForegroundColor Red; return }
    Write-Host ("{0}{1} = {2:N2} K = {3:N2} C = {4:N2} F" -f $num,
$unit.ToUpper(), $k,
        (From-Kelvin $k "C"), (From-Kelvin $k "F"))
}

Write-Host "=== Конвертер температур ==="
Write-Host "Поддерживаемые единицы: K (Кельвин), C (Цельсий), F
(Фаренгейт)"
Write-Host "Формат ввода: 100K, 100C или 100F"
Write-Host ""

if ($FileIn) {
    # Режим ввода из файла: Read-Host игнорирует перенаправленный
stdin,
    # поэтому для автоматического прогона источник входа задаётся
явно.
    Write-Host "--- Режим ввода из файла: $FileIn ---"
    foreach ($line in (Get-ConsoleLines $FileIn)) {
        if ($line -match '^(выход|exit|quit)$') { break }
        Convert-Value $line
    }
} elseif ($values.Count -eq 0) {
    Write-Host "--- Интерактивный режим, для выхода введите выход
---"
    while ($true) {
        $line = Read-Host "temp>"
        if ($line -match '^(выход|exit|quit)$') { Write-Host "Работа
завершена."; break }
        Convert-Value $line
    }
} else {
    foreach ($v in $values) { Convert-Value $v }
}
```

Листинг А.19 – Сценарий *tempconv.ps*1

```
# path_count.ps1 - подсчёт исполняемых и неисполняемых команд в PATH
(уровень 8)
$ErrorActionPreference = "SilentlyContinue"

Write-Host
"============================================================="
Write-Host " СТАТИСТИКА ПО КАТАЛОГАМ PATH"
Write-Host
"============================================================="

$env:PATH -split ';' | Where-Object { $_ -ne '' } | ForEach-Object {
    $dir = $_
    if (Test-Path $dir) {
        $items = @(Get-ChildItem -Path $dir -File)
        $exec = @($items | Where-Object { $_.Extension -match '^\.
(exe|com|bat|cmd|ps1)$' })
        $nonexec = $items.Count - $exec.Count
        $total = $items.Count
        $pct = if ($total -gt 0) { [math]::Round(100.0 *
$exec.Count / $total, 1) } else { 0 }
        "{0,-52} {1,8} {2,8} {3,6}%" -f $dir, $exec.Count, $nonexec,
$pct
    } else {
        "{0,-52} {1,8}" -f $dir, "нет такого каталога"
    }
}

Write-Host ""
Write-Host
"============================================================="
Write-Host(" ИТОГО ПО PATH")
Write-Host
"============================================================="

$allexec = 0; $allnonexec = 0; $found = 0
$env:PATH -split ';' | Where-Object { $_ -ne '' } | ForEach-Object {
    if (Test-Path $_) {
        $found++
        $items = @(Get-ChildItem -Path $_ -File)
        $allexec += @($items | Where-Object { $_.Extension -match
'^\.(exe|com|bat|cmd|ps1)$' }).Count
        $allnonexec += @($items | Where-Object { $_.Extension
-notmatch '^\.(exe|com|bat|cmd|ps1)$' }).Count
    }
}
Write-Host "Каталогов в PATH:        $found"
Write-Host "Исполняемых команд:      $allexec"
Write-Host "Неисполняемых файлов:    $allnonexec"
Write-Host "Всего файлов:            $($allexec + $allnonexec)"
if (($allexec + $allnonexec) -gt 0) {
    $share = [math]::Round(100.0 * $allexec / ($allexec +
$allnonexec), 1)
    Write-Host "Доля исполняемых:        $share%"
```

```
}
```

Листинг А.20 – Сценарий *path_count.ps*1

```
# tree_sizes.ps1 - вывод дерева каталогов с размерами файлов (уровень
8)
# Для каталогов - количество файлов внутри, для файлов - размер
# в KB/MB/GB (десятичные) и в KiB/MiB/GiB (двоичные).
param([string]$Path = ".")

$ErrorActionPreference = "SilentlyContinue"

function Format-Decimal($bytes) {
    if ($bytes -ge 1GB) { return "{0:N2} GB" -f ($bytes / 1GB) }
    if ($bytes -ge 1MB) { return "{0:N2} MB" -f ($bytes / 1MB) }
    if ($bytes -ge 1KB) { return "{0:N2} KB" -f ($bytes / 1KB) }
    return "$bytes B"
}

function Format-Binary($bytes) {
    if ($bytes -ge 1GB) { return "{0:N2} GiB" -f ($bytes / 1GB) }
    if ($bytes -ge 1MB) { return "{0:N2} MiB" -f ($bytes / 1MB) }
    if ($bytes -ge 1KB) { return "{0:N2} KiB" -f ($bytes / 1KB) }
    return "$bytes B"
}

Write-Host ""
Write-Host
"============================================================="
Write-Host " ДЕРЕВО КАТАЛОГОВ: $Path"
Write-Host " Формат размера файла: <десятичные> / <двоичные>"
Write-Host
"============================================================="

$items = @(Get-ChildItem -Path $Path -Recurse | Sort-Object
{ $_.FullName })
Write-Host "[DIR ] $($Path.PadRight(45)) размер: $(Format-Decimal
0) / $(Format-Binary 0)"

foreach ($i in $items) {
    $rel = $i.FullName.Replace((Get-Location).Path, ".")
    if ($i.PSIsContainer) {
        $count = @(Get-ChildItem -Path $i.FullName -File
-Recurse).Count
        Write-Host ("[DIR ] {0,-45} файлов: {1}, размер: {2} / {3}"
-f $rel.PadRight(45), $count, (Format-Decimal 0), (Format-Binary 0))
    } else {
        Write-Host ("[FILE] {0,-45} {1} / {2}" -f $rel.PadRight(45),
(Format-Decimal $i.Length).PadLeft(12), (Format-Binary
$i.Length).PadLeft(12))
    }
}
Write-Host
"============================================================="
```

Листинг А.21 – Сценарий *tree_sizes.ps*1

```
# top5.ps1 - монитор топ-5 процессов по загрузке процессора и памяти
(уровень 9)
# Параметр -Interval задаёт период обновления в секундах (по
умолчанию 5),
# -Count - сколько кадров вывести (0 = бесконечно, до Ctrl+C).
#
# Clear-Host при перенаправлении вывода очищает уже записанный буфер,
# поэтому в режиме с файлом вывода (-Count > 0) очистки нет: иначе
каждый
# кадр, кроме последнего, просто исчезает из вывода.
param(
    [int]$Interval = 5,
    [int]$Count = 0
)

if ($Interval -le 0) {
    Write-Host "Ошибка: интервал должен быть положительным целым
числом секунд" -ForegroundColor Red
    exit 1
}

function Format-Dep {
    param([long]$Value)
    if ($Value -ge 1GB) { return "{0:N2} GB" -f ($Value / 1GB) }
    if ($Value -ge 1MB) { return "{0:N2} MB" -f ($Value / 1MB) }
    if ($Value -ge 1KB) { return "{0:N2} KB" -f ($Value / 1KB) }
    return "$Value B"
}

$frame = 0
while ($true) {
    # Clear-Host только в бесконечном режиме на экране.
    if ($Count -eq 0) { Clear-Host }

    $frame++
    $now = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

    $byCpu = @(Get-Process | Sort-Object CPU -Descending | Select-
Object -First 5)
    $byMem = @(Get-Process | Sort-Object WorkingSet64 -Descending |
Select-Object -First 5)

    Write-Host
"============================================================"
    Write-Host " МОНИТОР ПРОЦЕССОВ - ТОП-5"
    Write-Host
"============================================================"
    Write-Host " Время:   $now"
    Write-Host " Интервал: $Interval c   Кадр: $frame"
    Write-Host
"============================================================"
    Write-Host ""
    Write-Host " ТОП-5 ПО ЗАГРУЗКЕ ПРОЦЕССОРА"
```

```
    Write-Host (" {0,-24} {1,12} {2,14}" -f "ИМЯ ПРОЦЕССА", "CPU
(c)", "ПАМЯТЬ")
    Write-Host ("-" * 52)
    foreach ($p in $byCpu) {
        $cpu = if ($null -ne $p.CPU) { [math]::Round($p.CPU, 1) }
else { 0 }
        Write-Host (" {0,-24} {1,12} {2,14}" -f $p.ProcessName, $cpu,
(Format-Dep $p.WorkingSet64))
    }

    Write-Host ""
    Write-Host " ТОП-5 ПО ИСПОЛЬЗОВАНИЮ ПАМЯТИ"
    Write-Host (" {0,-24} {1,14} {2,12}" -f "ИМЯ ПРОЦЕССА", "ПАМЯТЬ",
"CPU (c)")
    Write-Host ("-" * 52)
    foreach ($p in $byMem) {
        $cpu = if ($null -ne $p.CPU) { [math]::Round($p.CPU, 1) }
else { 0 }
        Write-Host (" {0,-24} {1,14} {2,12}" -f $p.ProcessName,
(Format-Dep $p.WorkingSet64), $cpu)
    }
    Write-Host ""

    if ($Count -gt 0 -and $frame -ge $Count) { break }
    Start-Sleep -Seconds $Interval
}

Write-Host "Монитор остановлен: выведено кадров - $frame"
```

Листинг А.22 – Сценарий *top*5.*ps*1